TEX BY TOPIC, A TEXNICIAN'S REFERENCE

VICTOR EIJKHOUT

版权所有 © 2007 Victor Eijkhout。
特此授权，任何人可在遵循 GNU 自由文档许可证（GNU Free Documentation License）第 1.2 版或自由软件基金会（Free Software Foundation）日后发布的任何更新版本的条款下，复制、分发和/或修改本文档；本文档无不变章节（Invariant Sections）、无封面文本（Front-Cover Texts）、无封底文本（Back-Cover Texts）。该许可证的副本收录在题为“GNU 自由文档许可证”的章节中。本文档基于著作《TEX by Topic》，版权所有 1991–2007 Victor Eijkhout。该书于 1991 年由 Addison-Wesley UK 出版，ISBN 0-201-56882-9，1993 年重印，pdf 版本于 2001 年首次免费公开。

目录

许可证 *13*
1 **TEX 处理器的结构** *18*
1.1 四个 TEX 处理器 *18*
1.2 输入处理器 *19*
1.2.1 字符输入 *19*
1.2.2 两级输入处理 *19*
1.3 展开处理器 *19*
1.3.1 展开的过程 *20*
1.3.2 特殊情形：\expandafter、\noexpand 与 \the *20*
1.3.3 展开处理器中的花括号 *20*
1.4 执行处理器 *21*
1.5 视觉处理器 *22*
1.6 例子 *22*
1.6.1 被跳过的空格 *22*
1.6.2 内部量及其表示 *22*
2 **类别码与内部状态** *23*
2.1 引言 *23*
2.2 初始处理 *23*
2.3 类别码 *24*
2.4 从字符到记号 *25*
2.5 作为有限状态自动机的输入处理器 *26*
2.5.1 *N* 状态：新行 *26*
2.5.2 *S* 状态：跳过空格 *26*
2.5.3 *M* 状态：行中 *26*
2.6 访问完整字符集 *26*
2.7 内部状态之间的转移 *26*
2.7.1 0：转义字符 *27*
2.7.2 1–4、7–8、11–13：非空白字符 *27*
2.7.3 5：行结束 *27*
2.7.4 6：参数 *27*
2.7.5 7：上标 *27*
2.7.6 9：忽略字符 *27*
2.7.7 10：空格 *27*
2.7.8 14：注释 *28*
2.7.9 15：无效 *28*
2.8 字母与其他字符 *28*
2.9 \par 记号 *29*
2.10 空格 *29*
2.10.1 被跳过的空格 *29*
2.10.2 可选空格 *29*

2.10.3 被忽略与被遵从的空格 30
2.10.4 更多被忽略的空格 31
2.10.5 ⟨space token⟩ 31
2.10.6 控制空格 31
2.10.7 ‘ ’ 31
2.11 再谈行尾 32
2.11.1 遵从行尾 32
2.11.2 改变 \endlinechar 32
2.11.3 关于行结束字符的更多说明 33
2.12 再谈输入处理器 33
2.12.1 把输入过程看作独立过程 33
2.12.2 不要把输入处理器看作独立过程 34
2.12.3 输入处理器的递归调用 34
2.13 @ 约定 34
3 字符 35
3.1 字符代码 35
3.2 用于字符的控制序列 36
3.3 指定需要排版的字符：\char 36
3.3.1 隐式字符记号：\let 37
3.4 重音 38
3.5 测试字符 38
3.6 大写与小写 39
3.6.1 大写代码与小写代码 39
3.6.2 大写命令与小写命令 39
3.6.3 关键字的大写与小写形式 40
3.6.4 \uppercase 与 \lowercase 的创造性用法 40
3.7 一个字符的各种代码 40
3.8 把记号转换为字符串 40
3.8.1 控制序列的输出 41
3.8.2 \string 的类别代码 41
4 字体 42
4.1 字体 42
4.2 字体声明 43
4.2.1 字体与 tfm 文件 43
4.2.2 查询当前字体与字体名 43
4.2.3 \nullfont 44
4.3 字体信息 44
4.3.1 字体维度 44
4.3.2 字距调整（kerning） 44
4.3.3 斜体修正（italic correction） 45
4.3.4 连字（ligatures） 45
4.3.5 边界连字（boundary ligatures） 45
5 盒子 46
5.1 *Boxes* 47
5.2 盒子寄存器 47
5.2.1 分配：\newbox 47
5.2.2 用法：\setbox、\box、\copy 48
5.2.3 测试：\ifvoid、\ifhbox、\ifvbox 48
5.2.4 \lastbox 48
5.3 盒子的自然尺寸 49

5.3.1 所构造水平盒子的尺寸 49
5.3.2 所构造竖直盒子的尺寸 49
5.3.3 例子 50
5.4 再谈盒子尺寸 51
5.4.1 预定尺寸 51
5.4.2 改变盒子尺寸 51
5.4.3 移动盒子 52
5.4.4 盒子尺寸与盒子放置 52
5.4.5 盒子与负胶 53
5.5 过满与过松盒子 53
5.6 打开与关闭盒子 54
5.7 拆开盒子 54
5.8 盒子中的文本 55
5.9 杂注 56
5.9.1 忘记 \box 56
5.9.2 特殊用途盒子 56
5.9.3 水平模式中竖直盒子的高 56
5.9.4 竖直盒子的更多微妙之处 56
5.9.5 把 \lastbox 挂回列表 57
5.9.6 用 \lastbox 解剖段落 57
6 **水平与竖直模式** 59
6.1 水平与竖直模式 59
6.1.1 水平模式 59
6.1.2 竖直模式 60
6.2 水平与竖直命令 60
6.3 内部模式 61
6.3.1 受限水平模式 61
6.3.2 内部竖直模式 61
6.4 盒子与模式 61
6.4.1 在什么模式中使用什么盒子？ 61
6.4.2 什么盒子中处于什么模式？ 61
6.4.3 盒子的模式相关行为 62
6.5 模式与胶 62
6.6 迁移材料 62
6.6.1 \vadjust 62
6.7 测试模式 63
7 **数** 65
7.1 数与 ⟨*number*⟩ 65
7.2 整数 65
7.2.1 表示法：整数 66
7.2.2 表示法：字符 66
7.2.3 内部整数 66
7.2.4 ⟨special integer⟩ 67
7.2.5 其他内部量：强制转换为整数 67
7.2.6 尾随空格 68
7.3 数的转换 68
7.4 整数寄存器 68
7.5 算术 69
7.5.1 算术语句 69
7.5.2 浮点算术 69

7.5.3 定点算术 69
7.6 数测试 69
7.7 备注 70
7.7.1 字符常量 70
7.7.2 展开过度/展开多少 70
8 **尺寸与胶** 71
8.1 ⟨*glue*⟩ 与 ⟨*dimen*⟩ 的定义 71
8.1.1 尺寸的定义 72
8.1.2 胶的定义 73
8.1.3 ⟨glue⟩ 到 ⟨dimen⟩ 的转换 73
8.1.4 `\dimen` 与 `\skip` 的寄存器 74
8.1.5 算术：加法 74
8.1.6 算术：乘法与除法 75
8.2 再谈尺寸 75
8.2.1 度量单位 75
8.2.2 尺寸测试 76
8.2.3 已定义的尺寸 76
8.3 再谈胶 76
8.3.1 拉伸与收缩 76
8.3.2 胶的设置 77
8.3.3 劣度 77
8.3.4 胶与断点 78
8.3.5 `\kern` 78
8.3.6 胶与模式 78
8.3.7 列表中的最后一个胶项：退格 78
8.3.8 退格示例 79
8.3.9 跟踪输出中的胶 79
9 **线段与引线** 81
9.1 线段 81
9.1.1 线段的尺寸 82
9.2 引线 82
9.2.1 线段引线 83
9.2.2 盒子引线 83
9.2.3 均匀间距的引线 84
9.3 杂注 84
9.3.1 线段与模式 84
9.3.2 用引线结束段落 84
9.3.3 引线与盒子寄存器 85
9.3.4 引线盒子中的输出 85
9.3.5 跟踪输出中的盒子引线 85
9.3.6 引线与偏移的边距 85
10 **分组** 86
10.1 分组机制 86
10.2 局部与全局赋值 87
10.3 分组定界符 87
10.4 再谈花括号 88
10.4.1 花括号计数器 88
10.4.2 作为记号的花括号 88
10.4.3 开与闭花括号控制符 89
11 **宏** 90

11.1 引言 90
11.2 宏定义的结构 90
11.3 前缀 91
11.4 定义类型 91
11.5 参数文本 92
11.5.1 未定界参数 92
11.5.2 定界参数 93
11.5.3 定界参数示例 93
11.5.4 空自变量 94
11.5.5 宏参数字符 94
11.5.6 花括号定界 95
11.6 控制序列的构造 95
11.7 通过 \let 与 \futurelet 进行记号赋值 96
11.7.1 \let 96
11.7.2 \futurelet 96
11.8 杂注 97
11.8.1 活动字符 97
11.8.2 宏与原语 97
11.8.3 尾递归 97
11.9 宏技巧 98
11.9.1 数目未知的自变量 98
11.9.2 检查自变量 98
11.9.3 用 \futurelet 实现可选宏参数 100
11.9.4 两步宏 100
11.9.5 注释环境 100
12 **展开** 103
12.1 引言 103
12.2 普通展开 103
12.3 颠倒展开顺序 104
12.3.1 一步展开：\expandafter 104
12.3.2 完全展开：\edef 104
12.3.3 \afterassignment 105
12.3.4 \aftergroup 106
12.4 阻止展开 106
12.4.1 \noexpand 107
12.4.2 \noexpand 与活动字符 107
12.5 \relax 107
12.5.1 \relax 与 \csname 108
12.5.2 用 \relax 阻止展开 108
12.5.3 TEX 插入一个 \relax 108
12.5.4 非宏的值：\the 109
12.6 例子 109
12.6.1 后置展开 109
12.6.2 在 \edef 内部定义 110
12.6.3 展开与 \write 110
12.6.4 \edef 内部的受控展开 111
12.6.5 多层阻止展开 112
12.6.6 更多 \relax 示例 112
12.6.7 示例：类别码的保存与恢复 113
12.6.8 组合 \aftergroup 与盒子 113

12.6.9 更多展开 114
13 **条件语句** 115
13.1 条件语句的形式 115
13.2 字符与控制序列测试 116
13.2.1 \if 116
13.2.2 \ifcat 116
13.2.3 \ifx 116
13.3 模式测试 117
13.4 数值测试 117
13.5 其他测试 117
13.5.1 长度测试 117
13.5.2 盒子测试 118
13.5.3 输入输出测试 118
13.5.4 case 语句 118
13.5.5 特殊测试 118
13.6 \newif 宏 118
13.7 条件语句的求值 120
13.8 杂项说明 120
13.8.1 测试会吞掉记号 120
13.8.2 测试会试图吞掉 \else 或 \fi 120
13.8.3 宏与条件语句；\expandafter 的用法 121
13.8.4 不正确的匹配 122
13.8.5 条件语句与分组 122
13.8.6 一个技巧 123
13.8.7 条件语句中展开的更多例子 123
14 **记号表** 125
14.1 记号表 125
14.2 记号表的用法 125
14.3 ⟨*token parameter*⟩ 126
14.4 记号表寄存器 126
14.5 例子 126
14.5.1 记号表上的操作：栈宏 127
14.5.2 执行记号表 127
15 **基线间距** 129
15.1 行间胶 129
15.2 盒子的感知深度 131
15.3 术语 131
15.4 补充说明 131
16 **段落的开始** 133
16.1 段落何时开始 133
16.2 段落开始时发生了什么 134
16.3 杂注 134
16.3.1 用一个盒子开始段落 134
16.3.2 用一个组开始段落 134
16.4 例子 134
16.4.1 可拉伸的缩进 134
16.4.2 抑制缩进 135
16.4.3 一种缩进方案 135
16.4.4 段间距方案 135
17 **段落的结束** 137

17.1 段落结束的方式 137
17.1.1 \par 命令与 \par 记号 137
17.1.2 段落填充：\parfillskip 138
17.2 杂注 138
17.2.1 同时结束段落和组 138
17.2.2 用 \hfill\break 结束段落 138
17.2.3 用线段结束段落 139
17.2.4 段落之间不允许分页 139
17.2.5 有限的 \parfillskip 139
17.2.6 对不缩进段落的一个预防措施 139
18 段落形状 140
18.1 文本行的宽度 140
18.2 形状参数 140
18.2.1 悬挂缩进 140
18.2.2 一般段落形状：\parshape 141
18.3 杂注 142
18.3.1 居中的最后一行 142
18.3.2 向页边方向缩进 142
18.3.3 把段落悬挂在某个对象旁 142
18.3.4 实现悬挂缩进的另一种方法 143
18.3.5 悬挂缩进与 \leftskip 平移的对比 143
18.3.6 更多例子 144
19 断行 145
19.1 段落断行代价的计算 146
19.1.1 劣度 146
19.1.2 罚值与其他断点位置 146
19.1.3 不良度 147
19.1.4 段落的行数 147
19.1.5 行与行之间 148
19.2 断行的过程 148
19.2.1 三遍 148
19.2.2 容差值 148
19.3 *discretionary* 项 148
19.3.1 连字符与 discretionary 149
19.3.2 discretionary 的例子 149
19.4 断字 150
19.4.1 单词的开头 150
19.4.2 单词的结尾 150
19.4.3 TEX2 与 TEX3 的对比 150
19.4.4 模式与例外词 151
19.4.5 切换断字模式 151
20 间距 153
20.1 引言 153
20.2 自动词间空格 153
20.3 用户设定的词间空格 154
20.4 控制空格与连字符（*tie*） 154
20.5 空格因子补遗 155
20.5.1 空格因子的赋值 155
20.5.2 标点 155
20.5.3 其他非字母字符 156

20.5.4 影响空格因子的其他因素 156
21 **数学模式中的字符** 157
21.1 数学字符 157
21.2 分隔符 158
21.2.1 分隔符码 158
21.2.2 显式 \delimiter 命令 159
21.2.3 寻找分隔符；后继字符 159
21.2.4 \big、\Big、\bigg 和 \Bigg 分隔符宏 160
21.3 根号 160
21.4 数学重音符号 160
22 **公式中的字体** 162
22.1 确定数学模式中字符的字体 162
22.2 初始字体族设置 162
22.3 字体族的定义 163
22.4 若干具体的字体更改 163
22.4.1 更改普通字符和大写希腊字母的字体 163
22.4.2 独立于文本字体地更改大写希腊字母 163
22.4.3 更改小写希腊字母和数学符号的字体 164
22.5 杂项说明 164
22.5.1 公式中的新字体 164
22.5.2 评估字体族 164
23 **数学排版** 165
23.1 数学模式 166
23.2 数学模式中的样式 166
23.2.1 上标与下标 167
23.2.2 样式的选择 167
23.3 数学对象的类别 167
23.4 大运算符及其上下限 168
23.5 竖直居中：\vcenter 169
23.6 数学间距：mu 黏合 169
23.6.1 mu 黏合的分类 169
23.6.2 数学胶（muskip）寄存器 170
23.6.3 数学模式中的其他间距 170
23.7 广义分式 171
23.8 下划线与上划线 171
23.9 数学公式中的断行 171
23.10 字体族 2 和 3 的字体尺寸 171
23.10.1 符号字体属性 172
23.10.2 扩展字体属性 172
23.10.3 例子：下标下移 173
24 **展示公式** 174
24.1 展示公式 174
24.2 段落中的展示公式 175
24.3 展示公式周围的竖直材料 175
24.4 展示数学列表的黏合设置 175
24.5 展示公式的居中：偏移量 176
24.6 公式编号 176
24.6.1 普通公式编号 176
24.6.2 单独成行的公式编号 177
24.7 非居中的展示公式 177

25 对齐 178
25.1 引言 178
25.2 水平对齐与竖直对齐 178
25.2.1 水平对齐：\halign 179
25.2.2 竖直对齐：\valign 179
25.2.3 行间材料：\noalign 179
25.2.4 对齐的尺寸 179
25.3 导言区 180
25.3.1 无限导言区 180
25.3.2 导言区中的花括号计数 180
25.3.3 导言区中的展开 180
25.3.4 \tabskip 180
25.4 对齐主体 181
25.4.1 读取条目 181
25.4.2 替代规格：\omit 182
25.4.3 跨越多列：\span 182
25.4.4 对齐中的线 183
25.4.5 行结束：\cr 与 \crcr 183
25.5 例子：数学对齐 184
26 页面形状 185
26.1 全局定位的参考点 185
26.2 \topskip 185
26.3 页面高度与深度 186
27 分页 187
27.1 当前页面与近期贡献 187
27.2 激活页面构造器 188
27.3 页面长度记录 188
27.4 断点 189
27.4.1 可能的断点 189
27.4.2 断点惩罚 189
27.4.3 断点计算 189
27.5 \vsplit 190
27.6 分页示例 191
27.6.1 填满页面 191
27.6.2 确定断点 191
27.6.3 段落后的页面构造器 192
28 输出例程 193
28.1 \output 记号列表 193
28.2 输出与 \box255 193
28.3 标记 194
28.4 杂注 195
28.4.1 非平凡输出例程中的隐患 195
28.4.2 页码 195
28.4.3 plain TEX 中的页眉和页脚 195
28.4.4 例子：避免寡行 196
28.4.5 例子：页面顶部不缩进 196
28.4.6 更多输出例程例子 197
29 插入 198
29.1 插入项 198
29.2 插入类声明 199

29.3 插入参数 199
29.4 从贡献列表中移动插入项 199
29.5 输出例程中的插入项 200
29.6 *plain* TEX 插入 200
30 **文件输入与输出** 202
30.1 包含文件：\input 与 \endinput 202
30.2 文件 I/O 202
30.2.1 打开与关闭流 203
30.2.2 用 \read 输入 203
30.2.3 用 \write 输出 203
30.3 *Whatsit* 项 204
30.4 杂注 204
30.4.1 检查输入 204
30.4.2 测试文件是否存在 204
30.4.3 时序问题 205
30.4.4 \message 与 \immediate\write16 的对比 205
30.4.5 竖直盒子中的 write 205
30.4.6 \write 和 \message 中的展开与空格 205
31 **寄存器的分配** 207
31.1 分配命令 207
31.1.1 \count、\dimen、\skip、\muskip、\toks 207
31.1.2 \box、\fam、\write、\read、\insert 208
31.2 宏编写者的基本规则 208
32 **运行 TEX** 209
32.1 作业 209
32.1.1 作业的开始 209
32.1.2 作业的结束 210
32.1.3 日志文件 210
32.2 运行模式 210
33 **TEX 与外部世界** 211
33.1 *TEX*、*IniTEX*、*VirTEX* 211
33.1.1 格式的装入 211
33.1.2 格式的转储 212
33.1.3 格式的预装 212
33.1.4 IniTEX 的知识 212
33.1.5 TEX 与 IniTEX 的内存大小 213
33.2 关于格式的进一步说明 213
33.2.1 兼容性 213
33.2.2 预装字体 213
33.2.3 plain 格式 213
33.2.4 LATEX 格式 213
33.2.5 数学格式 214
33.2.6 其他格式 214
33.3 dvi 文件 214
33.3.1 dvi 文件格式 214
33.3.2 页面标识 215
33.3.3 放大倍数 215
33.4 特殊命令 215
33.5 时间 215
33.6 字体 216

33.6.1 字体度量 216
33.6.2 虚拟字体 216
33.6.3 字体文件 216
33.6.4 Computer Modern 217
33.7 *TEX 与 web* 217
33.8 *TEX 用户组织* 217
34 **追踪与统计** 219
34.1 含义与内容：\show、\showthe、\meaning 219
34.2 显示盒子：\showbox、\tracingoutput 220
34.3 总体统计 222
35 **错误、灾难与求助** 223
35.1 错误信息 223
35.2 溢出错误 224
35.2.1 缓冲区大小 (500) 224
35.2.2 例外词典 (307) 224
35.2.3 字体内存 (20 000) 224
35.2.4 分组层级 224
35.2.5 散列表大小 (2100) 224
35.2.6 字符串数目 (3000) 225
35.2.7 输入栈大小 (200) 225
35.2.8 主存大小 (30 000) 225
35.2.9 参数栈大小 (60) 225
35.2.10 模式内存 (8000) 225
35.2.11 每种语言的模式内存 op 数 225
35.2.12 字符串池大小 (32 000) 225
35.2.13 保存栈大小 (600) 226
35.2.14 语义嵌套栈大小 (40) 226
35.2.15 文本输入层级 (6) 226
36 **TEX 的语法** 227
36.1 记号 227
36.2 关键字 227
36.3 特定文法符号 228
36.3.1 ⟨equals⟩ 228
36.3.2 ⟨filler⟩、⟨general text⟩ 228
36.3.3 {} 与 ⟨left brace⟩⟨right brace⟩ 228
36.3.4 ⟨math field⟩ 229
36.4 *TEX 第 2 版与第 3 版的差异* 229
37 **TEX 原语术语表** 230
38 **表格** 241
38.1 字符表 242
38.2 计算机现代字体 244
38.3 *Plain TEX 数学符号* 249
38.3.1 数学字符代码 249
38.3.2 定界符代码 250
38.3.3 ⟨mathchardef⟩ 记号：普通符号 251
38.3.4 ⟨mathchardef⟩ 记号：大型算符 252
38.3.5 ⟨mathchardef⟩ 记号：二元运算 253
38.3.6 ⟨mathchardef⟩ 记号：关系符 254
38.3.7 \delimiter 宏 255

许可证 GNU 自由文档许可证

第 1.2 版，2002 年 11 月

版权所有 © 2000、2001、2002 自由软件基金会（Free Software Foundation, Inc.）51 Franklin St, Fifth Floor, Boston, MA 02110-1301 USA 特此允许复制和分发本许可证文档的逐字副本，但不允许对其进行任何修改。

0. 前言

本许可证的目的，是使一本手册、教科书或其他有用的功能性文档在自由的意义上成为“自由的”：确保每个人都能有效地自由复制和再分发它，无论是否修改、无论是否用于商业目的。其次，本许可证也为作者和出版者保留了一种获得署名的方式，同时又不至于让他们对他人所作的修改承担责任。

本许可证是一种“copyleft”（著佐权），这意味着本文档的衍生作品本身也必须在同样的意义下是自由的。它与 GNU 通用公共许可证（GNU General Public License）互为补充；后者是为自由软件设计的 copyleft 许可证。

我们设计本许可证，是为了把它用于自由软件的手册，因为自由软件需要自由的文档：一个自由程序应当附带享有同等自由的手册。但本许可证并不局限于软件手册；它可用于任何文本作品，无论主题为何，也无论是否以印刷书籍的形式出版。我们主要向那些用于教学或参考的作品推荐本许可证。

1. 适用范围与定义

本许可证适用于任何载体中的任何手册或其他作品，只要它包含版权所有者所放置的声明，说明该作品可在本许可证条款下分发。此类声明授予一项全球范围、免版税、期限不限的许可，允许在本文所述条件下使用该作品。下文所称“文档”（Document），即指任何此类手册或作品。公众中的任何成员都是被许可人，下称“你”（you）。如果你以版权法要求许可的方式复制、修改或分发该作品，即视为你接受本许可证。

文档的“修改版本”（Modified Version），指任何包含该文档或其一部分的作品，无论是逐字复制，还是经过修改和/或翻译成另一种语言。

“次级章节”（Secondary Section），指文档中一个具名的附录或前置部分章节，其内容专门讨论文档的出版者或作者与文档整体主题（或相关事项）之间的关系，且不包含任何可直接归入该整体主题的内容。（因此，如果文档部分内容是一本数学教科书，则次级章节不得解释任何数学内容。）这种关系可以是与该主题或相关事项在历史上的联系，也可以是关于它们在法律、商业、哲学、伦理或政治立场上的观点。

“不变章节”（Invariant Sections），指某些次级章节，其标题在说明本文档依本许可证发布的声明中被指定为不变章节。如果某一章节不符合上述次级章节的定义，则不得被指定为不变章节。文档可以不包含任何不变章节。如果文档没有指明任何不变章节，则意味着没有。

“封面文本”（Cover Texts），指某些在声明文档依本许可证发布的告示中，作为封面文本或封底文本列出的简短文字。封面文本至多可有 5 个词，封底文本至多可有 25 个词。

文档的“透明”（Transparent）副本，指一种机器可读的副本，其所采用格式的规格向公众公开，便于用通用文本编辑器（对于由像素构成的图像）通用绘图程序或（对于图形）广泛可得的绘图编辑器直接加以修订，并且适合作为文本排版程序的输入，或适合自动转换为各种可供文本排版程序输入的格式。如果某副本虽采用通常透明的文件格式，但其标记（或无标记状态）被安排成会妨碍或阻碍读者随后进

行修改，则它不是透明的。如果一种图像格式被用于承载任何数量较大的文字内容，则它不是透明的。非“透明”的副本称为“不透明”（Opaque）。

适合作为透明副本的格式示例包括：无标记的纯 ASCII、Texinfo 输入格式、LaTeX 输入格式、使用公开可用 DTD 的 SGML 或 XML，以及为人工修改而设计的、符合标准的简单 HTML、PostScript 或 PDF。透明图像格式的示例包括 PNG、XCF 和 JPG。不透明格式包括：只能由专有文字处理程序读取和编辑的专有格式、其 DTD 和/或处理工具并不普遍可得的 SGML 或 XML，以及某些文字处理程序仅为输出目的而生成的机器产生的 HTML、PostScript 或 PDF。

对于印刷书籍，“标题页”（Title Page）指标题页本身，加上其后紧接的、为清晰容纳本许可证要求出现在标题页上的材料所需的若干页。对于本身没有标题页的格式作品，“标题页”指作品标题最显著出现处附近、位于正文开始之前的那一段文字。

一个“题为 XYZ”（Entitled XYZ）的章节，指文档中一个具名的子单元，其标题恰为 XYZ，或在其翻译为另一语言的文字之后以括号注明 XYZ。（这里 XYZ 代表下文提及的某个具体章节名，例如“致谢”“题献”“背书”或“历史”。）修改文档时要“保留该章节标题”，即指它按照本定义仍然是一个“题为 XYZ”的章节。

文档可以在说明本许可证适用于本文档的告示旁边包含担保免责声明。这些担保免责声明被视为通过引用而纳入本许可证，但仅限于免除担保这一方面；这些担保免责声明可能具有的任何其他含义均属无效，且不影响本许可证的含义。

2. 逐字复制

你可以在任何载体上复制和分发本文档，无论是否用于商业目的，只要在所有副本中均照录了本许可证、版权声明以及说明本许可证适用于本文档的许可证告示，并且你未在本许可证的条件之上附加任何其他条件。你不得使用技术手段来妨碍或控制你所制作或分发的副本被他人阅读或进一步复制。不过，你可以为副本收取费用。如果你分发的副本数量足够大，还必须遵循第 3 节中的条件。

你也可以在上述相同条件下出借副本，并且可以公开展示副本。

3. 批量复制

如果你公开发行本文档的印刷副本（或以通常带有印刷封面为载体的副本）数量超过 100，且文档的许可证告示要求有封面文本，则你必须将这些副本装入封面，在封面上清晰可辨地印上所有这些封面文本：封面上印封面文本（Front-Cover Texts），封底上印封底文本（Back-Cover Texts）。两个封面还必须清晰可辨地标明你是这些副本的出版者。封面必须以标题中各词同等醒目、可见的方式呈现完整标题。你也可以在封面上附加其他材料。在仅限封面有所改动的情况下进行复制，只要这些改动保留了文档标题并满足上述条件，即可视为在其他方面属于逐字复制。

如果任一封面所需的文字过多，无法清晰地排下，你应把最先列出的文字（能合理容纳多少就印多少）放在实际封面上，其余部分续印到相邻的页面上。

如果你公开发行或分发本文档的不透明副本数量超过 100，则你必须在每份不透明副本中附上一份机器可读的透明副本，或在每份不透明副本中（或随其）声明一个计算机网络地址，使使用网络的公众能够通过公共标准网络协议从中下载到本文档的一份完整透明副本，且不附加任何额外材料。如果你采用后一种方式，则在开始批量分发不透明副本时，必须采取合理审慎的措施，以确保在你最后一次向公众分发该版本的不透明副本（直接或通过代理或零售商）之后至少一年内，该透明副本仍可在所述地址处获取。

我们建议但不强制要求你在大量再分发副本之前，尽早与文档作者联系，以便他们有机会向你提供文档的更新版本。

4. 修改

你可以在遵循上述第 2、3 节条件的前提下复制和分发文档的修改版本，但前提是你必须完全以本许可证发布该修改版本，并由该修改版本充当“文档”的角色，从而使该修改版本的任何持有者都获得分发和修改它的许可。此外，你还必须在修改版本中做到以下各点：

A. 在标题页（以及若有封面，在封面上）使用一个与文档不同、也与以往版本不同的标题（以往版本若有，应列在文档的“历史”章节中）。如果该版本的原出版者许可，你可以沿用以往版本的同一标题。B. 在标题页上，作为作者列出一个或多个对修改版本中的修改负有署名责任的个人或实体，同时至少列出文档的主要作者中的五位（若主要作者不足五位，则列出全部主要作者），除非他们免除你这一要求。C. 在标题页上标明修改版本出版者的名称，作为出版者。D. 保留文档的所有版权声明。E. 在其他版权声明旁边，为你所作的修改添加一条适当的版权声明。F. 在版权声明之后立即附上一条许可证告示，以如下文附录所示的格式，许可公众在本许可证条款下使用修改版本。G. 在该许可证告示中完整保留文档许可证告示所给出的不变章节清单和所需封面文本。H. 附上一份未经改动的本许可证副本。I. 保留题为“历史”的章节，保留其标题，并在其中添加一项，至少说明修改版本的标题、年份、新作者和出版者（如标题页所列）。如果文档中没有题为“历史”的章节，则应新建一个，按标题页所列写明文档的标题、年份、作者和出版者，然后再按上一句所述添加一项说明修改版本。J. 保留文档中为公开获取文档透明副本而给出的网络地址（若有），以及文档据以改编的以往版本所给出的网络地址。这些地址可以放在“历史”章节中。对于在文档本身之前至少四年出版的作品，或其所指版本的原出版者许可的情况下，你可以省略网络地址。K. 对于任何题为“致谢”或“题献”的章节，保留该章节标题，并在其中完整保留各位贡献者致谢和/或题献的实质内容与语气。L. 保留文档的所有不变章节，其文字与标题均不作改动。章节编号或等价物不视为章节标题的一部分。M. 删除任何题为“背书”（Endorsements）的章节。此类章节不得出现在修改版本中。N. 不得把任何已有章节重新命名为“背书”，也不得使其标题与任何不变章节相冲突。O. 保留任何担保免责声明。如果修改版本包含新的前置章节或附录，且它们符合次级章节的定义、且不包含从文档复制来的材料，则你可以自行选择把其中部分或全部章节指定为不变章节。为此，应在修改版本的许可证告示中把它们的标题加入不变章节清单。这些标题必须与其他章节标题相区别。

你可以添加一个题为“背书”的章节，只要其中只包含各方对你的修改版本所作的背书——例如同行评议声明，或某组织已将该文本批准为某标准的权威定义。

你可以在修改版本封面文本清单的末尾，添加一段至多五个词的文字作为封面文本，以及一段至多 25 个词的文字作为封底文本。任何一个实体（或通过其安排）只能添加一段封面文本和一段封底文本。如果文档已经包含了由你或由你所代表的同一实体先前添加的同一封面的封面文本，则你不得再添加另一段；但在得到先前添加该文本的出版者明确许可的情况下，你可以替换旧文本。

文档的作者和出版者并不通过本许可证许可他人使用其姓名为任何修改版本作宣传，或主张或暗示任何修改版本得到其背书。

5. 组合文档

你可以按上述第 4 节关于修改版本所定义的条款，把本文档与其他在本许可证下发布的文档组合在一起，前提是你在组合体中未经改动地纳入所有原始文档的全部不变章节，并在组合作品的许可证告示中把它们全部列为组合作品的不变章节，同时保留它们各自的所有担保免责声明。

组合作品只需包含一份本许可证副本；多个完全相同的不变章节可以合并为一份。如果存在多个同名但内容不同的不变章节，则应在每个此类章节标题末尾以括号加

注该章节原作者或出版者的名称（若可知），或加注一个唯一编号，使其标题互不相同。对组合作品许可证告示中不变章节清单里的章节标题，也应作同样处理。

在组合时，你必须把各原始文档中题为“历史”的章节合并为一个“历史”章节；同样合并题为“致谢”和题为“题献”的章节。你必须删除所有题为“背书”的章节。

6. 文档汇编

你可以把本文档与其他在本许可证下发布的文档汇编成一部合集，并把各文档中各自的本许可证副本替换为合集所收录的一份副本，前提是你在所有其他方面均遵循本许可证对其中每一部文档逐字复制的规则。

你也可以从这样的合集中抽出其中一部文档，单独在本许可证下分发，前提是你把本许可证的一份副本插入该抽出的文档，并在其他所有关于逐字复制该文档的方面遵循本许可证。

7. 与独立作品聚合

如果把本文档或其衍生作品与其他独立的文档或作品汇编（在存储或发行载体的一卷之中或之上）在一起，且汇编所产生的版权未被用来限制汇编使用者在各独立作品自身许可之外的法律权利，则这种汇编称为“聚合”（aggregate）。当本文档被纳入聚合时，本许可证不适用于聚合中本身并非本文档衍生作品的其他作品。

如果第 3 节关于封面文本的要求适用于这些本文档副本，那么当本文档不足整个聚合的一半时，本文档的封面文本可以放在聚合中把本文档夹在中间的封面上；若本文档为电子形式，则放在封面的电子等价物上。否则，它们必须出现在把整个聚合夹在中间的印刷封面上。

8. 翻译

翻译被视为一种修改，因此你可以按第 4 节条款分发本文档的译文。用译文替换不变章节需要其版权所有者的特别许可，但你可以在保留这些不变章节原文的同时，附上部分或全部不变章节的译文。你可以附上本许可证以及文档中所有许可证告示和任何担保免责声明的译文，前提是你同时附上本许可证的英文原文以及那些告示和免责声明的原文。若译文与本许可证原文或某告示、免责声明之间出现分歧，以原文为准。

如果文档中某一章节题为“致谢”“题献”或“历史”，那么第 4 节要求保留其标题（见第 1 节）通常意味着必须更改实际标题。

9. 终止

除非本许可证明确许可，你不得复制、修改、再许可或分发本文档。任何以其他方式复制、修改、再许可或分发本文档的尝试均属无效，并将自动终止你在本许可证下的权利。但是，凡已从你处依据本许可证获得副本或权利的各方，只要其始终完全合规，其许可证不会被终止。

10. 本许可证未来的修订

自由软件基金会可能会不时发布 GNU 自由文档许可证的新修订版本。新版本在精神上会与当前版本相近，但在细节上可能有所不同，以应对新的问题或关切。参见 http://www.gnu.org/copyleft/ 。

本许可证的每个版本都有一个区分性的版本号。如果文档指明适用本许可证的某一特定编号版本“或任何更新版本”，你可以选择遵循该指定版本，或遵循自由软件基金会日后发布（非草案）的任何更新版本的条款。如果文档未指明本许可证的版本号，则你可以选择自由软件基金会曾经发布（非草案）的任何版本。

前言 在外行看来，TEX 似乎并不是一个最先进的排版系统。它没有花哨的多层菜单，也没有对文本和图形的交互式操作来吸引旁观者的目光。然而，若不那么表面地看，TEX 是一个非常精密的程序：首先，它内置了诸如段落断行、数学公式排版等精巧算法；其次，它几乎是完全可编程的。正是这两点结合在一起，使 TEX 能够以高度自动化的方式实现几乎任何可以想象的版式。

但遗憾的是，这同时也意味着 TEX 拥有数量异常庞大的命令与参数，而用 TEX 编程也远非易事。任何想学习 TEX 编程的人——或许甚至连普通用户也是如此——似乎都需要两本书：一本入门教程，让人先对 TEX 的诸多基本零件有个初步印象；此后再需要一本系统、完整的参考手册。本书试图承担后一种职能。一个已经入门的 TEXer （借助市面上任何一本入门书）此后便应能长期使用本书。

本书把 TEX 的世界划分为大约四十个不同的主题，每个主题单独成章。每章开头先列出与该章主题相关的控制序列清单，随后展开讨论该主题的理论。大多数章节在结尾处附有评注与示例。

从整体上看，各章顺序如下。先讲基本机制的章节，其次是文本处理与数学的章节，最后是若干关于输出以及 TEX 与外部世界相连接的方面的章节。本书还附有一份 TEX 命令词汇表、若干表格，以及按示例、按控制序列、按主题编制的索引。主题索引对大多数概念都只指向一页，该页汇集了关于该主题的大部分信息，并给出相关信息所在位置的交叉引用。

本书不讨论任何特定的 TEX 宏包。书中所涉及的 plain 格式的内容，仅限于那些属于 plain TEX“核心”的部分：这些部分同样存在于例如 LATEX 之中。因此，关于 plain 格式的大多数论述，对 LATEX 以及其他大多数格式同样成立。换句话说，若正文提到 plain 格式，应理解为它是与纯粹的 IniTEX 相对而言的，而非与 LATEX 相对而言。作为示例，书中偶尔也会解释一些不属于核心的 plain TEX 宏。

致谢
我要感谢 Barbara Beeton、Karl Berry 和 Nico Poppelier，他们阅读了本书的旧版本。他们的意见帮助改进了本书的叙述。我还要感谢讨论邮件列表 TEXhax、TEX-nl 以及 `comp.text.tex` 的参与者。他们的问题与回答给了我许多思考的素材。最后，任何一本关于 TEX 的书在致谢中都应当提到 Donald Knuth，因为是他首先发明了 TEX。本书也不例外。

Victor Eijkhout
伊利诺伊州厄巴纳，1991 年 8 月
田纳西州诺克斯维尔，2001 年 5 月

第 1 章

TEX 处理器的结构

本书把 TEX 的各个方面分章加以讨论，每章都围绕一个相对小而边界清晰的主题展开。因此在本章中，我们将给出 TEX 工作方式的整体图景。这里势必会略去许多细节，但这些都会在后续章节中逐一讨论。另一方面，本章给出的少量例子也会在后面适当的地方重复出现；此处收录它们，是为了使本章自成一体。

1.1 四个 TEX 处理器

TEX 处理其输入的方式，可以看作发生在四个层次上。可以说，TEX 处理器被分成四个独立的单元，每个单元都接收上一阶段的输出，并产出下一阶段的输入。于是，第一阶段的输入就是 `.tex` 输入文件；最后一阶段的输出则是一个 `.dvi` 文件。

在许多场合下，把这四个处理层次看作依次发生、每一层次都接收上一层次已完成的输出，是最为方便也最富有启发性的。然而实际情况并非如此：所有层次都是同时活跃的，彼此之间存在交互。

这四个层次分别如下（大致对应于 Knuth 最初术语中的「眼睛」「嘴巴」「胃」和「肠」）。

1. 输入处理器（input processor）。这是 TEX 中从运行 TEX 的那台计算机的文件系统接收输入行、并把它们转换为记号（token）的部分。记号是 TEX 的内部对象：既有构成排版文本的字符记号，也有作为命令、交由后面两个层次处理的控制序列记号。
2. 展开处理器（expansion processor）。第一层次生成的记号中，有一部分 – 宏、条件式以及若干原始 TEX 命令 – 是会被展开的。展开就是把某些（序列的）记号替换为另一些（或不替换为任何）记号的过程。
3. 执行处理器（execution processor）。不可展开的控制序列是可执行的，这种执行发生在 TEX 处理器的第三层次。
   这里活动的一部分涉及对 TEX 内部状态的改动：赋值（包括宏定义）是这一类别的典型活动。本层次发生的另一大类事情，是构造水平表、竖直表和数学表。
4. 视觉处理器（visual processor）。在最后一个处理层次中，TEX 处理的视觉部分得以完成。在这里，水平表被拆成段落，竖直表被拆成页面，公式则由数学表构建而成。向 `dvi` 文件的输出也在这一层次进行。此处运作的算法对用户是不可见的，但可以通过若干参数加以影响。

1.2 输入处理器

TEX 的输入处理器是 TEX 中把从输入文件得到的一切字符翻译为记号的那一部分。这个处理器的输出是一串记号：一个记号表（token list）。大多数记号属于两大类之一：字符记号和控制序列记号。剩下的一类是参数记号；本章不讨论它们。

1.2.1 字符输入

对于简单的输入文本，字符会被做成字符记号。不过 TEX 可以忽略输入字符：输入中一连串空格通常只相当于一个空格。此外，TEX 自身也可以插入不对应于输入中任何字符的记号，例如行尾的空格记号，或是空行之后的 `\par` 记号。

并非所有字符记号都表示要排版的字符。字符分为十六个类别 – 每一类都规定了字符可以具有的某种功能 – 其中只有两类包含将要排版的字符。其余类别包含诸如 `{`、`}`、`&` 和 `#` 这样的字符。一个字符记号可以看作一对数字：字符代码 – 通常就是 ASCII 代码 – 以及类别码。与某个特定字符代码相关联的类别码是可以更改的。

当转义字符（默认是 `\`）出现在输入中时，TEX 构造记号的行为就更为复杂。基本上，TEX 会从输入中取若干字符，把它们捆成一个单一记号，从而构成一个控制序列。

TEX 输入处理器对类别码作出反应的行为，可以描述为一台在三种内部状态之间切换的机器：N，新行（new line）；M，行中（middle of line）；S，跳过空格（skipping spaces）。这些状态以及它们之间的转移将在第 2 章中讨论。

1.2.2 两级输入处理

TEX 的输入处理器本身其实是一个两级处理器。由于终端、编辑器或操作系统的限制，用户可能无法输入某些想要的字符。因此，TEX 提供了一种机制，用两个上标字符来访问所有可用的字符位置。这可以看作 TEX 处理中的一个独立阶段，发生在前面提到的三状态机器之前。

例如，序列 `^^+` 会被替换为 `k`，因为 `k` 与 `+` 的 ASCII 代码相差 64。由于这种替换发生在记号形成之前，书写`\vs^^+ip 5cm` 与`\vskip 5cm` 效果相同。比这更有用的例子也存在。

注意，这第一个阶段是字符到字符的转换，并不考虑类别码。类别码只在输入处理的第二个阶段才开始起作用，在那一阶段，通过把类别码与字符代码耦合起来，字符被转换为字符记号。

1.3 展开处理器

TEX 的展开处理器接收一串记号，并在可能时把这串记号逐一展开，直到只剩下不可展开的记号为止。宏展开是这一点最清楚的例子：如果某个控制序列是宏名，它就会（连同可能的参数记号）被替换为该宏的定义文本。

展开处理器的输入主要由输入处理器提供。来自 TEX 处理第一阶段的记号串要经过展开过程，其结果是一串不可展开的记号，再送入执行处理器。

不过，展开处理器也会在处理 `\edef` 或 `\write`（以及其他一些）命令时发挥作用。这些命令的参数记号表会被展开，其方式很像是这些记号表位于顶层、而不是某个命令的参数。

1.3.1 展开的过程

展开一个记号由以下步骤组成：

1. 查看该记号是否可展开。
2. 如果该记号不可展开，就把它传入当前正在构造的记号表，并取下一个记号。
3. 如果该记号可展开，就用它的展开式替换它。对于不带参数的宏，以及像 `\jobname` 这样少数原始命令，这确实是简单的替换。但通常 TEX 需要从记号串中吸收一些参数记号，才能构造出当前记号的替换式。例如，如果该记号是带参数的宏，就需要吸收足够多的记号，以形成与这些参数相对应的实参。
4. 从展开式的第一个记号开始，继续展开。

判断一个记号是否可展开是一个简单的判定。宏和活动字符、条件式，以及若干原始 TEX 命令（见第 103 页的列表）是可展开的，其他记号则不可。因此展开处理器用展开式替换宏，对条件式求值并剔除其中无关的部分；而像 `\vskip` 这样的记号以及字符记号（包括美元符、花括号之类字符）则原样放行。

1.3.2 特殊情形：\expandafter、\noexpand 与 \the

如前所述，一个记号被展开之后，TEX 会接着展开所得到的记号。乍看之下，`\expandafter` 命令似乎是这条规则的一个例外，因为它只展开一步。实际发生的是，序列

`\expandafter`⟨token$_1$⟩⟨token$_2$⟩

被替换为

⟨token$_1$⟩⟨*expansion of token*$_2$⟩

而这个替换结果事实上还会被展开处理器重新检查。

不过，真正的例外确实存在。如果当前记号是 `\noexpand` 命令，那么下一个记号暂时被视为不可展开：它被当作 `\relax` 来处理，并被传入正在构造的记号表。

例如，在宏定义

```
\edef\a{\noexpand\b}
```

中，替换文本`\noexpand\b` 会在定义时被展开。`\noexpand` 的展开结果就是下一个记号，并临时具有 `\relax` 的含义。于是当展开处理器处理下一个记号（即 `\b`）时，会认为它不可展开，直接把它传入正在构造的记号表，也就是该宏的替换文本。

另一个例外是：如果 `\the`⟨token variable⟩ 这一语句出现在 `\edef` 宏定义内部，其所得记号不会再被展开。

1.3.3 展开处理器中的花括号

前面说过，花括号是作为不可展开的字符记号放行的。一般来说确实如此。例如，`\romannumeral` 命令由展开处理器处理；当遇到

```
\romannumeral1\number\count2 3{4 ...
```

时，TEX 会一直展开直到遇到花括号：如果 `\count2` 的值为零，结果就是 `103` 的罗马数字表示。

另一个例子，

```
\iftrue {\else }\fi
```

由展开处理器处理，其方式完全类似于

`\iftrue` *a*`\else` *b*`\fi`

结果是一个字符记号，与其类别无关。

不过，在宏展开的上下文中，展开处理器会识别花括号。首先，一对配对的花括号会圈出一组记号，作为一个整体实参传递。如果一个宏有形参

```
\def\macro#1{ ... }
```

既可以用单个记号来调用它，如

```
\macro 1 \macro \$
```

也可以用一组被花括号括起来的记号

```
\macro {abc} \macro {d{ef}g}
```

其次，当读取带参数宏的实参时，不接受花括号不配对的表达式。在

```
\def\a#1\stop{ ... }
```

中，实参由直到第一个不在花括号内的 `\stop` 之前的所有记号组成：在

```
\a bc{d\stop}e\stop
```

中，`\a` 的实参是 `bc{d\stop}e`。这里只接受配对的表达式。

1.4 执行处理器

执行处理器构造各种表：水平表、竖直表和数学表。与这些表相对应，它工作在水平、竖直或数学模式下。这三种模式各有「内部」和「外部」两种变体。除了构造表之外，TEX 处理器的这一部分还执行与模式无关的处理，例如赋值。

从展开处理器出来的是一串不可展开的记号，交由执行处理器处理。从执行处理器的角度看，这串记号包含两类：

- 表示赋值（包括宏定义）的记号，以及其他表示与模式无关的动作的记号，例如 `\show` 和 `\aftergroup`。
- 构造表的记号：字符、盒子和黏胶。它们的处理方式取决于当前模式。

有些对象可在任何模式中使用；例如盒子可以出现在水平表、竖直表和数学表中。当然，这类对象的效果仍取决于模式。另一些对象则专属于某一种模式。例如，字符（更确切地说，类别为 11 和 12 的字符记号）与水平模式紧密相连：如果执行处理器在竖直模式中遇到一个字符，它就会切换到水平模式。

并非所有字符记号都表示要排版的字符：执行处理器还可能遇到数学切换字符（默认是 `$`）以及组的开始/结束字符（默认是 `{` 和`}`）。数学切换字符让 TEX 进入或退出数学模式，花括号则让它进入或退出新一层分组。

执行处理器处理的一个控制序列值得特别一提：`\relax`。这个控制序列不可展开，其执行却什么也不做。比较 `\relax` 在

```
\count0=1\relax 2
```

中的效果，与由

```
\def\empty{}
```

定义的 `\empty` 在

```
\count0=1\empty 2
```

中的效果。第一种情况下，正在构造数字的展开过程在 `\relax` 处停下，于是赋值的数字是 `1`；第二种情况下，`\empty` 展开为空，于是赋值的是 `12`。

1.5 视觉处理器

TEX 的输出处理器涵盖那些不受用户直接控制的算法：断行、对齐、分页、数学排版以及 dvi 文件的生成。各种参数控制着 TEX 这些部分的运作。

这些算法中有一部分以执行处理器能够处理的形式返回结果。例如，已经拆成行的段落会作为一串水平盒子（其间夹杂黏胶和罚值）加入主竖直表。又如，分页算法把结果存入 \box255，以便输出例程对其进行剖析。另一方面，数学公式不能拆成碎片，而且理所当然地，把盒子送往 dvi 文件是不可逆的。

1.6 例子

1.6.1 被跳过的空格

被跳过的空格正好说明了这样一种观点：TEX 的各个处理层次接收的是上一层次已完成的输入。考虑命令

```
\def\a{\penalty200}
\a 0
```

这并不等价于

```
\penalty200 0
```

后者会放置一个 200 的罚值，并排版数字 0。相反，前者展开为

```
\penalty2000
```

因为 \a 后面的空格在输入处理器中被跳过了。后续处理层次接收到的便是序列

```
\a0
```

1.6.2 内部量及其表示

TEX 使用各种内部量，例如整数和尺寸。这些内部量有一个外部表示，即一串字符，例如 4711 或 91.44cm。

内部值与外部表示之间的转换发生在两个不同的层次上，取决于转换的方向。在诸如

```
\pageno=12 \baselineskip=13pt
```

这样的赋值，或诸如

```
\vskip 5.71pt
```

这样的语句中，一串字符被转换为内部值，而所有这些语句都由执行处理器处理。

另一方面，把内部值转换为一串字符的表示，则由展开处理器处理。例如，

```
\number\pageno \romannumeral\year
\the\baselineskip
```

都是通过展开来处理的。

最后举一个例子，设\count2=45 ，考虑语句

```
\count0=1\number\count2 3
```

展开处理器处理\number\count2 ，得到字符 45，而 2 后面的空格并不结束正在赋值的数字：它只是充当 \count 寄存器编号的分隔符。在下一个处理阶段，执行处理器看到的将是语句

```
\count0=1453
```

并执行它。

第 2 章

类别码与内部状态

读取字符时，TEX 会给它们赋予类别码（category code）。读取机制有三种内部状态，而这些状态之间的转移由输入字符的类别码来触发。本章描述 TEX 如何读取其输入，以及字符的类别码如何影响读取行为。还将讨论空格与行尾。

`\endlinechar` 追加到输入行末尾的行尾字符的字符代码。IniTEX 默认值：13。
`\par` 结束段落并进入竖直模式的命令。由空行生成。
`\ignorespaces` 读取并展开，直到遇到不是⟨space token⟩ 的东西为止。
`\catcode` 查询或设置类别码。
`\ifcat` 测试两个字符是否具有相同的类别码。
`\ ` 控制空格。插入与 `\spacefactor` = 1000 时一个空格记号所产生的等量空格。
`\obeylines` plain TEX 中使行尾有意义的宏。
`\obeyspaces` plain TEX 中使（大多数）空格有意义的宏。

2.1 引言

TEX 的输入处理器从文件或终端扫描输入行，并把字符做成记号。输入处理器可以看作一台简单的有限状态自动机，具有三种内部状态；根据状态的不同，其扫描行为也可能不同。这里将从内部状态以及支配转移的类别码两个角度来讨论这台自动机。

2.2 初始处理

来自文件的输入（或来自用户终端的输入，但大多数时候不再特别提及）是逐行处理的。下面讨论对 TEX 来说究竟什么是一个输入行。

不同计算机系统对输入行的确切定义并不相同。回车/换行序列是最常见的行结束方式，但有些系统只用换行，而有些采用定长记录（块）存储的系统根本没有行结束符。因此 TEX 有自己结束输入行的方式。

1. 从输入文件读取一个输入行（若有行结束符，则去掉它）。
2. 去掉行尾空格（这是为块存储系统准备的，也可避免混淆，因为这些空格在编辑器中很难看见）。
3. 追加，默认是⟨return⟩（代码 13）。如果 `\endlinechar` 的值为负或大于 255（在 TEX 第 3 版以前是 127；更多差异见第 229 页），则不追加任何字符。此时效果等同于该行以注释字符结尾。

计算机的字符编码也可能不同（最常见的是 ASCII 和 EBCDIC），因此 TEX 把从文件读到的字符转换为它自己的字符代码。此后一律使用这些代码，从而使 TEX 在任何系统上表现一致。详见第 3 章。

2.3 类别码

256 个字符代码（0–255）中的每一个都有一个相关联的类别码，不过并不一定总是同一个。共有 16 个类别，编号为 0–15。扫描输入时，TEX 由此形成「字符代码–类别码」对。输入处理器只看到这些对；由它们形成字符记号、控制序列记号和参数记号。这些记号随后被送入 TEX 的展开与执行过程。

字符记号就是原样放行的「字符代码–类别码」对。控制序列记号由一个或多个字符前面加一个转义字符构成；见下文。参数记号也在下文解释。

下面是各类别的列表及简要说明。更详尽的解释将在本章及后续章节中给出。

0. 转义字符（escape character）；标志着一个控制序列的开始。IniTEX 把反斜杠\ （代码 92）设为转义字符。
1. 组开始（beginning of group）；这样的字符使 TEX 进入新的一层分组。plain 格式把左花括号 `{` 设为组开始字符。
2. 组结束（end of group）；TEX 关闭当前这一层分组。plain TEX 以右花括号`}` 作为组结束字符。
3. 数学切换（math shift）；这是数学公式的开始与结束分隔符。plain TEX 用美元符号 `$` 来充当。
4. 对齐制表位（alignment tab）；用 `\halign`（`\valign`）排的表中列（行）之间的分隔符。在 plain TEX 中这是与号 `&`。
5. 行结束（end of line）；TEX 视为标志输入行结束的字符。IniTEX 把这个代码赋予⟨return⟩，即代码 13。并非巧合的是，13 也是 IniTEX 赋予 `\endlinechar` 参数的值；见前文。
6. 参数字符（parameter character）；表示宏的参数。在 plain TEX 中这是井号 `#`。
7. 上标（superscript）；在数学模式中位于上标表达式之前。它也用来表示无法在输入文件中键入的字符代码；见下文。在 plain TEX 中这是脱字符 `^`。
8. 下标（subscript）；在数学模式中位于下标表达式之前。在 plain TEX 中用下划线 `_` 来充当。
9. 忽略（ignored）；此类字符从输入中删去，因此对后续 TEX 处理没有影响。在 plain TEX 中这是⟨null⟩ 字符，即代码 0。
10. 空格（space）；空格字符受到特殊对待。IniTEX 把这个类别赋予 ASCII 的⟨space⟩ 字符，代码 32。
11. 字母（letter）；在 IniTEX 中只有 `a..z`、`A..Z` 属于此类。宏包常常把某个「秘密」字符（例如 `@`）设为字母。
12. 其他（other）；IniTEX 把所有不属于其他类别的字符都归入此类。因此它例如包括数字和标点。
13. 活动（active）；活动字符无需前置转义字符就可充当 TEX 命令。在 plain TEX 中只有连字符 ~ 是活动字符，它被定义为产生不可断开的空格；见第 154 页。
14. 注释字符（comment character）；从注释字符起，TEX 把输入行的剩余部分视为注释并忽略。在 IniTEX 中百分号`%` 被设为注释字符。
15. 无效字符（invalid character）；此类用于不应出现在输入中的字符。IniTEX 把 ASCII 的⟨delete⟩ 字符（代码 127）归入此类。

用户可以用 命令更改字符代码到类别码的映射（⟨equals⟩ 等概念的解释见第 36 章）：

\catcode⟨number⟩⟨equals⟩⟨number⟩.

在这种语句中，第一个数字常写成

`⟨character⟩ 或 `\⟨character⟩

两者都表示该字符的字符代码（见第 35 页与第 66 页）。

plain 格式定义了\active

```
\chardef\active=13
```

于是可以写出如下语句

```
\catcode`\{=\active
```

\chardef 命令在第 36 页和第 67 页讨论。

LaTeX 格式有如下控制序列

```
\def\makeatletter{\catcode`@=11 }
\def\makeatother{\catcode`@=12 }
```

用来开启和关闭「秘密」字符 @ （见下文）。

\catcode 命令也可用来查询类别码：在

```
\count255=\catcode`\{
```

中它给出一个数字，可以被赋值。

类别码可以用

\ifcat⟨token$_1$⟩⟨token$_2$⟩

来测试。TeX 会展开 \ifcat 之后的一切，直到找到两个不可展开的记号；然后比较它们的类别码。控制序列记号被视为具有类别码 16，这使它们彼此相等，而与所有字符记号都不同。条件式在第 13 章进一步讨论。

2.4 从字符到记号

TeX 的输入处理器从文件或用户终端扫描输入行，并把输入中的字符转换为记号。记号有三种类型。

- 字符记号：任何字符，只要它带着适当的类别码被单独传给 TeX 的后续处理层次，就是一个字符记号。
- 控制序列记号，有两种：一个转义字符 – 即类别为 0 的字符 – 后面跟着一串「字母」，会被捆成一个控制词（control word），它是单个记号。一个转义字符后面跟着一个不属于类别 11（字母）的字符，则构成一个控制符（control symbol）。如果控制词与控制符的区别无关紧要，二者统称为控制序列（control sequence）。
  转义字符后面\ 跟一个空格字符所得到的控制符称为控制空格（control space）。
- 参数记号：一个参数字符 – 即类别为 6 的字符，默认是 # – 后面跟一个数字 1..9，就被替换为一个参数记号。参数记号只允许出现在宏的上下文中（见第 11 章）。
  一个宏参数字符后面再跟另一个宏参数字符（字符代码不一定相同），会被替换为单个字符记号。这个记号的类别是 6（宏参数），字符代码是第二个参数字符的字符代码。最常见的情形是把 ## 替换为 #$_6$，其中下标表示类别码。

2.5 作为有限状态自动机的输入处理器

TEX 的输入处理器可以看作一台具有三种内部状态的有限状态自动机，也就是说，在任一时刻它处于三种状态之一，而转移到另一状态后，它不记得此前的状态。

2.5.1 *N* 状态：新行

N 状态在每个新输入行的开头进入，这也是 TEX 处于该状态的唯一时刻。在 *N* 状态中，所有空格记号（即类别为 10 的字符）都被忽略；行尾字符会被转换为 `\par` 记号。所有其他记号都使 TEX 进入 *M* 状态。

2.5.2 *S* 状态：跳过空格

S 状态在任何模式下、遇到控制词或控制空格之后进入（但遇到其他控制符后不进入）；或者在 *M* 状态中遇到空格之后进入。在此状态下，本行后续的所有空格或行尾字符都被丢弃。

2.5.3 *M* 状态：行中

到目前为止最常见的状态是 *M*，即「行中」。在类别为 1–4、6–8 和 11–13 的字符之后，以及在控制空格以外的控制符之后，都会进入此状态。在此状态下遇到的行尾字符会产生一个空格记号。

2.6 访问完整字符集

严格地说，TEX 的输入处理器并不是一台有限状态自动机。这是因为在扫描输入行时，所有由两个相同的上标字符（类别码 7）和随后一个字符（字符码 < 128）组成的三元组，都会被替换为单个字符，其字符码在 0–127 范围内，与原字符相差 64。

例如，这个机制可用来访问字体中那些对应于无法输入的字符代码的位置，例如因为它们是 ASCII 控制字符。最明显的例子是 ASCII 的⟨return⟩ 和⟨delete⟩ 字符；字体中相应的位置 13 和 127 可以分别用 `^^M` 和 `^^?` 访问。不过，由于 `^^?` 的类别是 15（无效），在访问字符 127 之前必须先改变它。

在 TEX3 中，这个机制被修改并扩展到可访问 256 个字符：任何四元组 `^^xy`，其中 `x` 和 `y` 都是小写十六进制数字 `0`–`9`、`a`–`f`，都会被替换为 0–255 范围内的一个字符，即数字以十六进制表示为 `xy` 的那个字符。这对早期机制的适用性施加了一点限制：例如，如果键入 `^^a` 来产生字符 33，那么紧随其后的 `0`–`9`、`a`–`f` 就会被误解。

虽然这个过程使 TEX 的输入处理器比真正的有限状态自动机稍强一些，但它并不干扰其余的扫描过程。因此从概念上讲，更简单的做法是假定这种以 `^^` 开头的字符三元组或四元组替换是事先完成的。实际中这并不可能，因为一个输入行可能把类别码 7 赋予脱字符以外的某个字符，从而影响其后续处理。

2.7 内部状态之间的转移

现在讨论当输入中遇到某些类别码时，TEX 输入处理器的内部状态会受到什么影响。

2.7.1 0：转义字符

遇到转义字符时，TEX 开始构造一个控制序列记号。根据转义字符后面那个字符的类别码，会产生三种不同类型的控制序列。

- 如果转义后面的字符属于类别 11（字母），那么 TEX 把转义符、该字符以及随后所有类别为 11 的字符合并成一个控制词。此后 TEX 进入 *S* 状态，即跳过空格。
- 遇到类别为 10（空格）的字符时，会产生一个称为控制空格的控制符，且 TEX 进入 *S* 状态。
- 遇到任何其他类别码的字符时，产生一个控制符，且 TEX 进入 *M* 状态，即行中。

控制序列名中的字母必须全部在同一行上；如果当前行以注释符结尾，或者（通过把 `\endlinechar` 设到 0–255 范围之外）没有结束字符，控制序列名不会延续到下一行。

2.7.2 1–4、7–8、11–13：非空白字符

类别码为 1–4、7–8 和 11–13 的字符会被做成记号，且 TEX 进入 *M* 状态。

2.7.3 5：行结束

遇到行结束字符时，TEX 丢弃本行剩余部分，并以 *N* 状态开始处理下一行。如果当前状态是 *N*，也就是说本行到目前为止至多只有空格，则插入一个 `\par` 记号；如果状态是 *M*，则插入一个空格记号；而在 *S* 状态下什么也不插入。

注意，这里说的「行结束字符」是指类别码为 5 的字符。它不一定是 `\endlinechar`，也不一定出现在行尾。关于行尾的进一步说明见下文。

2.7.4 6：参数

参数字符 – 通常是 `#` – 后面在宏定义上下文中既可跟数字 `1..9` 也可跟另一个参数字符。第一种情况产生一个「参数记号」，第二种情况只把单个参数字符作为字符记号传给后续处理。两种情况下 TEX 都进入 *M* 状态。

参数字符也可以单独出现在对齐前导（preamble）中（见第 25 章）。

2.7.5 7：上标

上标字符的处理与大多数非空白字符相同，除非它后面跟着一个相同字符代码的上标字符。把这两个字符加上随后那个字符（在 TEX3 中可能是两个字符）替换为另一个字符的过程已在前面描述。

2.7.6 9：忽略字符

类别为 9 的字符被忽略；TEX 保持原状态不变。

2.7.7 10：空格

类别码为 10 的记号 – 不管其字符码为何，都称为⟨space token⟩ – 在 *N* 和 *S* 状态下被忽略（且状态不改变）；在 *M* 状态下，TEX 进入 *S* 状态，插入一个类别为 10、字符码为 32 的记号（ASCII 空格），也就是说，空格记号的字符码可能与实际输入的字符不同。

2.7.8 14：注释

注释字符使 TEX 丢弃本行剩余部分，包括注释字符本身。特别地，行结束字符不会被看到，所以即使在 *M* 状态下遇到注释，也不会插入空格记号。

2.7.9 15：无效

无效字符会产生错误信息。TEX 保持原状态不变。不过，在控制符的上下文中，无效字符是可接受的。因此\^^? 不会产生任何错误信息。

2.8 字母与其他字符

在大多数编程语言中，标识符可以由字母和数字（可能还包括下划线之类的其他字符）组成，但 TEX 中的控制序列只允许由类别为 11（字母）的字符构成。通常，数字和标点符号的类别是 12（其他字符）。不过，在某些上下文中，TEX 自身会生成一串字符，它们全都具有类别码 12，即使这并非它们通常的类别码。

当使用 `\string`、`\number`、`\romannumeral`、`\jobname`、`\fontname`、`\meaning` 和 `\the` 这些操作来生成一串字符记号时，就会发生这种情况。如果这种命令给出的字符中有任何一个是空格字符（即字符码 32），它就获得类别码 10（空格）。

对于十六进制数字被隐藏在控制序列中的极罕见情形，除了通常的 $\mathtt{A}_{11}$–$\mathtt{F}_{11}$ 之外，TEX 还允许 $\mathtt{A}_{12}$–$\mathtt{F}_{12}$ 作为十六进制数字（这里下标表示类别码）。

例如，

```
\string\end
```

给出四个字符记号 $\backslash_{12}\mathtt{e}_{12}\mathtt{n}_{12}\mathtt{d}_{12}$

注意，$\backslash_{12}$ 之所以出现在输出中，只是因为 `\escapechar` 的值是反斜杠的字符码。`\escapechar` 取另一个值会使 `\string` 的输出中出现另一个字符。`\string` 命令在第 3 章进一步讨论。

空格也可能混进控制序列：

```
\csname a b\endcsname
```

给出一个控制序列记号，其三个字符中有一个是空格。把这个控制序列记号转成字符串

```
\expandafter\string\csname a b\endcsname
```

会得到 $\backslash_{12}\mathtt{a}_{12}\ {}_{10}\mathtt{b}_{12}$。

更实际的例子是，假设存在一系列输入文件 `file1.tex`、`file2.tex`，我们想写一个宏，找出正在处理的输入文件的编号。一种做法是写

```
\newcount\filenumber  \def\getfilenumber file#1.{\filenumber=#1 }
\expandafter\getfilenumber\jobname.
```

其中宏参数文本中的字母 `file`（见第 11.5 节）会吞掉 jobname 的那一部分，只留下数字作为唯一参数。

不过，这略有不妥：由 `\jobname` 命令得到的字母 `file` 具有类别码 12，而 `\getfilenumber` 定义中的字母类别码是 11。可以这样修正：

```
{\escapechar=-1
 \expandafter\gdef\expandafter\getfilenumber
       \string\file#1.{\filenumber=#1 }
}
```

现在序列\string\file 给出四个字母 $\mathtt{f}_{12}\mathtt{i}_{12}\mathtt{l}_{12}\mathtt{e}_{12}$；\expandafter 命令使这在宏定义之前先执行；因为设了\escapechar=-1 ，反斜杠被省略。把这个值限制在一个组内，就必须使用 \gdef。

2.9 \par 记号

在 *N* 状态下遇到类别码为 5（行结束）的字符时，TEX 会把一个 记号插入输入。要清楚地认识到这究竟在何时发生：由于 TEX 遇到空格以外的任何记号时都会离开 *N* 状态，因此产生 \par 的那一行只能包含类别为 10 的字符。特别地，它不能以注释字符结尾。这个事实常常反过来用：如果为了输入排版想要一个空行，可以在那一行放一个注释符。

两个连续的空行产生两个 \par 记号。就实际效果而言，这等价于一个 \par，因为在第一个之后 TEX 进入竖直模式，而在竖直模式中，\par 只触发页面构造器，并清空段落形状参数。

当 TEX 在无限制水平模式中看到⟨vertical command⟩ 时，也会把 \par 插入输入。\par 被读取并展开之后，该竖直命令会被重新检查（见第 6 章和第 17 章）。

\par 记号也可能由结束 TEX 本次运行的 \end 命令插入；见第 28 章。

重要的是要认识到，TEX 遇到空行时所做的事（即结束段落）只是因为 \par 记号的默认定义。通过重新定义 \par，由空行和竖直命令引起的行为可以彻底改变，从而实现有趣的特殊效果。为了仍能触发通常与 \par 相关联的动作，plain 格式中提供了同义词 \endgraf。详见第 17 章。

\par 记号不允许出现在宏参数中，除非该宏被声明为 \long。非 \long 宏的参数中出现 \par，会促使 TEX 给出「runaway argument」（实参失控）信息。不过，用 \let 令其等于 \par 的控制序列（如 \endgraf）是允许的。

2.10 空格

本节讨论 TEX 初始处理阶段中空格字符与空格记号的某些方面。文字排版中的间距问题在第 20 章讨论。

2.10.1 被跳过的空格

从对 TEX 输入处理器内部状态的讨论可以清楚地看到，输入中的一些空格永远不会到达输出；事实上它们根本通不过输入处理器。例如输入行开头的空格，以及使 TEX 切换到 *S* 状态的那个空格之后的空格。

另一方面，行尾会产生（输入中并不存在的）空格，这些空格可能最终进入输出。还有第三种空格：那些通过了输入处理器、甚至在那里生成、但仍不会进入输出的空格。它们就是 TEX 语法在各处允许的⟨optional spaces⟩。

2.10.2 可选空格

TEX 的语法中有「可选空格」（optional spaces）和「一个可选空格」（one optional space）的概念：

⟨one optional space⟩ ⟶ ⟨space token⟩ | ⟨empty⟩
⟨optional spaces⟩ ⟶ ⟨empty⟩ | ⟨space token⟩⟨optional spaces⟩

一般来说，数字和粘连规格之后允许⟨one optional space⟩；而只要数字内部（例如减号与数字之间）或粘连规格内部（例如 `plus` 与 `1fil` 之间）可以出现空格，就允许⟨optional spaces⟩。此外，⟨equals⟩ 的定义允许在 `=` 号之前有⟨optional spaces⟩。

下面是一些可选空格的例子。

- 数字可以由⟨one optional space⟩ 界定。这能防止意外（见第 7 章），也加快处理速度，因为 TEX 更容易判断正在读的⟨number⟩ 在哪里结束。不过要注意，并非每个「数字」都是⟨number⟩：例如 `\magstep2` 中的 `2` 不是数字，而是 `\magstep` 宏参数的单个记号。因此它之后的空格或行尾是有意义的。另一个例子是参数编号，例如 `#1`：由于最多允许九个参数，扫描参数字符之后的一位数字就够了。
- 从 TEX 的语法可以推出，关键字 `fill` 和 `filll` 由 `fil` 与各自独立的 `l` 组成，每个 `l` 本身都是关键字（更详细的讨论见第 227 页），因此后面可以跟可选空格。于是像`fil L l` 这样的形式也是合法的。这是奇怪意外的潜在根源。在大多数情况下，附加一个 `\relax` 记号就能避免这类事故。
- 原始命令 作为宏定义中的最后一个命令会很方便。由于它吞掉可选空格，可以用来防止参数闭花括号之后的空格无意中进入输出。例如在

```
\def\item#1{\par\leavevmode
    \llap{#1\enspace}\ignorespaces}
\item{a/}one line \item{b/} another line \item{c/}
yet another
```

中，`\ignorespaces` 防止了第二、第三项中出现多余空格。不过，`\ignorespaces` 之后的空行仍会插入一个 `\par`。

2.10.3 被忽略与被遵从的空格

控制字之后的空格被忽略。这并不是可选空格的实例，而是由于 TEX 在控制字之后进入 *S* 状态、跳过空格。类似地，控制字之后的行结束字符也被跳过。

数字只由⟨one optional space⟩ 界定，但

`a\count0=3  b` 得到 'ab'，

因为 TEX 在第一个空格记号之后进入 *S* 状态。因此第二个空格在 TEX 的输入处理器中被跳过，它根本不会成为空格记号。

此外，当 TEX 处于 *N*（换行）状态时空格会被跳过。当 TEX 在竖直模式中处理时，空格记号（即未被跳过的空格）会被忽略。例如，下面第一个盒子之后（因行尾）插入的空格

```
\par
\hbox{a}
\hbox{b}
```

没有任何效果。

plain TEX 和 LATEX 都定义了命令 `\obeyspaces` 它使空格变得有意义：一个空格之后，其他空格不再被忽略。两者的基础都是

```
\catcode`\ =13 \def {\space}
```

不过两者有一个差别：在 plain TEX 中

```
\def\space{ }
```

而在 LATEX 中

```
\def\space{\leavevmode{} }
```

尽管那里的宏另有名目。

这两个宏的差别在 `\obeylines` 的上下文中会显现出来：那时每个行尾都是一个 `\par` 命令，意味着下一行都在竖直模式开始。活动空格被 plain 宏展开成一个空格记号，在竖直模式中被忽略。而 LaTeX 中的活动空格会立即切换到水平模式，于是每个空格都有意义。

2.10.4 更多被忽略的空格

还有三处 TeX 会忽略空格记号。

1. 当 TeX 寻找无界定符的宏参数时，它接受第一个非空格的记号（或组）。这在第 11 章讨论。
2. 在数学模式中，空格记号被忽略（见第 23 章）。
3. 在对齐制表符之后，空格被忽略（见第 25 章）。

2.10.5 ⟨space token⟩

空格在 TeX 中是异常的。例如，`\string` 操作把除空格之外的所有字符都赋予类别码 12；空格则获得类别 10。此外，如前所述，TeX 的输入处理器在 *M* 状态下把所有类别码为 10 的记号转换成真正的空格：它们的字符码变为 32。任何类别为 10 的字符记号都称为⟨space token⟩。字符码不等于 32 的空格记号称为「滑稽空格」（funny spaces）。

在
`\font\tenroman=cmr10 \tenroman`
之后，调用`\the\font` 与`\the\tenroman` 都得到 `\tenroman`。
调用`\fontname\tenroman` 则得到 `cmr10`。

字符码不是 32 的空格记号可以用例如 `\uppercase` 来创建。不过，「由于各种形式的空格记号在行为上几乎相同，没有必要细究细节」；见 [25] 第 377 页。

2.10.6 控制空格

「控制空格」命令`\ ` 贡献的间距量，与`\spacefactor` 为 1000 时一个⟨space token⟩所贡献的相同。控制空格不被当作空格记号，也不被当作展开为空格记号的宏（plain TeX 中 `\space` 就是这样定义的）。例如，TeX 会忽略输入行开头的空格，但控制空格是一个⟨horizontal command⟩，因此它会使 TeX 从竖直模式切换到水平模式（并插入一个缩进盒子）。关于空格因子，见第 20 章；关于水平与竖直模式，见第 6 章。

2.10.7 ‘ ’

用显式符号 ‘ ’ 表示空格，在 Computer Modern 打字机字体中就是字符 32。不过，切换到 `\tt` 还不足以让空格这样表示，因为空格在输入处理器中仍会受到特殊处理。

一种让空格由 ‘ ’ 排版出来的方法，是设置

```
\catcode`\ =12
```

此后 TeX 会把空格当作排版字符 32 的指令。此外，后续空格不会被跳过，而也会这样排版：*S* 状态只有在遇到类别码为 10 的字符之后才进入。类似地，通过改变空格字符的类别码，控制序列之后的空格也能变得可见。

2.11 再谈行尾

TEX 从输入文件读入各行，去掉所用的行终止符。因此 TEX 在这里的行为不依赖于操作系统及其所用的行终止符（CR-LF、LF，或块存储模式下根本没有）。输入行末尾的空格会被去掉。这样做的原因是历史性的；与 IBM 大型计算机的块存储模式有关。关于行结束字符在不同计算机上的一些特殊问题，见 [2]。

然后追加一个字符码为 \endlinechar 的终止字符，除非该参数的值为负或大于 255。注意，这个终止字符不必具有类别码 5（行结束）。

2.11.1 遵从行尾

有时希望输入中的行尾与输出中的行尾一一对应。下面这段代码就能做到：

```
\catcode`\^^M=13 %
\def^^M{\par}%
```

这里把 \endlinechar 字符设为 active，其含义变为 \par。注释符防止 TEX 看到本定义各行的终止符，从而因它是 active 而把它展开。

不过，把这段代码嵌入宏时需要小心。下面这个定义

```
\def\obeylines{\catcode`\^^M=13 \def^^M{\par}}
```

会被误解：TEX 会丢弃第二个 ^^M 之后的一切，因为它的类别码是 5。实际上这一行就变成了

```
\def\obeylines{\catcode`\^^M=13 \def
```

要修正这一点，定义本身必须在 ^^M 为 active 字符的上下文中执行：

```
{\catcode`\^^M=13 %
 \gdef\obeylines{\catcode`\^^M=13 \def^^M{\par}}%
}
```

这个定义没有考虑输入中的空行：空行会消失，因为（在这种情况下）两个连续的 \par 记号等价于一个。把行尾的定义稍作修改为

```
\def^^M{\par\leavevmode}
```

就能修正这一点：现在每个行尾都迫使 TEX 开始一个段落。对于空行，这就是一个空段落。

2.11.2 改变 \endlinechar

有时你可能想改变 \endlinechar，或普通行终止符 ^^M 的 \catcode，例如为了获得特殊效果，比如参数由行尾终止的宏。一个完整示例见第 100 页。

这里有几个陷阱。考虑下面这段：

```
{\catcode`\^^M=12 \endlinechar=`\^^J \catcode`\^^J=5
...
... }
```

这会意外输出字符 13（^^M）和字符 10（^^J），它们由第一行和最后一行的行终止符引起。

在第一行和最后一行末尾加注释是有效的做法，但把第一行替换为下面两行

```
{\endlinechar=`\^^J \catcode`\^^J=5
\catcode`\^^M=12
```

也是一种解决办法。

当然，在很多情况下不必替换另一个行结束字符；那时更简单的办法是写

```
\endlinechar=-1
```

这会把所有行都当作以注释结尾。

2.11.3 关于行结束字符的更多说明

TEX 在输入行末尾追加的那个字符与其他字符一样被处理。通常我们察觉不到这一点，因为它的类别码是特殊的，但有几种方式可以让它被以不寻常的方式处理。

> 用 `^^` 结束一个输入行（通常是 `\endlinechar` 为 13 时）会在输出中得到 'M'，它是字符码为 13+64 的 ASCII 字符。
>
> 如果定义了`\^^M` ，那么以反斜杠结束输入行会执行这个命令。plain 格式定义了
> `\def\^^M{\ }`
> 这使「控制回车」等价于一个控制空格。

2.12 再谈输入处理器

2.12.1 把输入过程看作独立过程

TEX 的各个处理层次是同时、增量地工作的，但从概念上常常可以把它们看作独立的过程，每个过程都接受上一阶段的完整输出。对空格的处理就是一个很好的例证。

考虑定义

```
\def\DoAssign{\count42=800}
```

和调用

```
\DoAssign 0
```

输入处理器（TEX 中构造记号的部分）在扫描这次调用时，跳过了零前面的空格，因此这次调用展开为

```
\count42=8000
```

如果推理说「先读入 `\DoAssign`，再展开，空格界定数字 800，于是 800 被赋值，零被打印」，那就错了。注意，零出现在下一行时结果也一样。

另一个例子表明，可选空格与被跳过的空格出现在不同的处理阶段：

```
\def\c.{\relax}
a\c. b
```

展开为

```
a\relax b
```

它输出

'a b'

因为 `\relax` 控制序列之后的空格只在第一次读入行时被跳过，展开时并不跳过。而片段

```
\def\c.{\ignorespaces}
a\c. b
```

则展开为

```
a\ignorespaces b
```

执行 `\ignorespaces` 命令会移除随后的空格记号，因此输出是

‘ab’。

在两个定义中，`\c` 后面的句点都是界定记号；这里用它来防止空格被跳过。

2.12.2 不要把输入处理器看作独立过程

把 TEX 的记号化看作一个独立过程是一种方便的看法，但有时会引起混淆。下面这一行

```
\catcode`\^^M=13{}
```

把行尾设为 active，随后又因这一行自身的行尾而给出「undefined control sequence」错误。这一行上命令的执行就这样影响了同一行的扫描过程。

相比之下，

```
\catcode`\^^M=13
```

不会给出错误。原因是 TEX 仍在扫描数字 13 时就读取了行尾；也就是说，在赋值尚未执行的时候。行尾随后被转换为界定待赋值数字的可选空格字符。

2.12.3 输入处理器的递归调用

前面把「把参数字符加数字替换为参数记号」这一动作，描述为与「把字母聚合成控制序列记号」类似。实际情况要比这复杂一些。TEX 的记号扫描机制既用于来自文件的输入，也用于来自记号列表（如宏定义）的输入。只有第一种情况下才适用内部状态这一术语。

不过，宏参数字符在两种情况下受到相同对待。若非如此，就无法写出像下面这样的东西：

```
\def\a{\def\b{\def\c####1{####1}}}
```

关于这种嵌套定义的解释，见第 94 页。

2.13 @ 约定

凡是翻阅过 plain 格式或 LATEX 格式的人都会注意到，许多控制序列中都含有一个「@」符号。这些控制序列是为普通用户不可访问而设的。

在格式文件开头附近有这样一条指令

```
\catcode`@=11
```

它把 @ 符号变成字母，意味着它可以用在控制序列中。在格式定义接近结尾处，@ 符号又被变回「其他」字符：

```
\catcode`@=12
```

那么，为什么用户不能直接调用带 @ 符号的控制序列，却可以调用包含大量这种「@ 定义」的宏呢？原因是：含 @ 的控制序列在定义时就被 TEX 内化，此后它们是一个记号，而不是一串字符。宏展开只是插入这样的记号，此时组成字符的类别码就不再重要了。

第 3 章

字符

在内部，TEX 用（整数形式的）字符代码（character code）来表示字符。本章讨论这些代码，以及能够访问它们的命令。

`\char` 显式指定一个需要排版的字符。
`\chardef` 将一个控制序列定义为某个字符代码的同义词。
`\accent` 用于放置重音（accent）字符的命令。
`\if` 测试字符代码是否相等。
`\ifx` 同时测试字符代码与类别代码是否相等。
`\let` 将一个控制序列定义为某个记号的同义词。
`\uccode` 查询或设置某个给定代码的大写形式所对应的字符代码。
`\lccode` 查询或设置某个给定代码的小写形式所对应的字符代码。
`\uppercase` 将⟨general text⟩ 参数转换为其大写形式。
`\lowercase` 将⟨general text⟩ 参数转换为其小写形式。
`\string` 将一个记号转换为由一个或多个字符组成的字符串。
`\escapechar` 当控制序列被转换为字符记号时，用作转义字符的那个字符的代码。IniTEX 默认值为 92（即 `\`）。

3.1 字符代码

从概念上讲，最容易理解的方式是认为 TEX 在内部处理的是字符，但实际上 TEX 处理的是整数，即“字符代码”。

字符在计算机中的编码方式可能因系统而异。因此 TEX 使用它自己的一套字符代码体系。从文件（或用户终端）读入的任何字符，都会按照字符代码表被转换为一个字符代码；然后再根据它赋予一个类别代码（见第 2 章）。对于小于 128 的数字，字符代码表以 7 位 ASCII 表为基础（见第 38 章）。

在字符（更准确地说是字符记号）与字符代码之间，可以通过反引号（grave，back quote）字符 ` 进行显式转换：凡是 TEX 期望出现⟨number⟩ 的地方，都可以使用反引号后跟一个字符记号，或一个单字符控制序列。因此`\count `a` 与`\count `\a` 都是`\count97` 的同义词。另见第 7 章。

在某些情况下必须允许使用单字符控制序列，例如

```
\catcode`\%=11  或  \def\CommentSign{\char`\%}
```

如果省略反斜杠，它们就会被误解。例如

```
\catcode`%=11
```

会把 =11 当作注释。任何类别代码的字符都可以构成单字符控制序列。

在转换为字符代码之后，与外部表示之间的一切联系都已消失。当然，对于大多数字符，可见输出会“等于”输入（即输入一个 a 会产生一个 ‘a’）。然而即使在常见符号中也存在例外。在 Computer Modern 罗马字体中没有“小于”和“大于”符号，因此输入 ‘<>’ 将在输出中得到 ‘<>’。

为了让 TEX 在输出端与机器无关，字符代码也用于 dvi 文件：操作码 $n = 0 \ldots 127$ 简单地表示指令“从当前字体中取出字符 n”。dvi 文件中操作码的完整定义可参见 [23]。

3.2 用于字符的控制序列

控制序列可以有多种方式来表示一个字符。\char 命令指定需要排版的字符；\let 命令则为一个字符记号（即字符代码与类别代码的组合）引入一个同义词。

3.3 指定需要排版的字符：\char

字符可以用数字方式表示，例如\char98 。该命令告诉 TEX 把当前字体的第 98 号字符添加到正在构造的水平列表中。

与十进制记法相比，使用八进制或十六进制记法往往更方便。八进制使用单引号：\char'142；十六进制使用双引号：\char"62。注意\char''62 是不正确的；把两个单引号替换为一个双引号的过程发生在处理的较后阶段（视觉处理器），而不是在数字扫描阶段（执行处理器）。

由于反引号可以显式地转换为字符代码，因此也可以得到一个 ‘b’ – 前提是所使用的字体在组织上与 ASCII 表有些相似 – ——用\char `b 或\char `\b。

\char 命令表面上有点像 ^^ 替换机制（第 2 章）。两种机制都不直接指明字符而访问字符。不过 ^^ 机制在处理的非常早阶段（TEX 的输入处理器中、但在类别代码赋值之前）运作；而 \char 命令则出现在处理的最终阶段。实际上它表示的是“排版第某某号字符”。

有一种结构可以让一个控制序列代表某个字符代码： 命令。其语法为

\chardef⟨control sequence⟩⟨equals⟩⟨number⟩,

其中数字可以是显式表示或一个计数器值，但也可以是用反引号命令得到的字符代码（见上文；⟨number⟩ 的完整定义见第 7 章）。在 plain 格式中，后一种用法被用于如下定义

```
\chardef\%=`\%
```

它等价于

```
\chardef\%=37
```

在这个命令中，单独使用的控制符号\% 是\char37 的同义词，即排版第 37 号字符（通常是百分号字符）的命令。

用 \chardef 命令定义的控制序列也可以用作⟨number⟩。这一特性被用于分配命令，如 \newbox（见第 7 章与第 31 章）。用 \mathchardef 定义的记号也可以这样使用。

3.3.1 隐式字符记号：\let

另一种让控制序列代表（除其他对象外）一个字符的结构是 \let：

\let⟨control sequence⟩⟨equals⟩⟨token⟩

在（可选的）等号右侧放一个字符记号。其结果称为隐式字符记号（implicit character token）。（关于 \let 的进一步讨论见第 96 页。）

在 plain 格式中，例如有左右花括号的同义词：

```
\let\bgroup={ \let\egroup=}
```

由此得到的控制序列称为“隐式花括号”（见第 10 章）。

用 \let 赋字符与用 \chardef 定义控制序列是不同的：\let 让控制序列代表一个字符代码与类别代码的组合。

举例来说

```
\catcode`|=2 % make the bar an end of group
\let\b=|  % make \b a bar character
{\def\m{...}\b \m
```

会给出“未定义的控制序列 \m”，因为 \b 关闭了定义 \m 所在的组。另一方面，

```
\let\b=| % make \b a bar character
\catcode`|=2  % make the bar character end of group
{\def\m{...}\b \m
```

会留下一个组未闭合，并打印一个竖线（或当前字体第 124 号位置上的任何字符）。前一个例子说明，即使花括号已被重新定义（例如为格式化 C 代码的宏而设为活动字符），组开始与组结束的功能仍可通过控制序列 \bgroup 和 \egroup 获得。

下面再举一个例子，说明隐式字符记号很难与真正的字符记号区分开。在上面的序列

```
\catcode`|=2 \let\b=|
```

中，测试

```
\if\b|
```

和

```
\ifcat\b}
```

都为真。

plain 格式中还有另一个例子：命令

```
\let\sp=^ \let\sb=_
```

让键盘上没有下划线或脱字符的人也能在数学中写下标与上标。例如：

x\sp2\sb{ij} 得到 x^2_{ij}

如果连编写该格式本身的人也没有这两个键，就需要一些进一步的技巧：

```
{\lccode`,=94 \lccode`.=95 \catcode`,=7 \catcode`.=8
\lowercase{\global\let\sp=, \global\let\sb=.}}
```

就能完成任务；小写代码的解释见下文。TEX 第 2 版中的 ^^ 方法（见第 26 页）在此处不能使用，因为它要求键入两个通常无法输入的字符。借助 TEX 第 3 版中的扩展，也可以写成

```
{\catcode`\,=7
\global\let\sp=,,5e \global\let\sb=,,5f}
```

以十六进制表示代码 94 和 95。

要弄清用 \let 定义的控制序列究竟是什么，可以使用 \meaning：序列

```
\let\x=3 \meaning\x
```

会给出 'the character 3'。

3.4 重音

重音可以通过⟨horizontal command⟩ 来放置：

\accent⟨8-bit number⟩⟨optional assignments⟩⟨character⟩

其中⟨character⟩ 是一个类别为 11 或 12 的字符、一条 \char⟨8-bit number⟩ 命令，或一个 \chardef 记号。如果这四类⟨character⟩ 都不出现，则该重音本身被当作一条 \char 命令；这会产生一个“悬在半空”的重音。否则重音就被放在紧随其后的字符之上。重音与字符之间的字体切换可以由⟨optional assignments⟩ 完成。

\accent 命令后面必须跟一个⟨character⟩，这一事实有一个令人不快的后果：无法把重音放在连字（ligature）上，也无法把两个重音叠在一起。在某些语言（如印地语或越南语）中确实存在这种双重音。不过在数学模式中是可以把重音叠放的。

带重音字符的宽度与不带重音时相同。TEX 假定字体文件中出现的重音是针对一个高度等于该字体 x 高的字符而正确定位的；对于高度不同的字符，它相应地降低或升高重音。

TEX 中没有真正的下方重音。它们是以低位放置的上方重音来实现的。更正确的处理方式是写一个宏，测量紧随其后的字符，并相应地升高或降低重音。plain TEX 中的下加符宏 \c大致就是这样做的。不过对于有下伸部（descender）的字符，它并不降低重音。

重音的水平位置由 \fontdimen1（每点倾斜量）控制。水平移动使用字距（kerning）。注意，虽然这些字距是自动插入的，但它们被归类为显式字距。因此它们会抑制该词在字距前后部分的断词（连字符化）。

作为重音字距的例子，下面给出一段水平列表的转储。

```
\setbox0=\hbox{\it \`l}
\showbox0
```

会给出

```
\hbox(9.58334+0.0)x2.55554
.\kern -0.61803 (for accent)
.\hbox(6.94444+0.0)x5.11108, shifted -2.6389
..\tenit ^^R
.\kern -4.49306 (for accent)
.\tenit l
```

注意重音被先放置，因此之后仍可使用最后一个字符的斜体修正（italic correction）。

3.5 测试字符

字符代码的相等性由 \if 测试：

\if⟨token$_1$⟩⟨token$_2$⟩

该条件式后面的记号会被持续展开，直到剩下两个不可展开的记号。如果这两个记号是具有相同字符代码的字符记号（无论其类别代码如何），则条件为真。

一个不可展开的控制序列被认为具有字符代码 256 和类别代码 16（因此它除了与另一个控制序列相等外，与任何东西都不等），除非它曾被 `\let` 到一个非活动字符记号。在那种情况下，它被认为具有该字符的字符代码与类别代码。这一点前面已经提到过。

对类别代码的测试 `\ifcat` 在第 2 章中已经提到；测试

`\ifx`⟨token$_1$⟩⟨token$_2$⟩

可用来同时测试类别代码与字符代码。该测试后面的记号不被展开。不过，如果它们是宏，TEX 会测试它们展开结果是否相等。

用 `\chardef` 定义的量可以用 `\ifnum` 测试：

```
\chardef\a=`x \chardef\b=`y \ifnum\a=\b % is false
```

这基于（见第 7 章）⟨chardef token⟩ 可以用作数字这一事实。

3.6 大写与小写

3.6.1 大写代码与小写代码

每个字符代码都对应一个大写代码和一个小写代码（更多代码见下文）。它们可以用

`\uccode`⟨number⟩⟨equals⟩⟨number⟩

和

`\lccode`⟨number⟩⟨equals⟩⟨number⟩.

来赋值。在 IniTEX 中，代码 `` `a..`z、`A..`Z `` 的大写代码 是 `` `A..`Z ``，小写代码是 `` `a..`z ``。所有其他字符代码的大写和小写代码都为零。

3.6.2 大写命令与小写命令

命令`\uppercase{...}` 和`\lowercase{...}` 会遍历它们的参数列表，把显式字符记号的所有字符代码分别替换为其大写代码和小写代码（前提是这些代码非零），而不改变类别代码。

`\uppercase` 和 `\lowercase` 的参数是一个⟨general text⟩，其定义为

⟨general text⟩ ⟶ ⟨filler⟩`{`⟨balanced text⟩⟨right brace⟩

（⟨filler⟩ 的定义见第 36 章），意思是左花括号可以是隐式的，但闭合的右花括号必须是一个类别代码为 2 的显式字符记号。TEX 会执行展开以找到开括号。

大写与小写在执行处理器中完成；它们不像 `\number` 或 `\string` 那样是“宏展开”活动。下面这个序列（试图产生 `\A`）

```
\expandafter\csname\uppercase{a}\endcsname
```

会给出错误（TEX 在 `\uppercase` 之前插入一个 `\endcsname`，因为 `\uppercase` 不可展开），但

```
\uppercase{\csname a\endcsname}
```

可以正常工作。

作为正确使用 `\uppercase` 的例子，下面是一个测试某字符是否为大写的宏：

```
\def\ifIsUppercase#1{\uppercase{\if#1}#1}
```

同样的测试也可以用`\ifnum `#1=\uccode`#1` 来完成。

以大写字符开头的词（即一个不等于其自身 `\lccode` 的字符）的断词受 `\uchyph` 参数约束：若该参数为正，则允许对首字母大写的词进行断词。另见第 19 章。

3.6.3 关键字的大写与小写形式

TEX 关键字中的每个字符（例如 `pt`）都可以用大写或小写形式给出。例如，`pT`、`Pt`、`pt` 和 `PT` 都有相同的含义。TEX 在此并不使用 `\uccode` 和 `\lccode` 表来确定小写形式。相反，它通过在字符代码上加 32——即大写与小写字符之间的 ASCII 差值 – – ——把大写字符转换为小写。这对非罗马字母的 TEX 实现有一些影响；见 the TEX book 第 370 页，[25]。

3.6.4 `\uppercase` 与 `\lowercase` 的创造性用法

`\uppercase` 和 `\lowercase` 不改变类别代码这一事实，有时可用来创造某些否则难以产生的“字符代码-类别代码”组合。例如参见第 13 章中对 `\newif` 宏的解释，以及第 37 页的另一个例子。

作为一个略有不同的应用，考虑下面这个问题（由 Rainer Schöpf 解决）：给定一个计数器`\newcount\mycount`，如何把第`\mycount` 号字符写到终端上。下面是一种解法：

```
\lccode`a=\mycount \chardef\terminal=16
\lowercase{\write\terminal{a}}
```

`\lowercase` 命令实际上把 `\write` 命令的参数从 ‘a’ 改成了它应该成为的样子。

3.7 一个字符的各种代码

每个字符代码都有若干⟨codename⟩（代码名）与之关联。这些整数处于不同的取值范围，决定了该字符在各种上下文中如何被处理，或者该字符的出现如何改变 TEX 在某些上下文中的运作方式。

代码名如下：

`\catcode` ⟨4-bit number⟩（0–15）：字符所属的类别。这在第 2 章中讨论。

`\mathcode` ⟨15-bit number⟩（0–`"7FFF`）或`"8000`：决定字符在数学模式中如何处理。见第 21 章。

`\delcode` ⟨27-bit number⟩（0–`"7FFFFFF`）：决定字符在数学模式中 `\left` 或 `\right` 之后如何处理。见第 158 页。

`\sfcode` 整数：决定该字符之后的间距如何受影响。见第 20 章。

`\lccode`, `\uccode` ⟨8-bit number⟩（0-255）：小写代码与大写代码 – 上面已经讨论过。

3.8 把记号转换为字符串

`\string` 命令取下一个记号，并把它展开为一串独立的字符。因此

```
\tt\string\control
```

会在输出中给出 \control，而

\tt\string$

会给出 $；但要注意，字符串化操作发生在记号化之后，所以

\tt\string%

不会给出 %，因为注释符号已被 TEX 的输入处理器删除。因此这条命令会把下一行的第一个记号“字符串化”。

\string 命令由展开处理器执行，因此它会被展开，除非被显式抑制（见第 12 章）。

3.8.1 控制序列的输出

在上面的例子中选用了打字机字体，因为Computer Modern 罗马字体没有反斜杠字符。不过，TEX 并非一定要用反斜杠字符来显示控制序列：它使用第 \escapechar 号字符。当控制序列用 \write、\message 或 \errmessage 输出时，也使用同一个值；\show、\showthe 和 \meaning 的输出中同样使用。如果 \escapechar 为负或大于 255，则不输出转义字符；默认值（在 IniTEX 中设置）为 92，即反斜杠字符的代码。

对于 \write 语句中的使用，在某些情况下 \string 可以用 \noexpand 代替（见第 110 页）。

3.8.2 \string 的类别代码

\string 命令产生的字符具有类别代码 12，但被字符串化的控制序列中的空格除外；它们具有类别代码 10。由于控制序列内部不存在类别代码，\string 产生的任何空格必然只是空格字符，即代码为 32 的字符。不过，TEX 的输入处理器会把所有字符代码不是 32 的空格记号转换为字符代码为 32 的字符记号，因此“古怪空格”最终进入控制序列的可能性相当小。

其他在类别代码方面与 \string 行为相同的命令有 \number、\romannumeral、\jobname、\fontname、\meaning 和 \the。

第 4 章

字体

在文本模式中，TEX 从一个“当前字体”中取字符。本章描述如何向 TEX 指定字体，以及一个字体可以具有哪些属性。

`\font` 声明一个字体的标识控制序列。
`\fontname` 一个字体的外部名称。
`\nullfont` TEX 在紧急情况下使用的空字体的名称。
`\hyphenchar` 一个字体的连字符编号。
`\defaulthyphenchar` 加载字体时 `\hyphenchar` 的值。Plain TEX 默认值：`` `\- ``.
`\fontdimen` 访问字体的各种参数。
`\char47` 斜体修正（italic correction）。
`\noboundary` 省略隐式边界字符。

4.1 字体

用 TEX 的术语来说，字体是指包含在一个外部字体文件中的字符集合。在处理过程中，TEX 决定某个字符应取自哪个字体。这个决定对文本模式和数学模式分别做出。

当 TEX 处理普通文本时，字符取自“当前字体”。外部字体文件名通过如下语句与控制序列关联：

```
\font\MyFont=myfont10
```

这会让 TEX 加载文件 `myfont10.tfm`。之后，要把当前字体切换为该文件所描述的字体，就执行：

```
\MyFont
```

当前字体的状态可以查询：序列

```
\the\font
```

会产生当前字体的控制序列。

数学模式完全忽略当前字体。它转而查看“当前族”（current family），该族可以包含三个字体：一个用于文本样式，一个用于脚本样式，一个用于脚本脚本样式。这在第 21 章中讨论。

关于字体（font）与字样（typeface）的一致术语，参见 [42]。

借助“虚拟字体”（virtual fonts，参见 [24]），在 TEX 看来像是一个字体的东西，可能实际存放在不止一个物理字体文件中。另见第 216 页。

4.2 字体声明

在 TEX 或 IniTEX 的某次运行中，必须在某个时刻建立内部标识控制序列与字体外部文件名之间的关联。该命令的语法为：

\font⟨control sequence⟩⟨equals⟩⟨file name⟩⟨at clause⟩

其中

⟨at clause⟩ −→ at ⟨dimen⟩ | scaled ⟨number⟩ | ⟨optional spaces⟩

字体声明是组局部的。

通过⟨at clause⟩，用户指定想要该字体的某个放大版本。⟨at clause⟩ 有两种形式：如果字体被指定为 scaled f，则 TEX 把该字体的所有字体维度乘以 $f/1000$；如果该字体的设计尺寸为 $d$pt，而⟨at clause⟩ 为 at $p$pt，则 TEX 把所有字体数据乘以 p/d。是否带有⟨at clause⟩ 对 TEX 为该字体读取的外部字体文件（即.tfm 文件）没有影响；它只是把字体维度乘以一个常数。

在这样的字体声明之后，使用所定义的控制序列就会把当前字体设置为该控制序列所对应的字体。

4.2.1 字体与 tfm 文件

该字体所需的外部文件是一个 tfm（TEX 字体度量）文件，它的取用与 \font 声明中的任何⟨at clause⟩ 无关。如果该 tfm 文件此前已经加载过（例如由 IniTEX 在构造格式文件时加载），则可以重新执行该字体文件的赋值，而无需再次读取 tfm 文件。

字体的设计尺寸记录在字体度量文件中。例如 cmr10 字体的设计尺寸为 10 point。然而，该字体中真正具有 10 point 大小的字符并不多：开括号与闭括号是两个例子，但大写字母要小得多。

4.2.2 查询当前字体与字体名

前面已经提到，设置当前字体的控制序列可以用命令\the\font 取回。这是下面这条规则的一个特例：

\the⟨font⟩

其中

⟨font⟩ −→ \font | ⟨fontdef token⟩ | ⟨family member⟩
⟨family member⟩ −→ ⟨font range⟩⟨4-bit number⟩
⟨font range⟩ −→ \textfont | \scriptfont | \scriptscriptfont

一个⟨fontdef token⟩ 是由 \font 定义的控制序列,或是预定义的控制序列 \nullfont。⟨family member⟩ 的概念仅在数学模式中才有意义。

此外，还可以取回字体的外部名称：

\fontname⟨font⟩

会给出一串类别为 12 的字符记号（但空格字符的类别为 10），拼写出字体文件名，并在适用时附上⟨at clause⟩。

在
\font\tenroman=cmr10 \tenroman
之后，调用\the\font 与\the\tenroman 都得到 \tenroman。
调用\fontname\tenroman 则得到 cmr10。

4.2.3 \nullfont

TEX 总是认识一个不含任何字符的字体：\nullfont。如果尚未指定字体，或者在数学模式中需要某个尚未指定的族成员，TEX 就会从 nullfont 中取字符。这个控制序列属于⟨fontdef token⟩：它的行为与其他代表字体的控制序列一样；只不过它没有关联的 tfm 文件。

4.3 字体信息

在 TEX 运行期间，关于字体所需的主要信息是各个字符的尺寸。TEX 在字体度量文件中找到这些信息，这类文件通常以.tfm 为扩展名。此类文件包含

- 全局信息：\fontdimen 参数以及一些其他信息；
- 各个字符的尺寸与斜体修正；以及
- 字符的连字与字距程序。

此外，字体的设计尺寸在 tfm 文件中指定；见上文。tfm 格式的定义可参见 [23]。

4.3.1 字体维度

文本字体至少需要有七个\fontdimen 参数（但 TEX 对未指定的参数取零）；数学符号字体与数学扩展字体则有更多参数（见第 171 页）。对文本字体而言，这七个最小参数包括：

1. 每点倾斜量（slant per point）；该维度用于重音符号的正确水平定位；
2. 词间空格：除非用户显式指定 \spaceskip，否则就使用此值；见第 20 章；
3. 词间延展：词间空格的延展分量；
4. 词间收缩：词间空格的收缩分量；
5. x 高（x-height）：即⟨internal unit⟩ ex 的值，通常约等于小写字母 ‘x’ 的高度；
6. 象限宽（quad width）：即⟨internal unit⟩ em 的值，约等于大写字母 ‘M’ 的宽度；以及
7. 额外空格：在句末加到词间空格上的空格（即当 \spacefactor ≥ 2000 时），除非用户显式指定 \xspaceskip。

第 1 和第 5 号参数纯粹是关于字体的信息，没有必要改动。其他参数的值可以修改，用以调整间距；关于修改第 2、3、4、7 号参数的例子，见第 20 章。

字体维度可以在⟨font assignment⟩ 中更改，这是一种⟨global assignment⟩（见第 87 页）：

\fontdimen⟨number⟩⟨font⟩⟨equals⟩⟨dimen⟩

关于⟨font⟩ 的定义见上文。

4.3.2 字距调整（kerning）

有些字符组合应当比其包围框直接相接时靠得更近一些。这种精细的间距称为字距调整（kerning）；恰当的字距对字体而言，与字母形状的设计同等重要。

作为例子

‘Vo’ 与未作字距调整的 ‘Vo’ 的对比

TEX 中的字距由 tfm 文件中的信息控制，因而不受用户左右。不过 tfm 文件是可以编辑的（见第 33 章）。

\kern 命令与字距现象几乎没有关系；它在第 8 章中解释。

4.3.3 斜体修正（italic correction）

原始控制符号\/ - 会插入前一个字符或连字的“斜体修正”。由于字符“包围框”的定义，这种修正有时是必要的。该框的边总是竖直的，而 TEX 所理解的字符宽度就是这两条竖边之间的距离。然而，为了让倾斜或斜体字样的间距恰当，字符完全可能伸出其包围框之外。当这样一个伸出的字符后面跟着一个非倾斜字样的字符时，就需要斜体修正。

例如，对比

‘*TEX* has’ 与 ‘*TEX* has’，

其中第二种写法是这样输入的：

```
{\italic\TeX\/} has
```

每个字符的斜体修正大小由字体度量文件中的字体信息决定；对 Computer Modern 字体而言，它约为字符“伸出量”的一半；见 [17]。斜体修正不同于 `\fontdimen1`（每点倾斜量）。那个字体维度只用于把重音符号放到字符上方的定位。

只有当 TEX 此前处理的一项是字符或连字时，才能插入斜体修正。因此，下面这种在斜体段落中插入正体文本的做法并不奏效：

```
{\italic Some text {\/\roman not} emphasized}
```

此处斜体修正不起作用，因为前一项是胶（glue）。

4.3.4 连字（ligatures）

用连字替换字符序列，是由字体的 `tfm` 文件中的信息控制的。连字由⟨character⟩ 命令形成：在某些字体中，诸如 `fi` 这样的序列会被替换为 ‘fi’。

其他传统上常用的连字出现在 `ff`、`ffi`、`fl`、`ffl` 之间；在一些较早的作品中还能见到 `ft` 与 `st`；与 `fl` 连字类似，`fk` 与 `fb` 也可能出现。

TEX 中的连字可以在显式字符记号、`\char` 命令以及⟨chardef token⟩ 之间形成。例如，序列`\char `f\char`i` 会被替换为 ‘fi’ 连字——只要该字体包含这样的连字。

不想要的连字可以用多种方式抑制：例如不希望出现的连字 ‘halflife’ 可以这样避免：

`half{}life`、`half{l}ife`、`half\/life`，或 `half\hbox{}life`

但使用斜体修正的那种做法与其他几种并不等价。

4.3.5 边界连字（boundary ligatures）

每个词的左右两侧各有一个边界字符（仅 TEX3）。这使得诸如希腊语中两个不同 sigma 的现象成为可能：一个用于词末，另一个用于其他所有位置。这可以通过与边界字符的连字来实现。在一个词之前或之后立即给出`\noboundary` 命令，就会抑制该处的边界字符。

总体而言，随着过渡到 TEX 第 3 版，连字机制变得更加复杂；见 [20]。

第 5 章

盒子

TEX 的水平盒子与竖直盒子是水平列表与竖直列表片段的容器。盒子可以存放在盒子寄存器中。本章讨论盒子寄存器，以及盒子的尺寸、其各组成部分相互之间的排列方式等方面。

`\hbox` 构造一个水平盒子。

`\vbox` 构造一个竖直盒子，其参考点取最后一项的参考点。

`\vtop` 构造一个竖直盒子，其参考点取第一项的参考点。

`\vcenter` 构造一个在数学轴上竖直居中的竖直盒子；该命令只能在数学模式中使用。

`\vsplit` 切分出一个竖直盒子的顶部。

`\box` 使用一个盒子寄存器，并清空它。

`\setbox` 把一个盒子赋给一个盒子寄存器。

`\copy` 使用一个盒子寄存器，但保留其内容。

`\ifhbox` `\ifvbox` 测试一个盒子寄存器中是否含有水平/竖直盒子。

`\ifvoid` 测试一个盒子寄存器是否为空。

`\newbox` 分配一个新的盒子寄存器。

`\unhbox` `\unvbox` 拆开一个含有水平/竖直盒子的盒子寄存器，把其内容追加到当前水平/竖直列表，并清空该寄存器。

`\unhcopy` `\unvcopy` 与 `\unhbox` / `\unvbox` 相同，但不清空该寄存器。

`\ht` `\dp` `\wd` 盒子寄存器中盒子的高/深/宽。

`\boxmaxdepth` 盒子允许的最大深度。Plain TEX 默认值： `\maxdimen`。

`\splitmaxdepth` 由 `\vsplit` 生成的盒子允许的最大深度。

`\badness` 最近构造的盒子的 badness。

`\hfuzz` `\vfuzz` TEX 在认为一个水平/竖直盒子过满之前所容忍的超出量。

`\hbadness` `\vbadness` TEX 报告某个水平/竖直盒子过松或过紧之前的容忍量。

`\overfullrule` 为指示过满的水平盒子而打印的规则线宽度。

`\hsize` 竖直盒子内排版文本时所用的行宽。

`\vsize` 页面盒子的高度。

`\lastbox` 存放当前列表中最后添加的一项（若它是盒子）的寄存器。

`\raise` `\lower` 在水平模式中调整一个盒子的竖直位置。

`\moveleft` `\moveright` 在竖直模式中调整一个盒子的水平位置。

`\everyhbox` `\everyvbox` 在水平/竖直盒子开头插入的记号列表。

5.1 Boxes

本章我们将考察盒子（box）。盒子是容纳水平列表或竖直列表片段的容器。需要多次使用的盒子可以存放在盒子寄存器（box register）中。

当 TEX 期望一个⟨box⟩ 时，以下任一形式都是允许的：

- `\hbox`⟨box specification⟩`{`⟨horizontal material⟩`}`
- `\vbox`⟨box specification⟩`{`⟨vertical material⟩`}`
- `\vtop`⟨box specification⟩`{`⟨vertical material⟩`}`
- `\box`⟨8-bit number⟩
- `\copy`⟨8-bit number⟩
- `\vsplit`⟨8-bit number⟩`to`⟨dimen⟩
- `\lastbox`

⟨box specification⟩ 定义为

⟨box specification⟩ ⟶ ⟨filler⟩
| `to` ⟨dimen⟩⟨filler⟩ | `spread` ⟨dimen⟩⟨filler⟩

一个⟨8-bit number⟩ 是范围在 0–255 之间的数。

包围盒子材料的花括号界定了一个组；它们可以是类别 1 与 2 的显式字符，或是用 `\let` 令其等同于这些字符的控制序列；另见下文。

一般来说，⟨box⟩ 可用于水平、竖直和数学模式，但 `\lastbox` 见下文。盒子与模式之间的联系在第 6 章进一步探讨。

由 `\vcenter` 产生的盒子——该命令只允许在数学模式中使用——并不是一个⟨box⟩。例如，它不能用`\setbox` 赋值；另见第 23 章。

`\vsplit` 操作在第 27 章讨论。

5.2 盒子寄存器

共有 256 个盒子寄存器，编号 0–255。一个盒子寄存器要么为空（'void'），要么含有一个水平或竖直盒子。本节专门讨论盒子寄存器；盒子的尺寸以及其内部材料的排列方式见下文。

5.2.1 分配：`\newbox`

Plain TEX 的`\newbox` 宏分配一个未使用的盒子寄存器：

```
\newbox\MyBox
```

之后可以使用

```
\setbox\MyBox=...
```

或

```
\box\MyBox
```

等等。对该宏的后续调用会给出后续的盒子编号；这样，宏集合可以分配自己的盒子，而不必担心与其他宏冲突。

盒子的编号由 `\chardef` 赋值（见第 31 章）。这意味着 `\MyBox` 等价于一个 ⟨number⟩，并可作为其使用。该控制序列 `\newbox` 是一个 `\outer` 宏。新分配的盒子寄存器最初为空。

5.2.2 用法：\setbox、\box、\copy

一个寄存器通过向其赋值一个⟨box⟩ 来填充：

\setbox⟨number⟩⟨equals⟩⟨box⟩

例如，⟨box⟩ 可以是显式的

\setbox37=\hbox{...} 或 \setbox37=\vbox{...}

也可以是一个盒子寄存器：

\setbox37=\box38

通常，盒子编号由 \newbox 命令赋值。

盒子寄存器中的盒子由命令 \box 与 \copy 追加到 TEX 正在构造的任何列表中：调用

\box38

会追加盒子 38。为节省内存，盒子寄存器在被使用时即变空：TEX 假定，在你于某种模式中调用\box*nn* 插入一个盒子之后，不再需要该寄存器的内容，于是清空它。如果你确实需要一个盒子寄存器的内容多次使用，可以用\copy 它。调用 \copy*nn* 在各方面都等价于 \box*nn*，只是不清空该寄存器。

可以用命令 \unhbox 与 \unvbox，以及它们的复制版本 \unhcopy 与 \unvcopy，把一个盒子寄存器的内容" 拆开"。盒子可在任意模式中使用，而拆开操作只能在相应模式中使用，因为实际上它们贡献的是一个不完整的水平或竖直列表（另见第 6 章）。关于拆开寄存器的更多信息见下文。

5.2.3 测试：\ifvoid、\ifhbox、\ifvbox

盒子寄存器可以按其内容测试：

\ifvoid⟨number⟩

当盒子寄存器为空时为真。注意，一个空的（即 'void'）盒子寄存器与一个含有空盒子的寄存器并不相同。空盒子仍是水平盒子或竖直盒子；而空寄存器两者兼可。

测试

\ifhbox⟨number⟩

当盒子寄存器含有水平盒子时为真；

\ifvbox⟨number⟩

当盒子寄存器含有竖直盒子时为真。对于空寄存器，两个测试都为假。

5.2.4 \lastbox

当 TEX 构造了一个不完整的列表时，该列表中的最后一个盒子可作为\lastbox 访问。它的行为像一个盒子寄存器，因此你可以通过把 \lastbox 赋给某个盒子寄存器而从列表中移除最后一个盒子。如果当前列表的最后一项不是盒子，\lastbox 的行为就像一个空的盒子寄存器。对于主竖直列表，无法取得最后一个盒子。此时 \lastbox 总是空的。

例如，语句

{\setbox0=\lastbox}

从当前列表中移除最后一个盒子，把它赋给盒子寄存器 0。由于该赋值发生在一个组内，寄存器在组结束时被清空。在段落开头，可以用它移除缩进盒子（见第 16 章）。\lastbox 的另一个例子见第 57 页。

由于\lastbox 在外部竖直模式中总是空的，无法取得已添加到页面上的盒子。然而，一旦页面在 \box255 中，就可以解剖它，例如在输出例程内部这样做：

```
\vbox{\unvbox255{\setbox0=\lastbox}}
```

如果竖直模式中的盒子被 \moveright 或 \moveleft 移动过，或水平模式中的盒子被 \raise 抬高或被 \lower 降低过，那么当从列表中取出 \lastbox 时，由这种命令引起的位移信息都会丢失。

5.3 盒子的自然尺寸

5.3.1 所构造水平盒子的尺寸

在一个\hbox 内部，所有组成部分依次并排排列，其参考点位于盒子的基线上，除非它们被 \lower 或 \raise 在竖直方向显式移动。

盒子的最终宽度是各组成部分宽度之和。因此

```
\hbox{\hskip1cm}
```

的宽度为正，而

```
\hbox{\hskip-1cm}
```

的宽度为负。举例来说，

```
a\hbox{\kern-1em b}--
```

的输出为

ba

这表明水平盒子可以有负宽度。

\hbox 的高与深，是组成盒子在盒子基线之上与之下伸出的最大量。盒子被创建时它们为非负。

命令 \lower 与 \raise 是在 \hbox 内部进行竖直移动的仅有手段（当然，在 \hbox 内部包含一个 \vbox 除外）；⟨vertical command⟩（例如 \vskip）不允许出现在水平盒子中；\par 虽然允许，但在水平盒子内部不做任何事。

5.3.2 所构造竖直盒子的尺寸

在一个\vbox 内部，竖直材料依次排列，其参考点位于穿过盒子参考点的竖直线上，除非各组成部分被\moveleft 或 \moveright 在水平方向显式移动。

竖直盒子的参考点总是位于盒子的左边界。于是竖直盒子的宽度，是盒子中任何材料在参考点右侧伸出的最大量。参考点左侧的材料不计入宽度。因此

```
a\vbox{\hbox{\kern-1em b}}--
```

的结果为

ba–

这应与上面的例子对比。

对用 \vbox 与 \vtop 构造的竖直盒子，高与深的计算是不同的。基本规则是：\vbox 的参考点位于其最后一个组成部分的基线上，而 \vtop 的参考点位于其第一个组成部分的基线上。一般地，若 \vbox（\vtop）的最后（首）一项是盒子或规则线，其深（高）可以非零。

\vbox 的高，是除最后一个组成部分外所有组成部分的高与深之和，再加上最后一个组成部分的高；\vbox 的深是其最后一个组成部分的深。\vtop 的深，是第一个组成部分的深与其后所有材料的高和深之和；其高是第一个组成部分的高。

然而，当 \vtop 的第一个组成部分或 \vbox 的最后一个组成部分不是盒子或规则线时，实际规则要复杂一些。如果 \vbox 的最后一个组成部分是 kern 或胶，该盒子的深为零；\vtop 的高为零，除非其第一个组成部分是盒子或规则线。（注意这些定义中的不对称性；说明这一点的例子见下文。）于是 \vtop 的深，等于其所包围全部材料的总高加深减去 \vtop 的高。

竖直盒子的深度有一个上限：如果按上述规则算出的 \vbox 或 \vtop 的深度会超过，则盒子的参考点向下移动超出量。更确切地说，超出的深度被加到盒子的自然高度上。如果盒子带有 to 或 spread 规格，则所有胶会重新设置，以把新的高度考虑进去。

通常，\boxmaxdepth 被设为 TEX 中可能的最大尺寸。例如，在 plain TEX 输出例程的某些计算中它会被减小；见第 28 章。

5.3.3 例子

水平盒子相对直接。其宽度是盒子 '开头' 与 '结尾' 之间的距离，因而宽度不一定为正。用

```
\setbox0=\hbox{aa} \setbox1=\hbox{\copy0 \hskip-\wd0}
```

时，\box1 的宽度为零；

```
/\box1/
```

给出 '/a̸a'

水平盒子的高与深不能为负：在

```
\setbox0=\hbox{\vrule height 5pt depth 5pt}
\setbox1=\hbox{\raise 10pt \box0}
```

中，\box1 的深为 0pt、高为 15pt。

竖直盒子比水平盒子更麻烦。我们先讨论它们的宽度。在

```
\setbox0=\hbox{\hskip 10pt}
```

之后，\box0 寄存器中的盒子宽度为 10pt。定义

```
\setbox1=\vbox{\moveleft 5pt \copy0}
```

时，\box1 的宽度为 5pt；参考点左侧的材料不计入竖直盒子的宽度。用

```
\setbox2=\vbox{\moveright 5pt \copy0}
```

时，\box2 的宽度为 15pt。

\vbox 的深是最后一项（若它是盒子）的深，因此

```
\vbox{\vskip 5pt \hbox{\vrule height 5pt depth 5pt}}
```

的高为 10pt、深为 5pt，而

```
\vbox{\vskip -5pt \hbox{\vrule height 5pt depth 5pt}}
```

的高为 0pt、深为 5pt。当胶或 kern 是盒子中的最后一项时，最终深度为零，因此

`\vbox{\hbox{\vrule height 5pt depth 5pt}\vskip 5pt}`

的高为 `15pt`、深为 `0pt`；

`\vbox{\hbox{\vrule height 5pt depth 5pt}\vskip -5pt}`

的高为 `5pt`、深为 `0pt`。

`\vtop` 的高相对于盒子第一项的行为，与 `\vbox` 的深相对于最后一项的行为（几乎）相同。把上面的例子改用 `\vtop`，结果如下：

`\vtop{\vskip 5pt \hbox{\vrule height 5pt depth 5pt}}`

的高为 `0pt`、深为 `15pt`，而

`\vtop{\vskip -5pt \hbox{\vrule height 5pt depth 5pt}}`

的高为 `0pt`、深为 `5pt`；

`\vtop{\hbox{\vrule height 5pt depth 5pt} \vskip 5pt}`

的高为 `5pt`、深为 `10pt`，以及

`\vtop{\hbox{\vrule height 5pt depth 5pt} \vskip -5pt}`

的高为 `5pt`、深为 `0pt`。

5.4 再谈盒子尺寸

5.4.1 预定尺寸

盒子的尺寸可以用⟨box specification⟩ 预先指定；语法见上文。盒子中的任何胶随后被设置，以达到所需尺寸。规定盒子尺寸是通过

`\hbox to` ⟨dimen⟩ `{...}`, `\vbox to` ⟨dimen⟩ `{...}`

完成的。如果盒子中存在可拉伸或可收缩的胶，它会被拉伸或收缩，使盒子达到指定尺寸。与这种胶设置相关的是一个 badness 值（见第 8 章）。如果不存在所需的拉伸或收缩——无论是哪一种——最终盒子将分别为过松或过满。关于过松/过满盒子的错误报告见下文。

另一个让盒子具有非自然尺寸的命令是

`\hbox spread` ⟨dimen⟩ `{...}`, `\vbox spread` ⟨dimen⟩ `{...}`

它告诉 TEX 这样设置胶：使盒子的尺寸比自然尺寸多出一个指定量。

`\vtop` 竖直盒子的盒子规格有些难以解释。TEX 通过先构造一个 `\vbox`（包括由⟨box specification⟩ 引起的胶设置）来构造 `\vtop`；然后按上述规则计算高与深。胶设置在第 8 章描述。

5.4.2 改变盒子尺寸

盒子寄存器的尺寸可由命令`\ht`、`\dp` 与 `\wd` 访问；例如 `\dp13` 给出盒子 13 的深。然而，不仅可以这样测量盒子；通过给这些尺寸赋值，甚至可以让 TEX 误以为盒子的尺寸与其实际不同。不过，改变盒子的尺寸不会改变其内容；尤其不会改变胶的设置方式。

各种格式在 ‘smash’ 宏中使用这一点：由

```
\def\smash#1{{\setbox0=\hbox{#1}\dp0=0pt \ht0=0pt \box0\relax}}
```

定义的宏放置其参数，但取消其高与深；也就是说，输出确实显示整个盒子，但 TEX 的后续计算表现得就像高与深为零。

盒子尺寸只能通过设置来改变。它们是⟨box dimen⟩，只能在⟨box size assignment⟩中设置，而不能例如用 `\advance` 修改。

注意，⟨box size assignment⟩ 是一种⟨global assignment⟩：它的作用超越它所出现的任何组（见第 10 章）。因此

```
\setbox0=\hbox{---} {\wd0=0pt} a\box0b
```

的输出为 'ab–'。

创建盒子时对尺寸施加的限制（见上文），对显式改变盒子大小并不成立：例如对一个 `\dp0=-2pt` 的赋值对于水平盒子是完全允许的。

5.4.3 移动盒子

在水平盒子中，所有组成元素依次排列，其参考点与盒子的参考点处于同一高度。水平盒子内的任何盒子都可以用宏 `\raise` 与 `\lower` 抬高或降低。

类似地，在竖直盒子中，所有组成元素依次上下排列，其参考点沿盒子的参考点排成一线。盒子现在可以用宏`\moveleft` 与 `\moveright` 侧向移动。

只有盒子可以这样移动；这些操作不能应用于例如字符或规则线。

5.4.4 盒子尺寸与盒子放置

TEX 通过维持一条参考线（reference line）以及该线上的一个当前位置（current position）来放置水平列表与竖直列表的组成成分。对于水平列表，参考线是外层 `\hbox` 的基线；对于竖直列表，参考线是穿过外层 `\vbox` 参考点的竖直线。

在水平模式中，一个组成成分按如下方式放置。当前位置最初与外层盒子的参考点重合。在此基础上，执行以下动作。

1. 如果该组成成分已被 `\raise` 或 `\lower` 移动，则相应地移动当前位置。
2. 如果该组成成分是水平盒子，则对其内容递归使用本算法；如果它是竖直盒子，则向上移动该盒子的高度，把新的当前位置放到 enclosed 竖直列表的该处，并使用下文竖直列表的算法放置其组成成分。
3. 将参考线上的当前位置向右移动该组成成分的宽度。

对于竖直盒子中的列表，TEX 的当前位置如上所述最初位于该盒子的左上角，而参考线是穿过该点的竖直线；它同时也穿过该盒子的参考点。被包围的组成成分随后按如下方式放置。

1. 如果某组成成分已用 `\moveleft` 或 `\moveright` 移动，则相应地移动当前位置。
2. 将该组成成分的左上角放在当前位置。
3. 如果该组成成分是竖直盒子，则对其内容递归使用本算法；如果它是水平盒子，其参考点可在当前位置下方一个盒子高度处找到。把该盒子的当前位置放在该处，并使用上文水平列表的算法。
4. 沿参考线向下移动该盒子的高加深度（即从盒子的左上角开始），然后继续竖直处理。

注意，上述过程并不描述盒子的构造。那会（例如）涉及为竖直盒子插入基线胶。相反，它描述的是一个已完成盒子的各组成部分在输出中的排列方式。

5.4.5 盒子与负胶

有时让盒子重叠而不是排成一线很有用。一个简单办法是使用负胶。在水平模式中

```
{\dimen0=\wd8 \box8 \kern-\dimen0}
```

放置盒子 8 而不移动当前位置。

更灵活的是宏`\llap` 与`\rlap`，定义为

```
\def\llap#1{\hbox to 0pt{\hss #1}}
```

和

```
\def\rlap#1{\hbox to 0pt{#1\hss}}
```

它们允许材料从当前位置向左或向右伸出。`\hss` 胶等价于`\hskip 0pt plus 1fil minus 1fil`，它吸收 `\llap` 或 `\rlap` 参数的任何正或负宽度。

> 序列
> `\llap{\hbox to 10pt{a\hfil}}`
> 实际上等价于
> `\hbox{\hskip-10pt \hbox to 10pt{a\hfil}}`
> 其总宽度为 `0pt`。

5.5 过满与过松盒子

如果盒子带有尺寸规格，TEX 会拉伸或收缩盒子中的胶。对于只有有限拉伸或收缩分量的胶，会计算拉伸或收缩的 *badness*（见第 19 章）。在 TEX 第 3 版中，最近构造的盒子的 badness 可供用户通过 `\badness` 参数查看。badness 的取值范围为 0–10 000，但若盒子过满则为 1 000 000。

当 TEX 认为 badness 太大时，会给出诊断信息。我们先考虑水平盒子的错误报告。

其胶需要拉伸的水平盒子，在`\hbadness` $\geq 10\,000$ 时从不报告；否则，若其 badness 大于 `\hbadness`，TEX 就报告它们为 ‘过松’。

胶的收缩可能导致 ‘过满’ 盒子：如果可用收缩量小于满足盒子规格所需的收缩量，则盒子称为过满。只有当收缩量之差大于 `\hfuzz`，或当 `\hbadness` < 100 时（而事实是用尽全部可用收缩量的 badness 为 100），才报告过满盒子。

> 设置`\hfuzz=1pt` 会让 TEX 忽略那些收缩得不够、但缺少量小于 `1pt` 的盒子。在
> `\hbox to 1pt{\hskip3pt minus .5pt}`
> `\hbox to 1pt{\hskip3pt minus 1.5pt}`
> 中，只有第一个盒子会给出错误信息：它大了 `1.5pt`，而第二个缺少`.5pt`，小于 `\hfuzz`。

此外，收缩但并不过满的盒子也可能被报告：如果一个盒子 ‘紧’，即它至少用掉一半可收缩量，那么当算出的 badness（介于 13 与 100 之间）大于 `\hbadness` 时，TEX 就报告这一事实。

对于水平与竖直盒子，这种错误报告几乎相同，使用参数 `\vbadness` 与 `\vfuzz`。区别在于：对于水平过满盒子，TEX 会在盒子右侧画一条与盒子同高、宽度为 `\overfullrule` 的规则线。如果 `\halign` 中的 `\tabskip` 胶收缩得不够，则不会出现过满规则线。

5.6 打开与关闭盒子

盒子的开括号与闭括号可以是显式的，即类别码为 1 与 2 的字符令牌，也可以是隐式的，即通过\let 指向这类字符的控制序列。在开括号处插入\everyhbox 或\everyvbox 令牌。如果该盒子出现在\setbox 赋值中，则任何\afterassignment 令牌甚至会先于 'everybox' 令牌被插入。

```
\everyhbox{b}
\afterassignment a
\setbox0=\hbox{c}
\showbox0
给出
> \box0=
\hbox(6.94444+0.0)x15.27782
.\tenrm a
.\tenrm b
.\kern0.27779
.\tenrm c
```

隐式括号可用于让一个宏打开或关闭盒子，例如：

```
\def\openbox#1{\setbox#1=\hbox\bgroup}
\def\closebox#1{\egroup\DoSomethingWithBox#1}
\openbox0 ... \closebox0
```

这个机制可用来 '舀取' 段落：

```
\everypar{\setbox\parbox=
    \vbox\bgroup
         \everypar{}
         \def\par{\egroup\UseBox\parbox}}
```

这里 \everypar 打开盒子并让文本在盒子中排版：例如开始

```
Begin a text ...
```

等价于

```
\setbox\parbox=\vbox{Begin a text ...
```

在盒子内部 \par 已被重新定义，因此

```
... a text ends.\par
```

等价于

```
... a text ends.}\Usebox\parbox
```

在这个例子中，\UseBox 命令只能把盒子作为整体处理；如果盒子的各组成部分需要以某种方式分别处理，则需要另一种方法。在

```
\everypar{\setbox\parbox=
  \vbox\bgroup\everypar{}%
       \def\par{\endgraf\HandleLines
                \egroup\box\parbox}}
\def\HandleLines{ ... \lastbox ... }
```

中，宏 \HandleLines 可以访问段落竖直列表中相继的元素。另见第 57 页的例子。

5.7 拆开盒子

盒子可以用命令\unhbox 与\unvbox，以及它们的复制版本\unhcopy 与\unvcopy 来拆开。这些是水平与竖直命令（见第 6 章），因为实际上它们贡献的是一个不完

整的水平或竖直列表。不能对含有 \vbox 的寄存器执行 \unhbox，反之亦然，但一个空的盒子寄存器既可以被 \unhbox，也可以被 \unvbox。

拆开盒子会取出盒子寄存器中盒子的内容，并把它们追加到周围列表；随后任何胶都可以重新设置。因此

```
\setbox0=\hbox to 1cm{\hfil} \hbox to 2cm{\unhbox0}
```

完全等价于

```
\hbox to 2cm{\hfil}
```

而不等价于

```
\hbox to 2cm{\kern1cm}
```

\unhbox 的内在水平性质被用来定义

```
\def\leavevmode{\unhbox\voidb@x}
```

该命令从竖直模式切换到水平模式，而不向水平列表添加任何东西。然而，由这一转换引起的后续 \indent 会添加一个缩进盒子。在水平模式中，\leavevmode 命令不起作用。注意，这里不必使用 \unhcopy，因为该寄存器反正为空。

注意下面这个微妙之处：在竖直模式中拆开盒子，不会在盒子内容与任何前一项之间添加行间胶。此外，\prevdepth 的值不被改变，因此盒子内容与任何后一项之间的胶，只有在盒子之前确有某物时才会出现；行间胶将基于那个前一项的深度。类似地，在水平模式中拆开盒子不影响 \spacefactor。

5.8 盒子中的文本

水平与竖直盒子都可以含有文本。然而，文本的处理方式不同。在水平盒子中，文本被排成一条直线，盒子的宽度原则上是其中所含文本（及其他项）的自然宽度。水平盒子内部不允许有⟨vertical command⟩，此时 \par 不做任何事。

对于竖直盒子,情况截然不同。一旦在竖直盒子中遇到一个字符或任何其他⟨horizontal command⟩ （见第 60 页），TEX 就在非受限水平模式中开始构造一个段落，也就是说，就像该段落直接是页面的一部分一样。在遇到一个⟨vertical command⟩ （见第 60 页）时，或在盒子结束时，段落会用诸如 \hsize 等参数的当前值断行。

因此

```
\hbox to 3cm{\vbox{some reasonably long text}}
```

不会给出宽度为 3 厘米的段落（如果 \hsize > 3cm，它会给出一个过满的水平盒子）。然而，

```
\vbox{\hsize=3cm some reasonably long text}
```

的宽度将为 3 厘米。

竖直盒子中的一段文本被断成行，这些行打包成水平盒子。这些盒子随后在内外部竖直模式中堆叠，可能用 \baselineskip 与 \lineskip 分隔它们（这在第 15 章讨论）。页面上的文本也使用这一过程；此时盒子在外部竖直模式中堆叠。

如果内部竖直列表为空，则在段落开头不添加 \parskip 胶。

由于水平盒子中的文本不断行，受限与非受限水平模式中的文本之间还有进一步区别。在受限水平模式中，不插入断行结点和会改变当前语言值的 whatsit 项。如果随后文本被拆开以构成段落的一部分，这可能带来问题。

关于这些项的解释见第 19 章，绕过这一问题的方法见 [7]。

5.9 杂注

5.9.1 忘记 \box

在\newcount\foo 之后，可以单独使用 \foo 来取得 \foo 计数器。但对于盒子，必须使用\box\foo 来取得 \foo 盒子。原因是不存在单独的 \boxdef 命令，于是使用 \chardef（见第 31 章）。

> 假设\newbox\foo 分配盒子寄存器 25；那么键入 \foo 等价于键入\char25 .

5.9.2 特殊用途盒子

有些盒子寄存器有特殊用途：

- \box255 由 TEX 在内部使用，用于把页面交给输出例程。
- \voidb@x 是在 plain.tex 中分配的一个盒子寄存器的编号；它应当总是空的。它用于 \leavevmode 等宏中。
- 当用 plain TEX 的 \newinsert 宏创建一个新的 \insert 时，一个与该 insert 同号的 \count、\dimen、\skip 与 \box 都会被预留。这些寄存器的编号从 254 向下计数。

5.9.3 水平模式中竖直盒子的高

在水平模式中，一个竖直盒子放置时，其参考点与周围盒子的参考点竖直对齐。TEX 随后从左上角开始遍历其内容；也就是说，从参考点上方距离为盒子高度处开始。改变盒子的高，意味着盒子的内容被放在不同的高度。

作为例子

```
\hbox{a\setbox0=\vbox{\hbox{b}}\box0 c}
```

它给出

> abc

以及

```
\hbox{a\setbox0=\vbox{\hbox{b}}\ht0=0cm \box0 c}
```

它给出

> $\mathrm{a}_{\mathrm{b}}\,\mathrm{c}$

相比之下，改变在竖直模式中放置的盒子的宽度，对其放置没有影响。

5.9.4 竖直盒子的更多微妙之处

由于有两种竖直盒子——\vbox 与 \vtop——把这两种嵌套使用可能导致令人困惑的结果。例如，

```
\vtop{\vbox{...}}
```

完全等价于

```
\vbox{...}
```

上文已经说明：如果最后一项是 kern 或胶，\vbox 的深为零；而 \vtop 的高为零，除非其中第一项是盒子。上面的例子用 kern 作为那个首项或末项，但在 \vtop 的情形中，如果该项不是胶或 kern，人们容易忽略它对周围盒子的影响。例如，

```
\vtop{\write16{...}...}
```

的高为零，因为 write 指令被打包成一个 'whatsit' 项，放在当前（即竖直）列表上。补救办法是

```
\vtop{\leavevmode\write16{...}...}
```

它把 whatsit 放在段落开头，而不是段落上方。

竖直列表中各项的放置有时有点棘手。例如，竖直盒子与水平盒子在竖直列表中的处理方式是不同的。请看下面的例子。在 `\offinterlineskip` 之后，第一个例子

```
\vbox{\hbox{a}
      \setbox0=\vbox{\hbox{(}}
      \ht0=0pt \dp0=0pt \box0
      \hbox{ b}}
```

给出

a
(b

而一个稍加改动的版本

```
\vbox{\hbox{a}
      \setbox0=\hbox{(}
      \ht0=0pt \dp0=0pt \box0
      \hbox{ b}}
```

给出

a(
b

这一区别源于：水平盒子按其参考点放置，而竖直盒子按其左上角放置。

5.9.5 把 `\lastbox` 挂回列表

你可以从一个在（内部）竖直模式中排好的竖直列表中取出最后一个盒子。然而，如果试图把它挂回列表，竖直间距可能会乱掉。如果直接挂回，

```
\setbox\tmpbox=\lastbox
\usethetmpbox \box\tmpbox
```

baselineskip 胶会被第二次添加。如果在挂回盒子之前先 'unskip'，

```
\setbox\tmpbox=\lastbox \unskip
\usethetmpbox \box\tmpbox
```

事情会以更微妙的方式出错。⟨internal dimen⟩ `\prevdepth`（它控制行间胶；见第 15 章）的值将基于最后一个盒子，但正确的行间胶所需要的是基于再前一个盒子的深度。解决办法不是 unskip，而是指定 `\nointerlineskip`：

```
\setbox\tmpbox=\lastbox
\usethetmpbox \nointerlineskip \box\tmpbox
```

5.9.6 用 `\lastbox` 解剖段落

反复应用 `\last...` 与 `\un...` 宏可用来把一段文本拆开。下面是一个这样的例子。

在排版广告文案时，近年来一种两端对齐段落的方式流行起来，它介于右对齐与参差不齐之间。会拉伸超过某些限度的行，其胶按自然宽度设置。本段就是这种做法的示例；其宏见下。

```
\newbox\linebox \newbox\snapbox
\def\eatlines{
    \setbox\linebox\lastbox    % check the last line
```

```
\ifvoid\linebox
\else                       % if it's not empty
\unskip\unpenalty           % take whatever is
{\eatlines}                 % above it;
                            % collapse the line
\setbox\snapbox\hbox{\unhcopy\linebox}
                    % depending on the difference
\ifdim\wd\snapbox<.98\wd\linebox
      \box\snapbox % take the one or the other,
\else \box\linebox \fi
\fi}
```

该宏可以这样调用：

```
\vbox{ ... some text ... \par\eatlines}
```

或者用 `\everypar` 自动插入；见 [10]。

在宏 `\eatlines` 中，`\lastbox` 取自一个竖直列表。如果列表为空，最后一个盒子在 `\ifvoid` 测试中为真。这些含有段落中各行的盒子实际上是水平盒子：对它们施加 `\ifhbox` 测试会得到真结果。

第 6 章

水平与竖直模式

在处理过程中的任意时刻，TEX 都处于某种模式。共有六种模式，分为三类：

1. 水平模式与受限水平模式，
2. 竖直模式与内部竖直模式，以及
3. 数学模式与展示数学模式。

数学模式将在别处讨论（见第 166 页）。这里我们考察水平与竖直模式、相应列表中可以出现的对象种类，以及专属于某一模式的命令。

`\ifhmode` 测试当前模式是否为（可能是受限的）水平模式。
`\ifvmode` 测试当前模式是否为（可能是内部的）竖直模式。
`\ifinner` 测试当前模式是否为内部模式。
`\vadjust` 在水平模式中为外围竖直列表指定竖直材料。
`\showlists` 把所有模式中当前正在构造的不完整列表的内容写入日志文件。

6.1 水平与竖直模式

不排版数学时，TEX 处于水平或竖直模式，分别构造水平或竖直列表。水平模式通常用于生成文本行；竖直模式通常用于把段落各行上下堆叠。注意，这些模式与 TEX 输入处理器的内部状态不同（见第 26 页）。

6.1.1 水平模式

水平模式的主要活动是构造文本行。页面上的文本以及 `\vbox` 或 `\vtop` 中的文本都在水平模式中构造（这可称为‘段落模式’）；如果文本在 `\hbox` 中，则只有一行文本，相应模式是受限水平模式。

在水平模式中，所有材料都被添加到水平列表。如果该列表在非受限水平模式中构造，它随后会被断成行并添加到周围的竖直列表。

水平列表的每个元素属于以下之一：

- 一个盒子（字符、连字、`\vrule`，或一个⟨box⟩），
- 一个断行结点，
- 一个 whatsit（见第 30 章），
- 用 `\mark`、`\vadjust` 或 `\insert` 包围的竖直材料，
- 胶或引线、kern、惩罚，或一个数学开/关项。

最后一项中的所有元素都是可丢弃的。可丢弃项之所以如此称呼，是因为它们在断行时消失。水平列表的断行在第 19 章讨论。

6.1.2 竖直模式

竖直模式可用于把各项上下堆叠。大多数时候，这些项是含有段落各行的盒子。

堆叠材料可以发生在竖直盒子内部，但堆叠的各项也可以单独出现在页面上。在后一种情形中，TEX 处于竖直模式；在前一种情形中，即在竖直盒子内部，TEX 以内部竖直模式运作。

在竖直模式中，所有材料都被添加到竖直列表。如果该列表在外部竖直模式中构造，它随后会在形成页面时被断开。

竖直列表的每个元素属于以下之一：

- 一个盒子（水平或竖直盒子，或一个 `\hrule`），
- 一个 whatsit，
- 一个 mark，
- 胶或引线、kern，或惩罚。

最后一项中的所有元素都是可丢弃的。竖直列表的断行在第 27 章讨论。

竖直列表开头有几个例外条件：`\prevdepth` 的值被设为 `-1000pt`。此外，内部竖直列表顶部不添加 `\parskip` 胶；在主竖直列表顶部（'当前页面' 顶部）不添加胶或其他可丢弃项，而当第一个盒子放到该列表上时会添加 `\topskip` 胶（见第 26 章与第 27 章）。

6.2 水平与竖直命令

有些命令本质上就是水平或竖直的，它们会在可能时迫使 TEX 进入相应模式。迫使 TEX 进入水平模式的命令称为⟨horizontal command⟩；类似地，迫使 TEX 进入竖直模式的命令称为⟨vertical command⟩。

然而，并非所有模式转换都是可能的：TEX 可以从两种竖直模式转换到（非受限）水平模式，再通过水平与竖直命令返回，但不能与受限水平模式互相转换（除非把水平盒子包围在竖直盒子中，或反过来）。因此，受限水平模式中的竖直命令会给出错误；受限水平模式中的 `\par` 命令不起作用。

水平命令如下：

- 任何⟨letter⟩、⟨otherchar⟩、`\char`、由 `\chardef` 定义的控制序列，或 `\noboundary`；
- `\accent`、`\discretionary`、断行连字符 `\-` 与控制空格 `\ `；
- `\unhbox` 与 `\unhcopy`；
- `\vrule` 以及⟨horizontal skip⟩ 命令 `\hskip`、`\hfil`、`\hfill`、`\hss` 与 `\hfilneg`；
- `\valign`；
- 数学切换符（`$`）。

竖直命令如下：

- `\unvbox` 与 `\unvcopy`；
- `\hrule` 以及⟨vertical skip⟩ 命令 `\vskip`、`\vfil`、`\vfill`、`\vss` 与 `\vfilneg`；
- `\halign`；
- `\end` 与 `\dump`。

注意，竖直命令不包括 `\par`；`\indent` 与 `\noindent` 也不是水平命令。

盒子与模式之间的联系在下文探讨；规则与模式之间的联系见第 9 章。

6.3 内部模式

受限水平模式与内部竖直模式分别是水平模式与竖直模式在 \hbox 与 \vbox（或 \vtop、\vcenter）内部的变体。然而，受限水平模式在本质上比内部竖直模式受限得多。第三种内部模式是非展示数学模式（见第 23 章）。

6.3.1 受限水平模式

受限水平模式（\hbox 中的模式）与非受限水平模式（竖直盒子中及页面上段落构造时的模式）之间的主要区别是：你无法跳出受限水平模式——\par 在该模式中不做任何事。此外，受限水平模式中的⟨vertical command⟩ 会给出错误。在非受限水平模式中，它会导致插入一个 \par 记号并进入竖直模式（另见第 17 章）。

6.3.2 内部竖直模式

内部竖直模式是 \vbox 内部的竖直模式，它与外部竖直模式（页面构造时的模式）非常相似。内部竖直模式中的⟨horizontal command⟩ 例如是完全有效的：TEX 随后就在非受限水平模式中开始构造段落。

一个区别是：命令 \unskip 与 \unkern 在外部竖直模式中不起作用，\lastbox 在外部竖直模式中总是空的。另见第 48 页与第 79 页。

对齐的各项（见第 25 章）在内部模式中处理：\halign 的各项用受限水平模式，\valign 的各项用内部竖直模式。\vadjust 与 \insert 项中的材料也在内部竖直模式中处理；此外，TEX 在处理 \output 记号列表时进入此模式。

命令 \end 与 \dump（后者仅存在于 IniTEX）不允许在内部竖直模式中使用；此外，\dump 不允许出现在一个组内部（见第 33 章）。

6.4 盒子与模式

有水平盒子与竖直盒子，也有水平与竖直模式。不出所料，盒子与模式之间存在联系。可以从两个角度来问这一联系。

6.4.1 在什么模式中使用什么盒子？

这是错误的问题。水平盒子与竖直盒子都可以在水平与竖直模式中使用。它们的放置由当时所处的模式决定。

6.4.2 什么盒子中处于什么模式？

这是正确的问题。当 \hbox 开始时，TEX 处于受限水平模式。因此水平盒子中的所有内容水平排成一线。

当 \vbox 开始时，TEX 处于内部竖直模式。两种盒子及其他各项随后上下堆叠。

6.4.3 盒子的模式相关行为

任何⟨box⟩（完整定义见第 5 章）都可以在水平、竖直和数学模式中使用。然而，拆开命令是水平或竖直模式专有的。`\unhbox` 与 `\unhcopy` 都是⟨horizontal command⟩，因此它们可以使 TEX 从竖直模式切换到水平模式；`\unvbox` 与 `\unvcopy` 都是⟨vertical command⟩，因此它们可以使 TEX 从水平模式切换到竖直模式。

在水平模式中，放置一个盒子之后 `\spacefactor` 被设为 1000。在竖直模式中，`\prevdepth` 被设为所放置盒子的深度。这两条对拆开命令都不成立：`\unhbox` 或 `\unhcopy` 之后 spacefactor 不被改变，`\unvbox` 或 `\unvcopy` 之后 `\prevdepth` 保持不变。毕竟，这些命令添加的不是盒子，而是一个（水平或竖直）列表的片段。

命令 `\raise` 与 `\lower` 只能应用于水平模式中的盒子；类似地，`\moveleft` 与 `\moveright` 只能在竖直模式中应用。

6.5 模式与胶

在水平与竖直模式中，TEX 都可以插入胶项，其大小由列表中前一个对象决定。

对于水平模式，为空格记号插入的胶量取决于列表中前一个对象的 `\spacefactor`。这在第 20 章讨论。

在竖直模式中，TEX 插入胶以使盒子彼此保持一定距离。该胶受当前项高度与前一项深度的影响。各项的深度记录在 `\prevdepth` 参数中（见第 15 章）。

`\prevdepth` 与 `\spacefactor` 这两个量使用 TEX 的同一个内部寄存器。因此 `\prevdepth` 只能在竖直模式中使用，而 `\spacefactor` 只能在水平模式中使用。

6.6 迁移材料

三个控制序列 `\insert`、`\mark` 与 `\vadjust` 可以在段落中给出（前两个也可以出现在竖直模式中），用于指定最终放到外围竖直列表上的材料。注意，这不一定是主竖直列表：它可以是含有一段文本的竖直盒子。在这种情况下，`\mark` 或 `\insert` 命令不会到达页面断开算法。

当在某一行文本中指定了多个迁移项时，它们放到外围竖直列表上时保持从左到右的顺序。这些项直接放在包含该行文本的水平盒子之后：它们出现在任何自动插入的惩罚或胶项之前（见第 148 页）。

6.6.1 `\vadjust`

命令

`\vadjust`⟨filler⟩`{`⟨vertical mode material⟩`}`

只允许在水平与数学模式中使用（但它不是⟨horizontal command⟩）。由 `\vadjust` 指定的竖直模式材料从给出该命令的水平列表移到外围竖直列表，直接放在它所出现的盒子之后。

- 在当前这一行 中，放置了一个 `\vadjust` 项，以把圆点放到边距中。

`\vadjust` 项中的任何竖直材料都在内部竖直模式中处理，即使它最终会到达主竖直列表。例如，`\ifinner` 测试在 `\vadjust` 中为真，并且在竖直材料开头 `\prevdepth=-1000pt`。

6.7 测试模式

三个条件命令 \ifhmode、\ifvmode 与 \ifinner 可以区分 TEX 中四种非数学模式。当 TEX 处于受限水平模式或内部竖直模式（或非展示数学模式）时，\ifinner 测试为真。例外情况：在 \write 期间，TEX 处于 '无模式' 状态。此时 \ifhmode、\ifvmode 与 \ifmmode 测试都为假。

检查所有当前列表（包括 '近期贡献'，见第 27 章）可以通过命令\showlists 进行。该命令把给出命令时刻正在构造的所有列表的内容写入日志文件。

考虑例子

```
a\hfil\break b\par
c\hfill\break d
\hbox{e\vbox{f\showlists
```

这里第一段已被断成两行，并已添加到当前页面。第二段尚未结束或断行。

日志文件显示如下。TEX 正在构造一个段落（以一个宽 20pt 的缩进盒子开头）：

```
### horizontal mode entered at line 3
\hbox(0.0+0.0)x20.0
\tenrm f
spacefactor 1000
```

该段落位于一个竖直盒子内部：

```
### internal vertical mode entered at line 3
prevdepth ignored
```

该竖直盒子又在一个水平盒子中，

```
### restricted horizontal mode entered at line 3
\tenrm e
spacefactor 1000
```

而该水平盒子属于一个尚未完成的段落：

```
### horizontal mode entered at line 2
\hbox(0.0+0.0)x20.0
\tenrm c
\glue 0.0 plus 1.0fill
\penalty -10000
\tenrm d
etc.
spacefactor 1000
```

注意无限胶与 \break 惩罚仍然是水平列表的一部分。

最后，第一段已被断成行并添加到当前页面：

```
### vertical mode entered at line 0
### current page:
\glue(\topskip) 5.69446
\hbox(4.30554+0.0)x469.75499, glue set 444.75497fil
.\hbox(0.0+0.0)x20.0
.\tenrm a
.\glue 0.0 plus 1.0fil
.\penalty -10000
.\glue(\rightskip) 0.0
\penalty 300
```

```
\glue(\baselineskip) 5.05556
\hbox(6.94444+0.0)x469.75499, glue set 464.19943fil
.\tenrm b
.\penalty 10000
.\glue(\parfillskip) 0.0 plus 1.0fil
.\glue(\rightskip) 0.0
etc.
total height 22.0 plus 1.0
 goal height 643.20255
prevdepth 0.0
```

第 7 章

数

本章讨论整数及其表示法、双向的转换、\count 寄存器的分配与使用，以及整数算术。

\number 把一个⟨number⟩ 转换为十进制表示。
\romannumeral 把一个正⟨number⟩ 转换为小写罗马数字表示。
\ifnum 测试数之间的关系。
\ifodd 测试一个数是否为奇数。
\ifcase 枚举情形语句。
\count 计数器寄存器的前缀。
\countdef 定义一个控制序列作为某个 \count 寄存器的同义词。
\newcount 分配一个未使用的 \count 寄存器。
\advance 对⟨numeric variable⟩ 进行加或减的算术命令。
\multiply 对⟨numeric variable⟩ 进行乘的算术命令。
\divide 对⟨numeric variable⟩ 进行除的算术命令。

7.1 数与⟨number⟩

TEX 语法的一个重要部分是⟨number⟩ 的严格定义：即在语义上需要一个整数时 TEX 所期望的语法实体。这一定义将占本章的大部分篇幅。在末尾，将讨论 \count 寄存器、算术以及数的测试。

为便于讨论，这里区分整数与数，但注意一个⟨number⟩ 既可以是 '整数' 也可以是 '数'。'整数' 指数学上的数：一个可以相加或相乘的量。'数' 指一个整数的打印表示：换句话说，即一串数字。

7.2 整数

相当多种不同的对象都可以在 TEX 中充当整数。本节将讨论它们全部，并附上 TEX 语法中的相关条目。

首先，整数可以为正或为负：

⟨number⟩ ⟶ ⟨optional signs⟩⟨unsigned number⟩
⟨optional signs⟩ ⟶ ⟨optional spaces⟩
| ⟨optional signs⟩⟨plus or minus⟩⟨optional spaces⟩

无符号整数的第一种可能是十进制、八进制或十六进制表示的数字串。连同字母常量，这些在此统称为⟨integer denotation⟩。整数的另一种可能是内部整数量，即一个⟨internal integer⟩；连同表示法，它们构成⟨normal integer⟩。最后，整数可以是一个⟨coerced integer⟩：即一个转换为整数值的内部⟨dimen⟩ 或⟨glue⟩ 量。

⟨unsigned number⟩ ⟶ ⟨normal integer⟩ | ⟨coerced integer⟩
⟨normal integer⟩ ⟶ ⟨integer denotation⟩ | ⟨internal integer⟩
⟨coerced integer⟩ ⟶ ⟨internal dimen⟩ | ⟨internal glue⟩

这些可能性将依次讨论。

7.2.1 表示法：整数

任何看起来像数字的东西都可用作⟨number⟩：因此 `42` 是一个数。此外也可以使用十进制以外的进制：

`'123`

是 $1 \times 8^2 + 2 \times 8^1 + 3 \times 8^0 = 83$ 的八进制表示，而

`"123`

是 $1 \times 16^2 + 2 \times 16^1 + 3 \times 16^0 = 291$ 的十六进制表示。

⟨integer denotation⟩ ⟶ ⟨integer constant⟩⟨one optional space⟩
| '⟨octal constant⟩⟨one optional space⟩
| "⟨hexadecimal constant⟩⟨one optional space⟩

八进制数字为 `0`–`7`；跟在八进制表示之后的数字 `8` 或 `9` 不属于该数：在

```
\count0='078
```

之后，`\count0` 的值为 7，而数字 `8` 被排印出来。

十六进制数字为 `0`–`9`、`A`–`F`，其中 `A`–`F` 可以具有类别码 11 或 12。后者有一个颇为牵强的理由：由 `\string` 操作产生的字符具有类别码 12。小写 `a`–`f` 不是十六进制数字，尽管（在 TEX3 中）它们用于通过 ‘音调符号法’ 访问所有字符码的十六进制表示（见第 3 章）。

7.2.2 表示法：字符

一个字符记号由字符码（一个范围在 0–255 的数）与类别码组成。这两种码都可访问，并可用作⟨number⟩。

字符记号（或单个字母控制序列）的字符码可通过左引号命令访问：`` `a `` 与 `` `\a `` 都表示 `a` 的字符码，可作为整数使用。

⟨integer denotation⟩ ⟶ `⟨character token⟩⟨one optional space⟩

为强调访问字符码在某种意义上是一种使用表示法的方式，TEX 的语法允许在这样的 ‘字符常量’ 之后有一个可选空格。左引号必须具有类别 12。

7.2.3 内部整数

⟨internal integer⟩ 这一类可分为五部分。⟨codename⟩ 与⟨special integer⟩ 将在下文单独讨论；此外还有以下几种。

- `\count` 寄存器的内容;既可显式写出例如 `\count23`,也可通过由 `\countdef` 定义的控制序列引用该寄存器：在
  ```
  \countdef\MyCount=23
  ```
  之后，`\MyCount` 称为一个⟨countdef token⟩，它完全等价于 `\count23`。

- TEX 中持有整数值的所有参数；这包括显而易见的如 `\linepenalty`，但也包括诸如 `\hyphenchar`⟨font⟩ 与 `\parshape` 等参数（如果已为 n 行定义了段落形状，则在⟨number⟩ 上下文中使用 `\parshape` 会给出此 n 值）。
- 由 `\chardef` 或 `\mathchardef` 定义的记号。在
  `\chardef\foo=74`
  中，控制序列 `\foo` 可以单独使用以表示 `\char74`，但在需要⟨number⟩ 的上下文中它也可用来表示 74：
  `\count\foo`
  等价于`\count74` 。这一事实在寄存器分配例程中被利用（见第 31 章）。
  由 `\chardef` 这样定义的控制序列称为⟨chardef token⟩；如果由 `\mathchardef` 定义，则称为⟨mathchardef token⟩。

完整列表如下：

⟨internal integer⟩ ⟶ ⟨integer parameter⟩
| ⟨special integer⟩ | `\lastpenalty`
| ⟨countdef token⟩ | `\count`⟨8-bit number⟩
| ⟨chardef token⟩ | ⟨mathchardef token⟩
| ⟨codename⟩⟨8-bit number⟩
| `\hyphenchar`⟨font⟩ | `\skewchar`⟨font⟩ | `\parshape`
| `\inputlineno` | `\badness`
⟨integer parameter⟩ ⟶ | `\adjdemerits` | `\binoppenalty`
| `\brokenpenalty` | `\clubpenalty` | `\day`
| `\defaulthyphenchar` | `\defaultskewchar`
| `\delimiterfactor` | `\displaywidowpenalty`
| `\doublehyphendemerits` | `\endlinechar` | `\escapechar`
| `\exhypenpenalty` | `\fam` | `\finalhyphendemerits`
| `\floatingpenalty` | `\globaldefs` | `\hangafter`
| `\hbadness` | `\hyphenpenalty` | `\interlinepenalty`
| `\linepenalty` | `\looseness` | `\mag`
| `\maxdeadcycles` | `\month`
| `\newlinechar` | `\outputpenalty` | `\pausing`
| `\postdisplaypenalty` | `\predisplaypenalty`
| `\pretolerance` | `\relpenalty` | `\showboxbreadth`
| `\showboxdepth` | `\time` | `\tolerance`
| `\tracingcommands` | `\tracinglostchars`

7.2.4 ⟨special integer⟩

内部整数的一个子类是特殊整数。

⟨special integer⟩ ⟶ `\spacefactor` | `\prevgraf`
| `\deadcycles` | `\insertpenalties`

对其中任何一个的赋值称为⟨intimate assignment⟩，并且是自动全局的（见第 10 章）。

7.2.5 其他内部量：强制转换为整数

TEX 提供尺寸与整数之间的转换：如果期望一个整数，则在该上下文中使用的⟨dimen⟩ 或⟨glue⟩ 会通过取其（自然）尺寸（以 scaled points 为单位）来转换。然而，只有⟨internal dimen⟩ 与⟨internal glue⟩ 可以这样使用：尺寸或胶的表示法不能被强制转换为整数。

7.2.6 尾随空格

TEX 的语法定义整数表示法（十进制、八进制与十六进制）以及‘反引号’字符记号后可跟⟨one optional space⟩。这意味着 TEX 会读取该数之后的记号：如果它是空格记号则吸收它，如果不是则回退。

由于 TEX 的输入处理器在看到一个空格记号后进入‘跳过空格’状态，这种扫描行为意味着整数表示法后面可以跟输入中任意多个空格字符。此外，行尾也是允许的。然而，只允许一个空格记号。

7.3 数的转换

TEX 可以执行从数字串到整数的隐式转换。从十进制、八进制或十六进制表示的转换已在上面讨论。反方向的转换，即从一个⟨internal integer⟩ 到打印表示，则必须显式执行。TEX 提供两个转换例程：`\number` 与 `\romannumeral`。当 `\number` 后接一个内部整数时，它等价于 `\the`。这些命令在 TEX 的展开处理器中执行，也就是说，只要展开未被抑制，它们就会被展开。

两个命令都产生一串类别码为 12 的记号;其参数是一个⟨number⟩。因此`\romannumeral51` 、`\romannumeral\year` 与 `\number\linepenalty` 都是有效的，`\number13` 也是。对表示法应用 `\number` 有一些用途：它去掉前导零以及多余的正负号。

罗马数字是一串小写‘罗马数字字符’，它们是类别码 12 的字符。序列

```
\uppercase\expandafter{\romannumeral ...}
```

给出大写罗马数字。这是因为 TEX 会展开记号以找到`\uppercase` 参数的开括号。如果 `\romannumeral` 应用于负数，结果干脆为空。

7.4 整数寄存器

整数可以存储在`\count` 寄存器中：

`\count`⟨8-bit number⟩

是一个⟨integer variable⟩，也是一个⟨internal integer⟩。作为整数变量，它可用于⟨variable assignment⟩：

⟨variable assignment⟩ ⟶ ⟨integer variable⟩⟨equals⟩⟨number⟩
| ...

作为内部整数，它可用作⟨number⟩：

⟨number⟩ → ⟨optional signs⟩⟨internal integer⟩ | ...

`\count` 寄存器的同义词可通过`\countdef` 命令以⟨shorthand definition⟩ 引入：

`\countdef`⟨control sequence⟩⟨equals⟩⟨8-bit number⟩

这样定义的控制序列称为⟨countdef token⟩，它充当一个⟨internal integer⟩。

Plain TEX 宏`\newcount`（声明为 `\outer`）使用 `\countdef` 命令分配一个未使用的 `\count` 寄存器。计数器 0–9 是临时寄存器，就像所有编号为 0–9 的寄存器一样。然而，计数器 0–9 在 `dvi` 文件中用于页面标识（见第 33 章），因此它们应仅在一个组内部作为临时寄存器使用。计数器 10–22 用于 plain TEX 的寄存器分配簿记。计数器 255 也是临时的。

7.5 算术

用户可以在 TEX 中执行一些算术，TEX 内部也执行算术。用户算术只涉及整数；内部算术主要是定点量，仅在胶设置时涉及浮点数。

7.5.1 算术语句

TEX 允许用户对整数执行一些算术。语句

\advance⟨integer variable⟩⟨optional by⟩⟨number⟩

把⟨number⟩ 的值 – 可以为负 – 加到⟨integer variable⟩。类似地，

\multiply⟨integer variable⟩⟨optional by⟩⟨number⟩

乘⟨integer variable⟩ 的值，而

\divide⟨integer variable⟩⟨optional by⟩⟨number⟩

除一个⟨integer variable⟩。

乘法与除法也可用于任何所谓⟨numeric variable⟩：其最一般形式为

\multiply⟨numeric variable⟩⟨optional by⟩⟨number⟩

其中

⟨numeric variable⟩ ⟶ ⟨integer variable⟩ | ⟨dimen variable⟩
| ⟨glue variable⟩ | ⟨muglue variable⟩

算术运算的结果绝对值不应超过2^{30}。

整数除法得出整数；也就是说，余数被丢弃。这就产生了一个问题：当任一操作数为负时如何舍入。在这种情况下，TEX 用操作数的绝对值做除法，并且如果恰好一个操作数为负，则取结果的负值。

7.5.2 浮点算术

在内部，对浮点量执行一些算术即胶设置比率的计算中。然而，舍入的机器相关方面不能影响 TEX 的决策过程，因此在这方面保证了 TEX 的机器无关性（舍入的足够精度由 [21] 的 Trip 测试强制执行）。

7.5.3 定点算术

TEX 中的所有小数运算都以 '缩放整数' 的定点算术执行：即 2^{-16} 的倍数。这保证了 TEX 的机器无关性。缩放整数的打印表示舍入到 5 位小数。

在 TEX 的普通 32 位实现中，最大整数绝对值为 $2^{31}-1$。用户不允许指定绝对值大于 $2^{30}-1$ 的尺寸：两个这样的尺寸可以在 32 位系统上相加或相减而不溢出。

7.6 数测试

TEX 中最一般的整数测试是

\ifnum⟨number_1⟩⟨relation⟩⟨number_2⟩

其中⟨relation⟩ 是 <、> 或 = 字符，均为类别 12。

区分奇数与偶数用

\ifodd⟨number⟩

数值情形语句由

\ifcase⟨number⟩⟨case_0⟩\or...\or⟨case_n⟩\else⟨other cases⟩\fi

提供，其中 \else 部分是可选的。如果该数为 i，则处理⟨case$_i$⟩的记号；其他情形被跳过，这与条件命令中通常发生的情况类似（见第 13 章）。

7.7 备注

7.7.1 字符常量

在格式与宏集合中经常需要数值常量。在 TEX 中有几种方式实现它们。

第一种，

```
\newcount\SomeConstant \SomeConstant=42
```

这很浪费，因为它占用一个 \count 寄存器。

第二种，

```
\def\SomeConstant{42}
```

更好但容易出意外：TEX 必须展开以找到该数 – 这本身就是一点开销 – ，并且可能无意中展开本应不动的记号。

第三种，

```
\chardef\SomeConstant=42
```

这一种很好。⟨chardef token⟩ 与 \count 寄存器地位相同：两者都是⟨internal integer⟩。因此这样定义的数可以在任何可使用 \count 寄存器的地方使用。对于大数，\chardef 可替换为 \mathchardef，其上限为"7FFF = 32 767。注意⟨mathchardef token⟩ 通常只能出现在数学模式中，但在数的上下文中它可以出现在任何地方。

7.7.2 展开过度/展开多少

写一段 TEX 代码时一个常见错误是：TEX 在试图组合一个数时无意中展开了某物。例如：

```
\def\par{\endgraf\penalty200}
...\par \number\pageno
```

这里页码会被吸收到 penalty 的值中。

现在考虑

```
\newcount\midpenalty \midpenalty=200
\def\par{\endgraf\penalty\midpenalty}
...\par \number\pageno
```

这里页码不会被错误地舀取：TEX 在 \penalty 之后寻找⟨number⟩，它找到一个⟨countdef token⟩。这不会被转换为数字表示，因此页码永远不会被触及。

可以在赋值之前先把⟨countdef token⟩ 转换为数字表示：

```
\penalty\number\midpenalty
```

而这又带回了之前所有的展开问题。

第 8 章

尺寸与胶

在 TEX 中，竖直和水平方向的空白可以通过“拉伸”或“收缩”来调整自身。这种可调整的空白称为“胶”（glue）。本章讨论与尺寸和胶相关的所有技术概念，并解释如何计算拉伸或收缩某一数值时的劣度（badness）。

`\dimen` 尺寸寄存器前缀。

`\dimendef` 定义一个控制序列，作为某个 `\dimen` 寄存器的同义词。

`\newdimen` 分配一个未使用的尺寸寄存器。

`\skip` skip 寄存器前缀。

`\skipdef` 定义一个控制序列，作为某个 `\skip` 寄存器的同义词。

`\newskip` 分配一个未使用的 skip 寄存器。

`\ifdim` 比较两个尺寸。

`\hskip` 在水平模式中插入一个胶项。

`\hfil` 等价于`\hskip 0cm plus 1fil`.

`\hfilneg` 等价于`\hskip 0cm minus 1fil`.

`\hfill` 等价于`\hskip 0cm plus 1fill`.

`\hss` 等价于`\hskip 0cm plus 1fil minus 1fil`.

`\vskip` 在竖直模式中插入一个胶项。

`\vfil` 等价于`\vskip 0cm plus 1fil`.

`\vfill` 等价于`\vskip 0cm plus 1fill`.

`\vfilneg` 等价于`\vskip 0cm minus 1fil`.

`\vss` 等价于`\vskip 0cm plus 1fil minus 1fil`.

`\kern` 向当前水平或竖直列表添加一个字距（kern）项。

`\lastkern` 若当前列表的最后一项是字距项，则给出其大小。

`\lastskip` 若当前列表的最后一项是胶项，则给出其大小。

`\unkern` 若当前列表的最后一项是字距项，则移除它。

`\unskip` 若当前列表的最后一项是胶项，则移除它。

`\removelastskip` 追加 `\lastskip` 之负值的宏。

`\advance` 对某个 ⟨ 数值变量 ⟩ 做加法或减法的算术命令。

`\multiply` 对某个 ⟨ 数值变量 ⟩ 做乘法的算术命令。

`\divide` 对某个 ⟨ 数值变量 ⟩ 做除法的算术命令。

8.1 ⟨glue⟩ 与⟨dimen⟩ 的定义

本节给出⟨dimen⟩ 与⟨glue⟩ 这两个量的语法。下一节将讨论胶的实际方面。

遗憾的是，胶的术语稍有歧义。语法量 ⟨glue⟩ 是一个尺寸（一段距离），可能带有拉伸和/或收缩分量。为了向列表中加入一团”胶”（即一段空白），必须让一个⟨glue⟩ 前面有诸如 `\vskip` 之类的命令。

8.1.1 尺寸的定义

⟨dimen⟩ 是 TEX 在需要表示一个尺寸时期望看到的东西；它可以是正的或负的。

⟨dimen⟩ ⟶ ⟨optional signs⟩⟨unsigned dimen⟩

⟨dimen⟩ 的无符号部分可以是

⟨unsigned dimen⟩ ⟶ ⟨normal dimen⟩ | ⟨coerced dimen⟩
⟨normal dimen⟩ ⟶ ⟨internal dimen⟩ | ⟨factor⟩⟨unit of measure⟩
⟨coerced dimen⟩ ⟶ ⟨internal glue⟩

也就是说，有以下三种情形：

- 一个⟨internal dimen⟩（内部尺寸）；它是 TEX 中任何具有⟨dimen⟩ 值的寄存器或参数：

  ⟨internal dimen⟩ ⟶ ⟨dimen parameter⟩
  | ⟨special dimen⟩ | `\lastkern`
  | ⟨dimendef token⟩ | `\dimen`⟨8-bit number⟩
  | `\fontdimen`⟨number⟩⟨font⟩
  | ⟨box dimension⟩⟨8-bit number⟩
  ⟨dimen parameter⟩ ⟶ `\boxmaxdepth`
  | `\delimitershortfall` | `\displayindent`
  | `\displaywidth` | `\hangindent`
  | `\hfuzz` | `\hoffset` | `\hsize`
  | `\lineskiplimit` | `\mathsurround`
  | `\maxdepth` | `\nulldelimiterspace`
  | `\overfullrule` | `\parindent`
  | `\predisplaysize` | `\scriptspace`
  | `\splitmaxdepth` | `\vfuzz`
  | `\voffset` | `\vsize`

- 一个尺寸记号（dimension denotation），由⟨factor⟩ 与⟨unit of measure⟩ 组成，例如 `0.7\vsize`；或
- 一个⟨internal glue⟩（内部胶，见下文），通过略去拉伸与收缩分量而被强制转换为尺寸，例如 `\parfillskip`。

尺寸记号是一个相当复杂的实体：

- ⟨factor⟩（因子）是一个整数记号、一个十进制常数记号（带整数部分和小数部分的数），或一个⟨internal integer⟩（内部整数）

  ⟨factor⟩ ⟶ ⟨normal integer⟩ | ⟨decimal constant⟩
  ⟨normal integer⟩ ⟶ ⟨integer denotation⟩
  | ⟨internal integer⟩
  ⟨decimal constant⟩ ⟶ .12 | ,12
  | ⟨digit⟩⟨decimal constant⟩
  | ⟨decimal constant⟩⟨digit⟩

  内部整数是一个“真正的”整数参数（例如 `\count0`），而不是从尺寸或胶强制转换来的。各类整数的定义见第 7 章。

- ⟨unit of measure⟩（度量单位）可以是一个⟨physical unit⟩（物理单位），即普通单位如 `cm` （前面可加 `true`）；一个内部单位如 `em`；也可以是一个⟨internal integer⟩（通过转换为缩放点）、一个⟨internal dimen⟩，或一个⟨internal glue⟩。

  ⟨unit of measure⟩ ⟶ ⟨optional spaces⟩⟨internal unit⟩
  | ⟨optional `true`⟩⟨physical unit⟩⟨one optional space⟩
  ⟨internal unit⟩ ⟶ `em`⟨one optional space⟩
  | `ex`⟨one optional space⟩ | ⟨internal integer⟩
  | ⟨internal dimen⟩ | ⟨internal glue⟩

有些⟨dimen⟩ 称为⟨special dimen⟩（特殊尺寸）：

⟨special dimen⟩ ⟶ `\prevdepth`
| `\pagegoal` | `\pagetotal` | `\pagestretch`
| `\pagefilstretch` | `\pagefillstretch`
| `\pagefilllstretch` | `\pageshrink` | `\pagedepth`

对其中任何一个的赋值称为⟨intimate assignment⟩（直接赋值），它自动是全局的（见第 10 章）。这些尺寸的含义在第 27 章中解释，但 `\prevdepth` 例外，它在第 15 章中讨论。

8.1.2 胶的定义

⟨glue⟩ 要么是某种形式的胶变量，要么是一个显式指明了拉伸与收缩的胶记号。具体地，

⟨glue⟩ ⟶ ⟨optional signs⟩⟨internal glue⟩ | ⟨dimen⟩⟨stretch⟩⟨shrink⟩
⟨internal glue⟩ ⟶ ⟨glue parameter⟩ | `\lastskip`
| ⟨skipdef token⟩ | `\skip`⟨8-bit number⟩
⟨glue parameter⟩ ⟶ `\abovedisplayshortskip`
| `\abovedisplayskip` | `\baselineskip`
| `\belowdisplayshortskip` | `\belowdisplayskip`
| `\leftskip` | `\lineskip` | `\parfillskip` | `\parskip`
| `\rightskip` | `\spaceskip` | `\splittopskip` | `\tabskip`
| `\topskip` | `\xspaceskip`

胶记号中的拉伸与收缩分量是可选的，但当两者都给出时，必须按顺序给出；它们定义为

⟨stretch⟩ ⟶ `plus` ⟨dimen⟩ | `plus`⟨fil dimen⟩ | ⟨optional spaces⟩
⟨shrink⟩ ⟶ `minus` ⟨dimen⟩ | `minus`⟨fil dimen⟩ | ⟨optional spaces⟩
⟨fil dimen⟩ ⟶ ⟨optional signs⟩⟨factor⟩⟨fil unit⟩⟨optional spaces⟩
⟨fil unit⟩ ⟶ | `fil` | `fill` | `filll`

⟨fil unit⟩ 的实际定义是递归的（见第 36 章），但上述是仅有的合法可能。

8.1.3 ⟨glue⟩ 到⟨dimen⟩ 的转换

语法规则

⟨dimen⟩ ⟶ ⟨factor⟩⟨unit of measure⟩

会带来一些值得注意的推论，因为⟨unit of measure⟩ 完全不必看起来像一个“度量单位”（见上面的列表）。

例如，由这个定义可知，语句

```
\dimen0=\lastpenalty\lastpenalty
```

在语法上是正确的，因为 \lastpenalty 既可以作为整数，也可以通过取其缩放点值而作为⟨unit of measure⟩。在\penalty8 之后，这样定义的 \dimen0 将具有 64sp 的大小。
更重要的是，考虑⟨unit of measure⟩ 为⟨internal glue⟩ 的情形，即任何一种胶参数。在这样一个胶前面加一个数（即⟨factor⟩）就使它成为一个合法的⟨dimen⟩ 规格。因此

```
\skip0=\skip1
```

与下面的语句非常不同

```
\skip0=1\skip1
```

第一条语句使 \skip0 等于 \skip1;第二条则在赋值前把 \skip1 转换为⟨dimen⟩。换言之，第二条语句定义的 \skip0 没有拉伸或收缩。

8.1.4 \dimen 与 \skip 的寄存器

TEX 有用于存放⟨dimen⟩ 与⟨glue⟩ 值的寄存器:即\dimen 与\skip 寄存器。它们可由以下表达式访问

\dimen⟨number⟩

以及

\skip⟨number⟩

与 TEX 的所有寄存器一样，这些寄存器编号为 0–255。
寄存器的同义词可用\dimendef 与\skipdef 命令来建立。其语法为

\dimendef⟨control sequence⟩⟨equals⟩⟨8-bit number⟩

以及

\skipdef⟨control sequence⟩⟨equals⟩⟨8-bit number⟩

例如,在\skipdef\foo=13 之后,使用 \foo 等价于使用 \skip13。
plain TEX 中提供了宏\newdimen 与\newskip 用于分配一个未使用的 dimen 或 skip 寄存器。在 plain 格式中，这些宏被定义为 \outer。

8.1.5 算术：加法

与整数变量一样，也提供了针对dimen、glue 与 muglue（数学胶；见第 169 页）变量的算术运算。
下面的表达式

\advance⟨dimen variable⟩⟨optional by⟩⟨dimen⟩
\advance⟨glue variable⟩⟨optional by⟩⟨glue⟩
\advance⟨muglue variable⟩⟨optional by⟩⟨muglue⟩

对一个 dimen、glue 或 muglue 的大小做加法。
用⟨glue⟩ 对⟨glue variable⟩ 做 advance，是通过把自然大小以及拉伸、收缩分量分别相加来实现的。由于 TEX 会在⟨glue⟩ 与⟨dimen⟩ 之间转换，因此可以写出例如

```
\advance\skip1 by \dimen1
```

或

```
\advance\dimen1 by \skip1
```

在第一种情形中,\dimen1 被强制转换为不带拉伸或收缩的⟨glue⟩;
在第二种情形中,\skip1 通过取其自然大小而被强制转换为⟨dimen⟩。

8.1.6 算术：乘法与除法

对胶（glue）与尺寸也提供了乘法与除法运算。例如可以写出

```
\multiply\skip1 by 2
```

它把 `\skip1` 的自然大小以及拉伸、收缩分量都乘以 2。

`\multiply` 或 `\divide` 运算的第二个操作数只能是⟨number⟩，即一个整数。这里引入⟨numeric variable⟩（数值变量）的概念：

⟨numeric variable⟩ ⟶ ⟨integer variable⟩ | ⟨dimen variable⟩
| ⟨glue variable⟩ | ⟨muglue variable⟩

这些运算的形式为

`\multiply`⟨numeric variable⟩⟨optional `by`⟩⟨number⟩

以及

`\divide`⟨numeric variable⟩⟨optional `by`⟩⟨number⟩

胶（glue）与尺寸（dimen）可以乘以非整数的量：

```
\skip1=2.5\skip2
\dimen1=.78\dimen2
```

但在第一行中，`\skip2` 首先通过略去其拉伸与收缩而被强制转换为⟨dimen⟩ 值。

8.2 再谈尺寸

8.2.1 度量单位

在 TEX 中，尺寸可以用

厘米（centimetre） 记作 `cm`；

毫米（millimetre） 记作 `mm`；这些是 SI 单位（*Système International d'Unités*，即国际标准计量单位制）。

英寸（inch） 记作 `in`；在英美地区更常用。1 英寸合 2.54 厘米。

皮卡（pica） 记作 `pc`；1 皮卡合 12 点。

点（point） 记作 `pt`；英美印刷业通用的单位制。1 英寸合 72.27 点。

迪多点（didot point） 记作 `dd`；欧洲大陆印刷业通用的单位制。此外，1157 迪多点合 1238 点。

西塞罗（cicero） 记作 `cc`；1 西塞罗合 12 迪多点。

大点（big point） 记作 `bp`；1 英寸合 72 大点。

缩放点（scaled point） 记作 `sp`；这是 TEX 中最小的单位，所有度量都是一个缩放点的整数倍。1 点合 65 536 个缩放点。

小数可以同时采用英美式的小数点写法（例如 `1in=72.27pt`）和欧洲大陆式的小数逗号写法：`1in=72,27pt`。

在内部，TEX 以最小尺寸——缩放点——的倍数工作。（绝对值）大于 $2^{30}-1$`sp` 的尺寸，即约 5.75 米或 18.9 英尺，是非法的。

皮卡制与迪多制都源于法国：1737 年，铸字商 Pierre Simon Fournier 引入了基于法国尺的排版点。虽然他最初引入的是一个基于线（line）与点的系统，但后来他以点为单位：一英寸有 72 个点，而一英尺为 12 英寸。约 1770 年，另一位奠基人 François Ambroise Didot 引入了基于更常见、且稍长的 'pied du roi'（国王尺）的点。

8.2.2 尺寸测试

尺寸与胶的自然大小可以用 `\ifdim` 测试来比较。其形式为

`\ifdim`⟨dimen$_1$⟩⟨relation⟩⟨dimen$_2$⟩

其中关系可以是 `>`、`<` 或 `=` 令牌，类别码均为 12。

8.2.3 已定义的尺寸

`\z@ 0pt`

`\maxdimen 16383.99999pt`；最大的合法尺寸。

这些⟨dimen⟩ 在 plain 格式中是预定义的；例如

```
\newdimen\z@ \z@=0pt
```

对常用尺寸使用这样的缩写至少有两个优点。首先，如果这样的尺寸出现在宏中，它节省主存：一个控制序列是一个记号，而像 `0pt` 这样的串要占三个。其次，它节省处理时间，因为 TEX 无需进行转换即可得到正确类型的对象。

像 `\z@` 这样的控制序列只对那些更改了“@”符号类别码的用户可用。通常，这些控制序列只出现在诸如 plain 格式之类的宏包所定义的宏中。

8.3 再谈胶

胶项可以用以下命令之一加入竖直列表：`\vskip`⟨glue⟩、`\vfil`、`\vfill`、`\vss` 或 `\vfilneg`；胶项可以用以下命令之一加入水平列表：`\hskip`⟨glue⟩、`\hfil`、`\hfill`、`\hss` 或 `\hfilneg`。下面讨论胶的性质。

8.3.1 拉伸与收缩

在上面给出的语法中，⟨glue⟩ 被定义为具有

- 一个“自然大小”，它是一个⟨dimen⟩，以及可选地
- 一个由⟨fil dimen⟩ 构成的“拉伸”与“收缩”分量。

TEX 构建的每个列表都有一定量的拉伸与收缩（可能为零），它们是列表中各段胶的拉伸与收缩分量之和。当列表所处的上下文要求它取一个不同于其自然大小的尺寸时，就会用到拉伸与收缩。当拉伸与收缩分量为有限值时——即当⟨fildimen⟩ 不是 `fil`(`l`(`l`)) 时——二者的行为有一个重要区别。有限的收缩量确实是 TEX 所能采用的最大收缩：形如

`5pt minus 3pt`

所指定的胶量可以收缩到 `2pt`，但不能再小。与之相反，有限的拉伸量可以拉伸到任意远。但这种任意拉伸具有很大的“劣度”。劣度的计算在下面讨论。

自然大小为 `20pt` 的序列

```
\hskip 10pt plus 2pt \hskip 10pt plus 3pt
```

有 `5pt` 的拉伸，但没有收缩。在

```
\hskip 10pt minus 2pt \hskip 10pt plus 3pt
```

中有 `3pt` 的拉伸和 `2pt` 的收缩，因此它的最小尺寸为 `18pt`。

正的收缩并不等于负的拉伸：

```
\hskip 10pt plus -2pt \hskip 10pt plus 3pt
```

看起来很像上一个例子，但它不能被收缩，因为其中没有`minus`⟨dimen⟩ 规格。不过它确实有 `1pt` 的拉伸。
这是负的收缩与拉伸量的另一个例子。不可能通过（在技术意义上）收缩胶来（在日常意义上）拉伸胶：
```
\hbox to 5cm{a\hskip 0cm minus -1fil}
```
是一个过松的盒子，因为 TEX 在需要拉伸内容时会寻找 `plus` ⟨dimen⟩ 规格。
最后，
```
\hskip 10pt plus -3pt \hskip 10pt plus 3pt
```
既不能拉伸也不能收缩。只有拉伸可用这一事实意味着该序列不能收缩。然而，拉伸分量相互抵消：总拉伸为零。另一种看法是，第二个胶项每拉伸一点，第一个胶项就会“往回拉伸”一点。

任何无限的拉伸或收缩都会压倒所有可用的有限拉伸或收缩：
```
\hbox to 5cm{\hskip 0cm plus 16384pt
             text\hskip 0cm plus 0.0001fil}
```
使 `text` 位于盒子的最左端。“无限”有三个阶，每一阶都比前一阶无限地更强：
```
\hbox to 5cm{\hskip 0cm plus 16384fil
             text\hskip 0cm plus 0.0001fill}
```
以及
```
\hbox to 5cm{\hskip 0cm plus 16384fill
             text\hskip 0cm plus 0.0001filll}
```
都使 `text` 位于盒子的左端。

8.3.2 胶的设置

在“胶的设置”过程中，一个盒子的期望宽度（或高度）会与其内容的自然尺寸相比较，后者是所有盒子与各段胶的自然尺寸之和。若二者不同，就用任何可用的可拉伸性或可收缩性来弥合差距。为达到盒子的期望尺寸，只设置可用最高阶的胶：该阶的每一段胶都按相同的比例被拉伸或收缩。
例如，在
```
\hbox to 6pt{\hskip 0pt plus 3pt \hskip 0pt plus 9pt}
```
盒子的自然大小为 `0pt`，总拉伸为 `12pt`。为得到一个 `6pt` 的盒子，每个胶项都按 1/2 的拉伸比来设置。因此结果等价于
```
\hbox {\hskip 1.5pt \hskip 4.5pt}
```
只使用最高阶的拉伸或收缩：in
```
\hbox to 6pt{\hskip 0pt plus 1fil \hskip 0pt plus 9pt}
```
第二个胶将取其自然大小 `0pt`，只有第一个胶会被拉伸。
TEX 绝不会超过有限收缩量的最大值。无法充分收缩的盒子称为“过满”（overfull）。在困难情形下，可以超出有限可拉伸性以提供出路；但 TEX 很可能会就此给出 `Underfull \hbox` 消息（见第 53 页）。无限收缩的例子见第 53 页。

8.3.3 劣度

在拉伸或收缩一个列表时，TEX 会计算基于该行中实际拉伸量与现有拉伸量之比的劣度（badness）。关于劣度在段落算法中的应用，见第 19 章。
一个被拉伸（或收缩）的列表的劣度公式为

$$b = \min\left(10\,000, 100 \times \left(\frac{\text{actual amount stretched (shrunk)}}{\text{possible amount of stretch (shrink)}}\right)^3\right)$$

实际上，TEX 使用的是一个稍有不同的公式，它更易计算但表现相同。由于胶设置是 TEX 的主要活动之一，这必须尽可能高效地执行。

如果胶设置比率（即上式中的分数）相当小，则该公式使劣度成为一个相当小的数；但一旦该比率超过 1 就会迅速增大。如果胶必须收缩超过规定量，劣度就是无穷大；而把胶拉伸超过其最大值是可能的，这就为非常难排的文本行或页面提供了一条出路。

在 TEX3 中，`\badness` 参数记录最近一次形成的盒子的劣度。

8.3.4 胶与断点

TEX 可以在若干种位置断行和分页。其中一种位置就是胶项之前。此时该胶会被丢弃。断行在第 19 章讨论，分页见第 27 章。

plain TEX 中有两个宏`\hglue` 与`\vglue`，分别在水平和竖直模式中给出不会消失的胶。对于水平情形，这是通过放置

```
\vrule width 0pt \nobreak \hskip ...
```

由于 TEX 在胶的前端断点，这个胶会始终附着在竖线上，因而永远不会消失。实际的宏定义要稍复杂一些，因为它们还负责保持 `\spacefactor` 与 `\prevdepth`。

8.3.5 `\kern`

`\kern` 命令在 TEX 当前所处的任何模式中指定一个字距项。字距项很像一个没有拉伸或收缩的胶项。它与胶的不同之处在于，它一般不是合法的断点。因此在

```
.. text .. \hbox{a}\kern0pt\hbox{b}
```

中，TEX 不会在盒子之间断行；而在

```
.. text .. \hbox{a}\hskip0pt\hbox{b}
```

中，可以在盒子之间断行。

然而，如果字距后面跟着胶，TEX 就可以在该字距处断点（前提是不在数学模式中）。在水平模式中，字距与胶都会在断点处消失。在竖直模式中，当断点之前的材料由输出例程处理完毕后，它们被移到（空的）当前页时被丢弃（见第 27 章）。

8.3.6 胶与模式

所有水平 skip 命令都是⟨horizontal command⟩，所有竖直 skip 命令都是⟨vertical command⟩。这意味着，例如，`\hskip` 命令若在竖直模式中给出，会使 TEX 开始一个段落。`\kern` 命令在两种模式中都可给出。

8.3.7 列表中的最后一个胶项：退格

列表中的最后一个胶项可以被测量，并且除外部竖直模式外的所有模式中都可移除它。内部变量`\lastskip` 与`\lastkern` 可用于在所有模式中测量最后一段胶；若最后一个胶相应地不是 skip 或 kern，它们给出 `0pt`。在数学模式中，`\lastskip` 起⟨internal muglue⟩ 的作用，但一般而言它归类为⟨internal glue⟩。若列表中最后一个胶项或字距项的大小本就是 `0pt`，则 `\lastskip` 与 `\lastkern` 也为 `0pt`。

运算 \unskip 与\unkern 移除列表的最后一项，前提是该项相应地为胶或字距。它们在外部竖直模式中无效；此时最好的替代是\vskip-\lastskip 与 \kern-\lastkern。
在段落构建过程中，TEX 自身会执行一次重要的 \unskip：以一个空行结尾的段落，其末尾会被 TEX 的输入处理器插入一个空格记号。这个空格在插入 \parfillskip 胶（见第 17 章）之前由一次 \unskip 移除。
胶被 TEX 视为引线（leader）的一个特例，当对引线施加 \unskip 时这一点就很明显：它们会被移除。

8.3.8 退格示例

plain TEX 宏\removelastskip 定义为 as
```
\ifdim\lastskip=0pt \else \vskip-\lastskip \fi
```
若列表的最后一项是胶，该宏就会按其值退格，前提是其自然大小不为零。在所有其他情形中，不会向列表添加任何东西。
有时需要 \vskip 这类命令的“智能”版本，即两个相继的 skip 命令应只产生两者中较大的那个胶量。在第 135 页就用到了这样一个宏：
```
\newskip\tempskipa
\def\vspace#1{\tempskipa=#1\relax
    \ifvmode \ifdim\tempskipa<\lastskip
             \else \vskip-\lastskip \vskip\tempskipa
             \fi
    \else \vskip\tempskipa \fi}
```
首先，它测试当前模式是否为竖直；若不是，就可以安全地放置该参数。把参数复制到一个 skip 寄存器是必要的，因为 \vspace{2pt plus 3pt} 会在\ifdim#1 <\lastskip 测试中引发问题。
若周围模式是竖直的，则参数只有在不小于已存在的量时才应放置。如果测试写成下面这样，该宏就是不正确的
```
        \ifdim\tempskipa>\lastskip
            \vskip-\lastskip \vskip\tempskipa
        \fi
```
在这种情况下，序列
```
... last word.\par \vspace{0pt plus 1fil}
```
不会放置任何胶，因为在 \par 之后我们处于竖直模式，而 \lastskip 的值为 0pt。

8.3.9 跟踪输出中的胶

如果通过把 \tracingoutput 设为正数来跟踪 TEX 的工作，或者 TEX 把一个盒子写入日志文件（由于 \showbox 命令，或因为它过满或过松），胶就用控制序列 \glue 表示。这不是一个 TEX 命令；它只表示当前列表中存在胶。
TEX 由（例如 \showbox）生成的盒子表示，会在每个显式 \kern 之后插入一个空格，但在由字体 tfm 文件中的字距信息插入的隐式字距之后不插入空格。因此\kern 2.0pt 表示用户或宏插入的字距，而\kern2.0pt 表示一个隐式字距。
自动插入的胶（\topskip、\baselineskip 等）在 TEX 的跟踪输出中按名称表示。例如，盒子
```
\vbox{\hbox{Vo}\hbox{b}}
```

形如

```
\vbox(18.83331+0.0)x11.66669
.\hbox(6.83331+0.0)x11.66669
..\tenrm V
..\kern-0.83334
..\tenrm o
.\glue(\baselineskip) 5.05556
.\hbox(6.94444+0.0)x5.55557
..\tenrm b
```

注意 'V' 与 'o' 之间插入的隐式字距。

第 9 章

线段与引线

线段与引线是让 TEX 画出一条线的两种方式。引线比线段更通用：它们还能用某个盒子的副本填充可用空间。本章解释线段与引线如何工作，以及它们如何与模式相互作用。

`\hrule` 沿水平方向延伸的线段。

`\vrule` 沿竖直方向延伸的线段。

`\leaders` 用一条线段或盒子的若干副本填充指定大小的空间。

`\cleaders` 类似`\leaders`，但对于盒子引线，多余的空间在引线前后均分。

`\xleaders` 类似`\leaders`，但对于盒子引线，多余的空间在盒子之前、之后以及盒子之间均分。

9.1 线段

TEX 的线段命令给出带有水平边与竖直边的矩形黑色色块。多数时候，线段命令给出的输出看起来像一条线段，但■ 也可以由线段产生。

TEX 同时有水平线段和竖直线段，但这些名称并不一定意味着形状。不过，它们确实暗示了模式：`\hrule` 命令只能在竖直模式中使用，而`\vrule` 只能在水平模式中使用。事实上，`\hrule` 是⟨vertical command⟩，而 `\vrule` 是⟨horizontal command⟩，因此 TEX 在遇到这些命令时可能会切换模式。

那么为什么 `\vrule` 称为竖直线段呢？原因在于 `\vrule` 可以在竖直方向上任意延伸：如果没有显式指定其高度和深度，它就会占据周围环境所允许的那么大空间。

```
\hbox{\vrule\ text \vrule}
```

形如

| text |

以及

```
\hbox{\vrule\ A gogo! \vrule}
```

形如

| A gogo! |

对于 `\hrule` 命令，类似的说法也成立：水平线段可以延伸以取其周围环境的宽度。因此

```
\vbox{\hbox{One line of text}\hrule}
```

形如

One line of text

9.1.1 线段的尺寸

水平线段与竖直线段有一个默认粗细：

`\hrule` 等价于 `\hrule height.4pt depth0pt`

以及

`\vrule` 等价于 `\vrule width.4pt`

而如果其余那个尺寸未指定，线段就会沿该方向延伸以填满外层盒子。

下面是如何指定线段尺寸的形式化说明：

⟨vertical rule⟩ ⟶ `\vrule`⟨rule specification⟩
⟨horizontal rule⟩ ⟶ `\hrule`⟨rule specification⟩
⟨rule specification⟩ ⟶ ⟨optional spaces⟩
| ⟨rule dimensions⟩⟨rule specification⟩
⟨rule dimension⟩ ⟶ `width`⟨dimen⟩ | `height`⟨dimen⟩ | `depth`⟨dimen⟩

如果一个线段尺寸被指定两次，第二次优先于第一次。这使得覆盖默认尺寸成为可能。例如，在

```
\let\xhrule\hrule  \def\hrule{\xhrule height .8pt}
```

之后，宏 `\hrule` 给出的水平线段高度是原来的两倍，并且仍可以

```
\hrule height 2pt
```

来指定其他高度。

可以同时指定全部三个尺寸；此时

```
\vrule height1ex depth0pt width1ex
```

以及

```
\hrule height1ex depth0pt width1ex
```

看起来相同。尽管如此，它们各自只能在相应的模式中使用。

9.2 引线

线段与模式紧密相关，这使得很容易获得某些效果。例如，竖直线段的一个典型应用看起来像

```
\hbox{\vrule width1pt\ Important text! \vrule width 1pt}
```

它给出

| Important text! |

然而，人们可能想在水平模式中得到一条水平线段，以实现诸如下面这样的效果

⟵ 5cm ⟶
from here_____to there

`\hrule` 不能在水平模式中使用，而竖直线段不会自动延伸。

不过，确实有一种办法可以在水平模式中使用 `\hrule` 命令、在竖直模式中使用 `\vrule`，那就是使用“引线”（leader），之所以如此称呼，是因为它们引导你的目光横过页面。引线命令告诉 TEX 以所需数量的某个盒子或线段规格副本，填充指定的空间，无论它处于何种模式。例如，上面的例子写成

```
\hbox to 5cm{from here\leaders\hrule\hfil to there}
```

也就是说，用一个允许沿着 `\hfil` 拉伸的 `\hrule`。注意，该引线被赋予了一个水平 skip，与它出现时所处的水平模式相对应。

一般的引线命令看起来像

⟨leaders⟩⟨box or rule⟩⟨vertical/horizontal/mathematical skip⟩

其中⟨leaders⟩ 是 `\leaders`、`\cleaders` 或 `\xleaders`；⟨box or rule⟩ 是一个 ⟨box⟩、`\vrule` 或 `\hrule`；而水平与竖直 skip 的列表见第 6 章；数学 skip 要么是水平 skip，要么是 `\mskip`（见第 169 页）。因此引线可在三种模式中使用。当然，必须指定合适种类的 skip。
含引线的水平（竖直）盒子至少具有引线中所用⟨box or rule⟩ 的高度与深度（宽度），即使在盒子引线的情形中可能实际并没有放置任何引线。

9.2.1 线段引线

线段引线用一条沿所指定 skip 的方向延伸的线段填充指定大小的空间。所得线段引线的其他尺寸由所用线段的种类决定：各尺寸既可以显式指定，也可以使用默认值。
例如，

```
\hbox{g\leaders\hrule\hskip20pt f}
```

给出

g____f

因为水平线段的默认高度为.4pt。另一方面，

```
\hbox{g\leaders\vrule\hskip20pt f}
```

给出

g[redacted]f

因为竖直线段的高度与深度默认会填满外层盒子。
多余的线段尺寸会被忽略：在水平模式中

```
\leaders\hrule width 10pt \hskip 20pt
```

等价于

```
\leaders\hrule \hskip 20pt
```

如果 skip 或盒子的宽度或“高度加深度”为负值，TEX 就改用普通胶而非引线。

9.2.2 盒子引线

盒子引线用某个给定盒子的副本填充可用空间，而不是用线段。
对于以下所有例子，假设已经分配了一个盒子寄存器：

```
\newbox\centerdot  \setbox\centerdot=\hbox{\hskip.7em.\hskip.7em}
```

现在，以下内容的输出

```
\hbox to 8cm {here\leaders\copy\centerdot\hfil there}
```

得到

here . . . . . there

也就是说，该盒子寄存器的副本填满了可用空间。
像上面例子那样的点引线常用于目录。在这类应用中，希望后续各行上的点在竖直方向对齐。`\leaders` 命令会自动做到这一点：

```
\hbox to 8cm {here\leaders\copy\centerdot\hfil there}
\hbox to 8cm {over here\leaders\copy\centerdot\hfil over there}
```

给出

here . . . . . there
over here . . . . . over there

其背后的机制如下：TEX 表现得像有无限多个盒子排成一行，（不可见地）从外层盒子的左边缘开始，而实际放置的那一行副本只不过是这一行中未被盒子其他内容遮挡的部分。
换一种说法，盒子引线是望向无限长一行盒子的窗口，而这一行从外层盒子的左边缘开始。请看下面的例子：

```
\hbox to 8cm {\leaders\copy\centerdot\hfil}
\hbox to 8cm {word\leaders\copy\centerdot\hfil}
```
它给出

. . . . . .

word . . . . . .

引线盒子这一行一旦不与其他材料重合，就变得可见。
上面只讨论了水平模式中的引线。引线同样可以放在竖直模式中；对于盒子引线，这个“无限行”便从外层盒子的顶部开始。

9.2.3 均匀间距的引线

按上述方式对齐后续盒子引线，意味着引线前后的空白一般是不同的。如果竖直对齐不是问题，那么让引线均匀分布在审美上可能更悦目。`\cleaders` 命令类似 `\leaders`，区别在于它把引线前后的多余空间平分为两等份，使这一行盒子在可用空间中居中。

```
\hbox to 7.8cm {here\cleaders\copy\centerdot\hfil there}
\hbox to 7.8cm {here is\cleaders\copy\centerdot\hfil there}
```
给出

here . . . . . there

here is . . . . . there

“扩展引线”`\xleaders` 把多余空间均匀地分布在各盒子之间，在引线盒子之前、之后以及之间放置等量的胶。

```
\hbox to 7.8cm{here\hskip.7em
      \xleaders\copy\centerdot\hfil  \hskip.7em there}
```
给出

here . . . . . there

注意，这里引线盒子中的胶通过引线前后的显式胶达到平衡；略去这些胶项，如

```
\hbox to 7.8cm {here\xleaders\copy\centerdot\hfil there}
```
给出

here . . . . . there

这显然不是想要的结果。

9.3 杂注

9.3.1 线段与模式

上面解释了线段为何只能出现在相应的模式中。线段还会影响与模式相关的量：在竖直模式中，线段之前不会添加 baselineskip。为防止线段之后出现胶，TEX 把 `\prevdepth` 设为`-1000pt`（见第 15 章）。类似地，在水平模式中 `\vrule` 之后 `\spacefactor` 被设为 1000（见第 19 章）。

9.3.2 用引线结束段落

试图在段落末尾用下面的方式模拟一条 `\hrule`

```
\nobreak\leaders\hrule\hfill\par
```
并不奏效。原因在于 TEX 会在段落末尾执行一次 `\unskip`，从而移除引线。通常这个 `\unskip` 会移除输入处理器在最后一行之后插入的任何空格记号。补救办法：在引线末尾插入一个`\hbox{}`。

9.3.3 引线与盒子寄存器

上面的例子中，引线盒子是用 \copy 插入的。以下内容的输出

```
\hbox to 8cm {here\leaders\box\centerdot\hfil there}
\hbox to 8cm {over here\leaders\box\centerdot\hfil
                  over there}
```

得到

here . . . . . there
over here over there

第一个引线命令之后该盒子寄存器被清空，但在那第一个命令中就放置了不止一个副本。

9.3.4 引线盒子中的输出

出现在引线盒子中的任何 \write、\openout 或 \closeout 操作都会被忽略。否则这样的操作会对每个将被送出的盒子副本都执行一次。

9.3.5 跟踪输出中的盒子引线

从（例如）\tracingoutput 得到的转储盒子表示不会完整写出盒子引线：只转储总尺寸以及所用盒子的一个副本。特别是，引线前后周围的空白不予标出。

9.3.6 引线与偏移的边距

如果边距被偏移过，引线的外观可能有所不同，取决于偏移是如何实现的。为说明 \hangindent 与 \leftskip 如何影响引线的外观，请看下面的例子，其中

```
\setbox0=\hbox{K o }
```

引线上方的水平盒子用于指示引线这一行的起始位置。

首先

```
\hbox{\leaders\copy0\hskip5cm}
\noindent\advance\leftskip 1em
      \leaders\copy0\hskip5cm\hbox{}\par
```

给出

K o K o K o K o K o K o K o
K o K o K o K o K o K o

然后

```
\hbox{\kern1em\hbox{\leaders\copy0\hskip5cm}}
\hangindent=1em \hangafter=-1 \noindent
      \leaders\copy0\hskip5cm\hbox{}\par
```

给出（注意与上一例相比的偏移）

K o K o K o K o K o K o K o
K o K o K o K o K o K o K o

在第一段中，\leftskip 胶只遮挡了第一个引线盒子；在第二段中，悬挂缩进实际上移动了引线这一行的定位点。悬挂缩进在 TEX 中是通过对包含段落各行的盒子做 \moveright 来实现的。

第 10 章

分组

TEX 有一套分组机制，能够把大多数变化限制在某个局部范围内。本章解释哪些操作可以是局部的，以及分组如何形成。

\bgroup 隐式的分组开始字符。

\egroup 隐式的分组结束字符。

\begingroup 打开一个必须用 \endgroup 关闭的分组。

\endgroup 关闭一个用 \begingroup 打开的分组。

\aftergroup 保存下一个记号，待当前分组结束后插入。

\global 使赋值、宏定义与算术运算成为全局的。

\globaldefs 用于覆盖 \global 前缀的参数。IniTEX 默认值：0。

10.1 分组机制

分组是这样一个记号序列：它以一个“分组开始”记号开始，以一个“分组结束”记号结束，且其中所有这类记号都恰当地配对。TEX 的分组机制与普通编程语言的块结构并不相同。大多数带块结构的语言只能有局部定义。TEX 的分组机制更强：在分组内部做出的大多数赋值都只在该分组内有效，除非显式指明其他，而在分组之外旧值会被恢复。

局部定义的一个例子

```
{\def\a{b}}\a
```

会给出“未定义控制序列”消息，因为 \a 只在该分组内部定义。类似地，下面的代码

```
\count0=1 {\count0=2 } \showthe\count0
```

将显示值 1；在分组内部做出的赋值会在分组结束时被撤销。

对要在分组之外恢复的值的记录，是通过“保存栈”机制完成的。保存栈的溢出在第 35 章讨论。保存栈还用于其他一些场合：在诸如\hbox to 100pt{...} 这样的调用中，规格to 100pt 会被放入保存栈，然后才打开新的一层分组。

为避免保存栈出现诸多问题，IniTEX 不允许在分组内部转储格式。\end 命令可以出现在分组内部，但 TEX 会就此给出一条诊断消息。

\aftergroup 控制序列保存一个记号，待当前分组结束后插入。这个命令可以留出若干个记号，它们按书写时的从左到右顺序被插入。这在第 12 章讨论。

10.2 局部与全局赋值

赋值或宏定义通常通过在其前加\global 而成为全局的，但⟨integer parameter⟩ \globaldefs 的非零值会覆盖 \global 指定：若 \globaldefs 为正，每次赋值都隐式地带 \global 前缀；若 \globaldefs 为负，则 \global 被忽略。通常这个参数为零。

有些赋值总是全局的：⟨global assignment⟩ 包括

⟨**font assignment**⟩ 涉及 \fontdimen、\hyphenchar 与 \skewchar 的赋值。

⟨**hyphenation assignment**⟩ \hyphenation 与 \patterns 命令（见第 19 章）。

⟨**box size assignment**⟩ 用 \ht、\dp 与 \wd 改变盒子尺寸（见第 5 章）。

⟨**interaction mode assignment**⟩ TEX 作业的运行模式（见第 32 章）。

⟨**intimate assignment**⟩ 对⟨special integer⟩ 或⟨special dimen⟩ 的赋值；见第 67 与 73 页。

10.3 分组定界符

分组可以用类别码为 1 的记号（表示“分组开始”）和类别码为 2 的记号（表示“分组结束”）来定界，也可以用通过 \let 等同于这些字符的控制序列记号，即 plain TEX 中的 \bgroup 与 \egroup。隐式与显式花括号可以配对以定界一个分组。

分组也可以用\begingroup 与\endgroup 来定界。这两个控制序列必须配对使用：它们不能与隐式或显式花括号配对，也不能充当诸如环绕盒子材料的花括号。

用 \begingroup 与 \endgroup 定界可以提供一种有限形式的运行时错误检查。在这两个分组定界符之间，多余的开或闭花括号会导致

```
\begingroup ... } ... \endgroup
```

或

```
\begingroup ... { ... \endgroup
```

在这两种情形中，TEX 都会给出配对不当的错误消息。此处若用 \bgroup 与 \egroup，则由于会发生错误配对，错误将更难查找。若干格式的环境宏就用到了这一想法。

分组开始与分组结束所用花括号字符的选择并不是硬编码在 TEX 中的。在 plain 格式中它是这样安排的：

```
\catcode`\{=1 % left brace is begin-group character
\catcode`\}=2 % right brace is end-group character
```

隐式花括号也已在 plain 格式中定义：

```
\let\bgroup={ \let\egroup=}
```

特殊情形如下：

- 宏的替换文本必须用显式的分组开始与分组结束字符记号括起来。
- 盒子、\vadjust 与 \insert 的开、闭花括号可以是隐式的。这使得可以定义，例如

```
\def\openbox#1{\setbox#1=\hbox\bgroup}
\def\closebox#1{\egroup\box#1}
\openbox{15}Foo bar\closebox{15}
```

- 记号列表赋值的右侧，以及命令 `\write`、`\message`、`\errmessage`、`\uppercase`、`\lowercase`、`\special` 与 `\mark` 的参数，是一个⟨general text⟩，其定义为
  ⟨general text⟩ ⟶ ⟨filler⟩`{`⟨balanced text⟩⟨right brace⟩
  也就是说，左花括号可以是隐式的，但闭合的右花括号必须是类别码为 2 的显式字符记号。

在隐式左花括号即可满足要求、且展开未被显式抑制的情况下，TEX 会展开令牌，直到遇到左花括号为止。这正是诸如`\uppercase\expandafter{\romannumeral80}`之类结构的基础，它们在未展开的形式下并不符合语法。如果第一个不可展开的令牌不是左花括号，TEX 会给出错误信息。

TEX 的语法（见第 36 章）用⟨left brace⟩ 与⟨right brace⟩ 表示显式字符，即字符记号；而用 `{` 与 `}` 表示可能为隐式的字符，即通过 `\let` 等同于这些显式字符的控制序列。

10.4 再谈花括号

10.4.1 花括号计数器

TEX 有两个用于记录分组层级的计数器：即*主计数器*与*平衡计数器*。这两个计数器都是语法计数器：它们统计显式花括号字符记号，而不受隐式花括号（如 `\bgroup`）影响，尽管后者在语义上等价于一个显式花括号。

平衡计数器在除对齐之外的所有情形中处理花括号。其工作方式直观易懂：每遇到一个未被跳过的开花括号就加一，每遇到一个未被跳过的闭花括号就减一。因此

```
\iffalse{\fi
```

会使平衡计数器加一，如果该语句只是被扫描（例如出现在宏定义文本中）；如果该语句被执行，花括号就被跳过，因而对平衡计数器没有影响。

主计数器更微妙一些；在对齐中用它代替平衡计数器。这个计数器记录所有花括号，即使它们像`\iffalse{\fi` 中那样被跳过。对这个计数器而言，仍可能有不被计数的被跳过花括号：字母常量 `` `{ `` 与 `` `} `` 在被执行处理器用作⟨number⟩ 时对该计数器没有影响；而当它们被输入处理器看到（输入处理器只看到字符，看不到上下文）时，确实会影响该计数器。

10.4.2 作为记号的花括号

显式花括号是字符记号，因此它们不可展开。这意味着它们会一直存活到 TEX 处理的最后阶段。例如，

```
\count255=1{2}
```

会把 1 赋给 `\count255`，并打印 ‘2’，因为开花括号充当了数 1 的分隔符。类似地，

```
f{f}
```

会阻止 TEX 形成 ‘ff’ 连字。

由花括号不可展开这一事实，可以推出：它们的嵌套与条件语句的嵌套相互独立。例如

```
\iftrue{\else}\fi
```

会给出一个开花括号，因为条件语句是通过展开处理的。闭花括号只是作为⟨false text⟩ 的一部分被跳过；它对分组的任何影响要到 TEX 处理的后续阶段才发挥作用。

未定界的宏参数要么是单个记号，要么是用显式花括号括起来的记号组。因此显式的开或闭花括号不可能成为宏参数。然而，花括号可以用 `\let` 赋值，例如

```
\let\bgroup={
```

这在 plain `\footnote` 宏中使用（见第 106 页）。

10.4.3 开与闭花括号控制符

控制序列`\{` 与`\}` 其实并不属于本章，因为它们与分组无关。它们已用 `\let` 分别定义为 `\lbrace` 与 `\rbrace` 的同义词，而这两个控制序列是 `\delimiter` 指令（见第 21 章）。

Computer Modern Roman 字体没有花括号，但打字机字体中有花括号；而对于数学，扩展字体中有不同大小的花括号 – 以及可伸展的花括号 – 。

第 11 章

宏

宏是 TEX 对需要多次使用的命令序列的缩写机制，有点像普通编程语言中的过程。但 TEX 的参数机制相当不寻常。本章解释 TEX 宏如何工作。它还讨论 `\let` 与 `\futurelet` 命令。

`\def` 开始一个宏定义。

`\gdef` `\global\def` 的同义词。

`\edef` 开始一个宏定义；替换文本在定义时展开。该命令也在下一章讨论。

`\xdef` `\global\edef` 的同义词。

`\csname` 开始构造一个控制序列的名称。

`\endcsname` 停止构造一个控制序列的名称。

`\global` 使下一个定义、算术语句或赋值成为全局的。

`\outer` 前缀，表示正在定义的宏只能在“外层”级别使用。

`\long` 前缀，表示正在定义的宏的参数可以包含 `\par` 记号。

`\let` 定义一个控制序列，使其等价于下一个记号。

`\futurelet` 定义一个控制序列，使其等价于下下个记号。

11.1 引言

宏基本上是一串记号，它已被缩写成一个控制序列。以 `\def`（以及其他命令）开头的语句称为*宏定义*，而写出

```
\def\abc{\de f\g}
```

定义了宏 `\abc`，其*替换文本*为`\de f\g`。宏可以这样用来缩写需要多次给出的文本片段或命令序列。每当 TEX 的展开处理器遇到控制序列 `\abc`，就用替换文本替换它。

如果宏应当对其使用的上下文敏感，就可以带参数定义它：

```
\def\PickTwo#1#2{(#1,#2)}
```

取两个参数并用括号括起重现它们。调用 `\PickTwo 12` 得到‘(1,2)’。

用替换文本替换宏的这一行为称为*宏展开*。

11.2 宏定义的结构

一个宏定义依次由以下部分组成：

1. 任意数量的 `\global`、`\long` 与 `\outer` 前缀，
2. 一个⟨def⟩ 控制序列，或任何通过 `\let` 等同于它的东西，
3. 待定义的控制序列或活动字符，

4. 可能有一个⟨parameter text⟩，它除其他外还指定宏有多少个参数，以及
5. 一个用类别码为 1 与 2 的显式字符记号括起的替换文本，在 plain TEX 中默认为 { 与 }。

宏定义完成后，任何保存的 \afterassignment 记号（见第 12.3.3 节）都会被插入。

“展开式”定义 \edef 与 \xdef 在第 12 章讨论。

11.3 前缀

有三个前缀会改变宏定义的状态：

\global 如果定义出现在分组内部，此前缀使该定义成为全局的。此前缀也可用于宏定义之外的赋值；事实上，对于宏定义已有缩写，无需使用 \global：

\gdef\foo... 等价于 \global\def\foo...

；以及

\xdef\foo... 等价于 \global\edef\foo...

如果参数 \globaldefs 为正，则所有赋值都隐式地是全局的；如果 \globaldefs 为负，则任何 \global 前缀都被忽略，而 \gdef 与 \xdef 做出局部定义（见第 10 章）。

\outer “外层”宏的机制旨在便于（在其他错误中）定位不配对的花括号：\outer 宏应当只出现在非嵌套的上下文中。确切地说，它不允许出现在

- 宏替换文本中（但它可以出现在例如 \edef 中，位于 \noexpand 之后，以及 \meaning 之后），
- 参数文本中，
- 被跳过的条件文本中，
- 对齐前言中，以及
- \message、\write 等的⟨balanced text⟩ 中。

但对于某些应用，plain 宏中有一些是 outer 会带来不便，尤其是 \newskip 这类宏。一个补救办法是重新定义它们、去掉“outer”选项，例如 LATEX 就这么做；但也可以有更巧妙的技巧。

\long 通常，宏参数不应包含\par 记号。这个限制在定位被遗忘的闭花括号时很有用（比 \outer 定义有用得多）。例如，TEX 会对“失控的参数”抱怨 在下面的序列中：

```
\def\a#1{ ... #1 ... }
\a {This sentence should be in braces.

And this is not supposed to be part of the argument
```

空行生成一个 \par，这通常意味着遗忘了某个闭花括号。如果某个宏的参数应当允许包含 \par 记号，则该宏必须声明为 \long。

判断记号相等性的 \ifx 测试（见第 13 章）在测试两个记号是否有相同定义时，会把前缀考虑在内。

11.4 定义类型

TEX 中有四个⟨def⟩ 控制序列：\def、\gdef、\edef 与\xdef。控制序列 \gdef 是\global\def 的同义词，\xdef 是\global\edef

的同义词。“展开式定义”`\edef` 在第 12 章讨论。
各种宏定义类型之间的区别只在定义时才有意义。调用宏时无法判断它当初是如何定义的。

11.5 参数文本

在待定义的控制序列或活动字符与替换文本的开花括号之间，可以出现⟨parameter text⟩。它指定宏是否有参数、有多少个，以及它们如何定界。⟨parameter text⟩ 不能包含显式花括号。
一个宏最多可有九个参数。参数由参数记号（parameter token）指示，该记号由一个宏参数字符（即类别码为 6 的字符，在 plain TEX 中为 `#`）后接数字 1–9 构成。例如，`#6` 表示宏的第六个参数。参数记号不能出现在宏定义上下文之外。
在参数文本中，参数必须从 1 开始连续编号。参数记号之后的空格是有意义的，无论在参数文本中还是在替换文本中。
参数可以是定界的或未定界的。如果一个参数在⟨parameter text⟩中后接的是另一个参数，或后接替换文本的开花括号，它就称为未定界的；如果后接任何其他记号，它就称为定界的。
宏被展开（或“调用”时）替换某一参数的那些记号（零个或多个）称为对应该参数的“自变量”。

11.5.1 未定界参数

当一个带未定界参数的宏，例如一个带一个参数的宏 `\foo`

```
\def\foo#1{ ... #1 ...}
```

被展开时，TEX 会向前扫描（不做展开）直到找到一个非空白记号。如果该记号不是显式的⟨left brace⟩，它就被取作对应该参数的自变量。否则就通过扫描吸收一段⟨balanced text⟩，直到匹配的显式⟨right brace⟩。这段 balanced text 便构成该自变量。
下面是一个带三个未定界参数的例子：对于

```
\def\foo#1#2#3{#1(#2)#3}
```

宏调用 `\foo123` 给出 ‘1(2)3’；而`\foo 1 2 3` 也给出同样结果。在调用

```
\foo 1 2 3
```

中，第一个空格在 TEX 的输入处理器中被跳过。对应第一个参数的自变量于是为 `1`。为找到第二个参数，TEX 接着跳过所有空白，此处恰好一个。作为第二个参数，TEX 接着找到 `2`。类似地，第三个参数为 `3`。
为把若干记号作为一个未定界自变量传递，可以使用花括号。对于上面 `\foo` 的定义，调用`\foo a{bc}d` 给出 ‘a(bc)d’。当宏的自变量是一段 balanced text 而不是单个记号时，定界花括号不会在该自变量被插入替换文本时插入。例如：

```
\def\foo#1{\count0=1#1\relax}
\foo{23}
```

将展开为`\count0=123\relax` ，它把值 123 赋给该计数器。另一方面，the statement

```
\count0=1{23}
```

会赋 1 并打印 23。

11.5.2 定界参数

除了用花括号括起来之外，还有另一种方式把一串记号作为单个自变量传给宏，即使用定界参数。

⟨parameter text⟩ 中出现在某个宏参数之后的任何非参数记号（即参数字符之后的参数编号之后）都充当该参数的定界符。这包括空格记号：参数编号之后的空格是有意义的。定界记号也可以出现在待定义的控制序列与第一个参数记号 `#1` 之间。

在参数文本中充当定界符的字符记号会同时保存其字符码与类别码；实际自变量中的定界字符记号必须两者都匹配。这类字符的类别码可能包括一些通常只能出现在特殊上下文中的；例如，在定义

```
\def\foo#1_#2^{...}
```

之后，宏 `\foo` 可以在数学模式之外使用。

在寻找与定界参数对应的自变量时，TEX 吸收所有记号而不展开（但会配对花括号），直到遇到定界记号（的确切序列）为止。定界记号不是自变量的一部分；它们在宏调用期间从输入流中移除。

11.5.3 定界参数示例

举一个简单的例子，

```
\def\DoASentence#1#2.{{#1#2.}}
```

定义了一个宏，其第一个参数未定界，第二个参数以句点定界。在调用

```
\DoASentence \bf This sentence is the argument.
```

中，自变量为：

```
#1<-\bf
#2<-This sentence is the argument
```

注意，结尾的句点不在自变量中，但它已被吸收；它不再在输入流中。

一个常用的定界符是 `\par`：

```
\def\section#1. #2\par{\medskip\noindent {\bf#1. #2\par}}
```

这个宏的第一个参数以 '. ' 定界，第二个参数以 `\par` 定界。调用

```
\section 2.5. Some title

The text of the section...
```

将给出

```
#1<-2.5
#2<-Some title
```

注意，第二个自变量末尾有一个由行尾产生的空格。如果不想要这个空格，可以定义

```
\def\section#1. #2 \par{...}
```

用 `\par` 定界第二个自变量。但这种做法使用户无法显式写出 `\par`：

```
\section 2.5 Some title\par
```

摆脱这一两难的一个办法是：在定义文本中所有不希望出现尾随空格的地方都写上 `#2\unskip`。

充当定界符的控制序列无需已定义，因为它们被吸收时不做展开。因此

```
\def\control#1\sequence{...}
```

是一个有用的定义，即使 \sequence 未定义。
定界自变量中类别码的重要性由下面的例子说明：

```
\def\a#1 #2.{ ... }
\catcode`\ =12
\a b c
d.
```

它给出

```
\a #1 #2.-> ...
#1<- b c
#2<-d
```

解释：参数 1 与 2 之间的定界符是一个类别为 10 的空格。在 a 与 b 之间有一个类别为 12 的空格；第一个类别为 10 的空格就是由行尾产生的那个空格。
关于类别码匹配的“真实”应用，见第 13 章中对 \newif 的解释，以及第 28 页的例子。

11.5.4 空自变量

当 TEX 期待一个宏自变量时，如果用户在花括号中给出一段⟨balanced text⟩，该文本就用作自变量。因此，给出 {} 会得到一个由空记号列表构成的自变量；这称为“空自变量”。
空自变量也可因使用定界参数而产生。例如，在定义

```
\def\mac#1\ro{ ... }
```

调用

```
\mac\ro
```

将得到一个空自变量。

11.5.5 宏参数字符

当 TEX 的输入处理器扫描一段宏定义文本时，它会为每一处宏参数字符后接数字的出现插入一个参数记号。实际上，替换文本中的一个参数记号表示” 在此处插入第某某号参数”。连续两个参数字符会被替换为一个。
后一事实可用于嵌套宏定义。因此

```
\def\a{\def\b#1{...}}
```

会给出错误消息，因为 \a 定义时没有参数，但其替换文本中却有一个参数记号。
下面这个

```
\def\a#1{\def\b#1{...}}
```

定义了一个宏 \a，它又定义一个宏 \b。但 \b 仍然没有任何参数：调用

```
\a z
```

会定义一个不带参数的宏 \b，其后必须跟一个 z。注意，这并不会试图定义宏 \bz，因为在读取那一行输入时，控制序列 \b 早已在 TEX 的输入处理器中形成。
最后，

```
\def\a{\def\b##1{...}}
```

定义了一个带一个参数的宏 \b。
让我们较详细地考察参数字符的处理。考虑

```
\def\a#1{ .. #1 .. \def\b##1{ ... }}
```

当它作为输入被读取时，输入处理器

- 把字符 #1 替换为⟨parameter token$_1$⟩，并

- 把字符 ## 替换为 #

随后 \a 的一次宏调用会让输入处理器扫描

```
\def\b#1{ ... }
```

其中两个字符 #1 被替换为一个参数记号。

11.5.6 花括号定界

通常，定义的⟨parameter text⟩ 中不能出现左或右花括号。但有一个特殊机制，可以使宏的最后一个参数表现得像由一个开花括号定界。

如果最后一个参数记号后接一个参数字符（#），而它又后接替换文本的开花括号，TEX 就使最后一个参数由一个分组开始字符定界。此外，与参数文本中的其他定界记号不同，这个开花括号不会从输入流中移除。

请看一个例子。假设我们想要一个宏 \every，它能如下填充记号列表：

```
\every par{abc} \every display{def}
```

这个宏可以定义为

```
\def\every#1#{\csname every#1\endcsname}
```

在上面第一个调用中，对应该参数的自变量是 abc，因此该调用展开为

```
\csname everypar\endcsname{abc}
```

它给出期望的结果。

11.6 控制序列的构造

命令\csname 与\endcsname 可用于构造一个控制序列。例如

```
\csname hskip\endcsname 5pt
```

等价于\hskip5pt .

在这个构造过程中，\csname 与 \endcsname 之间的所有宏和其他可展开控制序列都会照常展开，直到只剩下不可展开的字符记号。上面例子的一个变体，

```
\csname \ifhmode h\else v\fi skip\endcsname 5pt
```

会根据模式执行 \hskip 或 \vskip。展开的最终结果应当只由字符记号构成，但它们的类别码无关紧要。此处一个不可展开的控制序列会给出错误：TEX 会在它正前方插入一个 \endcsname 作为错误恢复的尝试。

用 \csname 可以构造通常无法写出的控制序列，因为组成它的字符记号可能具有类别而非 11（字母）。这一原理可用于把宏包的内部控制序列对用户隐藏。

```
\def\newcounter#1{\expandafter\newcount
    \csname #1:counter\endcsname}
\def\stepcounter#1{\expandafter\advance
    \csname #1:counter\endcsname 1\relax}
```

在第二个定义中，\expandafter 是多余的，但它无害，且在概念上更清晰。

由 \newcounter 创建的实际计数器的名称含有一个冒号，因此写出这个控制序列需要费些周折。实际上，该计数器现在对用户隐藏，用户只能通过 \stepcounter 之类的控制序列访问它。顺便一提，宏 \newcount 在 plain 格式中被定义为 \outer，因此

上面 \newcounter 的定义只能在 \newcount 被重新定义之后写出。
如果用\csname...\endcsname 形成的控制序列此前尚未定义，其含义就被设为 \relax。因此如果\xx 是一个未定义的控制序列，命令

```
\csname xx\endcsname
```

不会给出错误消息，因为它等价于\relax 。此外，在执行完\csname...\endcsname 语句之后，控制序列\xx 本身就等价于 \relax，因此它不再会给出“未定义控制序列”错误（另见第 108 页）。

11.7 通过 \let 与 \futurelet 进行记号赋值

TEX 中有两种⟨let assignment⟩。其语法为

\let⟨control sequence⟩⟨equals⟩⟨one optional space⟩⟨token⟩
\futurelet⟨control sequence⟩⟨token⟩⟨token⟩

在 \futurelet 赋值的语法中不出现可选的等号。

11.7.1 \let

原语命令\let 把某个记号的当前含义赋给一个控制序列或活动字符。
例如，在 plain 格式中 \endgraf 定义为

```
\let\endgraf=\par
```

这使宏编写者可以重新定义 \par，而仍能使用原语 \par 命令的功能。例如，

```
\everypar={\bgroup\it\def\par{\endgraf\egroup}}
```

待赋值的⟨token⟩ 不是控制序列、而是字符记号的情形，已在第 3 章讨论。

11.7.2 \futurelet

如前所述，带 \let 的序列

\let⟨control sequence⟩⟨token$_1$⟩⟨token$_2$⟩⟨token$_3$⟩⟨token$\cdots$⟩

把⟨token$_1$⟩（的含义）赋给该控制序列，而剩余输入流看起来像

⟨token$_2$⟩⟨token$_3$⟩⟨token$\cdots$⟩

也就是说，⟨token$_1$⟩ 已从输入流中消失。
命令\futurelet 的工作方式略有不同：给定输入流

\futurelet⟨control sequence⟩⟨token$_1$⟩⟨token$_2$⟩⟨token$_3$⟩⟨token$\cdots$⟩

它把⟨token$_2$⟩（的含义）赋给该控制序列，而剩余输入流看起来像

⟨token$_1$⟩⟨token$_2$⟩⟨token$_3$⟩⟨token$\cdots$⟩

也就是说，⟨token$_1$⟩ 与⟨token$_2$⟩ 都没有被从输入流中取出。然而现在⟨token$_1$⟩ “知道”⟨token$_2$⟩ 是什么，而不必把它作为宏参数吸收。见下面的例子。
如果一个字符记号已通过 \futurelet 赋给某个控制序列，其类别码就固定了。随后的⟨token$_1$⟩ 不能再改变它。

11.8 杂注

11.8.1 活动字符

类别为 13 的字符记号，即“活动字符”，可以像控制序列一样定义。如果该字符的定义出现在某个宏内部，则在定义那个宏时，该字符必须是活动的。

例如，考虑下面的定义（取自第 2 章）：

```
{\catcode`\^^M=13 %
 \gdef\obeylines{\catcode`\^^M=13 \def^^M{\par}}%
}
```

^^M 字符这种不寻常的类别必须在定义 \obeylines 时设置，否则 TEX 会认为那一行在 \def 之后就结束了。

11.8.2 宏与原语

原语命令与用户宏之间的区别在 TEX 中远不如在其他编程语言中那么重要。

- 用户可以用不同的名字使用原语命令：
  `\let\StopThisParagraph=\par`
- 原语命令的名字可用于用户宏：
  `\def\par{\hfill$\bullet$\endgraf}`
- 用户宏与若干个 TEX 原语都会被展开，例如所有条件语句，以及 \number、 \jobname 这类命令。

11.8.3 尾递归

TEX 中的宏，就像大多数现代编程语言中的过程一样，允许递归：即宏的定义中可以包含对同一个宏的调用，或对另一个会调用本宏的宏的调用。如果这种宏的太多“化身”同时处于活动状态，递归宏往往会塞满 TEX 的内存。但 TEX 能够在一种常见的递归情形中防止这一点：尾递归。

为理解这里发生的事情，需要一些背景知识。当 TEX 开始执行一个宏时，它吸收参数，并在输入栈上放置一个指向替换文本的项，以便扫描器接下来被导向该替换。一旦它被处理完，输入栈上的该项就可以移除。但如果宏的定义文本中还含有其他宏，这个过程会对它们重复：即使第一个宏尚未完成，也可能有新的项被放到输入栈上，引导扫描器转向其他宏。

一般而言，这种“栈堆积”是一种必要的恶，但如果嵌套宏调用是原宏替换文本中的最后一个记号，就可以避免。最后一个记号之后无需再考虑其他记号，因此不妨在放入新项之前把输入栈顶项清除。TEX 正是这样做的。

plain TEX 的\loop 宏很好地示例了 这一原理。其定义为

```
\def\loop#1\repeat{\def\body{#1}\iterate}
\def\iterate{\body \let\next=\iterate
    \else \let\next=\relax\fi \next}
```

这个宏例如可以如下调用：

```
\loop \message{\number\MyCount}
    \advance\MyCount by 1
    \ifnum\MyCount<100 \repeat
```

宏 \iterate 可以自我调用；当它这样做时，递归调用是由列表中的最后一个记号执行的。本可以把 \iterate 定义为

```
\def\iterate{\body \iterate\fi}
```

但那样 TEX 就无法解析该递归,因为调用 \iterate 不是 \iterate 替换文本中的最后一个记号。此处赋值\let\next=\iterate 就是让递归调用成为列表中最后一个记号的一种办法。
解析尾递归的另一种办法是使用 \expandafter（见第 121 页）：在
```
\def\iterate{\body \expandafter\iterate\fi}
```
中它移除 \fi 记号。如果列表中的最后若干个记号是递归宏的自变量，尾递归同样会被解析。
附带一提：把 \iterate 定义为
```
\def\iterate{\let\next\relax
    \body \let\next\iterate \fi \next}
```
就可以写出
```
\loop ... \if... ... \else ... \repeat
```

11.9 宏技巧

11.9.1 数目未知的自变量

在某些应用中，需要一个可以拥有数目未预先指定的自变量的宏。考虑把国际象棋棋盘上的一个位置翻译出来的问题（完整的宏与字体见 [37] 与 [47]），给定如
```
\White(Ke1,Qd1,Na1,e2,f4)
```
把它转换成一排排版指令
```
\WhitePiece{K}{e1} \WhitePiece{Q}{d1} \WhitePiece{N}{a1}
\WhitePiece{P}{e2} \WhitePiece{P}{f4}
```
注意，对于兵，位置列表中省略了 'P'。
第一个问题是棋子列表长度可变，因此我们追加一个终止棋子：
```
\def\White(#1){\xWhite#1,xxx,}
\def\endpiece{xxx}
```
我们可以对它进行测试。接着，宏 \xWhite 从列表中取出一个位置，测试它是否为终止符；若不是，就对它做一次测试，看它是否为兵。
```
\def\xWhite#1,{\def\temp{#1}%
    \ifx\temp\endpiece
    \else \WhitePieceOrPawn#1XY%
          \expandafter\xWhite
    \fi}
```
需要 \expandafter 命令来移除 \fi（见第 121 页），以便 \xWhite 拿到下一个位置作为自变量，而不是 \fi。
位置长度为两个或三个字符。对四参数宏 \WhitePieceOrPawn 的调用追加了一个终止串 XY。因此在兵的情形中，自变量 3 是字符 X，自变量 4 为空；对其他所有棋子，自变量 1 是棋子，2 与 3 是位置，自变量 4 是 X。
```
\def\WhitePieceOrPawn#1#2#3#4Y{
    \if#3X \WhitePiece{P}{#1#2}%
    \else  \WhitePiece{#1}{#2#3}\fi}
```

11.9.2 检查自变量

有时可能需要测试某个宏的自变量是否包含某个元素。举一个实际的例子,考虑下面的情况（另见第 177 页上的 \DisplayEquation 例子）。

假设一篇文章的标题与作者给定为

```
\title{An angle trisector}
\author{A.B. Cee\footnote*{Research supported by the
Very Big Company of America}}
```

多位作者时给定为

```
\author{A.B. Cee\footnote*{Supported by NSF grant 1}
        \and
        X.Y. Zee\footnote{**}{Supported by NATO grant 2}}
```

进一步假设宏 \title 与 \author 定义为

```
\def\title#1{\def\TheTitle{#1}}  \def\author#1{\def\TheAuthor{#1}}
```

它们将如下使用

```
\def\ArticleHeading{ ... \TheTitle ... \TheAuthor ... }
```

对某些期刊，要求文章的作者名与标题全部大写。其实现可以是

```
\def\ArticleCapitalHeading
   { ...
    \uppercase\expandafter{\TheTitle}
     ...
    \uppercase\expandafter{\TheAuthor}
     ...
   }
```

现在 \expandafter 命令会把标题与作者名展开成实际文本，而 \uppercase 命令会把它们转为大写。但对作者名而言这是错的，因为 \uppercase 命令也会把脚注文本转为大写。于是问题在于：只把标题中位于脚注之间的部分转为大写。

作为第一次尝试，我们取只有一位作者的情形，并令基本调用为

```
\expandafter\UCnoFootnote\TheAuthor
```

这展开为

```
\UCnoFootnote A.B. Cee\footnote*{Supported ... }
```

这个宏

```
\def\UCnoFootnote#1\footnote#2#3{\uppercase{#1}\footnote{#2}{#3}}
```

会正确地分析它：

```
#1<-A.B. Cee
#2<-*
#3<-Supported ...
```

但如果没有脚注，这个宏就完全错了。

作为第一次改进，我们自己加一个脚注，只是为了确保存在一个：

```
\expandafter\UCnoFootnote\TheAuthor\footnote 00
```

现在我们必须测试找到的是哪一种脚注：

```
\def\stopper{0}
\def\UCnoFootnote#1\footnote#2#3{\uppercase{#1}\def\tester{#2}%
    \ifx\stopper\tester
    \else\footnote{#2}{#3}\fi}
```

我们用 \ifx 把定界脚注符号与实际遇到的符号比较。注意，下面这种解法

```
\ifx0#2
```

在脚注符号由多个记号构成时是错的，例如 {**}。

到目前为止，该宏在没有脚注时是正确的，但若有脚注，它就错了：终止记号仍有待处理。下面的版本处理了它们：

```
\def\stopper{0}
\def\UCnoFootnote#1\footnote#2#3{\uppercase{#1}\def\tester{#2}%
```

```
    \ifx\stopper\tester
    \else\footnote{#2}{#3}\expandafter\UCnoFootnote
    \fi}
```

重复调用 \UCnoFootnote 会移除定界记号（\expandafter 先移除 \fi），而且作为额外收获，这个宏对多位作者也是正确的。

11.9.3 用 \futurelet 实现可选宏参数

\futurelet 的一个标准应用是实现宏的可选参数。一般做法如下：

```
\def\Com{\futurelet\testchar\MaybeOptArgCom}
\def\MaybeOptArgCom{\ifx[\testchar \let\next\OptArgCom
                  \else \let\next\NoOptArgCom \fi \next}
\def\OptArgCom[#1]#2{ ... }\def\NoOptArgCom#1{ ... }
```

注意，虽然 \ifx 测试的是字符，这里仍用了它。原因当然是：如果省略了可选自变量，\Com 后面可能有一个可展开的控制序列。宏 \Com 现在有一个可选自变量和一个常规自变量；它可以调用为

```
\Com{argument}
```

或调用为

```
\Com[optional]{argument}
```

通常，不带可选自变量的调用会插入某个默认值：

```
    \def\NoOptArgCom#1{\OptArgCom[default]{#1}}
```

这种机制在 LaTeX 与 LAMSTeX 等格式中被广泛使用；另见 [49]。

11.9.4 两步宏

在用户看来像是一个宏的东西，实际上往往是一个两步过程：一个宏设置条件，第二个宏完成工作。

例如，下面是一个宏 \PickToEol 其自变量由行尾定界。首先我们写一个不带参数的宏，它改变行尾的类别码，然后调用第二个宏。

```
\def\PickToEol{\begingroup\catcode`\^^M=12 \xPickToEol}
```

于是第二个宏可以把直到行尾的一切作为自变量：

```
\def\xPickToEol#1^^M{ ... #1 ... \endgroup}
```

这个定义有一个问题：^^M 字符应具有类别 12。于是我们得到下面的写法：

```
\def\PickToEol{\begingroup\catcode`\^^M=12 \xPickToEol}
{\catcode`\^^M=12 %
 \gdef\xPickToEol#1^^M{ ... #1 ... \endgroup}%
}
```

其中 ^^M 的类别码是为了定义 \xPickToEol 而改变的。注意，\PickToEol 中的 ^^M 出现在一个控制符号中，因此那里的类别码无关紧要。所以那个定义可以放在重新定义 ^^M 类别码的分组之外。

11.9.5 注释环境

作为上面两步宏思想的一个应用，并为了说明尾递归，下面是用于“注释”环境的宏。

常常需要临时去掉 TeX 输入的一部分。为此会想写出

```
\comment
...
\endcomment
```

最简单的实现

```
\def\comment#1\endcomment{}
```

有不少缺点。例如，它无法处理 outer 宏，也无法处理花括号不平衡的输入。但其最严重的缺点在于：它把整段注释文本当作宏自变量读取。这把注释的大小限制在 TEX 输入缓冲区的大小。更好的做法是每次一行地处理被注释掉的文本。为此我们想写出一个基本结构如下的递归宏

```
\def\comment#1^^M{ ... \comment }
```

为了能够写出这个定义，必须改变行尾的类别码；如上所述我们会写出

```
\def\comment{\begingroup \catcode`\^^M=12 \xcomment}
{\catcode`\^^M=12 \endlinechar=-1 %
 \gdef\xcomment#1^^M{ ... \xcomment}
}
```

改变 `\endlinechar` 仅仅是为了避免在定义的每一行末尾都加注释字符。

当然，这个过程必须在某个时候停止。为此，我们考察被舀取为宏自变量的那一行：

```
{\catcode`\^^M=12 \endlinechar=-1 %
 \gdef\xcomment#1^^M{\def\test{#1}
    \ifx\test\endcomment \let\next=\endgroup
    \else \let\next=\xcomment \fi
    \next}
}
```

并且我们必须定义 `\endcomment`：

```
\def\endcomment{\endcomment}
```

这个命令永远不会被执行：它只是为了测试是否已到达该环境的末尾。

我们可能想把语法上不正确的文本注释掉。因此在注释时切换到 verbatim 模式下面这个宏见于 plain TEX：

```
\def\dospecials{\do\ \do\\\do\{\do\}\do\$\do\&%
  \do\#\do\^\do\^^K\do\_\do\^^A\do\%\do\~}
```

我们用它如下定义 `\comment`：

```
\def\makeinnocent#1{\catcode`#1=12 }
\def\comment{\begingroup
    \let\do=\makeinnocent \dospecials
    \endlinechar`\^^M \catcode`\^^M=12 \xcomment}
```

除了上面提到的可以注释掉语法上不正确的文本（例如因花括号不匹配）之外，这个解决方案还能处理 outer 宏。前一种 `\xcomment` 实现若在注释文本中遇到错误，就会引发 TEX 错误。然而，使用 verbatim 模式带来了如何结束该环境的问题。注释的最后一行现在不再是控制序列 `\endcomment`，而是构成它的那些字符。于是我们必须测试这些字符：

```
{\escapechar=-1
 \xdef\endcomment{\string\\endcomment}
}
```

序列`\string\\` 给出一个反斜杠。我们本不能使用

```
\edef\endcomment{\string\endcomment}
```
因为那样词 `endcomment` 的字母会具有类别码 12，而不是注释最后一行上那些字母所具有的 11。

第 12 章

展开

TEX 中的展开与大多数编程语言中的过程调用颇为不同。本章讨论与展开相关的命令，并给出若干（非平凡的）例子。

`\relax` 什么也不做。

`\expandafter` 取下两个记号，把第二个的展开结果放到第一个之后。

`\noexpand` 不展开下一个记号。

`\edef` 开始一个宏定义；替换文本在定义时被展开。

`\aftergroup` 保存下一个记号，以便在当前分组结束后插入。

`\afterassignment` 保存下一个记号，以便在下一次赋值或宏定义之后执行。

`\the` 把 TEX 中各种量的值展开成一串字符记号。

12.1 引言

TEX 的展开处理器接受从输入处理器出来的记号流，其结果又是一串记号，它把这串记号喂给执行处理器。对输入处理器而言有两类记号：可展开的与不可展开的。后一类原样传递，主要包含赋值与可排版材料；前一类被展开，展开的结果再重新被检查。

12.2 普通展开

下面列出那些会被展开的构造，除非展开被抑制：

- 宏
- 条件语句
- `\number`, `\romannumeral`
- `\string`, `\fontname`, `\jobname`, `\meaning`, `\the`
- `\csname ... \endcsname`
- `\expandafter`, `\noexpand`
- `\topmark`, `\botmark`, `\firstmark`, `\splitfirstmark`, `\splitbotmark`
- `\input`, `\endinput`

下面是展开被抑制的所有情形：

- 当 TEX 正在读取要由……定义的记号时
  - 一个⟨let assignment⟩，即由 `\let` 或 `\futurelet` 定义的
  - 一个⟨shorthand definition⟩，即由 `\chardef` 或 `\mathchardef` 定义的，或⟨register def⟩，即 `\countdef`、`\dimendef`、`\skipdef`、`\muskipdef` 或 `\toksdef`，

  - 一个⟨definition⟩,即宏定义用 `\def`、`\gdef`、`\edef` 或 `\xdef` 的，
  - ⟨simple assignment⟩ 中的 `\read` 与 `\font`；
- 当正在读取⟨parameter text⟩ 或宏自变量时；也当正在读取由 `\def`、`\gdef` 或 `\read` 定义的控制序列的替换文本时；
- 当正在读取⟨token variable⟩ 或 `\uppercase`、`\lowercase`、`\write` 的记号列表时；但 `\write` 的记号列表会在输出页面时展开；
- 当错误恢复期间正在删除记号时；
- 当正在跳过条件语句的一部分时；
- 在两种 TEX 必须知道其后跟什么的情形中
  - 在一个用于表示整数的上下文中的左引号之后（因此在`\catcode` `\a` 中 `\a` 不展开），或
  - 在开启数学模式的数学移位字符之后以查看是否跟有另一个数学移位字符（若是，则开启一个行间显示）；
- 当正在扫描对齐 preamble 时；但在这种情形下，由 `\span` 前置的记号以及 `\tabskip` 赋值中的记号仍会展开。

12.3 颠倒展开顺序

有时你需要改变记号展开的正常顺序。TEX 为此提供了若干机制。本节中的某些控制序列并非严格与展开相关。

12.3.1 一步展开：`\expandafter`

颠倒展开顺序最直接的工具是`\expandafter`。序列

`\expandafter`⟨token$_1$⟩⟨token$_2$⟩

展开为

⟨token$_1$⟩⟨*token$_2$ 的展开*⟩

注意以下几点。

- 如果⟨token$_2$⟩ 是宏，它被替换为其替换文本，而不是其最终展开结果。因此，如果

  ```
  \def\tokentwo{\ifsomecondition this \else that \fi}
  \def\tokenone#1{ ... }
  ```

  the call

  ```
  \expandafter\tokenone\tokentwo
  ```

  会把 `\ifsomecondition` 作为参数交给 `\tokenone`：

  ```
  \tokenone #1-> ...
  #1<-\ifsomecondition
  ```
- 如果 `\tokentwo` 是一个带一个或多个参数的宏，将吸收足够多的后续令牌以形成替换文本。

12.3.2 完全展开：`\edef`

宏通常用 `\def` 定义，但对于希望替换文本反映当前条件（而非调用时刻的条件）的情形，有一个” 展开式定义”`\edef`，它在把替换文本赋给控制序列之前，将其中的所有内容全部展开。

```
\edef\modedef{This macro was defined in
    `\ifvmode vertical\else \ifmmode math
     \else horizontal\fi\fi' mode}
```

模式测试将在定义时刻执行，因此替换文本将是一个单一字符串。

作为一个更有用的例子，假设在一个将要被 \input 的文件中， @ 的类别码会被改变。于是可以写出
\edef\restorecat{\catcode`@=\the\catcode`@}
在开头，以及
\restorecat
在末尾。完整示例见第 113 页。

与 \expandafter 的“一步展开”不同，\edef 内部的展开是完全的：它一直进行，直到只剩下不可展开的字符与控制序列记号。这种完全展开有两个例外：

- 任何由 \noexpand 前置的控制序列不被展开；以及
- 如果 \sometokenlist 是一个记号列表，表达式
  \the\sometokenlist
  会展开为该列表的内容，但这些内容不再进一步展开（例子见第 14 章）。

在某些场合，\edef 可以方便地被“借用”：此时并不想定义一个控制序列，而只关心展开的结果。例如，对于下面的定义
```
\def\othermacro{\ifnum1>0 {this}\else {that}\fi}
\def\somemacro#1{ ... }
```
the call
```
\expandafter\somemacro\othermacro
```
给出参数赋值
```
#1<-\ifnum
```
可以通过调用下面的写法来修正
```
\edef\next{\noexpand\somemacro\othermacro}\next
```
条件语句在 \edef 内部被完全展开，因此 \next 的替换文本将由序列
```
\somemacro{this}
```
构成；随后调用 \next 即执行该语句。

12.3.3 \afterassignment

命令取一个记号，把它留存起来，以便在下一次赋值或宏定义之后插入记号流。如果第一次赋值是把一个盒子赋给盒子寄存器，该记号会被插入在该盒子的开花括号之后（见第 54 页）。

用这种方式只能保存一个记号；随后由 \afterassignment 保存的记号会覆盖前一个。

让我们看一个使用 \afterassignment 的例子。常常希望有这样一个宏，它会

- 把自变量赋给某个变量，然后
- 基于该变量的新值做一点计算。

下面的例子展示了直接做法：
```
\def\setfontsize#1{\thefontsize=#1pt\relax
    \baselineskip=1.2\thefontsize\relax}
\setfontsize{10}
```
用 \afterassignment 可以得到更优雅的解法：
```
\def\setbaselineskip
   {\baselineskip=1.2\thefontsize\relax}
\def\fontsize{\afterassignment\setbaselineskip
    \thefontsize}
\fontsize=10pt
```

现在这个宏看起来像一次赋值：等号甚至是可选的。实际上它的展开以一个待赋值的变量结尾。控制序列 \setbaselineskip 被留存，以便在对 \thefontsize 的赋值之后执行。
plain TEX 中 \afterassignment 的例子有 \magnification 与 \hglue 宏。该命令另一个创造性应用见 [31]。

12.3.4 \aftergroup

可以用

\aftergroup⟨token⟩

命令留存若干记号，以便在当前分组结束后插入。这些记号按 \aftergroup 命令给出的顺序在分组结束后插入。分组可以由隐式或显式花括号定界，也可以由 \begingroup 与 \endgroup 定界。

```
{\aftergroup\a \aftergroup\b}
```

等价于

```
\a \b
```

这个命令有很多应用。一个例子见第 113 页的 \textvcenter 宏；另一个例子由 plain TEX 的脚注机制提供。
plain TEX 的脚注命令形式为

\footnote⟨footnote symbol⟩{⟨footnote text⟩}

，看起来像一个带两个自变量的宏。但不应把脚注文本“舀”进来，因为这会禁止例如脚注内部的类别码改变。
plain 脚注宏中发生的事情（从宏观看）如下。

- \footnote 命令打开一个 insert，
  ```
  \def\footnote#1{ ...#1... %treat the footnote sign
      \insert\footins\bgroup
  ```
- 在 insert 盒子中打开一个分组，并给出一个 \aftergroup 命令，以正确地收尾该 insert：
  ```
  \bgroup\aftergroup\@foot
  ```
  这个命令用于在用户键入、用以结束脚注文本的闭花括号之后收尾；用户脚注文本的开花括号必须由下面这行移除
  ```
  \let\next=}%end of definition \footnote
  ```
  它把下一个记号（即那个花括号）赋给 \next。
- 脚注文本在这个 insert 盒子中作为普通文本排版。
- 脚注之后，将执行由下面定义的命令 \@foot
  ```
  \def\@foot{\strut\egroup}
  ```
  。

12.4 阻止展开

有时需要在通常会发生展开的地方阻止展开。为此可用控制序列\string 与\noexpand。
\string 的用法相当有限，因为它把一个控制序列记号转换成一串字符，并用 \escapechar 的值作为类别码为 0 的那个字符。它特别适合用在 \write 中，用以输出控制序列名（另见第 30 章）；另一应用见第 13 章中对 \newif 的解释。
由 \string 产生的所有字符都具有类别码 12（other），空格字符除外；它们取得码 10。另见第 3 章。

12.4.1 \noexpand

命令 \noexpand 是可展开的，它的展开结果就是紧随其后的那个记号。该记号的含义被临时设为等于 \relax，因而它不能再被进一步展开。

\noexpand 最重要的应用大概是在 \edef 命令中（但在 write 语句中它常可替代 \string）。例如考虑

```
\edef\one{\def\noexpand\two{\the\prevdepth}}
```

若没有 \noexpand，TEX 会试图展开 \two，从而给出“未定义控制序列”错误。

下面这个（相当无意义的）例子说明：\noexpand 有效地把紧随其后的记号变成一个 \relax：

```
\def\a{b}
\noexpand\a
```

这不会产生任何输出，因为 \noexpand 的效果是把控制序列 \a 临时设为等于 \relax。

12.4.2 \noexpand 与活动字符

组合 \noexpand⟨token⟩ 等价于 \relax，即使该记号是一个活动字符。因此，

```
\csname\noexpand~\endcsname
```

不会与 \char`\~ 相同。相反，它会给出错误消息，因为不可展开命令 – 例如 \relax – 不允许出现在 \csname 与 \endcsname 之间。解决办法是改用 \string；例子见第 113 页。

但在另一种上下文中，序列 \noexpand⟨active character⟩ 等价于该字符，但处于不可展开形式。这发生在使用条件语句 \if 与 \ifcat 时（其解释见第 13 章）。比较

```
\if\noexpand~\relax % is false
```

其中测试的是波浪号的字符码；而

```
\def\a{ ... } \if\noexpand\a\relax % is true
```

中测试的是两个控制序列。

12.5 \relax

控制序列\relax 不可展开，但它被执行时什么也不发生。

这句话听起来有点矛盾，因此看一个例子。给定计数器

```
\newcount\MyCount
\newcount\MyOtherCount \MyOtherCount=2
```

。在赋值

```
\MyCount=1\number\MyOtherCount3\relax4
```

中，命令 \number 是可展开的，而 \relax 不可。当 TEX 构造待赋值的数字时，它会展开所有命令，直到找到一个非数字，或遇到一个不可展开命令为止。于是它读取 1；展开序列\number\MyOtherCount ，得到 2；读取 3；看到 \relax，由于它不可展开便停下。待赋值的数字于是为 123，整个调用被展开为

```
\MyCount=123\relax4
```

由于 \relax 记号在执行时没有效果，这一行的结果是 123 被赋给 \MyCount，而数字 4 被打印出来。

\relax 可用于标示命令结束的另一个例子 是

```
\everypar{\hskip 0cm plus 1fil }
\indent Later that day, ...
```
这会被误解：TEX 会把它看成
```
\hskip 0cm plus 1fil L
```
而fil L 是一个合法的、写 fill 的（古怪）方式（见第 36 章）。
一种补救办法是写出
```
\everypar{\hskip 0cm plus 1fil\relax}
```

12.5.1 \relax 与 \csname

如果一个\csname ... \endcsname 命令构成了某个先前未定义的控制序列的名字，则该控制序列被置为与 \relax 相等，且整个语句也等价于 \relax （另见第 95 页）。
然而，这个 \relax 的赋值是仅局部的：
```
{\xdef\test{\expandafter\noexpand\csname xx\endcsname}}
\test
```
会为未定义的控制序列 \xx 给出错误信息。
例如考虑 LATEX 环境，它们由以下形式定界：
```
\begin{...} ... \end{...}
```
begin 与 end 命令（在本质上）定义如下：
```
\def\begin#1{\begingroup\csname#1\endcsname}
\def\end#1{\csname end#1\endcsname \endgroup}
```
因此，对于 list 环境，命令 \list 与 \endlist 被定义，但任何命令都可以用作环境名，即使还没有定义相应的 \end...。例如，
```
\begin{it} ... \end{it}
```
等价于
```
\begingroup\it ... \relax\endgroup
```
关于为何使用 \begingroup 与 \endgroup 而不是 \bgroup 与 \egroup 的理由，见第 87 页。

12.5.2 用 \relax 阻止展开

因为 \relax 不能被展开，所以可以通过将一个控制序列暂时置为与 \relax 相等，来阻止它被展开（例如在 \edef 或 \write 中）：
```
{\let\somemacro=\relax \write\outfile{\somemacro}}
```
会把字符串 '\somemacro' 写入输出文件。如果省略了 \let 语句，它将写入宏 \somemacro 的展开（或者如果该宏未定义则给出错误信息）。

12.5.3 TEX 插入一个 \relax

TEX 自身在某些场合会插入 \relax。例如，当 TEX 仍在确定测试范围时遇到 \or、\else 或 \fi，就会插入 \relax。

```
\ifvoid1\else ... \fi
```
在内部被改为
```
\ifvoid1\relax \else ...\fi
```
。
类似地，如果测试 \if、\ifcat 之一只给出一个比较对象，如
```
\if1\else ...
```
则会插入一个 \relax 令牌。因此该测试等价于
```
\if1\relax\else ...
```

\relax 被使用的另一个场合如下。当一个控制序列正在一个⟨shorthand definition⟩中被定义时 – 即一个⟨registerdef⟩、\chardef 或 \mathchardef – 它的含义会被暂时置为与 \relax 相等。这使得可以写出\chardef\foo=123\foo 。

12.5.4 非宏的值；\the

展开在 TEX 中是一项精确定义的活动。上面已经给出了可被展开的完整令牌列表。不在上述列表中的其他令牌可能在非正式意义上具有" 展开"。例如，人们可能希望把 \parindent" 展开" 成它的值，比如 20pt。

把（其中包括）⟨integer parameter⟩、⟨glue parameter⟩、⟨dimen parameter⟩ 或⟨token parameter⟩ 的值转换为一串字符令牌，是由展开处理器完成的。命令 \the 只要展开未被抑制它就会展开，它取各种参数的值。它的结果（在大多数情形下）是一串类别码为 12 的记号，但空格的类别码为 10。

下面列出所有可在前面加 \the 的东西。

⟨**parameter**⟩ **或**⟨**register**⟩ 如果该参数或寄存器的类型是 integer、glue、dimen 或 muglue，其值作为一串字符记号给出；如果它是记号列表类型（例如 \everypar 或 \toks5），结果是一串记号。此处排除盒子寄存器。

⟨**codename**⟩⟨**8-bit number**⟩ 见第 40 页。

⟨**special register**⟩ 整数寄存器 \prevgraf、\deadcycles、\insertpenalties、\inputlineno、\badness、\parshape、\spacefactor（仅在水平模式），或 \prevdepth（仅在竖直模式）。尺寸寄存器 \pagetotal、\pagegoal、\pagestretch、\pagefilstretch、\pagefillstretch、\pagefilllstretch、\pageshrink 或 \pagedepth。

字体属性： \fontdimen⟨parameter number⟩⟨font⟩, \skewchar⟨font⟩, \hyphenchar⟨font⟩.

最近量： \lastpenalty, \lastkern, \lastskip.

⟨**defined character**⟩ 由 \chardef 或 \mathchardef 定义的任何控制序列；结果是十进制值。

在某些情形下，\the 可以给出一个控制序列记号或这类记号的列表。

⟨**font**⟩ 结果是代表该字体的控制序列。

⟨**token variable**⟩ 记号列表寄存器与⟨token parameter⟩ 可在前面加 \the；结果是它们的内容。

让我们看一个使用 \the 的例子。如果在一个将要被 \input 的文件中，某个字符（例如 @ 符号）的类别码被改变，可以写出

```
\edef\restorecat{\catcode`@=\the\catcode`@}
```

并在文件末尾调用 \restorecat。如果类别码原为 11，\restorecat 就被定义为等价于

```
\catcode`@=11
```

更精致的、用于保存与恢复类别码的宏见第 113 页。

12.6 例子

12.6.1 后置展开

\expandafter 最直接的用途是越过一个控制序列：

```
\def\stepcounter
    #1{\expandafter\advance\csname
            #1:counter\endcsname 1\relax}
\stepcounter{foo}
```

这里 \expandafter 让 \csname 命令形成控制序列 \foo:counter；\expandafter 完成后，该语句已归约为

```
\advance\foo:counter 1\relax
```

也可以越过控制序列以外的记号：例如

```
\uppercase\expandafter{\romannumeral \year}
```

它在开花括号的另一边展开 \romannumeral。

可以在两个控制序列之后展开：

```
\def\globalstepcounter
    #1{\expandafter\global\expandafter\advance
            \csname #1:counter\endcsname 1\relax}
```

如果把 \expandafter 想成颠倒两个控制序列的求值顺序，则可以通过下面的写法颠倒三个：

```
\expandafter\expandafter\expandafter\a\expandafter\b\c
```

它越过这三个控制序列

```
        \expandafter          \a          \b
```

以先展开 \c。

在条件语句中，\expandafter 甚至还有一个出人意料的用法：对于

```
\def\bold#1{{\bf #1}}
```

序列

```
\ifnum1>0 \bold \fi {word}
```

不会给出粗体的 'word'，而

```
\ifnum1>0 \expandafter\bold \fi {word}
```

会。\expandafter 让 TEX 在处理宏 \bold 之前先看到 \fi 并把它移除（另见第 121 页）。

12.6.2 在 \edef 内部定义

有一个 TEX 命令是被执行而非被展开的，值得明确指出：原语命令 \def（以及所有其他⟨def⟩ 命令）不被展开。

因此调用

```
\edef\next{\def\thing{text}}
```

会给出 \thing 的“未定义控制序列”错误，尽管在 \def 之后展开通常是被抑制的（见第 103 页）。在

```
\edef\next{\def\noexpand\thing{text}}
```

之后，\next 的“含义”将为

```
macro: \def \thing {text}
```

定义

```
\edef\next{\def\noexpand\thing{text}\thing}
```

会再次给出 \thing 的“未定义控制序列”错误（这一次是在它第二次出现时），因为它只会在调用 \next 时被定义，而不是在定义 \next 时。

12.6.3 展开与 \write

\write 的自变量记号列表的处理方式与 \edef 的替换文本大致相同；即可展开控制序列与活动字符被完全展开。不可展开控制序列被 \write 当作前面加了 \string 处理。

由于 \write 会做展开，用 \write 输出控制序列时必须小心。更复杂的是：\write 自变量的展开只在它被输出到页面时才进行。下面是一个完整示例。
假设 \somecs 是一个宏，而你想把字符串

    \def\othercs{*the expansion of* \somecs}

写入文件。
第一次尝试是

```
\write\myfile{\def\othercs{\somecs}}
```

这会给出 \othercs 的“未定义控制序列”错误，因为 \write 会试图展开那个记号。注意 \somecs 也被展开了，所以那一部分是对的。
下一次尝试是

```
\write\myfile{\def\noexpand\othercs{\somecs}}
```

这几乎对了，但还不完全。写入的语句是

    \def\othercs{*expansion of* \somecs}

看起来是对的。
但是，write – 及其自变量的展开 – 并不是当场执行的，而是被保存起来，直到它们所在的那页被输出（见第 30 章）。因此，在此期间 \somecs 的值可能已经改变。换句话说，写出的值可能并不是发出 \write 命令时的值。所以必须想办法把当前的展开结果插入 write 命令中。
下面是一种修正尝试：

```
\edef\act{\write\myfile{\def\noexpand\othercs{\somecs}}}
\act
```

现在 write 命令将为

    \write\myfile{\def\othercs{*value of*\somecs}}

\noexpand 阻止了 \edef 展开 \othercs，但在定义之后它已经消失，因此执行 write 时会再次给出未定义控制序列。最终解法是

```
\edef\act{\write\myfile
          {\def \noexpand\noexpand \noexpand\othercs{\somecs}}}
\act
```

在这种情形下，由 \act 的展开引发的 write 命令将为

    \write\myfile{\def\noexpand\othercs{*current value of* \somecs}}

而实际写出的字符串是

    \def\othercs{*current value of* \somecs}

这种机制是若干宏包中交叉引用宏的基础。

12.6.4 \edef 内部的受控展开

有时你可能需要 \edef 来求值当前条件，但你只想把替换文本中的某样东西展开到一定层次。假设给定

```
\def\a{\b} \def\b{c} \def\d{\e} \def\e{f}
```

，而你想把 \g 定义为 \a 展开一步、再后跟完全展开的 \d。下面的写法可行：

```
\edef\g{\expandafter\noexpand\a \d}
```

解释：\expandafter 越过 \noexpand 把 \a 展开一步，之后剩下序列\noexpand\b 。
当你需要在 \edef 内部用 \csname 构造一个控制序列时，这个技巧很方便。\edef 内部的下面这一序列

\expandafter\noexpand\csname name\endcsname
会恰好展开为 \name，而不再继续。例如，假设给定
\def\condition{true}
，那么
\edef\setmycondition{\expandafter\noexpand
          \csname mytest\condition\endcsname}
会让 \setmycondition 展开为 \mytesttrue。

12.6.5 多层阻止展开

如前所述，在命令前加 \noexpand 可阻止它在 \edef、\write 这类命令中被展开。但是，如果一串记号要经过不止一个展开命令，就需要更强的措施。
可以使用下面的技巧：为保护一个命令不被展开，可在它前面加\protect。在不希望展开的处理阶段，\protect 的定义为
\def\protect{\noexpand\protect\noexpand}
之后，当真正需要该命令时，\protect 定义为
\def\protect{}
这为什么有效？
\protect\somecs
的展开先是
\noexpand\protect\noexpand\somecs
在 \edef 内部，这个序列被进一步展开，随后的展开为
\protect\somecs
也就是说，展开结果与原序列相同。

12.6.6 更多 \relax 示例

上面给出了第一个例子，其中 \relax 用于防止 TEX 扫描得太远。下面再举几个用 \relax 界定数字的例子。
在
\countdef\pageno=0 \pageno=1
\def\Par{\par\penalty200}
序列
\Par\number\pageno
会被误解为
\par\penalty2001
在这种情况下，定义
\def\Par{\par\penalty200 }
就足够了，因为一个数字后允许跟⟨optional space⟩。
然而有时这种简单的转义并不可能。考虑定义
\def\ifequal#1#2{\ifnum#1=#2 1\else 0\fi}
问题在于 #2 后的那个空格是必要的、多余的，还是根本错误的。诸如\ifequal{27}{28} 之类比较两个数字（表示法）的调用会正确地给出 1 或 0，该空格对于防止误读是必要的。
然而，\ifequal\somecounter\othercounter 在计数器相等时会给出 1；在这种情形下本可以省掉那个空格。在两种情形下都可行的解法是
\def\ifequal#1#2{\ifnum#1=#2\relax 1\else 0\fi}
注意 \relax 不被展开，因此
\edef\foo{1\ifequal\counta\countb}
会把 \foo 定义为 1\relax1 或 10。

12.6.7 示例：类别码的保存与恢复

在许多应用中，需要在执行某段代码期间改变某个字符的类别码。如果该代码的作者也是周围代码的作者，可以简单地来回改变类别码。但如果周围代码出自另一位作者，就必须保存并恢复该类别码的值。

于是会想写出

```
\storecat@
... some code ...
\restorecat@
```

或者

```
\storecat\%
```

用于那些可能是注释字符（或被忽略、或无效）的字符。基本思路是定义

```
\def\storecat#1{%
    \expandafter\edef\csname restorecat#1\endcsname
        {\catcode`#1=\the\catcode`#1}}
```

这样例如`\storecat$` 就会把单个控制序列 '`\restorecat$`'（一个控制序列）定义为

```
\catcode`$=3
```

宏 `\restorecat` 则可实现为

```
\def\restorecat#1{%
    \csname restorecat#1\endcsname}
```

不幸的是，事情没那么简单。

问题出在活动字符（active character）上，因为它们会在`\csname ... \endcsname` 对之间被展开。人们可能会忍不住到处写`\noexpand#1`，但这是错误的。如上所述，这本质上等价于 `\relax`，而后者不可展开，因此当它出现在 `\csname` 与 `\endcsname` 之间时会导致错误信息。正确的解法是使用`\string#1` 。对于自变量以控制符号给出的情形（例如 `\%`），需要暂时关闭转义字符。

下面是完整的宏。`\storecat` 宏给其自变量一个默认类别码 12。

```
\newcount\tempcounta % just a temporary
\def\csarg#1#2{\expandafter#1\csname#2\endcsname}
\def\storecat#1%
   {\tempcounta\escapechar \escapechar=-1
    \csarg\edef{restorecat\string#1}%
          {\catcode`\string#1=
                \the\catcode\expandafter`\string#1}%
    \catcode\expandafter`\string#1=12\relax
    \escapechar\tempcounta}
\def\restorecat#1%
   {\tempcounta\escapechar \escapechar=-1
    \csname restorecat\string#1\endcsname
    \escapechar\tempcounta}
```

12.6.8 组合 `\aftergroup` 与盒子

有时，人们想构造一个盒子，并在它刚构造好之后立刻对它做些处理。`\aftergroup` 命令可用于把创建盒子的命令与处理盒子的命令放进同一个宏。

例如，下面是一个宏 `\textvcenter` 它定义了 `\vcenter` 盒子的一个变体（见第 169 页），可在数学模式之外使用。

```
\def\textvcenter
   {\hbox \bgroup$\everyvbox{\everyvbox{}%
    \aftergroup$\aftergroup\egroup}\vcenter}
```

其思路是：该宏插入`\hbox {$`，而配对的 `$}` 由 `\aftergroup` 命令插入。为了把 `\aftergroup` 命令放进盒子内部，使用了 `\everyvbox` 命令。

这个宏甚至可以配合⟨box specification⟩ 使用（见第 47 页），例如

```
\textvcenter spread 8pt{\hbox{a}\vfil\hbox{b}}
```

又因为它实际上只是一个 `\hbox`，它也可用于 `\setbox` 赋值。

12.6.9 更多展开

纯粹靠展开工作的宏有一种特别的魅力。参见以下文章：[11]、[16] 与 [32]。

第 13 章

条件语句

条件语句是编写强大宏的不可或缺的工具。TEX 拥有大量条件语句，用于查询诸如类别码或处理模式之类的信息。本章列出各种条件语句的清单，并详细讨论条件语句的求值过程。
`\if` 测试字符代码是否相等。
`\ifcat` 测试类别码是否相等。
`\ifx` 测试宏展开是否相等，或测试字符代码与类别码是否相等。
`\ifcase` 枚举式 case 语句。
`\ifnum` 测试数字之间的关系。
`\ifodd` 测试一个数字是否为奇数。
`\ifhmode` 测试当前模式是否为（可能受限的）水平模式。
`\ifvmode` 测试当前模式是否为（可能内部的）垂直模式。
`\ifmmode` 测试当前模式是否为（可能为展示式的）数学模式。
`\ifinner` 测试当前模式是否为内部模式。
`\ifdim` 比较两个长度。
`\ifvoid` 测试某个盒子寄存器是否为空。
`\ifhbox` 测试某个盒子寄存器是否包含一个水平盒子。
`\ifvbox` 测试某个盒子寄存器是否包含一个垂直盒子。
`\ifeof` 测试输入流是否结束，或文件是否不存在。
`\iftrue` 一个恒为真的测试。
`\iffalse` 一个恒为假的测试。
`\fi` 所有条件语句的结束分隔符。
`\else` 选择条件语句的⟨false text⟩，或 `\ifcase` 的默认分支。
`\or` `\ifcase` 各分支之间的分隔符。
`\newif` 创建一个新的测试。

13.1 条件语句的形式

TEX 中的条件语句有以下两种形式之一

`\if...`⟨test tokens⟩⟨true text⟩`\fi`
`\if...`⟨test tokens⟩⟨true text⟩`\else`⟨false text⟩`\fi`

其中⟨test tokens⟩ 是零个或多个记号，具体取决于所用的条件语句；⟨true text⟩ 是在测试为真时需要处理的一串记号，而⟨false text⟩ 是在测试为假时需要处理的一串记号。⟨true text⟩ 与⟨false text⟩ 都可以为空。
TEX 如何展开条件语句的确切过程将在下文讨论。

13.2 字符与控制序列测试

共有三种测试用于检测字符记号和控制序列记号。

13.2.1 \if

字符代码是否相等可以用下面的方式测试：

\if⟨token$_1$⟩⟨token$_2$⟩

为了让记号也可以是控制序列，TEX 给控制序列赋予字符代码 256，这是不属于任何字符记号字符代码的最小正整数（请记住合法的字符代码是 0–255）。

因此就 \if 而言，所有控制序列彼此相等，而它们与所有字符记号都不相等。举例来说，可以利用这一事实来定义

```
\def\ifIsControlSequence#1{\if\noexpand#1\relax}
```

它用于检测一个记号是控制序列记号而非字符记号（若参数是一个 {...} 组，其结果不可预测）。

在 \if 之后，TEX 会不断展开，直到得到两个不可展开的记号为止；因此当用 \if 检测可展开的控制序列和活动字符时，有必要在它们前面加上 \noexpand。

在

```
\catcode`\b=13 \catcode`\c=13 \def b{a} \def c{a} \let\d=a
```

我们发现：

\if bc 为真，因为 b 与 c 都展开为 a，
\if\noexpand b\noexpand c 为假，以及
\if b\d 为真，因为 b 展开为字符 a，而 \d 是一个隐式字符记号 a。

13.2.2 \ifcat

\if 测试会忽略类别码；类别码可以用由

\ifcat⟨token$_1$⟩⟨token$_2$⟩

来测试。

这个测试与 \if 非常相似：TEX 在它之后会不断展开，直到只剩下不可展开的记号。对于本测试，控制序列被认为具有类别码 16（通常类别码的范围是 0–15），这使得它们彼此全都相等，而与所有字符记号不同。

13.2.3 \ifx

记号是否相等，由下面的命令以比上述更强的意义来测试：

\ifx⟨token$_1$⟩⟨token$_2$⟩

- 对于 \ifx，若字符记号具有相同的字符代码和类别码，则它们相等。
- 若控制序列记号表示同一个 TEX 原语,或由 \font、\countdef 等命令以类似方式定义，则它们相等。例如，

```
\let\boxhor=\hbox \ifx\boxhor\hbox %is true
\font\a=cmr10 \font\b=cmr10 \ifx\a\b %is true
```

- 若控制序列是具有相同参数文本和替换文本的宏，并且在 \outer 与 \long 方面的状态相同，则它们也相等。例如，

```
\def\a{z} \def\b{z} \def\c1{z} \def\d{\a}
\ifx\a\b %is true
\ifx\a\c %is false
\ifx\a\d %is false
```

本测试之后的记号不会被展开。
作为 \ifx 用法的一个例子，考虑字符串比较。在 TEX 中实现字符串比较的一个简单方法如下：

```
\def\ifEqString#1#2{\def\testa{#1}\def\testb{#2}%
    \ifx\testa\testb}
```

这两个字符串被用作两个宏的替换文本，然后测试这两个宏是否相等。这差不多就是字符串比较所能达到的最高效率：TEX 会遍历宏 \testa 与 \testb 的定义文本，而这恰好产生了正确的效果。
作为另一个例子，可以用下面的方式检测一个控制序列是否已定义：

```
\def\ifUnDefinedCs#1{\expandafter
    \ifx\csname#1\endcsname\relax}
\ifUnDefinedCs{parindent} %is not true
\ifUnDefinedCs{undefined} %is (one hopes) true
```

这利用了如下事实:若某个控制序列此前从未定义过,则\csname...\endcsname 命令等价于 \relax。不幸的是，若某个控制序列已被 \let 为 \relax，本测试也会判定为真。

13.3 模式测试

为了确定 TEX 当前正处于六种模式（见第 6 章）中的哪一种，可以使用\ifhmode、\ifvmode、\ifmmode 以及\ifinner 这几个测试。

- 若 TEX 处于水平模式或受限水平模式，则 \ifhmode 为真。
- 若 TEX 处于垂直模式或内部垂直模式，则 \ifvmode 为真。
- 若 TEX 处于数学模式或展示式数学模式，则 \ifmmode 为真。

若 TEX 处于三种内部模式中的任何一种，即受限水平模式、内部垂直模式或非展示式数学模式，则 \ifinner 测试为真。

13.4 数值测试

⟨number⟩ 之间的数值关系可以用下面的形式测试：下面的形式（disp 见上）⟨number$_1$⟩⟨relation⟩⟨number$_2$⟩
其中关系符是一个类别码为 12 的字符 <、= 或 >。

诸如黏滞度之类的量，通过换算成缩放点，也可以在这里当作数字使用；为了得到这两个⟨number⟩，TEX 会进行展开。

数字是奇数还是偶数，可以用\ifodd 来测试：下面的测试

\ifodd⟨number⟩

若⟨number⟩ 为奇数则为真。

13.5 其他测试

13.5.1 长度测试

⟨dimen⟩ 值之间的关系（第 8 章）可以用\ifdim 来测试，所使用的三种关系与 \ifnum 中相同。

13.5.2 盒子测试

盒子寄存器的内容（第 5 章）可以用以下命令测试：

\ifvoid⟨8-bit number⟩

若该寄存器不包含任何盒子则为真；

\ifhbox⟨8-bit number⟩

若该寄存器包含一个水平盒子则为真；以及

\ifvbox⟨8-bit number⟩

若该寄存器包含一个垂直盒子则为真。

13.5.3 输入输出测试

输入流的状态（第 30 章）可以用文件结束测试\ifeof⟨number⟩ 来检测：只有当该数字在 0–15 范围内、且对应的输入流已打开且尚未读完时，它才为假。特别地，如果连接到该输入流（通过 \openin）的文件名并不对应一个已存在的文件，则本测试为真。参见第 204 页的例子。

13.5.4 case 语句

TEX 的 case 语句称为 \ifcase；其语法为

\ifcase⟨number⟩⟨case$_0$⟩\or...\or⟨case$_n$⟩\else⟨other cases⟩\fi

其中对于 n 个分支，共有 $n-1$ 个 \or 控制序列。每个⟨case$_i$⟩ 部分都可以为空，而 \else⟨other cases⟩ 部分是可选的。

13.5.5 特殊测试

\iftrue 与 \iffalse 这两个测试分别恒为真和恒为假。它们主要用作宏中的工具。

例如，序列

```
\iftrue{\else}\fi
```

以及

```
\iffalse{\else}\fi
```

分别产生一个左花括号和一个右花括号，但它们的花括号是配对平衡的，因此可以用在宏的替换文本内部。

下面将要讨论的 \newif 宏，给出了 \iftrue 与 \iffalse 的另一种用法。在 the TEX book 第 260 页上，这些控制序列也以一种有趣的方式被使用。

13.6 \newif 宏

plain 格式定义了一个（outer）宏\newif，用户可借此定义新的条件语句。若用户定义

```
\newif\iffoo
```

TEX 会定义三个新的控制序列：\footrue 与 \foofalse 供用户设置条件，以及用于测试 'foo' 条件的 \iffoo。

宏调用\newif\iffoo 展开为

```
\def\footrue{\let\iffoo=\iftrue} \def\foofalse{\let\iffoo=\iffalse}
\foofalse
```

其实际定义，尤其是保证 \iffoo 确实以 \if 开头的那一部分，相当精巧。下面加以解释。这里用到了第 11 章与第 12 章的概念。

宏 \newif 的开头如下：

```
\outer\def\newif#1{\count@\escapechar \escapechar\m@ne
```

这把当前的转义字符保存到 \count@ 中，并把 \escapechar 的值设为 -1。后一动作使得 \string⟨control sequence⟩ 的输出中不使用转义字符。

辅助宏\if@ 由下面的方式定义：

```
{\uccode`1=`i \uccode`2=`f \uppercase{\gdef\if@12{}}}
```

由于 uppercase 命令只改变字符代码而不改变类别码，宏 \if@ 之后必须紧跟类别码为 12 的字符 if。通常这些字符的类别码是 11。实际上，这个宏会吃掉这两个字符；若它们不存在，TEX 会报错。

接着还有一个宏\@if ，由下面的方式定义：

```
\def\@if#1#2{\csname\expandafter\if@\string#1#2\endcsname}
```

它将以\@if\iffoo{true} 和\@if\iffoo{false} 的形式被调用。

让我们考察调用\@if\iffoo{true} 。

- \expandafter 越过\if@ ，先展开 \string。由于不打印转义字符且所有字符的类别码都是 12，部分\string\iffoo 展开为 iffoo。
- \if@ 吃掉其中前两个字符 $\mathtt{i}_{12}\mathtt{f}_{12}$。
- 结果，\@if\iffoo{true} 的最终展开即为
  \csname footrue\endcsname

现在我们可以讨论 \newif 本身的相关部分了：

```
\expandafter\expandafter\expandafter
   \edef\@if#1{true}{\let\noexpand#1=\noexpand\iftrue}%
```

这三个 \expandafter 命令看起来可能有些吓人，我们一步一步来。

- 需要一个 \expandafter 越过 \edef，使\@if 得以展开：
  \expandafter\edef\@if\iffoo{true}
  得到
  \edef\csname footrue\endcsname
- 然后还需要另一个 \expandafter 来激活 \csname：
  \expandafter \expandafter \expandafter \edef \@if ...
  %   new   old   new
- 这使得最终展开为
  \edef\footrue{\let\noexpand\iffoo=\noexpand\iftrue}

此后是针对 false 情形的类似语句：

```
   \expandafter\expandafter\expandafter
   \edef\@if#1{false}{\let\noexpand#1=\noexpand\iffalse}%
```

该条件语句初始为假，并且转义字符必须复位：

```
  \@if#1{false}\escapechar\count@}
```

13.7 条件语句的求值

TEX 的条件语句与普通编程语言中的条件语句行为不同。在很多情况下你可能察觉不到这种差异，但在某些场合下，准确地知道究竟发生了什么非常重要。

当 TEX 求一个条件语句的值时，它首先确定要测试的是什么。这一过程本身可能涉及一些展开；正如我们在上一章看到的，只有在 `\ifx` 测试之后 TEX 才不展开。在所有其他测试之后，TEX 都会不断展开记号，直到确定了测试的范围以及待测试的记号为止。根据该测试的结果，⟨true text⟩ 与⟨false text⟩ 要么被展开，要么被跳过。

为了说明条件语句各部分的处理方式，我们分别考察几种情形。

- `\if... ... \fi` 且测试结果为假。测试之后，TEX 会开始跳过内容：不展开、不计数花括号，但会平衡嵌套的条件语句，直到遇到一个 `\fi` 记号为止。若找不到 `\fi`，则在文件末尾产生一条错误信息：
  ```
  Incomplete \if...; all text was ignored after line ...
  ```
  其中所指示的行号是 TEX 开始跳过之处所在的行，也就是该条件语句出现的那一行。
- `\if... \else ... \fi` 且测试结果为假。条件与 `\else` 之间的所有内容都会被跳过：不展开、不计数花括号，但会平衡嵌套的条件语句。
  `\fi` 记号可以是展开的结果；若它始终不出现，TEX 会给出一条诊断信息
  ```
  \end occurred when \if... on line ... was incomplete
  ```
  这类错误在输出中是看不见的。
  这一点连同上一点可以合起来描述如下：条件为假时，TEX 一直跳过，直到找到 `\else` 或 `\fi`；`\else` 与 `\fi` 之间的任何内容都会被处理。
- `\if... ... \fi` 且测试结果为真。TEX 会开始处理条件之后的内容。与上面一样，`\fi` 记号也可以由某个宏的展开而插入。
- `\if... \else ... \fi` 且测试结果为真。条件之后的所有内容都会被处理，直到找到 `\else`；此后 TEX 会跳过一切，直到找到与之配对的 `\fi`。
  这一点连同上一点可以描述如下：测试为真时，TEX 开始处理内容，直到找到 `\else` 或 `\fi`；若找到的是 `\else`，TEX 会继续跳过，直到找到配对的 `\fi`。

13.8 杂项说明

13.8.1 测试会吞掉记号

一个常见的错误是写出下面这样的代码：

```
\ifnum\x>0\someaction \else\anotheraction \fi
```

其结果是`\someaction` 会被展开，无论测试是否成功。原因在于：TEX 会对输入流求值，直到确信已经找到待测试的参数为止。在这种情况下，`\someaction` 完全可能产生一个数字，因此它会被展开。解决办法是在待测试数字的最后一位数字之后，插入一个空格或一个 `\relax` 控制序列。

13.8.2 测试会试图吞掉 `\else` 或 `\fi`

上一点中现象背后的同一机制，若在 TEX 仍忙于确定测试范围时恰好遇到`\else` 、`\or` 或`\fi` ，还会导致更加出人意料的效果。

回想一下，`\pageno` 是`\count0` 的同义词，请考虑下面的例子：

```
\newcount\nct \nct=1\ifodd\pageno\else 2\fi 1
```

以及

```
\newcount\nct \nct=1\ifodd\count0\else 2\fi 1
```

第一个例子会给 \nct 赋 11 或 121，而第二个例子会赋 1 或 121。原因在于：在第二个这样的情形中，当测试尚未确定界限时就遇到了\else ，此时会插入一个\relax 。若\count0 为奇数，结果就会是\relax ，于是该例子产生

```
\nct=1\relax2
```

这会把 1 赋给 \nct，并打印出 2。

13.8.3 宏与条件语句；\expandafter 的用法

请看下面的例子：

```
\def\bold#1{{\bf #1}} \def\slant#1{{\sl #1}}
\ifnum1>0 \bold \else \slant \fi {some text} ...
```

这不仅会把 'some text'，而且会把*其后的所有*文本都变成粗体。此外，在作业结束时还会提示 'end occurred inside a group at level 1'。打开 \tracingmacros 可以发现，\bold 的参数是\else 。这意味着，在\bold 展开之后，输入流看起来是这样的：

```
\ifnum1>0 {\bf \else }\fi {some text} rest of the text
```

因此，那个右花括号作为⟨false text⟩ 的一部分被跳过了。于是实际上，最终的记号流是

```
{\bf {some text} rest of the text
```

它是不配对平衡的。

这类问题的一种解决办法是写成

```
\ifnum1>0 \let\next=\bold \else \let\next=\slant \fi \next
```

不过，也可以用 \expandafter 来解决：

```
\ifnum1>0 \expandafter \bold \else \expandafter \slant \fi
```

这之所以可行，是因为 \expandafter 命令让 TEX 得以确定⟨true text⟩ 与⟨false text⟩ 的边界。

事实上，第二种方案可能比第一种更可取，因为条件语句是由展开处理器处理的，而 \let 语句只能由执行处理器处理；也就是说，它们不可展开。因此第二种方案（而第一种方案不会）可以在例如 \edef 内部正常工作。

另一个用到 \expandafter 的例子是下面的序列：

```
\def\get#1\get{ ... }
\expandafter \get \ifodd1 \ifodd3 5\fi \fi \get
```

这得到

```
#1<- \ifodd3 5\fi \fi
```

以及

```
\expandafter \get \ifodd2 \ifodd3 5\fi\fi \get
```

得到

```
#1<-
```

这再次说明：求条件语句的值所得的结果并不是最终展开，而是⟨true text⟩ 或⟨false text⟩ 展开的开端，具体取决于测试的结果。

有一个细节需要注意：使用 \expandafter 时，有可能在⟨true text⟩ 尚未完全展开之前就遇到\else 。这就引出了展开与跳过究竟何时进行的问题。在下面的例子中，

```
\def\hello{\message{Hello!}}
\ifnum1>0 \expandafter \hello \else \message{goodbye} \bye
```

由缺少\fi 引起的错误信息被给出，而\hello 根本没有被展开过。由此可知，⟨false text⟩ 一旦被定位就会立即被跳过，即使此时⟨true text⟩ 尚未完全展开。

13.8.4 不正确的匹配

TEX 对\if 、\else 和\fi 的匹配很容易被打乱。例如，the TEX book 警告你不要在一个条件语句内部写

```
\let\ifabc=\iftrue
```

因为若这段文本被跳过，TEX 至少会看到一个\if 需要匹配。

原因在于：当 TEX 正在跳过内容时，它会识别所有 \if...、\or、\else、\fi 记号，以及任何通过 \let 被声明为这类记号同义词的东西。在\let\ifabc=\iftrue 中，TEX 至少会把 \iftrue 看作一个条件语句的开头；而如果 \ifabc 当前的含义例如是 \iffalse，它也会被视为一个条件语句的开头。

又如，如果

```
\csname if\sometest\endcsname \someaction \fi
```

作为条件文本的一部分被跳过时，\fi 会意外地关闭外层的条件语句。

把这类有潜在危险的构造包在一个组里并没有用，因为分组与条件语句结构是相互独立的。把这类命令埋在宏内部才是最安全的做法。

不过有时也可以采用另一种解决办法。plain TEX 的 \loop 宏（见第 97 页）按下面的方式使用：

```
\loop ... \if ... \repeat
```

其中 \repeat 并不是一个真正可执行的命令，而只是一个分隔符：

```
\def\loop#1\repeat{ ... }
```

因此，通过声明

```
\let\repeat\fi
```

\repeat 就会与终止循环的那个 \if... 相配对，从而可以在被跳过的条件文本中使用循环。

13.8.5 条件语句与分组

前面已经提到，TEX 中组的嵌套与条件语句的嵌套是相互独立的。原因在于：条件语句是由 TEX 的展开部分处理的；在那个阶段，花括号只是不可展开的记号，无需特殊处理。分组只在稍后的执行处理阶段才进行。

下面给出这种独立性的一个例子。你可以写一个宏，让它产生条件语句的一部分：

```
\def\elsepart{\else \dosomething \fi}
```

反过来，下面这两个宏分别产生一个左花括号和一个右花括号：

```
\def\leftbrace{\iftrue{\else}\fi}
\def\rightbrace{\iffalse{\else}\fi}
```

注意这些定义中的花括号是正确嵌套的。

13.8.6 一个技巧

在某些场合下，要恰当地摆脱 `\else` 或 `\fi` 记号可能很困难。上面那种使用 `\expandafter` 的办法，只在涉及的记号数量有限时才管用。在其他情况下，下面这个技巧也许能帮你脱身：

```
\def\hop#1\fi{\fi #1}
```

把它用作

```
\if... \hop ⟨lots of tokens⟩\fi
```

它会把这些记号移到条件语句之外。例如 [11] 中就用到了这一点。

作为这类技巧的又一个例子，请考虑这样一个问题（由 Alan Jeffrey 向我提出并解决）：实现一个条件语句`\ifLessThan#1#2#3#4` ，使得根据 `#1` 是否小于 `#2`，分别产生与 `#3` 或 `#4` 对应的内容。

这里的问题在于如何摆脱 `\else` 与 `\fi`。一个 – 或者说，一种 – 解决办法是把它们当作宏的分隔符来吸收：

```
\def\ifLessThan#1#2{\ifnum#1<#2\relax\taketrue \else \takefalse \fi}
\def\takefalse\fi#1#2{\fi#2}
\def\taketrue\else\takefalse\fi#1#2{\fi#1}
```

注意，`\ifLessThan` 只有两个参数（即待比较的双方）；不过它的结果是一个宏，会在接下来的两个参数之间做出选择。

13.8.7 条件语句中展开的更多例子

前面给出了宏 `\ifEqString`，它用于比较两个字符串：

```
\def\ifEqString#1#2%
   {\def\csa{#1}\def\csb{#2}\ifx\csa\csb }
```

然而这个宏依赖于 `\def`，而后者并不是一个可展开命令。如果我们需要一个例如能在 `\edef` 内部工作的字符串比较器，就需要再多一点巧思（本解法取自 [11]）。这个解法的基本原理是逐个字符地比较两个字符串。其中用到了以 `\fi` 作为宏分隔符的做法；前面已作解释。

首先，`\ifEqString` 调用被替换为序列`\ifAllChars ...\Are ...\TheSame` ，并且两个字符串都用一个美元符号定界，而该符号本不应出现在字符串内部。

```
\def\ifEqString
    #1#2{\ifAllChars#1$\Are#2$\TheSame}
```

字符相等性测试首先判断两个字符串是否有一个已经结束。若两者都已结束，则原字符串相等；若只有一个结束，则二者长度不等，因而不相等。若两者都未结束，就测试首字符是否相等；若相等，就递归调用以测试字符串的剩余部分。

```
\def\ifAllChars#1#2\Are#3#4\TheSame
   {\if#1$\if#3$\say{true}%
          \else \say{false}\fi
    \else \if#1#3\ifRest#2\TheSame#4\else
                \say{false}\fi\fi}
\def\ifRest#1\TheSame#2\else#3\fi\fi
   {\fi\fi \ifAllChars#1\Are#2\TheSame}
```

`\say` 宏应当对`\say{true}` 给出 `\iftrue`，对`\say{false}` 给出 `\iffalse`。注意到对这个宏的所有调用都处于两层条件语句之内，我们按如下方式使用前面解释过的 ‘hop’ 技巧。

```
\def\say#1#2\fi\fi
   {\fi\fi\csname if#1\endcsname}
```

与上面的例子类似，我们来写一个宏，用于测试两个字符串的字典序（‘词典’ 顺序）先后：

```
\let\ex=\expandafter
\def\ifbefore
    #1#2{\ifallchars#1$\are#2$\before}
\def\ifallchars#1#2\are#3#4\before
   {\if#1$\say{true\ex}\else
     \if#3$\say{false\ex\ex\ex}\else
      \ifnum`#1>`#3 \say{false%
         \ex\ex\ex\ex\ex\ex\ex}\else
       \ifnum`#1<`#3 \say{true%
         \ex\ex\ex\ex\ex\ex\ex
         \ex\ex\ex\ex\ex\ex\ex\ex}\else
       \ifrest#2\before#4\fi\fi\fi\fi}
\def\ifrest#1\before#2\fi\fi\fi\fi
   {\fi\fi\fi\fi
    \ifallchars#1\are#2\before}
\def\say#1{\csname if#1\endcsname}
```

在这个宏中，使用了一种略有不同的 `\say` 实现。

简化地说，一次对 `\ifbefore` 的调用最终会导致如下情形（在 ‘为真’ 的情形下）：

```
\ifbefore{...}{...}
       \if... %% some comparison that turns out true
          \csname iftrue\expandafter\endcsname
       \else .... \fi
   ... %% commands for the `before' case
\else
   ... %% commands for the `not-before' case
\fi
```

当比较结果为真时，TEX 会开始处理⟨true text⟩，并在心里记下：一旦看到 `\else` 记号，就把`\else ... \fi` 部分移除。于是，序列

```
\csname iftrue\expandafter\endcsname \else ... \fi
```

会被替换为

```
 \csname iftrue\endcsname
```

因为 `\else` 是在 TEX 仍在处理`\csname...\endcsname` 时被看到的。

对 `\say` 的调用发生在嵌套的条件语句内部，因此所需的 `\expandafter` 命令数量可能大于 1：第二层为 3，第三层为 7，第四层为 15。这个宏也存在更紧凑一些的实现。

第 14 章

记号表

TEX 只有一种数据结构：记号表。有可供用户使用的记号表寄存器，此外 TEX 还有一些特殊的记号表：`\every...` 变量、`\errhelp` 以及 `\output`。

`\toks` 记号表寄存器的前缀。
`\toksdef` 把一个控制序列定义为某个 `\toks` 寄存器的同义词。
`\newtoks` 用于分配记号表寄存器的宏。

14.1 记号表

记号表是 TEX 所知的唯一一种数据结构。它们可以包含字符记号和控制序列记号。记号表中的空格是有意义的。对记号表仅有的两种操作是赋值与解包。

TEX 有 256 个记号表寄存器`\toks` nnn，它们可以用宏`\newtoks` 来分配，或通过 `\toksdef` 显式指定；见下文。

14.2 记号表的用法

记号表通过⟨variable assignment⟩ 赋值，在这种情况下取以下形式之一：

⟨token variable⟩⟨equals⟩⟨general text⟩
⟨token variable⟩⟨equals⟩⟨filler⟩⟨token variable⟩

这里⟨token variable⟩ 是一个显式的 `\toks`nnn 寄存器、某个由 `\toksdef`（可能藏在 `\newtoks` 中）定义为该寄存器的控制序列，或是下面这些特殊的⟨token parameter⟩ 之一。⟨general text⟩ 有一个显式的右花括号，但左花括号可以是隐式的。

记号表的例子如下（前两行等价）：

```
\toks0=\bgroup \a \b cd}
\toks0={\a \b cd}
\toks1=\toks2
```

记号表的解包由命令 `\the` 完成：`\the`⟨token variable⟩ 的展开结果就是记号表中原来的那串记号。

记号表在 `\edef` 中有特殊行为：当前面加了`\the` 时它们会被解包，但所得记号不会再被进一步求值。

```
\toks0={\a \b} \edef\SomeCs{\the\toks0}
```

得到

```
\SomeCs: macro:-> \a \b
```

这与通常在 `\edef` 中发生的情况不同；见第 104 页。

14.3 ⟨token parameter⟩

在 TEX 中有若干个记号表，它们会在特定位置被自动插入。这些⟨token parameter⟩如下：

`\output` 每当 TEX 判定已有足够内容排成一页时，或者用户在垂直模式中通过一个惩罚值 $\leq -10\,000$ 强制激活时，都会插入这个记号表（见第 28 章）；

`\everypar` 当 TEX 从外部或内部垂直模式切换到不受限水平模式时插入（见第 16 章）；

`\everymath` 在启动一个公式的单个数学切换字符之后插入；

`\everydisplay` 在启动一个展示式公式的双数学切换字符之后插入；

`\everyhbox` 当一个 `\hbox` 开始时插入（见第 5 章）；

`\everyvbox` 当一个垂直盒子开始时插入（见第 5 章）；

`\everyjob` 当一个作业开始时插入（见第 32 章）；

`\everycr` 在对齐排版中，于 `\cr` 或一个非冗余的 `\crcr` 之后插入（见第 25 章）；

`\errhelp` 包含用于补充 `\errmessage` 的记号（见第 35 章）。

⟨token parameter⟩ 的行为与一个显式的 `\toks`*nnn* 记号表、或由 `\toksdef` 定义的量完全相同。

14.4 记号表寄存器

记号表可以存放在`\toks` 寄存器中：

`\toks`⟨8-bit number⟩

它是一个⟨token variable⟩。记号表寄存器的同义词可以通过⟨registerdef⟩ 命令`\toksdef`，以⟨shorthand definition⟩ 的方式来建立：

`\toksdef`⟨control sequence⟩⟨equals⟩⟨8-bit number⟩

以这种方式定义的控制序列称为⟨toksdef token⟩，它也是一个记号变量（记号变量剩下的第三种是⟨token parameter⟩）。

plain TEX 宏`\newtoks` 使用 `\toksdef` 来分配尚未使用的记号表寄存器。这个宏是 `\outer` 的。

14.5 例子

记号表大概是 TEX 中最不显而易见的组件之一：大多数 TEX 用户可能永远不会用到它们，但格式设计者和其他宏编写者却能发现有趣的应用。下面是一些例子，说明用记号表可以做哪些事情。

14.5.1 记号表上的操作：栈宏

记号表可用的原语操作数量相当有限：只有赋值和解包。然而这已足以实现诸如追加之类的其他操作。

假定我们已经分配了一个记号寄存器

```
\newtoks\list \list={\c}
```

并且想给它添加记号，使用如下语法

```
\Prepend \a \b (to:)\list
```

使得

```
\showthe\list
```

得到

```
> \a \b \c .
```

为此，原来的表必须先解包，然后把新记号接在旧内容之前，再重新赋值给该寄存器。解包可以在 `\edef` 内部用 `\the` 完成，于是我们得到下面这个宏：

```
\def\Prepend#1(to:)#2{\toks0={#1}%
    \edef\act{\noexpand#2={\the\toks0 \the#2}}%
    \act}
```

注意，要添加的记号先被打包进一个临时记号表，然后再在 `\edef` 内部解包。若直接把它们包含进来，就会导致它们被展开。

接下来我们想把记号表当作一种栈来使用：我们希望有一个 'pop' 操作，把第一个元素从表中移除。具体地，

```
\Pop\list(into:)\first
\show\first \showthe\list
```

应当给出

```
> \first=macro:
->\a .
```

而对于剩余的表

```
> \b \c .
```

这里我们创造性地利用了定界参数与非定界参数。通过 `\edef` 把表解包，辅助宏 `\SplitOff` 把元素吸收为一个非定界参数（即第一个元素）和一个定界参数（即其余元素）。

```
\def\Pop#1(into:)#2{%
    \edef\act{\noexpand\SplitOff\the#1%
              (head:)\noexpand#2(tail:)\noexpand#1}%
    \act}
\def\SplitOff#1#2(head:)#3(tail:)#4{\def#3{#1}#4={#2}}
```

14.5.2 执行记号表

前面用于解包记号表的 `\the` 操作只在 `\edef` 内部使用过。若单独使用它，其效果是把表中的记号送入 TEX 的展开机制。如果加入表中的记号采用统一的语法，就会产生一些有趣的可能性。

设想我们正在为某个格式实现外部文件的簿记。这类外部文件可用于目录、插图清单等等。如果这类对象的存在由用户控制，我们就需要一些通用例程来打开和关闭文件，并记录应我们应要求打开了哪些文件。

这里只描述一些簿记例程。假定有一个辅助文件表和一个辅助计数器：

```
\newtoks\auxlist \newcount\auxcount
```

首先必须有一个添加辅助文件的操作：

```
\def\NewAuxFile#1{\AddToAuxList{#1}%
    % plus other actions
    }
\def\AddToAuxList#1{\let\\=\relax
    \edef\act{\noexpand\auxlist={\the\auxlist \\{#1}}}%
    \act}
```

这以统一格式把文件名加入表中：

```
\NewAuxFile{toc} \NewAuxFile{lof}
\showthe\auxlist
> \\{toc}\\{lof}.
```

其中使用了保持未定义状态的控制序列\\ 。

现在这个控制序列例如可以用来统计表中元素的个数：

```
\def\ComputeLengthOfAuxList{\auxcount=0
    \def\\##1{\advance\auxcount1\relax}%
    \the\auxlist}
\ComputeLengthOfAuxList \showthe\auxcount
> 2.
```

这个结构的另一种用途如下：在作业结束时，我们现在可以一次性关闭所有辅助文件，方法是

```
\def\CloseAuxFiles{\def\\##1{\CloseAuxFile{##1}}%
    \the\auxlist}
\def\CloseAuxFile#1{\message{closing file: #1. }%
    % plus other actions
    }
\CloseAuxFiles
```

这产生输出

```
closing file: toc.  closing file: lof.
```

第 15 章

基线间距

文本行在大多数情况下并不具有相同的高度或深度。因此 TEX 会添加行间胶，使各基线之间保持均匀的距离。本章讨论这种行间胶是如何计算的。

`\baselineskip` 垂直表上相邻盒子之间的 '理想' 基线距离。plain TEX 默认值：`12pt`。

`\lineskiplimit` 垂直表上相邻盒子的底边与顶边之间应维持的距离。plain TEX 默认值：`0pt`。

`\lineskip` 当相邻盒子的底边与顶边之间的距离小于 `\lineskiplimit` 时所添加的胶。plain TEX 默认值：`1pt`。

`\prevdepth` TEX 所感知到的、最近加入垂直表的那个盒子的深度。

`\nointerlineskip` 一次性阻止行间胶插入的宏。

`\offinterlineskip` 从此全局阻止行间胶的宏。

`\openup` 把 `\baselineskip`、`\lineskip` 和 `\lineskiplimit` 都增加指定的量。

15.1 行间胶

TEX 试图在加入垂直表的各个盒子的参考点之间保持一定距离；特别是，它试图把普通文本的基线保持在一个恒定的距离上，即`\baselineskip`。实际上，`\baselineskip` 是一个⟨glue⟩，所以行距可以拉伸或收缩。不过，所有行之间的自然尺寸以及拉伸量和收缩量都是相同的。

当盒子（无论是段落的行还是显式盒子）被附加到垂直表时，通常会添加胶，使前一个盒子的深度与当前盒子的高度之和等于 `\baselineskip`。这样就把后续各行的参考点保持在规则的间隔上。

然而,这一过程可能使两个相邻盒子的底边与顶边之间的距离小于 \lineskiplimit:

在这种情况下，会添加 \lineskip 胶：注意，这通常会把各盒子基线之间的距离增大到超过 \baselineskip。

确切的过程如下：

- 若 \prevdepth 为 -1000pt 或更小，则不添加胶；否则
- TEX 把前一个盒子底边与当前盒子顶边之间的距离,计算为 \baselineskip 的自然宽度减去 \prevdepth（即上一个盒子的深度），再减去当前盒子的高度；
- 若该距离至少为\lineskiplimit，则以算得的距离作为自然尺寸、并取 \baselineskip 的拉伸量与收缩量来添加胶；
- 否则添加\lineskip 胶。
- \prevdepth 被设为当前项的深度。

有两种例外情形：在一个线段（rule）之前和之后都不添加行间胶；而且 \unvbox 或 \unvcopy 命令不会更新 \prevdepth。在线段之后，\prevdepth 被设为 -1000pt，以此阻止行间胶。

无论盒子之间是否插入了额外的胶，上述过程都会照常进行。因此在垂直模式下，盒子之间的一个跳跃不会影响按基线距离算出的距离，从而也不会影响 baselineskip 胶的量。对于在段落内部用 \vadjust 添加的胶，情况同样如此。

```
\baselineskip=10pt \lineskiplimit=2pt \lineskip=2pt
\setbox0=\vbox{\hbox{\vrule depth4pt}
               \hbox{\vrule height 3pt}}
\showbox0
得到
\box0=
\vbox(10.0+0.0)x0.4
.\hbox(0.0+4.0)x0.4
..\rule(*+4.0)x0.4
.\glue(\baselineskip) 3.0
.\hbox(3.0+0.0)x0.4
..\rule(3.0+*)x0
把各盒子摆到彼此相距 \lineskiplimit 以内，即
\setbox0\vbox{\hbox{\vrule depth4pt}
              \hbox{\vrule height 5pt}}
\showbox0
得到
\box0=
\vbox(11.0+0.0)x0.4
.\hbox(0.0+4.0)x0.4
..\rule(*+4.0)x0.4
.\glue(\lineskip) 2.0
.\hbox(5.0+0.0)x0.4
..\rule(5.0+*)x0.4
这里插入的是 \lineskip 胶，而不是通常的 \baselineskip 胶。
```

plain TEX 的默认值为

```
\lineskiplimit=0pt lineskip=1pt
```

因此，当盒子开始相互接触时，它们会被挪开 1 个 point。

15.2 盒子的感知深度

行间胶的决策过程把\prevdepth 用作垂直表上前一个盒子的感知深度。\prevdepth 参数只能在垂直模式中使用。

\prevdepth 被设为加入垂直表的盒子的深度，但它不受 \unvbox 或 \unvcopy 影响。在 \hrule 之后它被设为 -1000pt，以阻止在下一个盒子之前添加行间胶。

在垂直表的开头，\prevdepth 被设为 -1000pt；但在 \halign 及其所包含的 \noalign 代码中除外，那里它从周围的表沿用而来。在对齐结束时，由最后一个对齐行设定的 \prevdepth 值会被带到外层表中。

要想只阻止一次行间胶，只需改变 \prevdepth 即可。

```
\def\nointerlineskip{\prevdepth=-1000pt}
```

\offinterlineskip 宏要激烈得多：它从被调用的那一刻起阻止所有行间胶；若它用在一个段落内部，则从该段落的开头起阻止。它的定义是

```
\baselineskip=-1000pt \lineskip=0pt
\lineskiplimit\maxdimen
```

其中第二行才是关键：它使 TEX 总是添加 \lineskip 胶（其值为零）。此后对 \baselineskip 的设置就不再起作用了。

\offinterlineskip 宏在对齐排版中有重要应用（见第 25 章）。

通过设置

```
\lineskiplimit=-\maxdimen
```

你可以强制 TEX 总是套用 \baselineskip，而不管这是否会让盒子彼此靠得太近，甚至是否会使它们重叠。

15.3 术语

在热金属活字排版时代，某个特定字体的所有字母都放在同一尺寸的 ‘字身’ 上。因此每一行铅字都有相同的高度和深度，所得的基线间距也就是该字号下某个合适的值。如果由于某种原因需要更大的间距（对此的讨论见 [52]），就会插入铅条。这额外的距离称为 ‘leading’（读作 ‘ledding’）。

到了照相排版时代，基线间距有时被称为 ‘film transport’，这一术语就变得含混起来，‘leading’ 一词也被用来指基线间距。这种混乱在 TEX 中也存在：参数 \baselineskip 规定基线间距，但在跟踪输出中（见上面的例子），为使基线间距等于 \baselineskip 而插入的胶也叫做 \baselineskip。

15.4 补充说明

一般来说，对于超过一页的文档，全文保持相同的基线间距是可取的。但对于单页文档，你可以给 baselineskip 增加拉伸性，例如当文本需要上下两端对齐时。

只想增大某一对行之间的距离，可以用 \vadjust。该命令的参数是垂直内容，它会被插入到垂直表中、紧接在给出本命令那一行之后。例如，本段的第二行就含有命令\vadjust{\kern2pt} 。

行距不能在一个段落的中途改变，因为实际使用的 \baselineskip 值，是段落最终被断行并加入主垂直表那一刻的当前值。\lineskip 与 \lineskiplimit 也是如此。

plain TEX 宏\openup 把 \baselineskip、\lineskip 和 \lineskiplimit 都增加该宏参数指定的量。实际上，这会把行距增大这个量，而无论这些行距是由 \baselineskip 还是 \lineskip 所决定的。

第 16 章

段落的开始

在段落开始时，TEX 会插入一个垂直跳跃，作为与上一段的间隔；同时插入一个水平跳跃，作为当前段落的缩进。本章解释这一确切的动作序列，并讨论如何改变 TEX 的这些决定。

`\indent` 切换到水平模式，并插入一个宽度为 `\parindent` 的盒子。
`\noindent` 以一个空水平表切换到水平模式。
`\parskip` 段落开始时，加到周围垂直表中的胶的量。plain TEX 默认值：`0pt plus 1pt`。
`\parindent` 加到段落前面的缩进盒子的尺寸。plain TEX 默认值： `20pt`。
`\everypar` 插入到段落文本前面的记号表；
`\leavevmode` 在必要时切换到水平模式的宏。

16.1 段落何时开始

每当 TEX 从垂直模式切换到（不受限的）水平模式时，它就开始一个段落。这一切换可以由 `\indent` 和 `\noindent` 这两个命令之一来实现，例如

```
{\bf And now~\dots}
\vskip3pt
\noindent It's~\dots
```

或由任何⟨horizontal command⟩来实现。水平命令包括字符、行内公式和水平跳跃，但不包括盒子。请看下面的例子。字符 'I' 就是一个水平命令：

```
\vskip3pt
It's~\dots
```

单个 `$` 也是一个水平命令：

```
$x$ is supposed~\dots
```

控制序列 `\hskip` 是一个水平命令：

```
\hskip .5\hsize Long indentation~\dots
```

水平命令的完整清单见第 60 页。

当在垂直模式中识别到一个水平命令时，TEX 会先执行一次 `\indent` 命令（以及与之相关的所有动作；见下文），然后再重新考察那个水平命令，这一次真正执行它。

16.2 段落开始时发生了什么

\indent 与\noindent 命令会引发段落开始。\indent 命令既可以由用户或宏显式给出，也可以在垂直模式中出现⟨horizontal command⟩ 时由 TEX 自动插入；而 \noindent 命令只能显式给出。

遇到这两个命令之一后，除非 TEX 正处于内部垂直模式且该垂直表为空（例如在 \vbox 或 \vtop 的开头），否则会把\parskip 胶附加到周围的垂直表上。随后 TEX 以一个空的水平表切换到不受限水平模式。若是 \indent（它也可以是隐式插入的），则在水平表的开头放置一个宽度为\parindent 的空 \hbox；而在 \noindent 之后不插入缩进盒子。

随后把\everypar 这个⟨token parameter⟩ 的内容插入输入（其应用见下文）。之后，页面构造器开始工作（见第 27 章）。注意这是在水平模式中发生的：目的是把 \parskip 胶移到当前页上。

如果在 TEX 已处于水平模式时给出 \indent 命令，缩进盒子同样会被插入。这其实没什么用。

16.3 杂注

16.3.1 用一个盒子开始段落

\hbox 并不意味着进入水平模式，因此试图用一个盒子来开始段落，例如

```
\hbox to 0cm{\hss$\bullet$\hskip1em}Text ....
```

会使盒子后面的文字落到盒子下面一行。必须显式地切换到水平模式，例如用 \noindent 或 \leavevmode。后者是用 \unhbox 定义的，而 \unhbox 是一个水平命令。

16.3.2 用一个组开始段落

如果段落的第一个⟨horizontal command⟩ 被花括号括起，则 \everypar 会在该组内部求值。这可能产生出乎意料的结果。请看下面的例子：

```
\everypar={\setbox0=\vbox\bgroup\def\par{\egroup}}
{\bf Start} a paragraph ... \par
```

启动本段的那个⟨horizontal command⟩ 是字符 'S'，因此在插入 \everypar 之后，输入本质上是

```
{\bf \indent\setbox0=\vbox\bgroup
    \def\par{\egroup}Start} a paragraph ... \par
```

这等价于

```
{\bf \setbox0=\vbox{Start} a paragraph ... \par
```

其效果与原先的意图大相径庭。此外，TEX 很可能会在一个组内部结束作业。

16.4 例子

16.4.1 可拉伸的缩进

考虑到 \parindent 是一个⟨dimen⟩ 而不是⟨glue⟩，你无法声明

```
\parindent=1cm plus 1fil
```

来在段落开头得到可变的缩进。这个问题可以通过放入

```
\everypar={\nobreak\hskip 1cm plus 1fil\relax}
```

`\nobreak` 用于（在少数情况下）防止在这个可拉伸的胶处断行。

16.4.2 抑制缩进

在段落开头向水平表中插入 `{\setbox0=\lastbox}` 即可去除缩进：缩进就是一个盒子，它可以通过 `\lastbox` 取得。对它赋值实际上就把它从表中移除了。

然而，这一命令序列必须在 TEX 已经切换到水平模式的那一刻插入，所以把这些命令显式地放在段落第一个⟨horizontal command⟩ 之前并不奏效。插入 `\everypar` 记号的那个时刻是更好的选择：指定

```
\everypar={{\setbox0=\lastbox}}
```

就会得到不缩进的段落，即使 `\parindent` 并不为零。

16.4.3 一种缩进方案

上面让 `\everypar` 移除缩进盒子的想法，可以用到一种系统化的缩进处理方法中：用两个条件语句

```
\newif\ifNeedIndent %as a rule
\newif\ifneedindent %special cases
```

来控制段落通常是否应当缩进，以及在特殊情形下是否需要缩进。本节取自 [8]。

我们设定一个固定的 `\everypar`：

```
\everypar={\ControlledIndentation}
```

它在某些情形下执行宏 `\RemoveIndentation`

```
\def\RemoveIndentation{{\setbox0=\lastbox}}
```

`\ControlledIndentation` 的实现如下：

```
\def\ControlledIndentation
   {\ifNeedIndent \ifneedindent
                  \else \RemoveIndentation\needindenttrue \fi
    \else \ifneedindent \needindentfalse
          \else         \RemoveIndentation
    \fi   \fi}
```

为了在整个文档范围内规范缩进，用户只需一次性地（例如）指定

```
\NeedIndenttrue
```

来表明原则上所有段落都应当缩进。诸如 `\section` 之类的宏随后可以在个别情形下阻止缩进：

```
\def\section#1{ ... \needindentfalse}
```

16.4.4 段间距方案

如前所述，用 `\everypar` 控制缩进的做法，可以推广到段间距。

段落之间的可见空白可以由 `\parskip` 参数产生；但一旦把这个参数设为某个值，就很难在某些地方优雅地阻止段间距。通常，环境和章节标题上下方的空白应当

能独立于段间距来指定。本节勾勒了一种做法：在某些构造的正上方和正下方把 \parskip 设为零，而用 \everypar 恢复原先的值。本节取自 [9]。

首先，这里有两个工具。控制序列 \csarg 只在其他宏内部使用；典型的调用形如

```
\csarg\vskip{#1Parskip}
```

其定义如下：

```
\def\csarg#1#2{\expandafter#1\csname#2\endcsname}
```

接下来是 \vskip 的一个推广：如果前一个胶项更大，宏 \vspace 就不放置它的参数；否则它会消除前面的胶，再放置其参数。

```
\newskip\tempskipa
\def\vspace#1{\tempskipa=#1\relax
    \ifvmode \ifdim\tempskipa<\lastskip
             \else \vskip-\lastskip \vskip\tempskipa \fi
    \else    \vskip\tempskipa \fi}
```

现在假定任何带有上下空白的构造 foo,其开头和结尾分别是宏调用\StartEnvironment{foo} 和\EndEnvironment{foo} 。进一步假定这个环境对应三个胶寄存器:\fooStartskip（环境上方的胶）、\fooParskip（环境内部的段间距），以及 \fooEndskip（环境下方的胶）。

为了恢复段间距的值，需要一个条件语句和一个胶寄存器：

```
\newskip\TempParskip \newif\ifParskipNeedsRestoring
```

环境的起始宏与结束宏的基本序列于是为

```
\TempParskip=\parskip\parskip=0cm\relax
\ParskipNeedsRestoringtrue
```

现在实现如下：

```
\def\StartEnvironment#1{\csarg\vspace{#1Startskip}
    \begingroup % make changes local
    \csarg\TempParskip{#1Parskip} \parskip=0cm\relax
    \ParskipNeedsRestoringtrue}
\def\EndEnvironment#1{\csarg\vspace{#1Endskip}
    \endgroup % restore global values
    \ifParskipNeedsRestoring
    \else \TempParskip=\parskip \parskip=0cm\relax
          \ParskipNeedsRestoringtrue
    \fi}
```

\EndEnvironment 宏需要略加说明：如果一个环境用在另一个环境内部，并且它出现在该环境的第一个段落之前，那么外层环境的段间距值已经被保存了。因此在这种情况下无需再做其他动作。

注意这两个宏都以一个垂直跳跃开头。这可以避免 \begingroup 与 \endgroup 语句出现在段落中。

现在来到要点：如有必要，\everypar 会恢复段间距的值。

```
\everypar={\ControlledIndentation\ControlledParskip}
\def\ControlledParskip
   {\ifParskipNeedsRestoring
       \parskip=\TempParskip \ParskipNeedsRestoringfalse
    \fi}
```

第 17 章

段落的结束

TEX 结束段落的机制巧妙而高效。本章解释这一机制、\par 在其中所起的作用，并给出若干实用说明。

\par 结束一个段落并进入垂直模式。
\endgraf \par 的同义词：\let\endgraf=\par
\parfillskip 放在段落最后一个元素与行末之间的胶。plain TEX 默认值： 0pt plus 1fil。

17.1 段落结束的方式

段落由原语 \par 命令终止；它可以由用户显式键入（或由宏展开插入）：

```
... last words.\par
A new paragraph ...
```

它也可以由 TEX 的输入处理器通过一个空行隐式生成（见第 2 章）：

```
... last words.

A new paragraph ...
```

\par 还可能因为在不受限水平模式中出现了一个⟨vertical command⟩ 而被插入：

```
... last words.\vskip6pt
A new paragraph ...
```

此外，若在 \vbox、\insert 或 \output 内部的水平模式中遇到一个右花括号，段落也会结束。

遇到 \par 命令后，TEX 进入垂直模式并启动页面构造器（见第 192 页）。如果这个 \par 是因为水平模式中出现了一个垂直命令而插入的，那么随后会重新考察那个垂直命令。\par 本身不插入任何垂直胶或罚值。\par 命令还会清空前一段落形状参数（见第 18 章）。

17.1.1 \par 命令与 \par 记号

必须区分 \par 记号与那个原语 \par 命令——后者是该记号最初的含义。当输入处理器看到一个空行，或执行处理器在水平模式中发现一个⟨vertical command⟩ 时，就会插入 \par 记号；而真正闭合段落的是 \par 命令。把记号与命令解耦，是在段落中实现特殊效果的重要手段（例子见第 5 章和第 9 章）。

17.1.2 段落填充：\parfillskip

在段落的最后一个元素之后，TEX 会隐式地插入相当于

```
\unskip \penalty10000 \hskip\parfillskip
```

\unskip 的作用是去掉段落末尾任何多余的胶，例如当 \par 是由输入处理器插入时，由行末产生的那个空格。例如：

```
end.

\noindent Begin
```

所得的记号为

```
end. \par Begin
```

把 \par 插入的那个序列加进来之后，就变成

```
end. \unskip\penalty10000\hskip ...
```

这又得到

```
end.\penalty ...
```

在 plain TEX 中,\parfillskip 是一阶无限的(0pt plus 1fil),因此用\hfil$\bullet$\par 结束段落时,会在最后一个单词与行末之间的正中得到一个项目符号;而用\hfill$\bullet$\par 时它会右对齐。

17.2 杂注

17.2.1 同时结束段落和组

如果段落是在一个组里排版的，可能需要确保结束该段落的那个 \par 发生在组内部。影响段落排版的参数，例如 \leftskip 和 \baselineskip，只有在段落完成时才被查看。因此若用下面这种方式结束段落

```
... last words.}\par
```

就会使用组外、而不是组内生效的那些值。更好的结束段落的方式是

```
... last words.\par}
```

或

```
... last words.\medskip}
```

在第二个例子中，垂直命令 \medskip 会促使 \par 记号被插入。

17.2.2 用 \hfill\break 结束段落

序列\hfill\break 是在段落内部强制换行的一种办法。不过，若你用它来结束一个段落，很可能会得到一条 Underfull \hbox 错误。奇怪的是，这个 underfull 盒子并不是那条被断开的行 – 毕竟它是被填满的 – ，而是它后面那个完全空的盒子(实际上它里面确实含有 \leftskip 和 \rightskip)。究竟发生了什么？段落以下面的内容结束：

```
\hfill\break\par
```

它变成

```
\hfill\break\unskip\nobreak\hskip\parfillskip
```

\unskip 找不到前面的胶，所以 \break 后面跟着一个罚值项和一个胶项；在 \break 处选定断行之后，这两项都会消失。然而，TEX 已经决定应当多出一行，即一个\hbox to \hsize。而没有任何东西可以填充它，于是就产生了一个 underfull 盒子。

17.2.3 用线段结束段落

关于用线段引线（rule leaders）而非默认的 \parfillskip 空白来结束段落，见第 84 页。

17.2.4 段落之间不允许分页

\par 命令不在垂直表中插入任何胶；所以在如下序列中

```
 ... last words.\par \nobreak \medskip
\noindent First words ...
```

段落之间不会发生分页。所生成的垂直表为

```
\hbox(6.94444+0.0)x ...        % last line of paragraph
\penalty 10000                 % \nobreak
\glue 6.0 plus 2.0 minus 2.0   % \medskip
\glue(\parskip) 0.0 plus 1.0   % \parskip
\glue(\baselineskip) 5.05556   % interline glue
\hbox(6.94444+0.0)x ...        % first line of paragraph
```

TEX 不会在 \medskip 上方断开这个垂直表，因为那个罚值禁止这样做；它也不会在任何其他地方断开，因为只有在某个胶前面有一个不可丢弃的项时，才能在该胶处断开。

17.2.5 有限的 \parfillskip

在 plain TEX 中，\parfillskip 有一个（一阶）无限拉伸分量。于是段落最后一行中的所有其他胶都会按自然宽度设定。如果 \parfillskip 只有有限的（甚至可能为零的）拉伸量，那么其他胶就会被拉伸或收缩。一个段落若其最后行是这种情形，则其中的展示公式将由 \abovedisplayskip 与 \belowdisplayskip 环绕，即使本应使用 \abovedisplayshortskip 胶。

这样做的原因是：胶的设定在一定程度上依赖于具体机器，这类过程应当排除在 TEX 的全局决策之外。

17.2.6 对不缩进段落的一个预防措施

如果你排版的文本，其段落缩进和段落间空白都为零，那么当上一段的最后一行几乎或完全右对齐时，新段落的开头可能难以辨认，这就有风险。一个合理的预防措施是把 \parfillskip 设为，例如

```
 \parfillskip=1cm plus 1fil
```

而不是通常的 0cm plus 1fil。

另一方面，你也许会被 [46] 说服，认为段落总是应当缩进。

第 18 章

段落形状

本章讨论影响段落形状的参数和命令。

\parindent 加到段落前面的缩进盒子的宽度。plain TEX 默认值： 20pt。
\hsize 排版一个段落时所用的行宽。plain TEX 默认值： 6.5in。
\leftskip 放在段落所有各行左侧的胶。
\rightskip 放在段落所有各行右侧的胶。
\hangindent 若为正，表示从左页边起的缩进量；若为负，其绝对值是从右页边起的缩进量。
\hangafter 若为正，表示开始缩进之前的行数；若为负，其绝对值表示从段落第一行起缩进的行数。默认值： 1。
\parshape 用于一般段落形状的命令。

18.1 文本行的宽度

当 TEX 吸收完一个段落时，它就形成了一个水平表；该表以一个缩进盒子开头，以 \parfillskip 胶结尾。随后这个表被断成长度为 \hsize 的若干行。段落的每一行左右两端都用一定量的胶（即 \leftskip 与 \rightskip）填充，这些胶在凑成 \hsize 时都被计入。

\leftskip 与 \rightskip 的值在线断行算法中被计入。因此 plain TEX 的\raggedright 宏以及 LATEX 的 'flushleft' 环境的要点在于：它们把 \rightskip 设为零加某个拉伸量。

命令 \parshape 与 \hangindent 也影响行宽。它们的做法是先改变 \hsize，然后平移容纳各行的盒子。

18.2 形状参数

18.2.1 悬挂缩进

一种简单而常见的段落形状，是让开头或末尾若干行缩进。TEX 可以用两个参数实现这种形状：\hangafter 与 \hangindent。二者都可取正值和负值。

\hangindent 控制缩进的量：

- \hangindent > 0：段落从左页边缩进这个量。
- \hangindent < 0：段落从右页边缩进该量的绝对值。

例如（设 \parindent=0pt），

```
a a a a a a a a a a a a ...

\hangindent=10pt
a a a a a a a a a a a a ...

\hangindent=-10pt
a a a a a a a a a a a a ...
```

得到

a a a a a
a a a a a
a a …
a a a a a
a a a a
a a a …
a a a a a
a a a a
a a a …

\hangindent 的默认值是 0pt。

\hangafter 参数决定有多少行要缩进：

- \hangafter ≥ 0：在这么多行之后，其余各行将被缩进；换句话说，从段落开头起的这么多行不缩进。
- \hangafter < 0：其绝对值表示从段落开头起要缩进的行数。

例如，

```
a a a a a a a a a a a a ...

\hangindent=10pt \hangafter=2
a a a a a a a a a a a a ...

\hangindent=10pt \hangafter=-2
a a a a a a a a a a a a ...
```

a a a a a
a a a a a
a a …
a a a a a
a a a a a
a a …
a a a a
a a a a
a a a a …

\hangafter 的默认值是 1。

两个参数都可取正负，于是就有四种悬挂缩进的方式。向页边方向悬挂缩进（'outdent'）见下文。

悬挂缩进的实现方式如下。对于需要缩进的行，悬挂缩进量从 \hsize 中减去；段落被断成若干水平盒子之后，应在左侧缩进的行再向右平移。

尺寸为 \parindent 的常规缩进不受悬挂缩进影响。因此，若不想要额外的缩进，就应当用 \noindent 显式地开始这个带悬挂缩进的段落。

\hangindent 和 \hangafter 的默认值在每次 \par 命令之后都会恢复。

18.2.2 一般段落形状：\parshape

使用\parshape 可以实现相当一般的段落形状。用这个命令可以指定一个段落前 n 行的行宽和缩进。因此该命令带 $2n+1$ 个参数：行数 n，随后是 n 对缩进和行宽。

\parshape⟨equals⟩ $n\ i_1\ \ell_1\ \ldots\ i_n\ \ell_n$

如果后面的段落超过 n 行，则最后一行的规格会重复使用。若不足 n 行，剩余的规格就被忽略。默认值（理所当然）是 \parshape = 0。

若同时指定了 \parshape 和 \hangindent，则 \parshape 优先。当 \parshape 生效时，常规的 \parindent、\leftskip 和 \rightskip 仍然被遵守。和 \hangindent、\hangafter、\looseness（见第 19 章）一样，\parshape 参数在一次 \par 命令之后被清空。由于每个空行都会产生一个 \par 记号，所以不要在段落形状（或悬挂缩进）的声明与随后的段落之间留下空行。控制序列\parshape 是一个⟨internal integer⟩：它的值就是设定它时所用的行数 n。

18.3 杂注

18.3.1 居中的最后一行

\leftskip 和 \rightskip 取相等的拉伸量和收缩量，就得到每行都居中的文本。要让段落的第一行和最后一行也正确居中，必须把 \parindent 和 \parfillskip 设为零。不过这样一来页边是参差不齐的。\leftskip 与 \rightskip 的一个出人意料的用法是得到页边对齐而最后一行居中的段落。

```
\leftskip=0cm plus 0.5fil \rightskip=0cm plus -0.5fil
\parfillskip=0cm plus 1fil
```

对段落中除最后一行外的所有行，各拉伸分量加起来为零，所以插入的 \leftskip 和 \rightskip 都是零。而在最后一行上，\parfillskip 又加上了plus 1fil 的拉伸量；因此该行左右两端各共有plus 0.5fil 的拉伸量。

本来，若指定下面这样就是错误的：

```
\leftskip=0cm plus 0.5fil \rightskip=0cm minus 0.5fil
```

TEX 会就此报错：它抱怨出现了 '无限收缩'。

不仅让最后一行居中，还要让段落第一行也居中，可以通过下面这些参数设置来实现

```
\parindent=0pt \everypar{\hskip 0pt plus -1fil}
\leftskip=0pt plus .5fil
\rightskip=0pt plus -.5fil
```

这一次，由 \everypar 插入的一个水平跳跃与 \leftskip 相结合，使段落第一行左右两侧具有相同的拉伸量。

18.3.2 向页边方向缩进

假设你想在段落开头两行之后，让其后各行向左页边方向悬挂缩进 1cm。指定\hangindent=-1cm 将从右页边起悬挂缩进 1 厘米，所以必须另想办法。下面这段可以做到：

```
 \leftskip=-1cm \hangindent=1cm \hangafter=-2
```

这样做唯一的问题是：段落结束后需要重置 leftskip。恰当地重定义 \par 就能消除这个不便：

```
\def\hangintomargin{\bgroup
    \leftskip=-1cm \hangindent=1cm \hangafter=-2
    \def\par{\endgraf\egroup}}
```

\par 的重定义在这里只作用于那个需要向页边外突出的段落。

另一个优雅的方案使用 \parshape：

```
\dimen0=\hsize \advance\dimen0 by 1cm
\parshape=3        % three lines:
    0cm\hsize      % first  line specification
    0cm\hsize      % second line specification
    -1cm\dimen0    % third  line specification
```

18.3.3 把段落悬挂在某个对象旁

LATEX 格式有一个宏 \@hangfrom，用来让一段文字悬挂在某个对象（通常是一个盒子或一小段文字）旁边。

Example 本段就是下面定义的 \hangfrom 宏的一个例子。在 LaTeX 文档样式中，\@hangfrom 宏（与这个类似）被用于多行章节标题。

那么，考虑宏 \hangfrom：

```
\def\hangfrom#1{\def\hangobject{#1}\setbox0=\hbox{\hangobject}%
    \hangindent \wd0 \noindent \hangobject \ignorespaces}
```

由于默认的 \hangafter=1，这会产生一行宽度为 \hsize 的行，此后段落的其余各行将从左侧缩进 \hangobject 的宽度。

18.3.4 实现悬挂缩进的另一种方法

悬挂缩进也可以通过左页边平移与向外突出相结合来实现。例如，项目列表就可以这样实现：

```
\newdimen\listindent
\def\itemize{\begingroup
    \advance\leftskip by \listindent
    \parindent=-\listindent}
\def\stopitemize{\par\endgroup}
\def\item#1{\par\leavevmode
    \hbox to \listindent{#1\hfil}\ignorespaces
    }
```

如果一个条目要包含多个段落，其实现可以是

```
\newdimen\listindent \newdimen\listparindent
\def\itemize{\begingroup
    \advance\leftskip by \listindent
    \parindent=\listparindent}
\def\stopitemize{\par\endgroup}
\def\item#1{\par\noindent
    \hbox to 0cm{\kern-\listindent #1\hfil}\ignorespaces
    }
```

```
\itemize\item{1.}First item\par
Is two paragraphs long.
\item{2.}Second item.\stopitemize
```

得到

1. First item

   Is two paragraphs long.

2. Second item.

18.3.5 悬挂缩进与 \leftskip 平移的对比

从上面的例子看，悬挂缩进与修改 \leftskip、\rightskip 似乎可以互换。它们确实可以，但只在一定程度上如此。

把一个段落的 \leftskip 设为某个正值，意味着 \hsize 保持不变，但每一行都以一个胶项开头。而悬挂缩进的实现方式是：对需要悬挂的行减小 \hsize 值，然后在周围的垂直表中水平平移那些已经排好的水平盒子。

两种方法的差异主要体现在：当改变 \leftskip 时，展示公式不会被平移。关于页边平移如何影响引线，例子见第 9 章。

18.3.6 更多例子

段落形状（通过各种手段实现）的更多例子见 [10]。该文章中的一个例子见第 57 页。

第 19 章

断行

本章讨论断行，以及 TEX 用来决定如何把段落断成各行、或在哪里分页的 'badness'（劣度）概念。本章还讨论影响断行代价的各种罚值，以及断字（连字符）。分页见第 27 章。

`\penalty` 指定不在此处断开的可取程度。

`\linepenalty` 与每次断行相关联的罚值。plain TEX 默认值： `10`。

`\hyphenpenalty` 一般情形下，在一个 discretionary 项处断开所关联的罚值。plain TEX 默认值： `50`。

`\exhyphenpenalty` 在断行前文本为空这种特殊情形下，在一个 discretionary 项处断开水平行的罚值。plain TEX 默认值： `50`。

`\adjdemerits` 相邻两行视觉上不协调时的罚值。plain TEX 默认值： `10 000`。

`\doublehyphendemerits` 相邻多行都以连字符结尾时的罚值。plain TEX 默认值： `10 000`。

`\finalhyphendemerits` 当段落倒数第二行以连字符结尾时附加的罚值。plain TEX 默认值： `5000`。

`\allowbreak` 通过插入 `\penalty0` 来创建断点的宏。

`\pretolerance` 不做断字时一个段落的容差值。plain TEX 默认值： `100`。

`\tolerance` 做断字时段落中各行的容差值。plain TEX 默认值： `200`。

`\emergencystretch`（仅 TEX3）段落各行中假定的额外拉伸量。

`\looseness` 本段应当比理想情况下多出的行数。

`\prevgraf` 最近一次加入垂直表的那个段落的行数。

`\discretionary` 指定在断行处如何拆分一个字符序列。

`\-` 自由断字连字符；等价于`\discretionary{-}{}{}` 。

`\hyphenchar` 某个字体中连字符的编号。

`\defaulthyphenchar` 装入一个字体时 `\hyphenchar` 的值。plain TEX 默认值：`` `\- ``。

`\uchyph` 取正值则允许对以大写字母开头的单词断字。plain TEX 默认值： `1`。

`\lefthyphenmin`（仅 TEX3）断字处之前最少保留的字符数。plain TEX 默认值：`2`。

`\righthyphenmin`（仅 TEX3）断字处之后最少保留的字符数。plain TEX 默认值：`3`。

`\patterns` 为 `\language` 的当前值定义一组断字模式；只允许在 IniTEX 中使用。

`\hyphenation` 为 `\language` 的当前值定义断字例外词表。

`\language` 选择一组断字模式与例外词表。

`\setlanguage` 重置当前语言。

19.1 段落断行代价的计算

段落被断开时，要使与之相关的不良度（*demerits*） d 最小。一个段落的不良度总量，是各行不良度之和，再加上可能的某些额外罚值。把段落作为整体来考虑、而不是逐行断开，可以得到更好的断行效果：TEX 可以在段落开头选一行略欠美观的行，以避免后面出现更大的麻烦。

对每一行，不良度由把该行拉伸或收缩到断点时的 *badness*（劣度） b，以及与该断点相关的罚值 p 计算得出。劣度不允许超过某个规定的容差。

除了断开各行所产生的不良度之外，TEX 还会对各行之间的组合方式赋予不良度；见下文。

TEX 段落断行算法的实现见 [27]。

19.1.1 劣度

根据一行中所含的拉伸量或收缩量，与实际取用的拉伸量或收缩量之比，可以算出在某点断行的 'badness'（劣度）。这个劣度是断行过程中的一个重要因素。劣度的计算公式见第 77 页。

本章只在断行的语境下讨论劣度。垂直表被拉伸或收缩时也会计算劣度（见第 27 章）。

描述劣度时使用如下术语：

紧 (3) 指任何被收缩、且劣度 $b \geq 13$ 的行，即至少用掉了其一半收缩量的行（计算见第 77 页）。

适中 (2) 指任何劣度 $b \leq 12$ 的行。

松 (1) 指任何被拉伸、且劣度 $b \geq 13$ 的行，即至少用掉了其一半拉伸量的行。

很松 (0) 指任何被拉伸、且劣度 $b \geq 100$ 的行，即用掉了其全部拉伸量甚至更多的行。请记住：胶可以拉伸，但收缩不能超过其允许的收缩量。

这些编号用于跟踪输出（第 34 章），也用于下面这个定义：若相邻两行的分类相差超过 1，就称这两行视觉上不协调。与此相关的 `\adjdemerits` 参数见下文。

超满的水平盒子和垂直盒子，如果其超出宽度或高度分别小于 `\hfuzz` 或 `\vfuzz`，就会被忽略；如果劣度小于 `\hbadness` 或 `\vbadness`，也不会报告（见第 5 章）。

19.1.2 罚值与其他断点位置

水平表中可以在以下位置断行：

1. 在一个罚值处。罚值就是在该处断行的 '美学代价'。负罚值被视为奖励。罚值为 10 000 或更大则禁止断开；罚值为 −10 000 或更小则强制断开。
   连续放置多个罚值，等价于只放其中值最小的那一个，因为它是断行的最佳候选。
   水平模式中的罚值由用户（或用户宏）插入。唯一的例外是在 `\parfillskip` 胶之前插入的那个 `\nobreak`。
2. 在一个胶处，前提是它不是数学公式的一部分，且它前面有一个不可丢弃的项（见第 6 章）。在胶处断开没有关联的罚值。
   关于不可丢弃前驱项的条件是必要的：否则就可能在两段胶之间断开，这会使段落的边缘参差不齐。
3. 在一个 kern 处，前提是它不是数学公式的一部分，且后面紧跟着胶。在 kern 处断开没有关联的罚值。

4. 在一个 math-off 处，前提是其后紧跟着胶。由于 math-off（和 math-on）起 kern 的作用（见第 23 章），这与上一种情形非常相似。在 math-off 处断开没有关联的罚值。
5. 在一个 discretionary 断点处。其罚值是 `\hyphenpenalty` 或 `\exhyphenpenalty`。这在下文讨论。

断点之后的任何可丢弃内容 – 胶、kern、math-on/off 以及罚值 – 都会被丢弃。如果认为在胶（kern、math-on/off）处的断行发生在该胶项的前端，那就意味着那段胶在断行处消失了。

19.1.3 不良度

一行的不良度由该行的劣度以及（若有的话）罚值算得。设 l 为`\linepenalty` 的值，b 为该行的劣度，p 为断点处的罚值；则不良度 d由下式给出：

$$d = \begin{cases} (l+b)^2 + p^2 & \text{若 } 0 \le p < 10\,000 \\ (l+b)^2 - p^2 & \text{若 } -10\,000 < p < 0 \\ (l+b)^2 & \text{若 } p \le -10\,000 \end{cases}$$

这个公式以及劣度公式，在 [27] 中都被称为 ‘相当任意’，但实践表明它们能给出好的结果。

一个段落的不良度是各行不良度之和，再加上

- 对任意两条视觉上不协调的相邻行（见上文），加`\adjdemerits`；
- 对任意两条都以连字符结尾的相邻行，加`\doublehyphendemerits`；
- 若段落倒数第二行以连字符结尾，加`\finalhyphendemerits`。

在段落开头，TEX 会假设前面有一条 ‘适中’ 的行。因此若第一行是 ‘很松’ 的，就会加上 `\adjdemerits`。此外，段落的最后一行通常也是 ‘适中’ 的 – 由于 `\parfillskip` 中的无限拉伸，所有空格都按自然宽度设定 – ；因此若前一行是 ‘很松’ 的，就会加上 `\adjdemerits`。

注意：选定断行处的罚值，其权重大约与该行的劣度相当，所以它们可以相对小一些。不过，那三个额外的不良度参数，必须达到罚值和劣度平方的量级，才能同样举足轻重。

19.1.4 段落的行数

当一个段落完成后（或在展示公式之前部分完成时），变量`\prevgraf` 记录该段落的行数。通过给这个变量赋值 – 由于它是一个⟨special integer⟩，这种赋值自动是全局的 – ，可以影响 TEX 的决策过程。这在与悬挂缩进或 `\parshape` 规格配合时可能有用（见第 18 章）。

断行过程对最终行数也有一些直接影响。其中一个因素是 `\linepenalty`，它被计入每一行的不良度。增大行罚值，就可以让 TEX 把一个段落的行数减到最少。

偏离最优行数（即由最优断行方式所得的行数），可以由用户通过`\looseness` 参数来强制。这个参数在每次形状参数被清空时都会重置（见第 18 章），它表示当前段落应比最优情况多出几行。`\looseness` 取负值时，会试图把段落缩短该行数的绝对值。

在 `\looseness` 影响下拉长或缩短段落时，TEX 仍会遵守 `\pretolerance` 和 `\tolerance` 的值（见下文）。因此，TEX 只会在不超过这些参数的前提下，尽量拉长或缩短段落。

19.1.5 行与行之间

TEX 的段落机制把各行打包成水平盒子，再附加到周围的垂直表上。所得的垂直项序列于是就是一个不断重复的序列：

- 一个装着一行文本的盒子；
- 可能迁移过来的垂直内容（见第 62 页）；
- 一个罚值项，反映在该点分页的代价，通常就是 `\interlinepenalty`（见第 27 章）；
- 行间胶，它根据 `\prevdepth` 自动计算（见第 15 章）。

19.2 断行的过程

TEX 试图把段落断行，使每一行的劣度都不超过某个容差。若这样的方案不止一个，就取不良度最小的那个。

把`\tracingparagraphs` 设为正值，就可以让 TEX 在日志文件中报告段落机制的计算过程。有些 TEX 实现可能禁用了这个选项，以便让 TEX 运行得更快。

19.2.1 三遍

首先尝试把段落断成各行而不做断字，即不插入自由断字连字符。如果没有任何一行的劣度超过`\pretolerance`，这次尝试就成功了。

否则进行第二遍：插入 discretionary，并使用`\tolerance`。如果 `\pretolerance` 为负，则跳过第一遍。

如果前两遍都失败，还可以让 TEX 进行第三遍。若`\emergencystretch` 是一个正的长度，则在计算劣度和不良度时，TEX 会假定每行都有这么多额外拉伸量。于是那些只略微超过给定容差的方案现在也变得可行了。然而实际上并不存在大小为 `\emergencystretch` 的胶，所以仍可能出现 underfull 盒子消息。

19.2.2 容差值

TEX 排版一段文本时会有多大麻烦，部分取决于容差值。因此，对劣度数值在视觉上意味着什么有个概念是有益的。

对于被拉伸的行，劣度等于拉伸比的立方再乘以 100。因此劣度 800 意味着拉伸比是 2。如果空格（如十点 Computer Modern 字体那样）是

```
3.33pt plus 1.67pt minus 1.11pt
```

则劣度 800 意味着空格被拉伸到

$$3.33\mathtt{pt} + 2 \times 1.67\mathtt{pt} = 6.66\mathtt{pt}$$

即恰好拉伸到其自然尺寸的两倍。是否觉得太大，由你自己判断。

19.3 discretionary 项

一个 discretionary 项`\discretionary{..}{..}{..}` 标记单词可以断开的位置。它的三个参数每个都是一段⟨general text⟩（见第 36 章），依次为：

- 断前文本，附加到单词断点之前的部分上；
- 断后文本，加到单词断点之后部分的前面；

- 不中断文本，当单词不在该 discretionary 项处断开时使用。

例如：`ab\discretionary{g}{h}{cd}ef` 就是单词abcdef，但它可以断字为断前是 `abg`、断后是 `hef`。注意这里没有自动添加连字符。

这三段文本都可以包含任意种类的记号，但其中的原语命令和宏应当展开成盒子、kern 和字符。

19.3.1 连字符与 discretionary

在内部，TEX 在每个可以断开单词的位置插入相当于

```
\discretionary{\char\hyphenchar\font}{}{}
```

的内容。如果`\hyphenchar\font` 不在 0–255 范围内，或它在字体中的位置没有字符，就不插入这样的 discretionary。装入字体时，其 `\hyphenchar` 值被设为`\defaulthyphenchar`。此后可以再改变这个 `\hyphenchar` 值。

在 plain TEX 中，`\defaulthyphenchar` 的值是 `` `\- ``，因此对所有字体，字符 45（即 ASCII 连字符）就是断字连字符，除非另行指定。

原语命令`\-`（称为‘自由断字连字符’）`\-`等价于上面的`\discretionary{\char\hyphenchar\font}{}{}`。在这样的 discretionary 处断行——无论是 TEX 隐式插入的还是用户显式插入的——其代价都是`\hyphenpenalty`。

在不受限水平模式中，凡字符代码等于该字体 `\hyphenchar` 值的字符之后，都会自动插入一个空的 discretionary `\discretionary{}{}{}`，从而允许在该处断字。在这种断前文本为空的 discretionary 处断行，其罚值是`\exhyphenpenalty`，即‘显式连字符’罚值。

如果一个单词含有 discretionary 断点（例如因为有显式连字符），TEX 就不会再对它做进一步断字。人们用如下技巧解决由此产生的问题：

```
\def\={\penalty10000 \hskip0pt -\penalty0 \hskip0pt\relax}
... integro\=differential equations...
```

连字符前后的跳跃使 TEX 把这个复合词的前半和后半当作两个独立的单词；第一个跳跃之前的罚值则禁止在连字符之前断开。

19.3.2 discretionary 的例子

有些语言（如德语、荷兰语）的单词在断字时会改变拼写（德语：‘backen’ 变成 ‘bak-ken’；荷兰语：‘autootje’ 变成 ‘auto-tje’）。这个问题可以用 TEX 的 discretionary 解决。

例如，对德语（做法受 [36] 启发）：

```
\catcode`\"=\active
\def"#1{\ifx#1k\discretionary{k-}{k}{ck}\fi}
```

这就使用户可以写成 `ba"ken`。

荷兰语还有一个问题，对此可以给出一个漂亮的系统化解法。变音符（荷兰语称为 ‘trema’）在断字处往往应当消失，例如 ‘na”apen’ 断字为 ‘na-apen’，‘onbe”invloedbaar’ 断字为 ‘onbe-invloedbaar’。一个解法（受 [5] 启发）是

```
\catcode`\"=\active
\def"#1{\ifx#1i\discretionary{-}{i}{\"\i}%
        \else  \discretionary{-}{#1}{\"#1}\fi}
```

这就使用户可以输入 `na"apen` 和 `onbe"invloedbaar`。

19.4 断字

TEX 的断字算法使用一组模式来判定一个候选断字的单词可以在哪些位置断开。断字中与这些模式相关的方面，见 the TEX book 附录 H；自动生成断字模式的方法见 [30]。人们也知道有人手工生成模式表，例如见 [28]。这种手工生成的表可能优于自动生成的表。

这里主要说明 TEX 如何判定一个单词是断字候选。这里的问题是：如何处理可能附着在单词上的标点以及引号之类的东西。此外，
em 隐式 kern（即因字体信息而插入的 kern）也必须妥善处理。

19.4.1 单词的开头

TEX 从胶项处（若不在数学模式中）开始寻找一个单词的*起始字母*：一个 `\lccode` 非零的字符，或以这种字符开头的连字。（大/小写代码见第 39 页。）在寻找这个起始字母时，TEX 会跳过任何隐式 kern、以及 `\lccode` 为零的字符（例如标点和引号），或以这种字符开头的连字。

如果没有找到合适的起始字母，也就是说找到的东西既不是字符也不是连字，TEX 就跳到下一个胶，重新开始这个算法。否则就收集一个试凑单词：它由随后所有与起始字母同字体、且 `\lccode` 非零的字符，或完全由这种字符构成的连字组成。字符与连字之间可以有隐式 kern。

如果起始字母所在字体的 `\hyphenchar` 值无效，或该字体中没有这个字符，就放弃对这个单词断字。如果起始字母是大写字母（即它不等于自己的 `\lccode`），除非`\uchyph` 为正，否则 TEX 放弃断字。这个参数在 plain TEX 中的默认值是 1，意味着首字母大写的单词也可以断字。

19.4.2 单词的结尾

试凑单词之后可以跟一些字符（来自另一个字体，或 `\lccode` 为零）、连字或隐式 kern。在这些项（若有）之后，必须是下列之一：

- 胶或一个显式 kern；
- 一个罚值；
- 一个 whatsit；
- 或一个 `\mark`、`\insert` 或 `\vadjust` 项。

特别地，如果单词后面跟着

- 盒子、
- 线段、
- 数学公式或
- 一个 discretionary 项，就不对它断字。

由于 discretionary 是插在字体的 `\hyphenchar` 之后的，一旦出现这个字符就会阻止进一步断字。此外，重音符号的放置是用显式 kern 实现的（见第 3 章），所以任何 `\accent` 命令都被视为单词的结尾，从而阻止对该单词断字。

19.4.3 TEX2 与 TEX3 的对比

TEX2 与 TEX3 在断字片段的处理上有一个明显差别。TEX2 坚持断前部分至少两个字符、断后部分至少三个字符长。从排版角度看这是一个合理的决定：这样一来，就不会有只有两个字符的单词片段孤零零地留在行末或行首。断前和断后都至少有三个字符。

在 TEX3 中引入了两个整数参数来控制这些片段的长度:\lefthyphenmin 和\righthyphenmin。在 TEX3 的 plain 格式中，它们分别设为 2 和 3。若这两个参数之和为 63 或更大，就禁止一切断字。

TEX3 的另一项新增内容——可以有多组断字模式——见下文。

19.4.4 模式与例外词

语句

\patterns⟨general text⟩
\hyphenation⟨general text⟩

都是⟨hyphenation assignment⟩，也就是⟨global assignment⟩。\patterns 命令用来指定一组断字模式，它只允许在 IniTEX 中使用（见第 33 章），而且所有模式都必须在排第一个段落之前指定。

断字例外词可以随时指定，例如用如下语句

```
\hyphenation{oxy-mo-ron gar-goyle}
```

来指定一个单词允许在哪些位置断字。后续的 \hyphenation 语句是累加的。

在 TEX3 中，这些语句被视为适用于 \language 参数的当前值所代表的语言。

19.4.5 切换断字模式

排段落时，TEX（第 3 版）可以使用若干组模式和断字例外词，最多支持 256 种语言。

如果给出了 \patterns 或 \hyphenation 命令（见上文），TEX 就把这些模式或例外词存到\language 参数的当前值之下。\patterns 命令只允许在 IniTEX 中使用，且模式必须在任何排版之前指定。而断字例外词可以累加指定，且不只限于 IniTEX。

除了用户可以设置的 \language 参数之外，TEX 内部还有一个‘当前语言’。每个段落开始时它都被设为零。对加入段落的每个字符，都把当前语言与 \language 的值比较；若二者不同，就向水平表添加一个 whatsit 元素，把当前语言重置为 \language 的值。

在段落开头，这个 whatsit 是在 \everypar 记号之后插入的，但 \lastbox 仍然能取到缩进盒子。

举例说，假设已经创建了一个格式，其中语言 0 是英语、语言 1 是荷兰语。如果用户不另行指定，就会使用英语断字；如果作业以

```
\language=1
```

开头，则整个文档都用荷兰语断字排版，因为 TEX 会在每个段落开头插入一个改变当前语言的命令。例如：

```
\language=1
T...
```

得到

```
.\hbox(0.0+0.0)x20.0           % indentation
.\setlanguage1 (hyphenmin 2,3) % language whatsit
.\tenrm T                      % start of text
```

也可以显式地插入这个 whatsit，而不改变\language 的值，方法是指定

\setlanguage⟨number⟩

。不过这几乎用不到。可能需要它的一种情形是：把一个水平盒子的内容解包成一个段落时——因为在这种盒子内部不会自动添加 whatsit，盒子内部也不会发生断字。水平盒子中文本的另一个问题见第 55 页。

第 20 章

间距

TEX 中通常的词间间隔是在字体信息中规定的，但用户可以覆盖它。本章解释 TEX 计算词间间隔所用的规则。

\ 控制空格。插入与空格记号相同量的空格，相当于 \spacefactor = 1000 时。
\spaceskip 非零时作为词间胶。
\xspaceskip 非零且 \spacefactor ≥ 2000 时作为词间胶。
\spacefactor 词间胶的拉伸（收缩）分量应当乘（除）以的那个比值的 1000 倍。
\sfcode 与某个字符相关联的 \spacefactor 值。
\frenchspacing 关闭标点后额外空格的宏。
\nonfrenchspacing 开启标点后额外空格的宏。

20.1 引言

在文本的单词之间，TEX 插入空格。这种空格有一个自然分量，外加拉伸和收缩分量，以便能够排版两端对齐（右对齐）的文本。在某些排版风格中，标点之后会有更多空格。本章讨论 TEX 实现这种效果的机制。

大致思路如下：

- 每个字符记号之后，\spacefactor 量都用该字符的空格因子代码更新。
- 插入空格时，其自然尺寸可以增大（若 \spacefactor ≥ 2000），一般地，其拉伸量乘以、收缩量除以 \spacefactor/1000。
- 还有其他规则，例如使...word.) And... 中的空格按句点、而不是按右括号来调整。

20.2 自动词间空格

在水平模式中，每个空格记号都会插入当前字体的词间胶，其拉伸和收缩分量都由 \fontdimen 参数决定。具体说，字体维度 2 是正常词间空格，维度 3 是词间空格的拉伸量，维度 4 是收缩量。字体维度 7 称为 '额外空格'；见下文（全部字体维度的清单见第 44 页）。

通常，（同一字体中）单词之间的所有空格都一视同仁。为了允许不同大小的空格 – 例如排版上模拟打字文档中标点后双空格的效果 – TEX 给每个字符都关联一个所谓的 'space factor'（空格因子）。

当一个字符被加入当前水平表时，该字符的空格因子代码（\sfcode）就被赋给空格因子\spacefactor。这条规则有两个例外：

- 当空格因子代码为零时，\spacefactor 不改变。这个机制使空格因子能透过括号之类保持不变；见第 20.5.3 节。
- 当最后一个字符的空格因子代码 >1000、而当前空格因子 <1000 时，空格因子变为 1000。这个机制防止首字母缩写后面出现拉长的空格；见第 20.5.2 节。

空格因子的最大值是 32 767。

词间空格的拉伸分量乘以空格因子除以 1000 的商；收缩分量则除以这个因子。当空格因子 ≥ 2000 时，额外空格（字体维度 7）会加到词间空格的自然分量上。

20.3 用户设定的词间空格

用户可以把\spaceskip 和\xspaceskip 设为非零值，来覆盖 \fontdimen 参数中所含的词间空格。若 \spaceskip 非零，就用它代替正常词间空格（\fontdimen2 加 \fontdimen3 减 \fontdimen4）；但若空格因子 ≥ 2000，则用非零的 \xspaceskip 作为词间空格。

若使用 \spaceskip，其拉伸分量和收缩分量分别乘以和除以 \spacefactor/1000。

注意：如果 \spaceskip 和 \xspaceskip 是用 em 定义的，它们会随字体变化。

设给定如下宏：

```
\def\a.{\vrule height10pt width4pt\spacefactor=1000\relax}
\def\b.{\vrule height10pt width4pt\spacefactor=3000\relax}
\def\c{\vrule height10pt width4pt\relax}
则
\vbox{
\fontdimen2\font=4pt % normal space
\fontdimen7\font=3pt % extra space
\a. \b. \c\par
% zero extra space
\fontdimen7\font=0pt
\a. \b. \c\par
% set \spaceskip for normal space
\spaceskip=2\fontdimen2\font
\a. \b. \c\par
% set \xspaceskip
\xspaceskip=2pt
\a. \b. \c\par
}
```

得到



在所有这些行中，胶都按自然宽度设定。第一行中，\b 之后很高的空格因子值使额外空格 \fontdimen7 被加上。若它为零（第二行），则不同空格因子值之间的唯一差别就是拉伸/收缩比。第三行中，对所有空格因子值都采用 \spaceskip。若 \xspaceskip 非零，则在空格因子取高值时（第四行）用它代替 \spaceskip。

20.4 控制空格与连字符（tie）

控制空格 \ 是一个水平命令，它插入一个空格，\ 其作用如同当前空格因子为 1000。但它并不改变 \spacefactor 的值。

控制空格主要有两个用途。第一，在控制序列之后使用它很方便：`\TeX\ is fun!`。第二，在 `\nonfrenchspacing`（见下文）生效时，它可以用在缩写之后。例如：

```
\hbox spread 9pt{\nonfrenchspacing
     The Reverend Dr. Drofnats}
```

得到

The Reverend Dr. Drofnats

而

```
\hbox spread 9pt{\nonfrenchspacing
     The Reverend Dr.\ Drofnats}
```

得到

The Reverend Dr. Drofnats

（这里用 `spread 9pt` 是为了让效果更明显。）

（plain 格式中的）活动字符波浪号 ~ 使用控制空格：它定义为

```
\catcode`\~=\active
\def~{\penalty10000\ }
```

这样一个活动波浪号称为一个 'tie'（不可断开空格）；它插入一个普通量的空格，并禁止在该空格处断开。

20.5 空格因子补遗

20.5.1 空格因子的赋值

某个字符的空格因子可以这样赋值：

`\sfcode`⟨8-bit number⟩⟨equals⟩⟨number⟩

IniTEX 把所有字符的空格因子代码都设为 1000，但大写字母除外，它们的空格因子代码是 999。然后 plain 格式给各种标点符号赋以大于 1000 的空格因子代码，例如`\sfcode `\.=3000`，它把句点之后的拉伸量和收缩量增至三倍。此外，对所有 ≥ 2000 的空格因子值都会加上额外空格；见上文。

20.5.2 标点

由于空格因子不能从低于 1000 的值直接跳到高于 1000 的值，大写字母之后的标点对词间空格的影响，就不同于小写字母之后标点的影响。

```
a% \sfcode`a=1000, space factor becomes 1000
.% \sfcode`.=3000, spacefactor becomes 3000
 % subsequent spaces will be increased.

A% \sfcode`A=999,  space factor becomes 999
.% \sfcode`.=3000, space factor becomes 1000
 % subsequent spaces will not be increased.
```

这样，首字母缩写就不会被误认为句末。如果大写字母确实是句子的结尾，例如

```
... and NASA.
```

则有几种解决办法：

```
... NASA\spacefactor=1000.
```

或者

```
... NASA\hbox{}.
```

这里利用了如下事实：盒子之后空格因子被设为 1000。LaTeX 宏 `\@` 等价于第一种办法。

plain 格式定义了两个宏，用来在均匀词间空格与标点后额外空格之间切换。宏 `\frenchspacing` 把所有标点的空格因子代码设为 1000；宏 `\nonfrenchspacing` 则把它们设为大于 1000 的值。

下面是 `plain.tex` 中的实际定义：

```
\def\frenchspacing{\sfcode`\.\@m \sfcode`\?\@m
  \sfcode`\!\@m \sfcode`\:\@m
  \sfcode`\;\@m \sfcode`\,\@m}
\def\nonfrenchspacing{\sfcode`\.3000 \sfcode`\?3000
  \sfcode`\!3000 \sfcode`\:2000
  \sfcode`\;1500 \sfcode`\,1250 }
```

其中

```
\mathchardef\@m=1000
```

是在 plain 格式中给出的。

法语式空格是一个有些争议的问题：《the TeX book》似乎把非法语式空格当作印刷的标准做法，但例如在 [14] 中可以看到：‘在正文中，所有句点之后都应使用行空格’。标点后加额外空格也许会被认为是 ‘打字机习惯’，但这并不完全正确。它过去比现在常见得多，而且也有反对它的理性论据：句末的句点（句号）——额外标点在那里最显眼——本身相当小，所以它上方自带一点额外的视觉空间。本书不在标点后使用额外空格。

20.5.3 其他非字母字符

空格因子代码取零值，会使既非字母又非标点的字符对空格因子 ‘透明’。

```
a% \sfcode`a=1000, space factor becomes 1000
.% \sfcode`.=3000, spacefactor becomes 3000
 % subsequent spaces will be increased.

a% \sfcode`a=1000, space factor becomes 1000
.% \sfcode`.=3000, space factor becomes 3000
)% \sfcode`)=0,    space factor stays 3000
 % subsequent spaces will be increased.
```

20.5.4 影响空格因子的其他因素

当 TeX 开始构造一个水平表时，空格因子为 1000，特别是在 `\indent`、`\noindent` 之后，以及展示公式刚结束时。它在 `\vrule`、一个重音符号或一个⟨box⟩（水平模式中）之后也是 1000，但不受 `\unhbox` 或 `\unhcopy` 命令影响。

在 `\valign` 的第一列中，周围水平表的空格因子被沿用；类似地，垂直对齐结束后，空格因子被设为最后一列所达到的值。

第 21 章

数学模式中的字符

在数学模式中，每个字符都通过它的 \mathcode 等信息来指定要访问某个字体中的哪个位置。对分隔符来说，情况要更复杂一些。本章解释数学码的概念，并说明 TEX 如何实现可变大小的分隔符。

\mathcode 一个字符的码，决定它在数学模式中如何处理。
\mathchar 一个数学字符的显式表示。
\mathchardef 把一个控制序列定义为某个数学字符码的同义词。
\delcode 指定一个字符如何用作分隔符的码。
\delimiter 一个分隔符的显式表示。
\delimiterfactor 1000 乘以 '被分隔公式应当被分隔符覆盖的比例'。plain TEX 默认值： 901
\delimitershortfall 被分隔公式中允许不被分隔符覆盖的那一部分的大小。plain TEX 默认值： 5pt
\nulldelimiterspace 空分隔符所占的宽度。plain TEX 默认值： 1.2pt
\left 把下一个字符用作开分隔符。
\right 把下一个字符用作闭分隔符。
\big 一行高的分隔符。
\Big 一行半高的分隔符。
\bigg 两行高的分隔符。
\Bigg 两行半高的分隔符。
\bigl etc. 左侧分隔符。
\bigm etc. 用作二元关系的分隔符。
\bigr etc. 右侧分隔符。
\radical 用来排根号之类符号的命令。
\mathaccent 在数学模式中放置一个重音符号。
\skewchar 后置重音符号在字体中的位置。
\defaultskewchar 装入字体时 \skewchar 的值。
\skew 显式地把重音符号在字符上方平移的宏。
\widehat 能容纳宽表达式的帽形重音符号。
\widetilde 能容纳宽表达式的波浪重音符号。

21.1 数学字符

256 个允许的字符代码中的每一个，都关联一个\mathcode；它可以这样赋值：

\mathcode⟨8-bit number⟩⟨equals⟩⟨15-bit number⟩

在数学模式中处理时，TEX 把类别 11 和 12 的所有字符，以及 \char 和 \chardef 字符，都替换为它们所关联的 mathcode。

这个 15 位数学码最方便用十六进制记为"xyzz ，其中

x ≤ 7 是类别（见第 167 页）；
y 是字体族编号（见第 22 章）；
zz 是该字符在字体中的位置。

数学码也可以直接用一个⟨math character⟩ 来指定，它可以是

- \mathchar⟨15-bit number⟩；
- 一个⟨mathchardef token⟩，即由
  \mathchardef⟨control sequence⟩⟨equals⟩⟨15-bit number⟩
  定义的控制序列；或
- 一个分隔符命令
  \delimiter⟨27-bit number⟩
  ，其最后 12 位被丢弃。

命令 \mathchar 与 \mathchardef 类似于文本模式中的 \char 和 \chardef。分隔符在下文讨论。一个⟨mathchardef token⟩ 甚至在数学模式之外也可以当作一个⟨number⟩ 使用。

在 IniTEX 中，所有字母得到 \mathcode"71zz ，所有数字得到"70zz ，其中"zz 是该字符在字体中的十六进制位置。因此，字母最初取自字体族 1（plain TEX 中是数学斜体），数字取自字体族 0（罗马体）。对所有其他字符，IniTEX 都赋以

\mathcode $x = x$,

从而把它们也归入字体族 0。

如果 mathcode 是"8000 ——它是最小的、不是⟨15-bit number⟩ 的整数——则该字符被当作一个具有原字符代码的活动字符。plain TEX 把空格、下划线和撇号的 \mathcode 都设为"8000 。

21.2 分隔符

在\left 和\right 命令之后，TEX 会寻找一个分隔符。一个分隔符要么是一个显式的 \delimiter 命令（或它的宏缩写），要么是一个具有非零分隔符码的字符。

\left 和 \right 命令隐式地圈出一个组，这个组被视为一个子公式。由于被圈住的公式可以任意大，寻找合适分隔符就是一件复杂的事：要在两种不同字体中查看各种变体、在一种字体中查看相互链接的变体链，以及用可重复的片段拼装出可伸缩的分隔符。

被\left...\right 圈住的组被当作独立子公式处理，这意味着该公式开头的上下标不被认为属于分隔符。例如，TEX 的处理如同\left(_2 等价于\left({}_2 。（\right 分隔符之后的下标是相对于该分隔符定位的。）

21.2.1 分隔符码

每个字符代码都对应一个分隔符码，由如下方式赋值：

\delcode⟨8-bit number⟩⟨equals⟩⟨24-bit number⟩

因此一个分隔符码由六个十六进制数字"uvvxyy 组成，其中

uvv 是分隔符的小号变体，xyy 是大号变体；
u、x 是各变体所在的字体族，vv、yy 是它们在那些字体中的位置。

分隔符码用于 \left 和 \right 命令之后。IniTEX 把所有分隔符码都设为 −1，除了 \delcode`.=0，它使句点成为一个空分隔符。在 plain TEX 中，分隔符通常有 u = 2 和 x = 3，即先试字体族 2；若找不到足够大的分隔符，再试字体族 3。

21.2.2 显式 \delimiter 命令

分隔符也可以用一个⟨27-bit number⟩ 显式表示，

```
\delimiter"tuvvxyy
```

其中 uvvxyy 如上所述是分隔符的小号和大号变体；额外的那位数字 t（它 < 8）表示类别（见第 167 页）。例如，宏 \langle 定义为

```
\def\langle{\delimiter "426830A }
```

这意味着它属于类别 4（开符号）。类似地，\rangle 属于类别 5（闭符号）；\uparrow 属于类别 3（关系符）。

在 \left 和 \right 之后 – 也就是当 TEX 正在寻找分隔符时 – 类别那位数字被忽略；否则 – 当 TEX 不在寻找分隔符时 – 最右边三位数字被忽略，剩下四位数字被当作一个 \mathchar；见上文。

21.2.3 寻找分隔符；后继字符

排一个分隔符是一件相当复杂的事。首先 TEX 确定要被覆盖的公式的尺寸 y，它是该公式高度与深度两者最大值的两倍。因此，如果公式本身不居中，排出来的分隔符可能看起来不是最佳。

分隔符的尺寸应至少为\delimiterfactor×y/1000，且至少为 y−\delimitershortfall。然后 TEX 先试小号变体；若它不令人满意（或分隔符的 uvv 部分是 000），就试大号变体。若试大号变体也不成功，TEX 就取这次搜索中遇到的最大分隔符；若根本没找到任何分隔符（当 xyy 部分也是 000 时可能发生），就取一个宽度为 \nulldelimiterspace 的空盒子。

考察一个变体依次意味着：

- 若当前样式（见第 166 页）是 scriptscriptstyle，就试该字体族的 \scriptscriptfont；
- 若当前样式是 scriptstyle 或更小，就试该字体族的 \scriptfont；
- 否则试该字体族的 \textfont。

plain 格式把 cmex10 字体放进字体族 3 的全部三种样式中。

在某个字体的某个位置寻找分隔符意味着：

- 若该字符足够大，就接受它；
- 若该字符可伸缩，就接受它；
- 否则，若该字符有后继字符——即它是同一字体中一串越来越大的分隔符链条中的一环——就试那个后继字符。

分隔符的后继字符和可伸缩性信息编码在该字体的度量文件中。一个可伸缩字符有一个顶部、一个底部、可能还有一个中部片段，以及一个直接重复在顶部片段下方、（若有中部片段）直接在底部片段上方的片段。

21.2.4 \big、\Big、\bigg 和 \Bigg 分隔符宏

为了能在\left...\right 上下文之外使用分隔符，或者指定一个不同于 TEX 自动选择尺寸的分隔符，有四个 'big' 分隔符宏：\big、\Big、\bigg 和 \Bigg。凡是能跟在 \left 或 \right 之后的东西，都可以配它们使用。

另外还有十二个宏（例如 \bigl、\bigm 和 \bigr）分别把这类分隔符强制用作开符号、二元关系和闭符号的上下文：

```
\def\bigl{\mathopen\big}
\def\bigm{\mathrel\big} \def\bigr{\mathclose\big}
```

这些 'big' 宏本身是把所请求的分隔符和一个空分隔符放在一个空垂直盒子的两侧：

```
\def\big#1{{\nulldelimiterspace=0pt \mathsurround=0pt
           \hbox{$\left#1\vbox to 8.5pt{}\right.$}}}
```

作为近似量度，Big 分隔符是 big 分隔符的一倍半大（11.5pt）；bigg 是两倍（14.5pt）；Bigg 是两倍半大（17.5pt）。

21.3 根号

根号（radical）是一个左分隔符与一个上方加线的数学表达式的组合。那个上方加线的表达式按周围样式的 cramped（紧缩）版本排版（见第 166 页）。

在 plain 格式和 Computer Modern 数学字体中，只有一种根号：平方根构造

```
\def\sqrt{\radical"270370 }
```

控制序列 \radical 后面跟一个⟨24-bit number⟩，如前所述，它指定左分隔符的小号和大号变体。把分隔符与横线接合起来的做法是：让分隔符有一个较大的深度，以及一个等于所需横线粗细的高度。这样横线就能放在当前基线上。分隔符与加了横线的表达式接合之后，整体再垂直平移，以达到通常的垂直居中（见第 23 章）。

21.4 数学重音符号

数学模式中的重音符号由

\mathaccent⟨15-bit number⟩⟨math field⟩

指定。把这个 15 位数记为"xyzz 时，只用字体族 y 和字符位置 zz：一个带重音的表达式起 \mathord 表达式的作用（见第 23 章）。

在数学模式中，整个表达式都可以加重音，而在文本模式中只能给字符加重音。因此在数学模式中重音可以叠加。不过，最上面那个重音在水平方向上可能（或者说很可能）位置不正确。为此 plain 格式有一个宏\skew，用来平移最上面的重音。它的定义是

```
\def\skew#1#2#3{{#2{#3\mkern#1mu}\mkern-#1mu}{}}
```

它的用法例如：

```
$\skew4\hat{\hat x}$
```

它给出 $\hat{\hat{x}}$.

为了把重音正确地放在单个字符上方，符号字体和扩展字体都有一个\skewchar：它是那个会增加带重音字符宽度的最大重音符号。任何重音的定位，都基于被加重音字符的宽度，再加上斜移字符。

Computer Modern 数学斜体字体和符号字体的斜移字符分别是字符"7F、‘ ’，和"30、‘0’。装入字体时，\skewchar 被赋为\defaultskewchar 的值。在 plain TEX 中这个值是 -1，所以通常字体没有 \skewchar。

数学重音可以自适应于被加重音表达式的大小：TEX 寻找重音后继字符的方式，与寻找分隔符后继字符的方式相同。在 Computer Modern 数学字体中，这个机制用于\widehat 和 \widetilde 宏。例如，

```
\widehat x, \widehat{xy}, \widehat{xyz}
```

分别给出

$\widehat{x}$, $\widehat{xy}$, $\widehat{xyz}$

。

第 22 章

公式中的字体

数学排版与文本排版不同：单一的当前字体是不够的。相反，TEX 使用若干个字体族，而每个字体族可以包含三种字体。本章解释字体族是如何组织的，以及 TEX 如何决定字符应取自哪个字体族。

`\fam` 当前字体族的编号。
`\newfam` 分配一个新的数学字体族。
`\textfont` 访问某个字体族的 textstyle（正文样式）字体。
`\scriptfont` 访问某个字体族的 scriptstyle（上下标样式）字体。
`\scriptscriptfont` 访问某个字体族的 scriptscriptstyle（二级上下标样式）字体。

22.1 确定数学模式中字符的字体

数学公式中的字符可以取自若干种不同的字体（或者更确切地说，字体族），而无需任何用户命令。例如，在 plain TEX 中，数学公式使用罗马字体、数学斜体字体、符号字体以及数学扩展字体。

为了确定一个字符应取自哪种字体，TEX 会考察公式中每个字符的 `\mathcode`（数学字符代码，见第 21 章）。`\mathcode` 是一个 15 位的数，形式为`"xyzz`，其中各位十六进制数字的含义如下：

x: 类别，
y: 字体族，
zz: 在字体中的位置。

一般而言，只有字体族决定字符应取自哪种字体。数学字符的类别主要用于控制间距以及排版的其他方面。典型的类别包括“关系符”“运算符”“定界符”。

类别 7 在这方面比较特殊：它被称为“可变字体族”。如果一个字符的 `\mathcode` 具有形式`"7yzz`，那么它取自字体族 y；除非参数 `\fam` 的值在 0–15 范围内，此时它取自字体族 `\fam`。

22.2 初始字体族设置

小写字母和大写字母都由 IniTEX 定义为具有数学代码`"71zz`，这意味着它们属于可变字体族，初始时取自字体族 1。由于 TEX 在公式开始时将 `fam=-1`，即一个无效值，字符确实取自字体族 1，在 plain TEX 中该字体族是数学斜体。

数字的数学代码为`"70zz`，因此它们初始时取自字体族 0，在 plain TEX 中即罗马字体。所有其他字符代码都由 IniTEX 赋予如下数学代码：

\mathcode $x = x$

，这将它们归入类别 0（普通字符）和字体族 0（在 plain TEX 中为罗马字体）。

在 plain TEX 中，诸如 \sl 之类的命令会同时设置一个字体和一个字体族：

```
\def\sl{\fam\slfam\tensl}
```

因此在公式中使用 \sl 会使所有字母、数字以及大写希腊字母变为斜体样式。

在大多数情况下，任何字体都可以分配给任何字体族，但 TEX 中有两个字体族具有特殊含义：即字体族 2 和 3。例如，它们的 \fontdimen 参数个数与通常的 7 个不同。字体族 2 需要 22 个参数，字体族 3 需要 13 个参数。这些参数在数学排版中的定位都有非常专门的含义。其含义将在下面解释，但完整内容请读者参阅 the TEX book 的附录 G。

22.3 字体族的定义

TEX 在数学模式中可以访问 16 个字体族；字体族编号为 0–15。当前字体族的编号记录在参数 \fam 中。

宏\newfam 给出一个未使用的字体族编号。该编号通过 \chardef 分配给相应的控制序列。

每个字体族可以有一个用于正文样式、上下标样式和二级上下标样式的字体。下面将解释 TEX 如何决定一个（子）公式应以何种样式排版。字体按如下方式分配给某个字体族：

```
\newfam\MyFam
\textfont\MyFam=\tfont \scriptfont\MyFam=\sfont
\scriptscriptfont\MyFam=\ssfont
```

分别用于字体族的正文、上下标和二级上下标字体。一般而言，不必填满一个字体族的全部三个成员（但字体族 3 例外）。如果 TEX 需要某个字体族成员中的字符，而该成员尚未填充，它会改用 \nullfont，这是一个没有字符（也没有.tfm 文件）的原始字体。

22.4 若干具体的字体更改

22.4.1 更改普通字符和大写希腊字母的字体

在 plain TEX 中，所有字母和大写希腊字母默认都属于类别 7（可变字体族），因此更改 \fam 会改变它们所取自的字体。例如

```
{\fam=9 x}
```

给出一个取自字体族 9 的 x。

大写希腊字母在 plain 格式中由 \mathchardef 语句定义为"70zz ，即可变字体族，初始时为罗马字体。因此，大写希腊字母也会随字体族而改变。

22.4.2 独立于文本字体地更改大写希腊字母

在 Computer Modern 字体布局中，大写希腊字母是罗马字体的一部分；参见第 244 页。因此，引入另一种文本字体（具有另一种布局）会改变大写希腊字母（甚至使其消失）。补救方法之一是引入一个新的字体族，将包含大写希腊字母的 cmr 字体置于其中。然后访问这些字符的控制序列必须重新定义：

```
\newfam\Kgreek
\textfont\Kgreek=cmr10 ...
\def\hex#1{\ifcase#10\or 1\or 2\or 3\or 4\or 5\or 6\or
    7\or 8\or 9\or A\or B\or C\or D\or E\or F\fi}
\mathchardef\Gamma="0\hex\Kgreek00 % was: "0100
\mathchardef\Beta ="0\hex\Kgreek01 % was: "0101
\mathchardef\Gamma ...
```

顺便注意，在 `\hex` 的定义中，`#1` 之后既没有空格也没有 `\relax` 标记。这意味着该宏只能以一个控制序列作为参数来调用。

22.4.3 更改小写希腊字母和数学符号的字体

小写希腊字母的数学代码为`"01zz`，这意味着它们始终取自数学斜体字体族。为了更改这一点，可以重新定义它们，例如`\mathchardef\alpha="710B`，使其成为可变字体族。plain TEX 中并未这样做，因为 Computer Modern 罗马字体没有小写希腊字母，尽管它有大写字母。

另一种方法是像`\mathchardef\alpha="0n0B` 这样重新定义它们，其中 `n` 是与数学斜体兼容的某个字体族的（十六进制）编号，该字体族例如包含粗体数学斜体字体。

22.5 杂项说明

22.5.1 公式中的新字体

在数学内部访问字体有两种方法。在 `\font\newfont=....` 之后，无法用 `$...{\newfont a}...$` 得到新字体的 'a'，因为 TEX 在数学模式中并不查看当前字体。可行的做法是

```
$ ... \hbox{\newfont a} ...$
```

但这使得新字体无法在上下标和二级上下标样式中使用。

正确的解决方法要多做一点工作：

```
\font\newtextfont=...
\font\newscriptfont=... \font\newsscriptfont=...
\newfam\newfontfam
\textfont\newfontfam=\newtextfont
\scriptfont\newfontfam=\newscriptfont
\scriptscriptfont\newfontfam=\newsscriptfont
\def\newfont{\newtextfont \fam=\newfontfam}
```

此后该字体即可在所有三种样式中这样使用：

```
$... {\newfont a_{b_c}} ...$
```

22.5.2 评估字体族

TEX 只有在整个公式结束时才会查看各个字体族的 `\textfont` 等成员中实际是什么。因此，在单个公式内部无法切换字体族中的字体。对于某些应用而言，16 个字体族的数量可能会显得不够。

第 23 章

数学排版

TEX 有两种数学模式：展示模式和非展示模式；四种样式：展示、正文、上下标和二级上下标样式；并且数学模式中的每个对象都属于八个类别之一。本章讨论这些概念。

`\everymath` 在非展示公式开头插入的记号列表。
`\everydisplay` 在展示公式开头插入的记号列表。
`\displaystyle` 选择数学排版的展示样式。
`\textstyle` 选择数学排版的正文样式。
`\scriptstyle` 选择数学排版的上下标样式。
`\scriptscriptstyle` 选择数学排版的二级上下标样式。
`\mathchoice` 为四种数学排版样式给出公式的四个变体。
`\mathord` 使随后的字符或子公式作为普通对象。
`\mathop` 使随后的字符或子公式作为大运算符。
`\mathbin` 使随后的字符或子公式作为二元运算。
`\mathrel` 使随后的字符或子公式作为关系符。
`\mathopen` 使随后的字符或子公式作为开符号。
`\mathclose` 使随后的字符或子公式作为闭符号。
`\mathpunct` 使随后的字符或子公式作为标点符号。
`\mathinner` 使随后的字符或子公式作为内部公式。
`\mathaccent` 在数学模式中放置重音符号。
`\vcenter` 构造一个竖直盒子，在数学轴上垂直居中。
`\limits` 在大运算符的上方和下方放置上下限。
`\nolimits` 将大运算符的上下限作为上下标表达式放置。
`\displaylimits` 恢复上下限的默认放置方式。
`\scriptspace` 上下标之后的额外间距。Plain TEX 默认值： `0.5pt`
`\nonscript` 如果下一个黏合项出现在 scriptstyle 或 scriptscriptstyle 中，则取消它。
`\mkern` 插入以 mu 单位度量的紧排。
`\mskip` 插入以 mu 单位度量的黏合。
`\muskip` 以 mu 单位度量的跳跃的前缀。
`\muskipdef` 将一个控制序列定义为某个 `\muskip` 寄存器的同义词。
`\newmuskip` 分配一个新的 muskip 寄存器。
`\thinmuskip` 少量 mu 黏合。
`\medmuskip` 中等量 mu 黏合。
`\thickmuskip` 大量 mu 黏合。
`\mathsurround` 在行间公式前后放置的紧排量。
`\over` 分式。

\atop 将对象上下叠放。
\above 指定分数线宽度的分式。
\overwithdelims 带定界符的分式。
\atopwithdelims 带定界符地将对象上下叠放。
\abovewithdelims 带定界符的广义分式。
\underline 在随后的⟨ 数学符号 ⟩ 或分组下方加下划线。
\overline 在随后的⟨ 数学符号 ⟩ 或分组上方加上划线。
\relpenalty 在未被包围在子公式中的二元关系符之后换行的惩罚。Plain TEX 默认值： 500
\binoppenalty 在未被包围在子公式中的二元运算符之后换行的惩罚。Plain TEX 默认值： 700
\allowbreak 创建断点的宏。

23.1 数学模式

当 TEX 在输入中遇到数学移位字符（类别 3）时，它进入数学模式。在这样一个开头的数学移位之后，它会（不展开地）查看下一个记号，以判断这是否是另一个数学移位。在后一种情况下，TEX 开始以展示数学模式处理，直到遇到成对的结束双数学移位：

```
.. $$ 展示公式 $$ ..
```

否则它以非展示数学模式开始处理一个行间公式：

```
.. $ 行间公式 $ ..
```

单个数学移位字符是一个⟨ 水平命令 ⟩。

例外：在受限水平模式中不能有展示公式，因此在 \hbox 内部，序列 $$ 是空数学公式，而不是展示公式的开始。

与两种数学模式相关联的是两个⟨ 记号参数 ⟩ 寄存器（另见第 14 章）：在行间公式开头插入\everymath 记号；在展示公式开头插入 \everydisplay 记号。展示数学将在下一章进一步讨论。

数学模式可以测试：\ifmmode 在展示和非展示数学模式中均为真，而 \ifinner 在非展示模式中为真，在展示模式中则不为真。

23.2 数学模式中的样式

数学公式以下列八种样式之一排版：

D 展示样式，
T 正文样式，
S 上下标样式，
SS 二级上下标样式，

以及这四种的四个“紧缩”变体 D'、T'、S'、SS'。紧缩样式的主要区别在于上标抬升的距离不如原始样式那么高。

23.2.1 上标与下标

如果前面的项是一个符号或分组，TEX 可以将一个符号或分组排版为前面符号或分组的上标（或下标），前提是前面的项尚未具有上标（下标）。上标（下标）由如下语法指定：

⟨ 上标 ⟩⟨ 数学场 ⟩

或

⟨ 下标 ⟩⟨ 数学场 ⟩

其中⟨ 上标 ⟩（⟨ 下标 ⟩）可以是类别 7（8）的字符，或是 `\let` 到此类字符的控制序列。plain 格式具有控制序列

```
\let\sp=^ \let\sb=_
```

作为隐式的上标和下标字符。

将上标（下标）表达式指定为空数学列表中的第一项，等价于将其指定为空表达式的上标（下标）。例如，

`$^{...}` 等价于 `${}^{...}`

在 TEX 的内部计算中，上标和下标表达式会通过`\scriptspace` 加宽；plain TEX 中该值为 `0.5pt`。

23.2.2 样式的选择

将四种样式 D、T、S 和 SS 排序，并将另外四种视为仅为变体，则数学模式的样式规则如下：

- 在任何样式中，上标和下标都取自下一个较小的样式。例外：在展示样式中它们以上标样式排出。
- 下标始终采用该样式的紧缩变体；上标仅在原始样式为紧缩时才紧缩。
- 在任何样式的 `{..\over..}` 公式中，分子和分母都取自下一个较小的样式。
- 分母始终采用紧缩样式；分子仅在原始样式为紧缩时才采用紧缩样式。
- `\sqrt` 或 `\overline` 之下的公式采用紧缩样式。

样式可以通过显式命令强制指定：即 `\displaystyle`、`\textstyle`、`\scriptstyle` 和 `\scriptscriptstyle`。

在展示样式和正文样式中，使用当前字体族的 `\textfont`；在 scriptstyle 中使用 `\scriptfont`；在 scriptscriptstyle 中使用 `\scriptscriptfont`。

原始命令

`\mathchoice{`D`}{`T`}{`S`}{`SS`}`

允许用户为公式的四种样式指定四个变体。TEX 会构造全部四个，并插入合适的那一个。

23.3 数学对象的类别

数学模式中的对象属于八个类别之一。根据类别不同，对象周围可能会有一定量的空白，或以某种方式被特殊处理。存在一些命令，可强制符号或符号序列表现为属于某一类别。在十六进制表示`"xyzz` 中，类别是⟨3 位数 ⟩ `x`。

下面是各类别以及强制归入这些类别的命令的列表。例子取自 plain 格式（参见从第 249 页开始的表格）。

1. 普通：小写希腊字母以及那些“只是符号”的符号；命令\mathord 强制归入此类。
2. 大运算符：积分号和求和号，以及诸如 \bigcap 或 \bigotimes 之类的“大”对象；命令\mathop 强制归入此类。作为大运算符的字符在竖直方向居中，并且它们在展示样式中的表现可能与在其他样式中不同；见下文。
3. 二元运算：加号和减号，以及诸如 \cap 或 \otimes 之类的符号；命令\mathbin 强制归入此类。
4. 关系符（也称二元关系）：等号、小于号、大于号，子集和超集符号，垂直、平行符号；命令\mathrel 强制归入此类。
5. 开符号：左花括号、方括号、圆括号、尖括号，地板、天花板符号；命令\mathopen 强制归入此类。
6. 闭符号：右花括号、方括号、圆括号、尖括号，地板、天花板符号；命令\mathclose 强制归入此类。
7. 标点:大多数标点符号,但：是关系符,\colon 是冒号标点;命令\mathpunct 强制归入此类。
8. 可变字体族：此类中的符号随 \fam 参数更改字体；在 plain TEX 中，大写希腊字母、普通字母和数字属于此类。

还有一个类别：内部子公式。没有字符可被指定为此类，但字符和子公式可通过\mathinner 强制归入此类。〈广义分式〉和\left...\right 分组都是内部公式。内部公式周围有一定空白；见下表。

除内部公式之外的其他子公式都被当作普通符号处理。特别地，花括号括起来的子公式是普通的：$a+b$ 看起来像 ‘$a+b$’，但 $a{+}b$ 看起来像 ‘$a+b$’。不过要注意，在 ${a+b}$ 中，整个子公式被当作一个普通符号，而不是它的各个成分；因此结果是 ‘$a+b$’。

23.4 大运算符及其上下限

Computer Modern 字体中的大运算符有两种尺寸：一种用于正文样式，一种用于展示样式。诸如 \sum 之类的控制序列只是通过 \mathchardef 定义为对应字体中的某个位置：

```
\mathchardef\sum="1350
```

但如果当前样式是展示样式，TEX 会查看该字符在字体中是否有后继字符。

正文样式中的大运算符表现得好像后面跟着 \nolimits，它把上下限作为运算符之后的上下标表达式放置：

$\sum_{k=1}^{\infty}$

在展示样式中，它们表现得好像后面跟着 \limits，把上下限放置在运算符的上方和下方：

$$\sum_{k=1}^{\infty}$$

后继字符机制（见第 159 页）让 TEX 在此处取该定界符的更大变体。

积分号在 plain TEX 中定义为

```
\mathchardef\intop="1352 \def\int{\intop\nolimits}
```

即使在展示样式中，这也把上下限放在运算符之后：

$$\int_0^\infty e^{-x^2}\,dx = \sqrt{\pi}/2$$

使用\limits\nolimits 或\nolimits\limits 时，最后一个指定优先；默认放置方式可通过\displaylimits 恢复。例如，

```
$ ... \sum\limits\displaylimits ... $
```

等价于

```
$ ... \sum ... $
```

而

```
$$ ... \sum\nolimits\displaylimits ... $$
```

等价于

```
$$ ... \sum ... $$
```

23.5 竖直居中：\vcenter

每个公式都有一条轴；对于行间公式，该轴大约位于周围文本 x 高度的一半处；其精确值是当前样式中字体族 2（符号字体）的 \fontdimen22。

分式中的分数线放置在轴上；大运算符、定界符和 \vcenter 盒子都以它为中心居中。

\vcenter 盒子是一种竖直盒子,其排布方式使其在数学轴上居中。可以为 \vcenter 盒子给出 spread 或 to 规格。

\vcenter 盒子只允许在数学模式中使用，它的行为与其他盒子不同；例如，它不能存入盒子寄存器。它算不上一个 ⟨ 盒子 ⟩。参见第 113 页的一个宏，可修复这一问题。

23.6 数学间距：mu 黏合

数学对象周围的间距以 mu 度量。一个 mu 是当前样式中字体族 2 字体的 \fontdimen6（即符号字体的“quad”值）的 1/18。

23.6.1 mu 黏合的分类

用户可以通过 \mkern 或 \mskip 指定 mu 间距，但大多数 mu 黏合是由 TEX 根据对象所属的类别（见上文）自动插入的。首先，这里有一些描述总体行为的经验法则。

- 在（二元）关系符周围插入 \thickmuskip（plain TEX 默认值：5mu plus 5mu），除非关系符前后是其他关系符或标点，或关系符后跟开符号、前跟闭符号。
- 在二元运算符周围放置\medmuskip （plain TEX 默认值：4mu plus 2mu minus 4mu）。
- \thinmuskip（plain TEX 默认值：3mu）跟在标点之后，并放置在内部对象周围，除非这些内部对象后跟闭符号或前跟开符号，或另一方是大运算符或二元关系符。
- 在开符号之后或闭符号之前不插入 mu 黏合，除非后者之前是标点；在标点之前也不插入 mu 黏合，除非前面的对象是标点或内部对象。

下表给出了数学对象之间 mu 黏合的完整定义。

	0: Ord	1: Op	2: Bin	3: Rel	4: Open	5: Close	6: Punct	Inner
0: Ord	0	1	(2)	(3)	0	0	0	(1)
1: Op	1	1	*	(3)	0	0	0	(1)
2: Bin	(2)	(2)	*	*	(2)	*	*	(2)
3: Rel	(3)	(3)	*	0	(2)	*	*	(2)
4: Open	0	0	*	0	0	0	0	0
5: Close	0	1	(2)	(3)	0	0	0	(1)
6: Punct	(1)	(1)	*	(1)	(1)	(1)	(1)	(1)
Inner	(1)	1	(2)	(3)	(1)	0	(1)	(1)

其中各符号的含义如下：

- 0，无间距；1，细间距；2，中等间距；3，粗间距；
- (·)：仅在正文和展示模式中插入，不在上下标或二级上下标模式中插入；
- 标有 * 的情形不会出现，因为如果 Bin 对象位于列表开头，或前面是 Bin、Op、Open、Punct、Rel，或后面是 Close、Punct、Rel，它会被转换为 Ord；此外，当 Rel 后跟 Close 或 Punct 时，也会被转换为 Ord。

可伸缩的 mu 黏合按照支配普通黏合的相同规则设置。但是，只有外层的 mu 黏合可以伸缩；任何被包围在分组中的 mu 黏合都按自然宽度设置。

23.6.2 数学胶（muskip）寄存器

与普通黏合一样，mu 黏合可以存入寄存器，即 \muskip 寄存器，TEX 中有 256 个。寄存器用

\muskip⟨8 位数 ⟩

表示，并可通过

\muskipdef⟨ 控制序列 ⟩⟨ 等于 ⟩⟨8 位数 ⟩

分配给一个控制序列；还有一个分配未使用寄存器的宏：

\newmuskip⟨ 控制序列 ⟩

mu 黏合的运算与普通黏合一样；见第 8 章。

23.6.3 数学模式中的其他间距

在数学模式中，空格记号被忽略；不过，在 plain TEX 中，空格字符的数学代码是"8000 ，因此如果把它的类别设为“字母”或“其他字符”，它在数学模式中会表现得像一个活动字符。另见第 158 页。

数学模式中允许的黏合为 ⟨ 数学跳跃 ⟩ 类型，它要么是⟨ 水平跳跃 ⟩（见第 6 章），要么是 \mskip⟨mu 黏合 ⟩。数学模式中的引导线（leaders）可用⟨ 数学跳跃 ⟩ 指定。

如果一个黏合项前面有\nonscript，且它出现在 scriptstyle 或 scriptscriptstyle 中，则被取消。

控制空格在数学模式中的作用与在水平模式中相同。

行间公式周围环绕着大小为\mathsurround 的紧排，即所谓的“进入数学”和“离开数学”项。如果离开数学紧排之后跟有黏合，则可在该紧排之前断行。

23.7 广义分式

类似分式的对象可用六个⟨ 广义分式 ⟩ 类型的原始命令排版。每个命令都取前后两个子公式，将它们上下叠放，必要时加分数线和定界符。

`\over` 是普通分式；线宽为扩展字体的 `\fontdimen8`：

```
$\pi\over2$
```

给出 ‘$\frac{\pi}{2}$’

`\atop` 等价于线宽为零的分式：

```
$\pi\atop2$
```

给出 ‘$\pi\atop 2$’

`\above`⟨ 尺寸 ⟩ 显式指定分数线的粗细：

```
$\pi\above 1pt 2$
```

给出 ‘$\frac{\pi}{2}$’

这三个命令各有对应的 `\...withdelims` 变体让用户为表达式指定定界符。例如，最一般的命令（其余五个命令都可以用它定义）是

`\abovewithdelims`⟨ 定界符 $_1$⟩⟨ 定界符 $_2$⟩⟨ 尺寸 ⟩。

这些广义分式中的定界符不会随所包围的表达式增高：在展示模式中，所取定界符的高度至少为 `\fontdimen20`；否则它必须至少为 `\fontdimen21` 高。这些尺寸取自当前样式中字体族 2（符号字体）的字体。

控制序列 `\over`、`\atop` 和 `\above` 是原始命令，尽管它们本可以用 `\...withdelims..`（即带两个空定界符）定义。由于这些隐含的包围空定界符，在这些简单广义分式前后各有一个大小为 `\nulldelimiterspace` 的紧排。

23.8 下划线与上划线

原始命令`\underline` 和`\overline` 取一个⟨ 数学场 ⟩ 参数，即一个⟨ 数学符号 ⟩ 或一个分组，并在其下方或上方画线。结果是一个“Under”或“Over”原子，被附加到当前数学列表。线宽是扩展字体的字体尺寸 8，它同时决定了线与⟨ 数学场 ⟩ 之间的间隙。

plain TEX 中还有各种其他 `\over...` 和 `\under...` 命令；它们都是使用 TEX `\halign` 命令的宏。

23.9 数学公式中的断行

行间公式可以在关系符和二元运算符之后断开。相应的惩罚分别是 `\relpenalty` 和 `\binoppenalty`。但是，只有当这些符号没有被花括号包围时，TEX 才会在其后断行。其他断点可用 `\allowbreak` 创建，它是 `\penalty0` 的缩写。

在水平或竖直模式中，连续放置两个惩罚等价于只放最小的那个；但在数学模式中，放在断点处（即关系符或二元运算符之后）的惩罚 – – 会有效地用新惩罚替换旧惩罚。

23.10 字体族 2 和 3 的字体尺寸

如果字体用于文本模式，TEX 会查看它的前 7 个 `\fontdimen` 参数（见第 44 页），例如用于控制间距。但在数学中，需要更多的字体尺寸。TEX 会查看字体族 2 中字体的前 22 个参数，以及字体族 3 中字体的前 13 个参数，以控制数学排版的各个方面。下面两个小节转引自 [3]。

23.10.1 符号字体属性

字体族 2 中字体的属性主要规定分式各部分、下标、上标等的初始竖直位置。应用这些属性所确定的位置还可能因其他条件而被进一步修改，例如存在分数线时。

一个文本字体尺寸，即第 6 号 quad，决定 mu 黏合的大小；见上文。

分式分子属性：广义分式中分子基线相对主基线的最小上移量，

1. num1：用于展示样式；
2. num2：当有分数线时，用于正文或更小样式；
3. num3：当无分数线时，用于正文或更小样式。

分式分母属性：广义分式中分母基线相对主基线的最小下移量，

1. denom1：用于展示样式；
2. denom2：用于正文或更小样式。

上标属性：上标基线相对主基线的最小上移量，

1. sup1：用于展示样式；
2. sup2：用于正文或更小样式，非紧缩；
3. sup3：用于正文或更小样式，紧缩。

下标属性：下标基线相对主基线的最小下移量，

1. sub1：当无上标时；
2. sub2：当有上标时。

上下标调整属性：仅用于非字形（即复合）对象。

1. sup_drop：上标基线低于核（nucleus）顶部的最大距离；
2. sub_drop：下标基线低于核底部的最小距离。

定界符跨度属性：包围广义分式的定界符的高加深度，

1. delim1：用于展示样式；
2. delim2：用于正文或更小样式。

一个用途广泛的参数：数学轴的高度，

1. axis_height：分数线、大定界符以及大多数运算符和关系符的中心高出基线的高度。此位置用于竖直居中操作。

23.10.2 扩展字体属性

字体族 3 中字体的属性主要规定大运算符上下限的设置方式。

第一个参数，即第 8 号 default_rule_thickness（默认线宽），用途广泛。它是上划线、下划线、根号延伸线（平方根）以及分数线所用线的粗细。各种间隙也用此尺寸规定：分数线与分子、分母之间，对象与由下划线、上划线或根号画出的线之间，以及上标底部与下标顶部之间。

大运算符周围的最小间隙如下：

1. big_op_spacing1：上限基线与大运算符顶部之间的最小间隙；见下文。
2. big_op_spacing2：大运算符底部与下限顶部之间的最小间隙。
3. big_op_spacing3：上限基线与大运算符顶部之间的最小间隙，计及上限的深度；见下文。
4. big_op_spacing4：大运算符底部与下限顶部之间的最小间隙，计及下限的高度；见下文。
5. big_op_spacing5：大运算符上方限之上或下方限之下的间隙。

运算符上方最终的间隙取参数 7，以及参数 11 减去上限深度两者中的较大值。运算符下方最终的间隙取参数 10，以及参数 12 减去下限高度两者中的较大值。

23.10.3 例子：下标下移

下标的位置取决于是否存在上标；例如

$$X_1 + Y_1^2 = 1$$

如果希望它看起来像

$$X_1 + Y_1^2 = 1,$$

只需指定

```
\fontdimen16\textfont2=3pt \fontdimen17\textfont2=3pt
```

这使两种情况下的下标下移量相等。

第 24 章

展示公式

展示公式单独排在一行上，通常位于段落中的某处。本章解释周围的空白（上下以及左右）是如何计算的。

`\abovedisplayskip` `\belowdisplayskip` 展示公式上方/下方的黏合。Plain TEX 默认值： `12pt plus 3pt minus 9pt`

`\abovedisplayshortskip` `\belowdisplayshortskip` 若展示公式前一行较短，展示公式上方/下方的黏合。Plain TEX 默认值分别为： `0pt plus 3pt` 和 `7pt plus 3pt minus 4pt`。

`\predisplaypenalty` `\postdisplaypenalty` 放在展示公式上方/下方竖直列表中的惩罚。Plain TEX 默认值分别为： `10 000` 和 `0`。

`\displayindent` 展示公式所在居中盒子因悬挂缩进而缩进的距离。

`\displaywidth` 展示公式所在居中盒子的宽度。

`\predisplaysize` 展示公式前一行的有效宽度。

`\everydisplay` 在展示公式开头插入的记号列表。

`\eqno` 在展示公式中放置右侧公式编号。

`\leqno` 在展示公式中放置左侧公式编号。

24.1 展示公式

当 TEX 连续遇到两个数学移位字符（类别 3 的字符，在 plain TEX 中为 `$`）时，它开始构造展示公式。另一对这样的字符（后面可能 跟一个可选空格）标志展示公式结束。

数学移位是一个⟨ 水平命令 ⟩，但展示公式只允许在非受限水平模式中使用（在受限水平模式中，`$$` 是空数学公式）。不过，展示公式本身是在周围（可能是内部的）竖直模式中开始的，以便计算诸如 `\prevgraf` 之类的量；展示公式的结果被附加到竖直列表。

展示公式上方的段落部分作为独立段落断行（但 `\prevgraf` 会被带过来；见下文），段落的剩余部分则从空列表开始、以 `\spacefactor` 等于 1000 排版。展示公式之后的段落部分不会插入 `\everypar` 记号，也不插入 `\parskip` 黏合。

就在展示公式的开头，插入`\everydisplay` 记号列表（但在计算 `\displayindent`、`\displaywidth` 和 `\predisplaysize` 之后）。使用 `\everydisplay` 的例子见第 177 页。

页面构造器在展示公式之前（但在插入 `\everydisplay` 记号之后）以及展示公式结束之后被调用。

数学排版的“展示样式”已在第 22 章讨论。

24.2 段落中的展示公式

展示公式在段落中的位置可能受悬挂缩进或 \parshape 规格影响。为此，TEX 使用 \prevgraf 参数（见第 18 章），并表现得好像展示公式占三行深。

如果展示公式开始时 \prevgraf 的值为 n – 即展示公式上方有 n 行文本 – 则段落恢复时 \prevgraf 被设为 $n+3$。展示公式仿佛占据第 $n+1$、$n+2$ 和 $n+3$ 行。展示公式的偏移和行宽就是第 $n+2$ 行所应有的偏移和行宽。

展示公式的偏移记录在 \displayindent；行宽记录在 \displaywidth。这些参数（以及下面解释的 \predisplaysize）在扫描到 $$ 后立即设置。通常它们分别等于零和 \hsize。用户可以更改这些参数的值；TEX 会在展示公式的数学列表处理完毕后所取的值。

注意，展示公式是竖直材料，因此不受 \leftskip 和 \rightskip 设置的影响。

24.3 展示公式周围的竖直材料

在竖直列表中，展示公式之前是

- 大小为 \predisplaypenalty 的惩罚（plain TEX 默认值 10 000），以及
- 大小为 \abovedisplayskip 或 \abovedisplayshortskip 的黏合；当 \leqno 公式编号单独占一行时（见下文），此黏合被省略。

展示公式之后是

- 大小为 \postdisplaypenalty 的惩罚（默认值 0），以及可能的
- 大小为 \belowdisplayskip 或 \belowdisplayshortskip 的黏合；当 \eqno 公式编号单独占一行时（见下文），此黏合被省略。

如果没有 \leqno 左侧公式编号，且展示公式上方段落的最后一行足够短，使得展示公式可以上提一点而不至于离该行太近，则采用黏合的“短”变体。为判断这一点，前一行的有效宽度保存在\predisplaysize 中。该值在扫描到展示公式开头的 $$ 后，连同上面解释的 \displaywidth 和 \displayindent 一起立即计算。

记住展示公式上方的段落部分已经断行，于是得到求最后一行有效宽度的方法如下。TEX 取列表的最后一个盒子（它是包含最后一行的水平盒子），并定位其中最后一个盒子的右边缘。于是 \predisplaysize 就是该最右边缘的位置，加上最后一行的任何偏移量，再加上当前字体的两个 em。

对此有两个例外。如果没有前一行，即展示公式开始了段落，或它跟在另一个展示公式之后，则 \predisplaysize 取为 −\maxdimen；如果最后一行中的黏合未按自然宽度设置（当 \parfillskip 只含有限伸缩时可能发生），则 \predisplaysize 取为 \maxdimen。最后一条的理由是，黏合设置略有机器依赖性，而这种依赖应排除在 TEX 的全局决策过程之外。

24.4 展示数学列表的黏合设置

展示公式必须放入 \displaywidth 中，但除公式外还可能有公式编号。公式与公式编号之间的最小间隔应为符号字体中的一个 em，即 \fontdimen6\textfont2。

如果公式连同任何公式编号和间隔都能放入 \displaywidth，则公式中的黏合按自然宽度设置。如果放不进去，但公式含有足够的收缩量，就将其收缩。否则 TEX 把公式编号单独放在一行，并调整公式中的黏合使其放入 \displaywidth。公式编号单独一行后，公式很可能就刚好能放入展示宽度；但是，如果公式非常长，它所在的盒子仍可能过满。TEX 从不打断展示公式。

24.5 展示公式的居中：偏移量

根据包含公式的盒子的宽度 – 它可能并未真正“包含”公式；可能过满 – TEX 尝试把公式在 \displaywidth 中居中，即不考虑公式编号。初始时计算一个偏移量，即 \displaywidth 与公式盒子宽度之差的一半。

但是，如果有一个不会单独成行的公式编号，且偏移量小于公式编号宽度的两倍，则重新计算偏移量。如果公式以黏合开头，新偏移量为零；否则它使公式盒子在公式编号所留出的空间中居中。

如果没有公式编号，或公式编号将单独成行，则公式盒子此时放置为向右偏移 \displayindent 加上上面算出的偏移量。

24.6 公式编号

用户可以通过在展示公式末尾加上

\eqno⟨ 数学模式材料 ⟩$$

来指定右侧公式编号，或用

\leqno⟨ 数学模式材料 ⟩$$

指定左侧公式编号。

24.6.1 普通公式编号

上面描述了当公式编号与公式同行时，TEX 如何根据展示公式和公式编号计算偏移量。

如果公式编号是 \leqno 编号，TEX 放置一个盒子，包含

- 公式编号，
- 大小为所算偏移量的紧排，以及
- 公式。

该盒子向右偏移 \displayindent。

如果公式编号是 \eqno 编号，TEX 放置一个盒子，包含

- 公式，
- 大小为所算偏移量的紧排，以及
- 公式编号。

该盒子向右偏移 \displayindent 加上所算偏移量。

24.6.2 单独成行的公式编号

由于展示公式可能变得相当大，TEX 可以（如上所述）决定把任何公式编号单独放在一行。左侧公式编号于是放在展示公式上方，在一个向右偏移 \displayindent 的盒子中；右侧公式编号放在展示公式下方，在一个盒子中，该盒子向右偏移 \displayindent 加 \displaywidth 减去公式编号盒子的宽度。

两种情况下，在公式编号盒子与公式之间都放置一个 10 000 的惩罚。

TEX 不在左侧公式编号上方或右侧公式编号下方放置额外黏合；此处 TEX 依赖于基线跳跃（baselineskip）机制。

24.7 非居中的展示公式

默认情况下，TEX 会将展示公式居中。要得到非居中的展示公式，需要一些宏技巧。

一种方法是编写宏 \DisplayEquation，其基本形式为

```
\def\DisplayEquation#1{%
    \par \vskip\abovedisplayskip
    \hbox{\kern\parindent$\displaystyle#1$}
    \vskip\belowdisplayskip \noindent}
```

但更理想的是，只需写

```
$$ ... \eqno ... $$
```

就让它成为左对齐的展示公式。

利用 \everydisplay 记号列表可以实现上述想法。基本思路是写

```
\everydisplay{\IndentedDisplay}
\def\IndentedDisplay#1$${ ...
```

这样宏 \IndentedDisplay 就会接收到公式，包括任何公式编号。第一步是在存在公式编号时将其提取出来。这创造性地使用了带定界符的宏参数：

```
\def\ExtractEqNo#1\eqno#2\eqno#3\relax
   {\def\Equation{#1}\def\EqNo{#2}}
\def\IndentedDisplay#1$${%
    \ExtractEqNo#1\eqno\eqno\relax
```

接下来应在可用空间 \displaywidth 中排公式：

```
    \hbox to \displaywidth
        {\kern\parindent
         $\displaystyle\Equation$\hfil$\EqNo$}$$
    }
```

注意该宏以结尾的 $$ 结束，以配对导致插入 \everydisplay 记号的开头美元符。这也意味着包含展示材料的盒子会自动被 \abovedisplayskip 和 \belowdisplayskip 黏合包围。本宏中无需任何地方使用 \displayindent，因为 TEX 自己会相应地偏移展示公式。

第 25 章

对齐

TEX 提供了一个用于制作表格的通用对齐机制。

\halign 水平对齐。
\valign 竖直对齐。
\omit 省略某个对齐条目所用的模板。
\span 连接两个相邻的对齐条目。
\multispan 连接若干相邻对齐条目的宏。
\tabskip \halign（\valign）各列（行）之间的黏合量。
\noalign 指定放在 \halign（\valign）各行（列）之间的竖直（水平）材料。
\cr 结束一行对齐。
\crcr 当一行对齐尚未被 \cr 结束时，结束它。
\everycr 在每个 \cr 或非冗余 \crcr 之后插入的记号列表。
\centering plain TEX 中用于使 \eqalign 和 \eqalignno 居中的黏合寄存器。值：0pt plus 1000pt minus 1000pt
\hideskip plain TEX 中使对齐条目不可见的黏合寄存器。值：-1000pt plus 1fill
\hidewidth 使前一个或后一个条目不可见的宏。

25.1 引言

TEX 有一个精致的对齐机制，它基于模板，每列或每行对应一个模板条目。模板可包含表条目中的任何共同元素，并且通常包含排版条目的指令。TEX 先计算所有条目的宽度（对 \halign）或高度（对 \valign）；然后它在每列（行）中使用该列（行）条目的最大宽度（高度）来排整个对齐。

25.2 水平对齐与竖直对齐

TEX 中的两个对齐命令是

\halign⟨ 盒子规格 ⟩{⟨ 对齐材料 ⟩}

用于列的水平对齐，以及

\valign⟨ 盒子规格 ⟩{⟨ 对齐材料 ⟩}

用于行的竖直对齐。\halign 是一个⟨ 竖直命令 ⟩，\valign 是一个⟨ 水平命令 ⟩。

花括号引入新的分组层级；它们可以是隐式的。

下面的讨论主要集中于水平对齐，但把“列”换成“行”（或反之），它同样适用于竖直对齐。

25.2.1 水平对齐：\halign

水平对齐产生一个水平盒子列表，即各行，它们被放置到周围的竖直列表上。对齐行加入竖直列表后，页面构造器被调用。对齐前的 \prevdepth 值用于第一行的基线跳跃；对齐之后 \prevdepth 被设为基于最后一行的值。

每个条目在自己的分组中、在受限水平模式中处理。

存在一类特殊的水平对齐：展示对齐，其指定形式为

```
$$⟨ 赋值 ⟩\halign⟨ 盒子规格 ⟩{...}⟨ 赋值 ⟩$$
```

此类对齐按 \displayindent 偏移（见第 24 章），并由 \abovedisplayskip 和 \belowdisplayskip 黏合包围。

25.2.2 竖直对齐：\valign

竖直对齐是“旋转”了的水平对齐：它们被放置到周围的水平列表上，产生一行列。\spacefactor 值的处理与水平对齐中 \prevdepth 的处理方式相同：对齐前的值用于第一列，最后一列之后达到的值用于对齐之后。列之间 \spacefactor 值为 1000。

每个条目在自己的分组中，在内部竖直模式中处理。

25.2.3 行间材料：\noalign

需要包含在对齐中、但不应被当作一个或一系列条目的材料，可由

```
\noalign⟨ 填充 ⟩{⟨ 竖直模式材料 ⟩}
```

（水平对齐）给出，或由

```
\noalign⟨ 填充 ⟩{⟨ 水平模式材料 ⟩}
```

（竖直对齐）给出。

例如

```
\noalign{\hrule}
```

用于在 \halign 的两行之间画一条水平线，以及

```
\noalign{\penalty100}
```

用于不鼓励在 \halign（\valign）的两行（列）之间分页（或断行）。

25.2.4 对齐的尺寸

⟨ 盒子规格 ⟩ 可用于给对齐指定预定尺寸：例如

```
\halign to \hsize{ ... }
```

对齐条目所含的黏合在此不起作用；所需的任何伸缩都取自 \tabskip 黏合。这在下面解释。

25.3 导言区

对齐中的每一行以 \cr 结束；第一行称为模板行。其形式为

u_1#v_1&...&u_n#v_n\cr

其中每个 u_i、v_i 是（可能为空的）任意记号序列，各模板条目由对齐定位符字符（在 plain TEX 中为 &）分隔，即任何类别 4 的字符。

u_i#v_i 序列是将用于第 i 列的模板：用户指定为该列条目的任何序列 α_i 都将插入参数字符处。然后处理序列 $u_i\alpha_i v_i$，得到当前行第 i 列的实际条目。详见下文。

模板行的长度 n 不必等于对齐的实际列数：模板只用于一行中指定的那么多项。例如

```
\halign{a#&b#&c#\cr 1&2\cr 1\cr}
```

它有一个三项模板，但各行只有一项或两项。其输出为

a1b2
a1

25.3.1 无限导言区

对于列数事先未知的情形——例如对齐要用于一个宏，由用户指定列——可以指定导言区的末尾一段可任意重复多次。在其前加 &，可将某个条目标记为导言区可重复部分的起点。见下面 \matrix 的例子。

当整个导言区都要重复时，第一个条目开头会有一个对齐定位符：

```
\halign{& ... & ... \cr ... }
```

如果导言区的开头部分要免于重复，则会出现双对齐定位符：

```
\halign{ ... & ... & ... && ... & ... \cr ... }
```

可重复部分不必使用整数次。对齐行可随时结束；导言区的剩余部分随后不再使用。

25.3.2 导言区中的花括号计数

对齐可以嵌套在对齐中，因此 TEX 用如下规则判定 & 或 \cr 控制序列属于哪个对齐：

一个对齐的所有定位符和 \cr 记号应处于同一分组层级。

由此可知，若定位符和 \cr 记号嵌套在花括号中，它们可以出现在条目内部。这使得嵌套对齐成为可能。

25.3.3 导言区中的展开

导言区中的所有记号 – 除定位符外 – 都被存储起来，用于插入对齐的条目；但前面带\span 的记号在扫描导言区时会被展开。\span 在对齐其余部分中的作用见下文。

25.3.4 \tabskip

对齐中的条目被设定为取其列中最大元素的宽度。分隔列的黏合可通过给\tabskip 赋值来指定。TEX 在每对列之间、以及第一列之前和最后一列之后插入此黏合。

对齐之外的 \tabskip 值用于第一列之前以及所有后续列之后，除非导言区含有对 \tabskip 的赋值。任何对 \tabskip 的赋值在 TEX 扫描导言区时执行；遇到定位

符时所取的值将用于每行的该位置，以及所有后续列之后，除非发生进一步赋值。遇到 \cr 时所取的 \tabskip 值用于最后一列之后。

导言区中对 \tabskip 的赋值对该对齐是局部的，但不限于给出它的条目。这些赋值是普通黏合赋值：它们去掉任何可选的尾部空格。

例如，在下面的表格中，第一列之前和最后一列之后没有 tabskip 黏合；所有列之间有可伸缩的 tabskip。

```
\tabskip=0pt \halign to \hsize{
    \vrule#\tabskip=0pt plus 1fil\strut&
    \hfil#\hfil& \vrule#& \hfil#\hfil& \vrule#& \hfil#\hfil&
    \tabskip=0pt\vrule#\cr
  \noalign{\hrule}
    &\multispan5\hfil Just a table\hfil&\cr
  \noalign{\hrule}
    &one&&two&&three&\cr &a&&b&&c&\cr
  \noalign{\hrule}
    }
```

其结果为

Just a table		
one	two	three
a	b	c

表格的所有竖线都在单独的一列中。这是让条目周围空间伸缩的唯一方法。

25.4 对齐主体

模板行之后可以跟任意多个以 \cr 结束的行。TEX 读取所有这些行，处理条目以找出每列（行）的最大宽度（高度）。由于所有条目都保存在内存中，过长的表格可能撑爆 TEX 的主存。对这类表格最好编写专用宏。

25.4.1 读取条目

对齐中的条目由模板的常量 u、v 部分和变量 α 部分组成。TEX 基本上是形成记号序列 $u\alpha v$ 并处理它。但有两种特殊情形，TEX 在形成该序列之前必须先展开。

上面描述了 \noalign 命令。由于它需要与其他对齐条目不同的处理，TEX 在读完一个 \cr 后，会展开下一行第一个 α 串的第一个记号，以查看它是否为（或展开为）\noalign。类似地，一行中所有条目的第一个记号都会被展开，以查看它是否为（或展开为）\omit。该控制序列将在下面描述。

以 \if... 条件或展开为该条件的宏开头的条目，可能因这种过早展开而被误解。例如，

```
\halign{$#$\cr \ifmmode a\else b\fi\cr}
```

会给出

$$b$$

因为条件是在数学模式建立之前求值的。解决方法与许多其他情形一样，插入一个 \relax 控制序列以停止展开。这里 \relax 必须插在对齐条目开头。

如果既未发现 \noalign，也未发现 \omit（见下文），TEX 将处理一个由 u 部分、α 记号（由 & 或 \span 定界，见下文）和 v 部分组成的输入流。

条目由 &、\span 或 \cr 定界，但仅当此类记号出现在同一分组层级时。这使得一个对齐可以作为另一个对齐的条目。

25.4.2 替代规格：\omit

模板行很少足以描述对齐的所有行。对于条目应不同排版的行，存在命令\omit：如果条目中的第一个记号是（或展开为）\omit，则使用平凡模板 #，而不是模板行所指定的内容。

下面的对齐对所有列使用同一模板，但第二列给出了 \omit 命令。

```
\tabskip=1em
\halign{&$<#>$\cr a&\omit (b)&c \cr}
```

其输出为

$$< a > \quad (\mathrm{b}) \quad < c >$$

25.4.3 跨越多列：\span

有时希望材料跨越若干列。最明显的例子是表格上方的标题。为此 TEX 提供了 \span 命令。

条目由 &、\cr 或\span 定界。在后一种情形，TEX 会省略通常跟在如此定界的条目之后的 tabskip 黏合，并把刚读取的材料与下一个条目放在共同可用的空间中排版。

例如，

```
\tabskip=1em
\halign{&#\cr a&b&c&d\cr a&\hrulefill\span\hrulefill&d\cr}
```

给出

a b c d
a ——— d

注意，两跨越列之间没有 tabskip 黏合，但第一列之前和最后一列之后有。

使用 \omit 命令，同一对齐也可生成为

```
\halign{&#\cr a&b&c&d\cr a&\hrulefill\span\omit&d\cr}
```

\span\omit 组合用于 plain TEX 宏 \multispan：例如

\multispan4 给出 \omit\span\omit\span\omit\span\omit

它跨越三个定位符，并去掉四个条目的模板。再重复上面的例子：

```
\halign{&#\cr a&b&c&d\cr a&\multispan2\hrulefill&d\cr}
```

\multispan 的参数是单个记号，不是数字，因此要跨越 9 列以上，参数应用花括号括起来，例如\multispan{12} 。此外，单位数参数后的一个空格会最终进入输出。

对于跨越列的“低成本”方案，plain TEX 有宏\hidewidth，定义为

```
\newskip\hideskip \hideskip=-1000pt plus 1fill
\def\hidewidth{\hskip\hideskip}
```

把 \hidewidth 放在对齐条目开头或结尾会使其宽度为零，条目中的材料分别向左或向右伸出。

25.4.4 对齐中的线

水平对齐内部的水平线大多横跨对齐的宽度。实现这一点最简单的方法是在对齐内部使用

```
\noalign{\hrule}
```

行。如果对齐包含在竖直盒子中，对齐上下的线可用

```
\vbox{\hrule \halign{...} \hrule}
```

指定。在对齐中得到水平线最一般的方法是使用

```
\multispan n\hrulefill
```

它可用于给任意相邻列加下划线。

对齐中的竖线需要多费些心思。由于水平对齐会拆成放在竖直列表上的水平盒子，TEX 会在对齐各行之间插入基线跳跃黏合。如果相邻行中的竖线要相接，就必须阻止基线跳跃黏合，例如用 `\offinterlineskip` 宏。

为确保各行仍有适当间距，就必须在导言区某处放置一个支柱（*strut*）。支柱是一个具有一定高度和深度的不可见对象。把它放进导言区可保证每一行至少有该高度和深度。在 plain 格式中`\strut` 静态定义为

```
\vrule height8.5pt depth3.5pt width0pt
```

因此使用其他字体或字号时必须更改它。

用一整列作竖线是个好主意，即在导言区写

```
\vrule#&
```

并把对齐中相应的条目留空。要省略竖线，可指定 `\omit`；要显式指定线的粗细，可在条目中放入例如`height 15pt`，而不留空。当然，此时竖线左右都会有 tabskip 黏合，因此导言区中可能需要一些额外的 tabskip 赋值。

25.4.5 行结束：\cr 与 \crcr

对齐中的所有行都由`\cr` 控制序列结束，包括最后一行。TEX 无法从 α 部分中的右花括号推断对齐已结束，因为不匹配的右花括号在对齐条目中完全合法；它可能与相应导言区条目的 u 部分中的左花括号配对。

TEX 有一个原始命令`\crcr`，它等价于 `\cr`，但如果它紧接在 `\cr` 之后，则不起作用。以 plain TEX 中`\cases` 的定义为例：

```
\def\cases#1{%
    \left\{\,\vcenter{\normalbaselines\m@th
              \ialign{ $##\hfil$& \quad##\hfil \crcr #1\crcr}}%
    \right.}
```

由于用户参数 `#1` 之后有 `\crcr`，该宏的以下两种用法

```
\cases{1&2\cr 3&4}  和  \cases{1&2\cr 3&4\cr}
```

都可行。第一种情形中，宏定义里的 `\crcr` 结束最后一行；第二种情形中，用户的 `\cr` 结束该行，而 `\crcr` 是冗余的。

在 `\cr` 之后以及非冗余 `\crcr` 之后，会插入⟨ 记号参数 ⟩ `\everycr`。这包括结束模板行的 `\cr`。

25.5 例子：数学对齐

plain 格式有若干在数学模式中起作用的对齐宏。例子之一是\matrix，定义为

```
\def\matrix#1{\null\,\vcenter{\normalbaselines\m@th
    \ialign{\hfil$##$\hfil && \quad\hfil$##$\hfil\crcr
            \mathstrut\crcr
         \noalign{\kern-\baselineskip}
            #1\crcr
            \mathstrut\crcr
         \noalign{\kern-\baselineskip}}}\,}
```

它使用重复（以 && 开头）的第二个导言区条目；每个条目前后各有一个 \hfil 使其居中，列之间有一个 \quad 的间距。这里没有使用 tabskip 黏合，因为矩阵前后不应有任何黏合。

在矩阵上下使用 \mathstrut 与\kern-\baselineskip 的组合，增大了竖直尺寸，使得行数相同的两个矩阵具有相同的高度和深度；否则如果其中一个矩阵最后一行有下标而另一个没有，情况就不是这样。\mathstrut 引起行间黏合的插入，并且由于它的尺寸等于 \baselineskip，负紧排实际上只留下行间黏合，从而缓冲第一行和最后一行的任何差异。当然这只到一定程度：大于左花括号的对象仍会导致矩阵高度或深度不同。

数学模式对齐的另一个更复杂的例子是 \eqalignno。

```
\def\eqalignno#1{\begin{disp}l@y \tabskip\centering
  \halign to\displaywidth{
    \hfil$\@lign\displaystyle{##}$%    -- first column
        \tabskip\z@skip
    &$\@lign\displaystyle{{}##}$\hfil% -- second column
        \tabskip\centering
    &\llap{$\@lign##$}%                -- third column
        \tabskip\z@skip\crcr   %   end of the preamble
       #1\crcr}}
```

首先，公式编号之后的 tabskip 设为零，因此该编号与右边距平齐。由于它由 \llap 放置，其有效宽度为零。其次，第一列与第二列之间的 tabskip 也为零，而第一列之前和第二列之后的 tabskip 是 \centering，即 0pt plus 1000pt minus 1000pt，因此第一列和第二列在 \hsize 中共同居中。注意，由于 minus 1000pt，这两列会很自然地超出左右边距，覆盖任何公式编号。

第 26 章

页面形状

本章讨论若干决定页面大小及其在纸上呈现方式的参数。

`\topskip` 页面盒子顶部与页面上第一个盒子基线之间的最小距离。Plain TEX 默认值： `10pt`

`\hoffset` `\voffset` 页面相对于参考点向右/向下偏移的距离。

`\vsize` 页面盒子的高度。Plain TEX 默认值： `8.9in`

`\maxdepth` 页面盒子的最大深度。Plain TEX 默认值： `4pt`

`\splitmaxdepth` 由 `\vsplit` 操作分出的盒子的最大深度。Plain TEX 默认值：`\maxdimen`

26.1 全局定位的参考点

输出设备驱动程序必须遵守一个 TEX 约定，即页面左上角距页面边缘一英寸。不幸的是，这可能带来很多麻烦，例如打印机（或它所用的页面描述语言）把左下角当作参考点，且出厂设置为美国纸张尺寸，却用于欧洲标准 A4 纸。

若给`\hoffset` 或`\voffset` 赋非零值，页面在纸上就会偏移：正值分别向右、向下偏移。

26.2 \topskip

`\topskip` 在一定程度上保证：即使在页面之间切换字号，或第一行没有上伸字母，每页的第一条基线位置都相同。

在每页第一个盒子之前会插入一些黏合。该黏合与 `\topskip` 有相同的伸缩量，但其自然大小为 `\topskip` 的自然大小减去第一个盒子的高度；若结果为负则取零。

Plain TEX 把 `\topskip` 设为 `10pt`。因此，如果字符顶部不超过十点，各页首行的基线就在同一位置。对于 Computer Modern 字体，当字号小于（约）13 点时满足此条件；字号更大时，首行基线会下降。

对于只含文本的页面（假设 `\parskip` 为零），页面盒子高度为 `\topskip` 加上若干倍 `\baselineskip`。因此可以定义一个宏，根据每页行数计算 `\vsize`：

```
\def\HeightInLines#1{\count@=#1\relax
    \advance\count@ by -1\relax
    \vsize=\baselineskip
    \multiply\vsize by \count@
    \advance\vsize by \topskip}
```

这样计算 \vsize 可避免纯文本页面出现 underfull 盒子。

当页面不是以一行文本开头（例如以一条线开头）时，topskip 可能产生不想要的效果。为避免这些，可用如下内容开始页面：

```
\hbox{}\kern-\topskip
```

后面跟你想放在顶部的内容。

与 \topskip 类似，对于由 \vsplit 操作生成的页面有 \splittopskip；见下一章。

26.3 页面高度与深度

TEX 试图把页面构造成高度为\vsize 的 \vbox；另见下一章的 \pagegoal。

如果页面上最后一项的深度过大，该页会明显比其他页长。为防止这种现象，TEX 用\maxdepth 作为页面盒子的最大深度。如果向页面添加一项会使深度超过此值，就把页面参考点下移，使深度恰好为 \maxdepth。

plain TEX 中通过给 \topskip 一定的有限伸缩量来获得“参差不齐底边”效果：10pt plus 60pt。于是盒子 255 到达输出例程时其自然高度可以变化。页面随后（大致）如下输出：

```
\dimen0=\dp255 \unvbox255
\ifraggedbottom \kern-\dimen0 \vfil \fi
```

\vfil 使 topskip 按自然宽度设置，因此效果是页面顶行固定、底行可变。

在 plain TEX 输出例程中解包 \box255 之前，\boxmaxdepth 被设为 \maxdepth，以便该盒子在与页面构造器拼合 \box255 时所用假设相同的条件下生成。

由 \vsplit 操作分出的盒子的深度由 \splitmaxdepth 参数控制。

第 27 章

分页

本章讨论“页面构造器”（page builder）：TEX 中决定在何处把主竖直列表断成页面的部分。页面构造器在输出例程之前运行，并把结果通过 \box255 交给输出例程。

\vsplit 把一个盒子的顶部切分出来。这与分页类似。
\splittopskip \vsplit 操作之后剩余部分的顶部与该盒子中第一项之间的最小距离。Plain TEX 默认值： 10pt
\pagegoal 页面盒子的目标高度。它起始于 \vsize，并因插入项的高度而减小。
\pagetotal 当前页面累积的自然高度。
\pagedepth 当前页面的深度。
\pagestretch 当前页面累积的零阶伸缩。
\pagefilstretch 当前页面累积的一阶伸缩。
\pagefillstretch 当前页面累积的二阶伸缩。
\pagefilllstretch 当前页面累积的三阶伸缩。
\pageshrink 当前页面累积的收缩量。
\outputpenalty 当前分页处的惩罚值；如果断点不在惩罚处，则为 10 000。
\interlinepenalty 在段落各行之间分页的惩罚。Plain TEX 默认值： 0
\clubpenalty 在段落第一行之后分页的附加惩罚。Plain TEX 默认值： 150
\widowpenalty 在段落最后一行之前分页的附加惩罚。Plain TEX 默认值： 150
\displaywidowpenalty 在展示公式上方最后一行之前分页的附加惩罚。Plain TEX 默认值： 50
\brokenpenalty 在断字行之后分页的附加惩罚。Plain TEX 默认值： 100
\penalty 在当前列表上放置一个惩罚。
\lastpenalty 如果列表上最后一项是惩罚，则为该惩罚的值。
\unpenalty 如果当前列表最后一项是惩罚，则移除它。

27.1 当前页面与近期贡献

TEX 的主竖直列表分为两部分：即“当前页面”和“近期贡献”列表。任何加入主竖直列表的材料都被附加到近期贡献；把近期贡献移到当前页面的动作称为“驱动页面构造器”。

每当有内容被移到当前页面时，TEX 就计算在该处分页的代价。如果它判定已经过了最优点，那么当前页面中直到目前最佳断点的部分被放入 \box255，而当前页面的剩余部分被移回近期贡献列表顶端。如果页面在惩罚处断开，该值被

记录在 \outputpenalty 中，并在近期贡献顶端放置大小为 10 000 的惩罚；否则\outputpenalty 被设为 10 000。

如果当前页面为空，从近期贡献移入的可丢弃项会被丢弃。这就是分页后以及第一页顶部黏合消失的机制。当第一个不可丢弃项被移到当前页面时，插入 \topskip 黏合；见上一章。

页面构造器的工作可通过把 \tracingpages 设为某个正值来显示（见第 34 章）。

27.2 激活页面构造器

页面构造器在以下情形发挥作用。

- 段落附近：在 \everypar 记号插入之后，以及段落加入竖直列表之后。本章末尾有一个例子。
- 展示公式附近：在 \everydisplay 记号插入之后，以及展示公式加入列表之后。
- 在 \par 命令、盒子、插入以及竖直模式中的显式惩罚之后。
- 在输出例程结束之后。

在这些地方，页面构造器把近期贡献移到当前页面。注意，驱动页面构造器时 TEX 不必处于竖直模式。在水平模式下，激活页面构造器用于把前面的竖直黏合（例如 \parskip、\abovedisplayskip）移到页面。

\end 命令 – 只允许在外部竖直模式中 – 结束一个 TEX 作业，但前提是主竖直列表为空且 \deadcycles = 0。若不是这种情况，则附加组合

\hbox{}\vfill\penalty−2^{30}

这会迫使输出例程执行。

27.3 页面长度记录

到达输出例程的页面盒子的高度和深度，由 \vsize、\topskip 和 \maxdepth 决定，如上一章所述。TEX 在第一个盒子放到当前页面时放置 \topskip 黏合；\vsize 和 \maxdepth 在页面上出现第一个盒子或插入项时读取。此后对这些参数的任何更改直到下一页（更严格地说，直到调用输出例程之后）才会显现。

当前页面上第一个盒子、线或插入项之后，\vsize 被记录到 \pagegoal，在 \output 被激活之前不会再查看其值。更改 \pagegoal 确实会影响当前页面。当页面为空时，pagegoal 为 \maxdimen，\pagetotal 为零。

累积的尺寸和伸缩量可在参数 \pagetotal、\pagedepth、\pagestretch、\pagefilstretch、\pagefillstretch、\pageshrink 和 \pagefilllstretch 中获得。它们由页面构造器设置。每次向页面添加黏合时，伸缩和收缩参数都会更新。如果最后一项是紧排或黏合，深度参数变为零。

这些参数是⟨ 特殊尺寸 ⟩；对它们中任何一个的赋值都是⟨ 紧密赋值 ⟩，并且自动是全局的。

27.4 断点

27.4.1 可能的断点

分页可发生在与断行相同类别的位置：

- 在前面有不可丢弃项的黏合处；
- 在紧接黏合之后的紧排处；
- 在惩罚处。

当段落各行加入竖直列表时，TEX 会插入行间黏合和各种行间惩罚，因此页面上通常有足够的断点。

27.4.2 断点惩罚

如果 TEX 决定在某个惩罚项处分页，该惩罚大多是自动插入段落各行之间的惩罚。

如果列表（不一定是竖直列表）上最后一项是惩罚，其值记录在参数`\lastpenalty`中。如果该项不是惩罚，该参数值为零。列表的最后一个惩罚可用命令`\unpenalty`移除。例子见第 5.9.6 节。

下面是这类惩罚的列表：

`\interlinepenalty` 在段落各行之间分页的惩罚。在 plain TEX 中此值为零，因此行之间不添加惩罚。TEX 随后可在 `\baselineskip` 黏合处找到有效断点。

`\clubpenalty` 在段落第一行之后分页的附加惩罚。在 plain TEX 中此值为 `150`。此值以及后续惩罚被加到 `\interlinepenalty` 上，并在包含段落第一行的 `\hbox` 之后插入一个 resulting 大小的惩罚，取代 `\interlinepenalty`。

`\widowpenalty` 在段落最后一行之前分页的附加惩罚。在 plain TEX 中此值为 `150`。

`\displaywidowpenalty` 在展示公式上方最后一行之前分页的附加惩罚。plain TEX 中的默认值为 `50`。

`\brokenpenalty` 在断字行之后分页的附加惩罚。plain TEX 中的默认值为 `100`。

如果结果惩罚为零，则不放置。

惩罚也可由用户插入。例如，plain 格式有一些宏来鼓励（可能强制）或禁止分页：

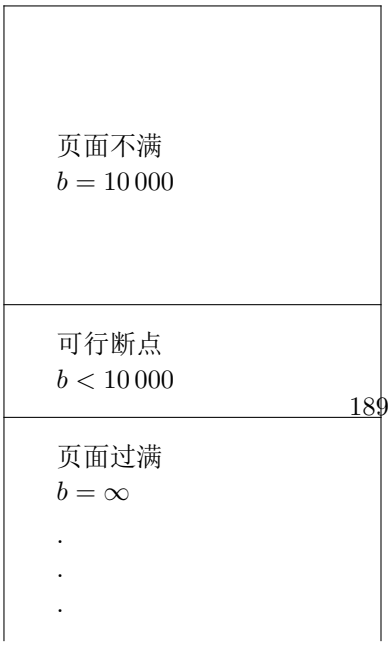
```
\def\break{\penalty-10000 }         % force break
\def\nobreak{\penalty10000 }        % prohibit break
\def\goodbreak{\par\penalty-500 }  % encourage page break
```

此外，`\vadjust{\penalty ... }` 是在竖直列表中获得惩罚的一种方式。它可用于不鼓励或鼓励在段落某行之后分页。

27.4.3 断点计算

每当一项被移到当前页面时，TEX 计算在该处分页相关的惩罚 p 和 badness（坏度）b。由惩罚和坏度计算分页代价 c。

代价最小的位置被记住；当代价为无穷大（即页面过满），或惩罚为 $p \leq -10\,000$ 时，当前页面在（最后记住的）代价最小处断开。断开的部分随后被放入 `\box255`，并插入输出例程记号列表。盒子 255 总是被赋予高度 `\vsize`，不论它含有多少材料。



坏度计算基于把页面装入高度为 \vsize、最大深度为 \maxdepth 的盒子所需的伸缩量。该计算与断行相同（见第 8 章）。坏度是值 $0 \le b \le 10\,000$，页面过满时除外；此时 $b = \infty$。

有些惩罚由 TEX 隐式插入，例如 \interlinepenalty，它被放在段落每一对行之间。其他惩罚可由用户或用户宏显式插入。惩罚值 $p \ge 10\,000$ 抑制断开；惩罚 $p \le -10\,000$（在外部竖直模式中）强制分页，并立即激活输出例程。

代价计算如下：

1. 当惩罚低到强制分页并立即调用输出例程，但页面未过满，即
   $b < \infty$ 且 $p \le -10\,000$
   时代价等于惩罚： $c = p$。
2. 当惩罚不强制任何操作，且页面未过满，即
   $b < \infty$ 且 $|p| < 10\,000$
   时代价为 $c = b + p$。
3. 对于非常坏的页面，即
   $b = 10\,000$ 且 $|p| < 10\,000$
   时代价为 $c = 10\,000$。
4. 过满页面，即
   $b = \infty$ 且 $p < 10\,000$
   给出无穷代价： $c = \infty$。此时 TEX 判定最优断点必定在更早处，并调用输出例程。超过 10 000 的 \insertpenalties 值（见第 29 章）也给出无穷代价。

惩罚 $p \le -10\,000$ 会激活输出例程这一事实在 LATEX 输出例程中被广泛使用：超出的 $|p| - 10\,000$ 是一个代码，指示调用输出例程的原因；另见下一章的第二个例子。

27.5 \vsplit

分页操作可通过\vsplit 操作提供给用户。

\setbox1 = \vsplit2 to \dimen3
把盒子 2 中大小为 \dimen3 的顶部部分赋给盒子 1。该材料实际上从盒子 2 中移除。把这与从当前页面切下大小为 \vsize 的一块相比较。

\vsplit⟨8 位数 ⟩to⟨ 尺寸 ⟩

提取出的结果是一个具有以下性质的盒子。

- 高度等于指定的⟨ 尺寸 ⟩；TEX 会遍历原盒子寄存器（它必须包含竖直盒子）以找到最佳断点。这可能导致 underfull 盒子。
- 深度至多为\splitmaxdepth；这类似于页面盒子的 \maxdepth，而不是适用于任何盒子的 \boxmaxdepth。
- 在 \splitfirstmark 和 \splitbotmark 寄存器中有首标记和尾标记。

\vsplit 操作的剩余部分是一个盒子，其中

- 所有可丢弃项已从顶部移除；
- 顶部插入了大小为\splittopskip 的黏合；若被切分的盒子是盒子 255，它顶部已有 \topskip 黏合；
- 其深度被强制至多为 \splitmaxdepth。

原盒子的底部总是 \vsplit 操作的有效断点。若采用此断点，剩余盒子寄存器为空。提取出的盒子可以为空；只有当原盒子为空或不是竖直盒子时它才为空。

通常，\vsplit 操作用于切下 \box255 的一部分。通过把 \splitmaxdepth 设为等于 \boxmaxdepth，结果就是 TEX 页面构造器本可造出的东西。修剪 \box255 顶部后，标记寄存器 \firstmark 和 \botmark 含有盒子 255 剩余部分上的首标记和尾标记。关于标记的更多信息见下一章。

27.6 分页示例

27.6.1 填满页面

假设某个竖直盒子太大，放不下页面剩余部分。那么

```
\vfil\vbox{ ... }
```

是填满页面并把盒子推到下一页的错误方法。TEX 只能在黏合开头断开，而 \vfil 在断开后被丢弃：结果是 underfull 或至少被过度拉伸的页面。另一方面，

```
\vfil\penalty0 % or any other value
\vbox{ ... }
```

才是正确方法：TEX 会在惩罚处断开，页面就被填满。

27.6.2 确定断点

在下面例子中使用 \vsplit 操作，它与分页有相同机制。

给定如下宏和参数设置：

```
\offinterlineskip \showboxdepth=1
\def\High{\hbox{\vrule height5pt}}
\def\HighAndDeep{\hbox{\vrule height2.5pt depth2.5pt}}
```

首先考虑一个竖直列表被简单拉伸以到达断点的例子。

```
\splitmaxdepth=4pt
\setbox1=\vbox{\High \vfil \HighAndDeep}
\setbox2=\vsplit1 to 9pt
```

给出

```
> \box2=
\vbox(9.0+2.5)x0.4, glue set 1.5fil
.\hbox(5.0+0.0)x0.4 []
.\glue 0.0 plus 1.0fil
.\glue(\lineskip) 0.0
.\hbox(2.5+2.5)x0.4 []
```

两个盒子总高度为 7.5pt，因此黏合需拉伸 1.5pt。

接下来，我们减小所得列表允许的深度。

```
\splitmaxdepth=2pt
\setbox1=\vbox{\High \vfil \HighAndDeep}
\setbox2=\vsplit1 to 9pt
```

给出

```
> \box2=
\vbox(9.0+2.0)x0.4, glue set 1.0fil
.\hbox(5.0+0.0)x0.4 []
.\glue 0.0 plus 1.0fil
.\glue(\lineskip) 0.0
.\hbox(2.5+2.5)x0.4 []
```

参考点下移半磅，伸缩量相应减小，但此移动不会导致比所指定更大的尺寸。

作为例子，考虑序列

```
\splitmaxdepth=3pt
\setbox1=\vbox{\High \kern1.5pt \HighAndDeep}
\setbox2=\vsplit1 to 9pt
```

这给出一个恰好 9 磅高、2.5 磅深的盒子。设置`\splitmaxdepth=2pt` 不会把高度增加半磅；相反，因为采用了更早的断点，结果是 underfull 盒子。

有时动作的时机很重要。TEX 先定位一个能达到所需高度的断点，然后检查配合 `\maxdepth` 或 `\splitmaxdepth` 是否不会违反该高度。

考虑这种时机的例子：在

```
\splitmaxdepth=4pt
\setbox1=\vbox{\High \vfil \HighAndDeep}
\setbox2=\vsplit1 to 7pt
```

中，结果不是一个 7 磅高、3 磅深的盒子。相反，

```
> \box2=
\vbox(7.0+0.0)x0.4
.\hbox(5.0+0.0)x0.4 []
```

它是 underfull 盒子。

27.6.3 段落后的页面构造器

段落之后，页面构造器把材料移到当前页面，但它不判定是否已找到断点。

```
\output{\interrupt \plainoutput}% show when you're active
\def\nl{\hfil\break}\vsize=22pt % make pages of two lines
a\nl b\nl c\par \showlists      % make a 3-line paragraph
```

会报告

```
### current page:
[...]
total height 34.0
 goal height 22.0
prevdepth 0.0, prevgraf 3 lines
```

尽管收集了绰绰有余的材料，`\output` 只有在下一段开始时才被调用：输入一个 `d` 会给出

```
! Undefined control sequence.
<output> {\interrupt
                     \plainoutput }
<to be read again>
                   d
```

此时 `\output` 在 `\everypar` 之后插入。

第 28 章

输出例程

页面处理的最后阶段由输出例程完成。页面构造器切下主竖直列表的某一部分，并通过 \box255 交给输出例程。本章讨论与输出例程相关的命令和参数，并解释输出例程如何通过标记接收信息。

\output 含有输出页面指令的记号列表。
\shipout 把一个盒子送到 dvi 文件。
\mark 指定一段标记文本。
\topmark 上一页的最后一个标记。
\botmark 当前页面的最后一个标记。
\firstmark 当前页面的第一个标记。
\splitbotmark 切分出页面的最后一个标记。
\splitfirstmark 切分出页面的第一个标记。
\deadcycles 计数器，记录输出例程被调用而未发生 \shipout 的次数。
\maxdeadcycles 输出例程被调用而未发生 \shipout 的最大允许次数。
\outputpenalty 当前分页处的惩罚值；如果断点不在惩罚处，则为 10 000。

28.1 \output 记号列表

通常说法是，当 TEX 找到断开主竖直列表的位置时，“输出例程被调用”。实际上，\output 不是宏，而是一个插入 TEX 命令流的记号列表。

\output 记号列表的插入发生在一个隐式打开的组内。此外，TEX 进入内部竖直模式。由于该组，非局部赋值（例如对页码的赋值）必须加前缀 \global。竖直模式意味着在输出例程运行期间空格大多无害。

\output 记号列表属于⟨ 记号参数 ⟩ 类。它们的行为与 \toks*nnn* 记号列表相同；见第 14 章。因此赋值输出例程可采取以下形式：

\output⟨ 等于 ⟩⟨ 一般文本 ⟩ 或 \output⟨ 等于 ⟩⟨ 填充 ⟩⟨ 记号变量 ⟩

28.2 输出与 \box255

TEX 的页面构造器在最优点断开当前页面，把该点以上的所有内容存入 \box255；然后把 \output 记号插入输入流。主竖直列表上任何剩余材料被推回近期贡献。如果页面在惩罚处断开，该值记录在 \outputpenalty 中，并在近期贡献顶端放

置大小为 10 000 的惩罚；否则 \outputpenalty 被设为 10 000。输出例程结束时，\box255 应当为空。若不为空，TEX 给出错误信息。

通常，输出例程会取出页面盒子，附加页眉和/或页脚，可能并入某些插入（如脚注），并把页面送到 dvi 文件：

```
\output={\setbox255=\vbox
           {\someheadline
            \vbox to \vsize{\unvbox255 \unvbox\footins}
            \somefootline}
         \shipout\box255}
```

当盒子 255 到达输出例程时，其高度已被设为 \vsize。然而其中材料的高度可能小得多。因此上述输出例程可能导致 underfull 盒子。这可用 \vfil 补救。

输出例程没有义务对 \box255 做任何有用之事；它可以清空它，或解包它让 TEX 再试一次寻找分页。输出例程推迟 \shipout 的次数记录在 \deadcycles 中：此参数由 \shipout 设为 0，并在每次 \output 之前加 1。

当死循环次数达到\maxdeadcycles 时，TEX 给出错误信息，并执行默认输出例程

```
\shipout\box255
```

而不是它正要开始的例程。LATEX 格式的 \maxdeadcycles 值比 plain TEX 高得多，因为 LATEX 中的输出例程常被调用以浮动体和边注的中间处理。

\shipout 命令可把任何⟨ 盒子 ⟩ 送到 dvi 文件；它不必是盒子 255，甚至不必是包含当前页面的盒子。它也不必在输出例程内部调用。

如果输出例程产生任何材料，例如通过调用

```
\unvbox255
```

这会被放到近期贡献顶端。

输出例程结束后，页面构造器被激活。特别地，由于当前页面已被清空，\vsize 被再次读取。因此在输出例程内部（用 \global）对此参数的更改会生效。

28.3 标记

信息可通过“标记”机制传给输出例程。用户可用

\mark{⟨ 标记文本 ⟩}

指定一个记号列表，它被放在当前竖直列表上的一个标记项中。标记文本像 \edef 那样会被展开。

如果标记在水平模式中给出，它像插入项一样迁移到周围的竖直列表（见第 62 页）；然而，如果这不是外部竖直列表，输出例程将找不到该标记。

标记是输出例程获取当前断开页面内容（特别是其顶部和底部）信息的主要机制。TEX 设置三个变量：

\botmark 当前页面上出现的最后一个标记；
\firstmark 当前页面上出现的第一个标记；
\topmark 上一页的最后一个标记，即上一页上 \botmark 的值。

如果尚未出现任何标记，三者都为空；如果当前页面上没有出现标记，三个标记变量都等于上一页的 \botmark。

对于由 \vsplit 命令生成的盒子（见上一章），\splitbotmark 和 \splitfirstmark 含有切出部分的标记；\firstmark 和 \botmark 反映寄存器中剩余部分的状态。

标记可用于把节标题放入页面的页眉或页脚。

```
\def\section#1{ ... \mark{#1} ... }
\def\rightheadline{\hbox to \hsize
   {\headlinefont \botmark\hfil\pagenumber}}
\def\leftheadline{\hbox to \hsize
   {\headlinefont \pagenumber\hfil\firstmark}}
```

这把左页上开始的第一节标题放在左页眉，把右页上开始的最后一节标题放在右页眉。把页眉放到页面上是输出例程的工作；见下文。

重要的是，在标记与放置标题的盒子之间不能发生分页：

```
\def\section#1{ ...
    \penalty\beforesectionpenalty
    \mark{#1}
    \hbox{ ... #1 ...}
    \nobreak
    \vskip\aftersectionskip
    \noindent}
```

再考虑另一个页眉例子：通常在章节开始的页面上省略页眉会更好看。可如下实现：

```
\def\endofchapter
\chapter#1{ ... \def\chtitle{#1}\mark{1}\mark{0} ... }
\def\theheadline{\expandafter\ifx\firstmark1
    \else \chapheadline \fi}
```

只有在章节开始的页面上标记才为 1，其他所有页面都放置页眉。

28.4 杂注

28.4.1 非平凡输出例程中的隐患

如果对输出例程的最后一次调用没有执行 `\shipout`，TEX 会无休止地调用输出例程，因为一次运行只有在竖直列表为空且 `\deadcycles` 为零时才会停止。输出例程可把 `\deadcycles` 设为零来防止这种情况。

28.4.2 页码

页码不是输出例程的内在属性；在 plain TEX 中它是 `\count0` 的值。当页面被输出时，输出例程负责增加页码。

除 `\count0` 外，计数器寄存器 1–9 也用于页面标识：输出页面时 TEX 把这十个计数器的值写入 `dvi` 文件（见第 33 章）。终端和日志文件输出只显示非零计数器，以及存在更高编号非零计数器的零计数器；即如果 `\count0` = 1 和 `\count3` = 5 是仅有的非零计数器，显示的计数器列表为 `[1.0.0.5]`。

28.4.3 plain TEX 中的页眉和页脚

Plain TEX 有记号列表 `\headline` 和 `\footline`；它们用于宏 `\makeheadline` 和 `\makefootline`。页面（大致）如下输出：

```
\vbox{\makeheadline\pagebody\makefootline}
```

页眉和页脚都插入在 `\line` 内。对于非标准页眉页脚，重定义宏 `\makeheadline` 和 `\makefootline` 比改动记号列表更容易。

28.4.4 例子：避免寡行

假设不允许寡行，但页面通常没有伸缩量，例如因为只含纯文本。一个解决办法是：如果某页结果在寡行处断开，就把页面长度增加一行。

TEX 的输出例程可执行这种技巧：如果把 \widowpenalty 设为某个可识别值，输出例程可通过 \outputpenalty 看出是否发生了寡行。此时输出例程可临时增加 \vsize，让页面构造器再试一次寻找断点。

下面是这种输出例程的骨架。不考虑页眉页脚。

```
\newif\ifLargePage \widowpenalty=147
\newdimen\oldvsize \oldvsize=\vsize
\output={
    \ifLargePage \shipout\box255
          \global\LargePagefalse
          \global\vsize=\oldvsize
    \else \ifnum \outputpenalty=\widowpenalty
             \global\LargePagetrue
             \global\advance\vsize\baselineskip
             \unvbox255 \penalty\outputpenalty
          \else  \shipout\box255
    \fi   \fi}
```

如果 \outputpenalty 等于 \widowpenalty，测试 \ifLargePage 被输出例程设为真。然后页面盒子被 \unvbox，以便页面构造器再次处理同样材料。

28.4.5 例子：页面顶部不缩进

有些输出例程可被归类为对输出例程机制的滥用。本节的输出例程就是一个好例子。

可以想见，人们希望段落若从页面顶部开始就不缩进。（这种版式有很多反对理由，但偶尔会用到。）此问题可用输出例程解决：检查页面是否仍为空，若是，则发出信号表明段落不应缩进。

注意，这里不能利用页面构造器在插入 \everypar 之后发挥作用这一事实：即使我们能强制输出例程在此激活，它也无法移除缩进盒子。

这里给出的解决办法让 \everypar 立即终止段落：

```
\par\penalty-\specialpenalty
```

这会激活输出例程。通过查看页面盒子是否为空（在移除空行和任何 \parskip 黏合之后），输出例程可设置一个开关，指示段落重试时是否应缩进。

下面的例程中有一些小细节，其含义留给读者思考。

```
\mathchardef\specialpenalty=10001
\newif\ifPreventSwitch
\newbox\testbox
\topskip=10pt

\everypar{\begingroup \par
    \penalty-\specialpenalty
    \everypar{\endgroup}\parskip0pt
    \ifPreventSwitch \noindent \else \indent \fi
```

```
    \global\PreventSwitchfalse
    }
\output{
    \ifnum\outputpenalty=-\specialpenalty
        \setbox\testbox\vbox{\unvbox255
                  {\setbox0=\lastbox}\unskip}
        \ifdim\ht\testbox=0pt \global\PreventSwitchtrue
        \else \topskip=0pt \unvbox\testbox \fi
    \else \shipout\box255 \global\advance\pageno1 \fi}
```

28.4.6 更多输出例程例子

大量输出例程例子可在 [38] 和 [39] 中找到。

第 29 章

插入

插入是 TEX 处理浮动信息的方式。TEX 的页面构造器计算哪些插入项以及多少个插入项能放在页面上；这些插入项随后被放入插入盒子，由输出例程处理。

`\insert` 开始一个插入项。
`\newinsert` 分配一个新的插入类。
`\insertpenalties` 切分插入项的惩罚总和。在输出例程内部，是延到下一页的插入项数量。
`\floatingpenalty` 切分插入项时加入的惩罚。
`\holdinginserts`（仅 TEX3）若此值为正，输出时插入项不放入其盒子。
`\footins` plain TEX 中脚注插入类的编号。
`\topins` 顶部插入类的编号。
`\topinsert` plain TEX 宏，用于开始一个顶部插入。
`\pageinsert` plain TEX 宏，用于开始一个占满整页的插入。
`\midinsert` plain TEX 宏：若有空间则放置其参数，否则转换为顶部插入。
`\endinsert` plain TEX 宏,用于结束一个以 `\topinsert`、`\midinsert` 或 `\pageinsert` 开始的插入项。

29.1 插入项

插入项包含浮动信息。处理插入项是用户、TEX 内部机制与输出例程之间的奇怪交互。首先用户指定一个插入，它是一定量的竖直材料；然后 TEX 的页面构造器决定哪些插入应放到当前页面上，并把这些插入放入插入盒子；最后输出例程必须对这些盒子做些处理。

插入项形如

`\insert`⟨8 位数 ⟩`{`⟨ 竖直模式材料 ⟩`}`

其中 8 位数不应为 255，因为 `\box255` 被 TEX 用于把页面传给输出例程。

插入项中围绕竖直模式材料的花括号可以是隐式的；它们意味着新的一层分组。竖直模式材料在内部竖直模式中处理。

`\splittopskip`、`\splitmaxdepth` 和 `\floatingpenalty` 的值与切分插入相关（见下文）；使用分组结束前当前的值。

插入项可出现在竖直模式、水平模式和数学模式中。对于后两种模式，它们必须迁移到周围的竖直列表（见第 62 页）。插入项放到竖直列表上之后，页面构造器被驱动。

29.2 插入类声明

在 plain 格式中，新插入类的编号由\newinsert 分配：

```
\newinsert\myinsert % new insertion class
```

它使用 \chardef 给该控制序列赋一个编号。

插入类从 254 向下分配编号。由于盒子 255 用于输出，这种分配方案使 \skip255、\dimen255 和 \count255 留作临时使用。

29.3 插入参数

每个插入类 n 分配四个寄存器：

- \box$n$ 输出例程激活时，此盒子含有应放在当前页面上的类 n 插入项。
- \dimen$n$ 这是每页类 n 插入项的最大分配空间。若超出此量，TEX 会切分插入项。
- \skip$n$ 当类 n 的插入项第一次加入当前页面时，加入此大小的黏合。这对于诸如用一条线把脚注与正文分开之类的现象很有用。
- \count$n$ 每个插入项都是一个竖直列表，因此有一定高度。然而有效高度（它对页面正文高度的影响量）可能不同于实际高度。\count$n$ 的值是 1000 乘以把高度换算为有效高度所乘的因子。
  考虑以下例子：
  - 边注不影响正文高度，因此因子应为 0。
  - 双栏模式下的脚注对页面的影响为其高度的一半：计数值应为 500。
  - 反之，双栏模式下放在页面下方、按页宽排版的脚注影响两栏，所以 – 假设双栏模式是通过对双高栏应用 \vsplit 实现的 – 计数值应为 2000。

29.4 从贡献列表中移动插入项

插入项最复杂的问题是把插入项加入主竖直列表并在必要时计算断点的算法。

TEX 从不更改 \vsize，但它把\pagegoal 减去将出现在分页之前的插入项的（有效）高度。因此输出例程收到的 \box255 高度为 \pagegoal，不一定是 \vsize。

1. 当某类 n 的第一个插入出现在当前页面上时，TEX 必须计入量 \skip$n$。此步仅在竖直列表上没有更早出现该类插入项时执行 – 这包括被切分的插入 – 但此时 \box$n$ 不必为空。
   如果 \box$n$ 不为空,其高度加深度乘以 \count$n$/1000,结果从 \pagegoal 中减去。然后 \pagegoal 减去 \skip$n$ 的自然分量。\skip$n$ 的任何伸缩量分别计入 \pagestretch 和 \pageshrink。
2. 如果页面上有类 n 的切分插入 – 此情形与算法前一步互斥 – 则\floatingpenalty 被加到\insertpenalties。切分插入是指为其计算了断点的插入项，因为它无法完整放在当前页面上。因此当前考虑的插入项肯定不会放到当前页面上。
3. 在前两点的预备动作之后，TEX 把实际插入项放到主竖直列表上当前贡献的末尾。首先它检查该项能否不被切分就放下。
   有两个条件要检查：
   - 把插入项（加上该类之前所有插入）加入 \box$n$ 不应使该盒子的高度加深度超过 \dimen$n$，且

- 或者插入项的有效高度为负，或者 \pagetotal 加 \pagedepth 减 \pageshrink 加插入项的有效尺寸应小于 \pagegoal。

若满足这些条件，\pagegoal 减去插入项的有效尺寸，即高度加深度乘以 \count$n$/1000。

4. 在算法前一步中两个条件之一不满足的插入项将被考虑切分。TEX 计算要从插入项切下的最大部分的尺寸，使得
   (a) 把此部分连同该类更早插入加入 \box$n$ 不会使盒子尺寸超过 \dimen$n$，且
   (b) 此部分的有效尺寸加入 \pagetotal 加 \pagedepth 后不超过 \pagegoal。注意这次不像前一步那样计入 \pageshrink。

   一旦确定了要切下的最大尺寸，TEX 在当前插入项中定位代价最小的断点，使结果盒子的高度等于该最大尺寸。与此断点相关的惩罚加到 \insertpenalties，\pagegoal 减去要从插入项切下的盒子的有效高度加深度。

29.5 输出例程中的插入项

当输出例程开始动作时 – 更准确地说：当 TEX 开始处理 \output 记号列表中的记号时 – 所有应放在当前页面上的插入项都已放入其盒子，输出例程负责把它们放到将要输出的盒子中的某处。

plain TEX 输出例程通过打包以下序列处理顶部插入和脚注：

```
\ifvoid\topins \else \unvbox\topins \fi
\pagebody
\ifvoid\footins \else \unvbox\footins \fi
```

解包插入盒子使页面各部分的黏合以一致方式伸缩。

在 TEX3 中插入机制略有扩展：参数 \holdinginserts 可用于指定插入项尚未放入其盒子。如果输出例程要重新计算 \vsize，或输出例程被调用做其他中间计算而非弹出页面，这非常有用。

输出例程运行期间，参数\insertpenalties 持有延到下一页的插入项数量。在 plain TEX 输出例程中，这在最后一页之后使用：

```
\def\dosupereject{\ifnum\insertpenalties>0
    % something is being held over
  \line{}\kern-\topskip\nobreak\vfill\supereject\fi}
```

29.6 plain TEX 插入

plain TEX 格式只有两个插入类：脚注和顶部插入。宏\pageinsert 生成被拉伸到恰好 \vsize 高的顶部插入。宏\midinsert 测试用户指定的竖直材料是否能放在页面上；若能则放在那里；若不能则转换为顶部插入。

脚注允许被切分，但一旦某脚注被切分，当前页面上不应再出现其他脚注。通过设置

```
\floatingpenalty=20000
```

达到此效果。如果一个插入项跟在同一类的切分插入项之后，\floatingpenalty 被加到 \insertpenalties。然而 \floatingpenalty $> 10\,000$ 有无穷代价，因此 TEX 会取更早的断点把页面从竖直列表切下。

顶部插入本质上只含一个竖直盒子，装着用户指定的内容。因此这种插入不能被切分。然而\endinsert 宏在盒子顶部放置 \penalty100，所以插入项可以在切分前带一个空部分被切分。效果是整个插入项被延到下一页。由于顶部插入的 \floatingpenalty 为零，任意多个这种插入项可前移，直到有一页空间足够。

插入宏的更多例子见 [40]。

第 30 章

文件输入与输出

本章讨论 TEX 从外部文件读入和写出的各种方式。

\input 读取指定文件作为 TEX 输入。
\endinput 在当前行之后结束当前文件的输入。
\pausing 指定 TEX 从文件读入每行后暂停。
\inputlineno 当前输入行的行号。
\write 把〈 一般文本 〉写到终端或文件。
\read 从流中读一行到一个控制序列。
\newread \newwrite 分配新输入/输出流的宏。
\openin \closein 打开/关闭输入流。
\openout \closeout 打开/关闭输出流。
\ifeof 测试文件是否已读完或不存在。
\immediate 前缀，使输出操作立即执行。
\escapechar 控制序列转换为字符记号时所用转义字符的编号。IniTEX 默认值：92。
\newlinechar 在 \write 语句中触发换行的字符编号。

30.1 包含文件：\input 与 \endinput

大型文档可在 TEX 中分块：把各部分放在单独文件中，并用 \input 装入主文件。文件名的确切语法与实现有关；大多数情况下若未给出显式扩展名，就假定为.tex 扩展名。文件名可用空格或 \relax 界定。\input 命令是可展开的。

如果 TEX 在输入文件中遇到\endinput 语句，它就表现得像该文件在该语句所在行之后结束一样。与 \endinput 同一行上的语句仍会执行。\endinput 语句是可展开的。

30.2 文件 I/O

TEX 支持输入输出流，用于逐行读写文件。

30.2.1 打开与关闭流

TEX 最多支持 16 个同时输入流和 16 个输出流。plain TEX 宏\newread 和\newwrite 给出未使用流的编号。该编号由 \chardef 命令分配。输入流与输出流完全独立。

输入流由

\openin⟨4 位数 ⟩⟨ 等于 ⟩⟨ 文件名 ⟩

打开，由

\closein⟨4 位数 ⟩

关闭。

输出流由

\openout⟨4 位数 ⟩⟨ 等于 ⟩⟨ 文件名 ⟩

打开，由

\closeout⟨4 位数 ⟩

关闭。

如果输出文件尚不存在，由 \openout 创建；如果已存在，\openout 会覆盖它。

输出操作 \openout、\closeout 和 \write 三者都可加前缀 \immediate；见下文。

30.2.2 用 \read 输入

除了读取整个文件的 \input 命令外，TEX 还有\read 操作，它从文件（或用户终端）读一行。read 命令的语法为

\read⟨ 编号 ⟩to⟨ 控制序列 ⟩

此语句的效果是从指定流读入一行输入，并把该控制序列定义为无参数宏，以该行作为替换文本。

如果输入行花括号不配对，TEX 会读取多行，继续读取直到获得平衡的记号列表为止。TEX 隐式地给每个输入流附加一个空行，因此对流的最后一次 \read 操作总会产生一个 \par 记号。

从 0–15 范围之外的任何流 – 或未与打开文件关联的流，或已到达文件末尾的流 – 进行 read 操作时，从终端读取。如果流编号为正，用户会看到提示，给出由 \read 语句正在定义的控制序列名。

```
\read16 to \data
显示提示
\data=
输入 'my name' 作为回应，使该 read 语句等价于
\def\data{my name }
输入末尾的空格来自行尾；为防止这一点，可写
{\endlinechar=-1 \global\read16 to \data}
```

30.2.3 用 \write 输出

TEX 的\write 命令

\write⟨ 编号 ⟩⟨ 一般文本 ⟩

把平衡的记号列表写到由 \openout 打开的文件、日志文件或终端。

向 0–15 之外的流 – 或未与打开文件关联的流 – 进行 write 操作时，写到日志文件；如果流编号为正，则同时写到终端和日志文件。

\write 的记号列表参数定义为

⟨ 一般文本 ⟩ ⟶ ⟨ 填充 ⟩{⟨ 平衡文本 ⟩⟨ 右花括号 ⟩

可以有隐式左花括号。该参数像 \edef 的替换文本一样被展开，因此其中出现的任何宏和条件语句都会被展开。但不执行任何命令。此展开发生在输出页面时；见下文。在此之前，参数记号列表存放在当前列表上的 whatsit 项中。关于写出期间展开的进一步讨论见第 12 章。

由 \write（或 \message）输出的控制序列带尾随空格表示，并使用字符编号 \escapechar 作为转义字符。IniTEX 对此的默认值是 92，即反斜杠的代码。尾随空格可通过在控制序列前加 \string 来避免。

30.3 Whatsit 项

输入与输出在执行上有本质区别：与输出相关的操作（\openout、\closeout、\write）不立即执行；相反，它们被保存到出现这些操作的盒子输出到 dvi 文件时为止。

write 及另外两个输出操作被放在当前正在构建的列表上的“whatsit”项中。实际操作发生在页面中含有该项的部分输出到 dvi 文件时。这种延迟输出是由 TEX 异步输出例程行为所要求的。完整示例见第 110 页。

\immediate\write – 或任何其他\immediate 输出操作 – 立即执行，不在当前列表上放置 whatsit 项。

\special 命令的参数（见第 215 页）也放在 whatsit 中。

引导盒子中的 whatsit 项被忽略。

30.4 杂注

30.4.1 检查输入

TEX 在⟨ 内部整数 ⟩ 参数\inputlineno（在 TEX3 中）中记录当前输入文件的当前行号。

如果参数\pausing 为正，TEX 在终端屏幕上显示每一行输入，并给用户插入命令的机会。这些可以是例如 \show 命令。插入的命令被当作直接在源文件中一样处理：例如不必像 TEX 因错误暂停时那样在前面加 ‘i’。

30.4.2 测试文件是否存在

当你让 TEX 输入一个不存在的文件时，它不是最友好的系统。因此可用以下命令序列防止麻烦：

```
\newread\instream \openin\instream= fname.tex
\ifeof\instream \message{File 'fname' does not exist!}
\else \closein\instream \input fname.tex
\fi
```

这里用给定文件名打开一个输入流。如果输入流不对应物理文件，文件结束测试也为真；因此如果此条件不为真，文件存在，可安全给出 \input 命令。

30.4.3 时序问题

一方面 write 操作与另一方面文件打开/关闭操作之间的同步可能是关键点。各种格式用来实现交叉引用的辅助文件就是很好的说明。

假设在一次 TEX 运行期间写出辅助文件，在运行结束时它必须因各种目的（例如查看引用是否改变）再次被输入。\input 命令立即执行，因此文件必须已用\immediate\closeout 关闭。然而，现在可能出现文件在所有对它的 write 完成之前就被关闭的情况。以下序列补救这一点：

```
\par\vfil\penalty -10000 \immediate\closeout\auxfile
```

前三个命令激活输出例程以关闭最后一页，因此在文件关闭之前所有 write 确实都已执行。

30.4.4 \message 与 \immediate\write16 的对比

给用户的消息可用\message⟨ 一般文本 ⟩ 给出，它写到终端。消息彼此附加；当达到行长度（TEX 的编译时常量）时换行。在 TEX 第 2 版中，每条消息最多写 1000 字符；这不是编译时常量，而是硬连到 TEX 程序中的。

每条用\immediate\write 给出的消息都从新行开始；用户可通过包含编号为\newlinechar 的字符在消息中强制换行。此参数在 \message 中也有效。

30.4.5 竖直盒子中的 write

由于 write 操作以 whatsit 形式落在竖直列表上，在 \vtop 开始处发出 write 很可能影响该盒子的高度（见第 5 章）。例如，

```
have the \vtop{\write\terminal{Hello!}\hbox{more text}}
dangling from
```

会有 $\underset{\text{more text}}{}$ 从基线悬挂（当本书用 TEX 排版时，消息 'Hello!' 出现在屏幕上）。

30.4.6 \write 和 \message 中的展开与空格

\write 和 \message 都像 \edef 的替换文本一样展开其参数。因此

```
\def\a{b}\message{\a}
```

会写出 'b'。

不可展开的控制序列带尾随空格显示（并加 \escapechar 前缀）：

```
\message{\hbox\vbox!}
```

会写出 '\hbox \vbox !'。未定义控制序列在此给出错误。

可展开控制序列需小心写出：

```
\message{\noexpand\ifx}
\message{\string\ifx}
{\let\ifx\relax \message{\ifx}}
```

都写出 '\ifx'。

但注意，可展开控制序列之后的空格在输入处理器中被移除，输入处理器在控制序列后进入状态 S。因此

```
\def\a{b}\def\c{d}
\message{\a \c}
```

写出 ‘bd’。插入空格可如下进行：

```
\def\space{ } % in plain TeX
\message{\a\space\c}
```

显示 ‘b d’。注意

```
\message{\a{ }\c}
```

不起作用：它显示 ‘b{ }d’，因为花括号是不可展开的字符记号。

第 31 章

寄存器的分配

TEX 有若干种类型的寄存器。其中一部分存在显式命令，可为某个寄存器定义别名；而对所有类型，plain 格式中都提供了宏，用于分配一个未使用的寄存器。本章讨论别名与分配命令，并就寄存器分配为宏编写者给出若干指导原则。

`\countdef` 为一个 `\count`（整数）寄存器定义别名。
`\dimendef` 为一个 `\dimen`（尺寸）寄存器定义别名。
`\muskipdef` 为一个 `\muskip`（数学间距）寄存器定义别名。
`\skipdef` 为一个 `\skip`（胶）寄存器定义别名。
`\toksdef` 为一个 `\toks`（记号列表）寄存器定义别名。
`\newbox` 分配一个未使用的 `\box`（盒子）寄存器。
`\newcount` 分配一个未使用的 `\count`（整数）寄存器。
`\newdimen` 分配一个未使用的 `\dimen`（尺寸）寄存器。
`\newfam` 分配一个未使用的数学族。
`\newinsert` 分配一个未使用的插入类。
`\newlanguage`（仅 TEX3）分配一个新的语言编号。
`\newmuskip` 分配一个未使用的 `\muskip`（数学间距）寄存器。
`\newskip` 分配一个未使用的 `\skip`（胶）寄存器。
`\newtoks` 分配一个未使用的 `\toks`（记号列表）寄存器。
`\newread` 分配一个未使用的输入流。
`\newwrite` 分配一个未使用的输出流。

31.1 分配命令

在 plain TEX 中，定义了一组 `\new...` 宏用于分配寄存器。TEX 的寄存器分为两类，其分配方式各不相同（详见下文）。

plain TEX 中的`\newlanguage` 宏并不分配任何寄存器，它只是从 0 开始依次赋一个编号。TEX （第 3 版）最多可容纳 256 套不同的断词模式。

plain TEX 中的 `\new...` 宏被定义为 `\outer`（外部）宏（确切解释见第 11 章），因此不能在其他宏内部调用这些分配宏。LATEX 格式因此重新定义了这些宏，去掉了 `\outer` 前缀。

31.1.1 `\count`、`\dimen`、`\skip`、`\muskip`、`\toks`

对这类寄存器，存在一个⟨registerdef⟩（寄存器定义）命令，例如 `\countdef`，用于将某个具体寄存器绑定到一个控制序列：

⟨registerdef⟩⟨control sequence⟩⟨equals⟩⟨8-bit number⟩

在定义

```
\countdef\MyCount=42
```

之后，所分配的寄存器即可这样使用：

```
\MyCount=314
```

或者

```
\vskip\MyCount\baselineskip
```

⟨registerdef⟩ 命令被用于 plain TEX 中那些分配未使用寄存器的宏，如 `\newcount` 等。在

```
\newcount\MyCount
```

之后，`\MyCount` 即可完全照上述两例那样使用。

31.1.2 `\box`、`\fam`、`\write`、`\read`、`\insert`

对这类寄存器，TEX 中没有⟨registerdef⟩ 命令，因此 plain TEX 中相应的宏（例如分配盒子寄存器的 `\newbox`）改用 `\chardef` 来实现。

使用 `\chardef` 意味着：所定义的控制序列并不代表寄存器本身，而只代表寄存器的编号。因此在

```
\newbox\MyBox
```

之后，必须写成

```
\box\MyBox
```

若漏掉 `\box`，则排版的就是当前字体中编号为 `\MyBox` 的那个字符。`\chardef` 命令在第 3 章中进一步讨论。

31.2 宏编写者的基本规则

plain TEX 的 `\new...` 宏被设计为宏宏包的基础，使得多个这样的宏宏包能在同一次 TEX 运行中互不冲突地工作。Knuth 在 the TEX book 附录 B 中列出了若干宏编写者应当遵守的基本规则：

1. `\new...` 宏不会分配编号为 0–9 的寄存器。因此这些寄存器可作为 '临时' 寄存器使用。但由于任何宏系列都可能使用它们，故不能假定其内容会长期保持。需要在两次调用之间传递的结果，应当放在专门分配的寄存器中。注意：count 寄存器 0–9 被用于在 `dvi` 文件中标识页码（见第 33 章），因此不应对它们做全局赋值。
2. `\count255`、`\dimen255` 和 `\skip255` 也是可用的。这是因为插入类从 254 向下分配，而且每次分配一个插入盒子时，还会连同分配一个编号相同的 count、dimen 和 skip 寄存器。由于 `\box255` 被输出例程使用（见第 28 章），编号为 255 的 count、dimen 和 skip 便空闲可用。
3. 对临时寄存器 0、2、4、6、8 和 255 的赋值应当是局部的；对寄存器 1、3、5、7、 9 的赋值应当使用 `\global`（`\count` 寄存器为例外）。这条规则可防止 '保存栈不断堆积'（见第 35 章）。
4. 任何寄存器都可以在一个分组内使用，因为 TEX 的分组机制会在分组之外恢复其值。这种用法有两个条件：不应对它做全局赋值；并且在该分组内，不可能激活其他会对该寄存器做全局赋值的宏。
5. 需要长期使用、或需要在不同宏的多次调用之间保留内容的寄存器，应当用 `\new...` 来分配。

第 32 章

运行 TEX

本章讨论 TEX 的各种运行模式，以及与正在处理的作业相关的一些其他命令。

\everyjob 在每个新作业开始时插入的记号列表。
\jobname 正在处理的主 TEX 文件的名称。
\end 结束一次 TEX 运行的命令。
\bye plain TEX 宏，用于强制输出最后内容。
\pausing 指定 TEX 每从文件读入一行后暂停。
\errorstopmode 遇到错误时 TEX 将请求用户输入。
\scrollmode TEX 自行修正错误，但在缺少文件时仍会请求用户处理。
\nonstopmode TEX 自行修正错误，在遇到严重错误（例如缺少输入文件）时紧急停止。
\batchmode TEX 自行修正错误，在遇到严重错误（例如缺少输入文件）时紧急停止，但不产生任何终端输出。

32.1 作业

TEX 为每次运行关联一个正在处理文件的名称即\jobname。如果 TEX 以交互方式运行 – 即在调用时没有给出文件名参数，由用户直接键入命令 – 则作业名为 texput。

\jobname 可用于生成本次运行期间要读写的辅助文件名。例如,对于文件 story.tex，\jobname 为 story，而写入

```
\openout\Auxiliary=\jobname.aux
\openout\TableOfContents=\jobname.toc
```

将创建文件 story.aux 与 story.toc。

32.1.1 作业的开始

TEX 通过在命令流开头插入\everyjob 记号列表来启动每个作业。在一次 TEX 运行期间设置该变量没有意义，但格式文件可以用它向用户标识自身。如果某个格式填充了该记号列表，则用该格式运行 TEX 时，其中的命令会自动执行。

32.1.2 作业的结束

TEX 作业由\end 命令终止。这可能首先要求输出例程处理所有剩余内容（见第 27 章）。如果作业结束时仍处于某个分组内，TEX 会给出一条诊断信息。\end 命令不允许出现在内部竖直模式中，因为那将位于某个竖直盒子内部。

通常需要给 \end 命令裹上一层 '糖衣'。例如 plain TEX 宏\bye 定义为

```
\def\bye{\par\vfill\supereject\end}
```

其中 \supereject 负责处理所有剩余的插入内容。

32.1.3 日志文件

每次运行 TEX 都会创建一个日志文件。通常该文件以 \jobname 的值命名，扩展名为.log；有些实现也使用.lis 等其他扩展名。该日志文件包含运行期间屏幕上显示的所有信息，但它会把某些信息展示得更详尽，还可能包含通常不在屏幕上显示的统计数据。如果参数 \tracingonline 为正值，则日志文件中的全部信息都会同时显示在屏幕上。

溢出盒子与不足盒子会报告到终端屏幕，并在日志文件中按参数 \showboxdepth 与 \showboxbreadth 转储（见第 34 章）。这两个参数也用于 \showbox 命令引起的盒子转储，以及当 \tracingoutput 设为正值时由 \shipout 写出的盒子转储。

由 \tracingparagraphs 等命令生成的统计数据会写入日志文件;若 \tracingonline 为正值，它们也会同时显示在屏幕上。

向未打开的流、或向编号不在 0–15 范围内的流执行输出操作时，内容会写入日志文件。如果流编号为正，它们还会同时送到终端。

32.2 运行模式

默认情况下，遇到错误时 TEX 进入 \errorstopmode：它停止运行并请求用户输入。有些实现在用户中断 TEX 时有办法强制 TEX 进入 errorstopmode，以便检查（并修改）TEX 的内部状态。关于在 TEX 被中断后切换运行模式的方法，见第 223 页。

通常，只要用户让 TEX 继续（一般是按回车键），TEX 往往能自行修正错误；但有时（例如在对齐中）TEX 要过一段时间才能重新回到正轨（而且有时再也回不来）。这种情况下用户可能希望开启\scrollmode，它指示 TEX 对出现的任何错误尽力自行修正，不再征求用户确认。这通常是在 TEX 请求输入时键入 's' 来完成的。

在 \scrollmode 下，TEX 在 \show... 命令之后也不再请求输入。然而，有些错误并不容易补救，例如 \input 找不到某个文件，此时仍会请求用户输入。

使用\nonstopmode 时，TEX 会一路滚过错误；若遇到无法恢复的错误，它会紧急停止并中止本次运行。此外，如果试图从终端执行 \read，TEX 也会中止运行。\batchmode 与 nonstopmode 的唯一区别在于：它只把消息写入日志文件，而不送到终端。

第 33 章

TEX 与外部世界

本章讨论与 dvi 文件和格式相关的命令。它给出关于 IniTEX、字体与格式文件、Computer Modern 字体以及 WEB 的一些总体信息。

\dump 转储一个格式文件；仅能在 IniTEX 中使用，不允许出现在分组内部。
\special 向 dvi 文件写入一段⟨balanced text⟩（平衡文本）。
\mag 文档放大倍数的 1000 倍。
\year 当前作业的年份。
\month 当前作业的月份。
\day 当前作业的日期。
\time 当前作业开始时刻距午夜的分钟数。
\fmtname 宏，包含所转储格式的名称。
\fmtversion 宏，包含所转储格式的版本。

33.1 TEX、IniTEX、VirTEX

按照 *TEX: the Program*[23] 中确立的术语，TEX 程序有三种变体。IniTEX 是可以生成格式的 TEX 版本；VirTEX 是不预装格式的生产版本；而 TEX 是预装了（plain）格式的生产版本。遗憾的是，这一术语并未被普遍遵守。许多系统并不使用预装格式（在某些操作系统上制作预装格式的过程可能根本无法进行），而把 ‘原生 TEX’ 直接称为 TEX。本手册也沿用这一惯例。

33.1.1 格式的装入

格式文件（扩展名通常为 .fmt）是 TEX 内部结构的紧凑转储。装入一个格式文件所需的时间，远少于装入构成该格式的字体信息与宏所需的时间。

TEX 与 IniTEX 都可以装入格式；用户可在命令行上给出名称

```
% tex &plain
```

或在 ** 提示符之后给出

```
% tex
This is TeX. Version ....
** &plain
```

并在其前加一个与号（在 UNIX 命令行上应写作\& ）。在这两处，格式名之后都可以再跟一个输入文件名。

IniTEX 不需要格式；但如果为（Vir）TEX 未指定格式，它会尝试装入 plain 格式，若找不到则停止。

33.1.2 格式的转储

IniTEX 是 TEX 唯一能转储格式的版本，因为它是 TEX 中唯一拥有命令\dump 的版本，该命令会把内部结构转储为一个格式。它也是 TEX 中唯一拥有命令 \patterns 的版本，该命令用于指定断词模式列表。

不允许在分组内转储，即

```
{ ... \dump }
```

是不允许的。这一限制可避免 TEX 保存栈出现问题。执行 \dump 命令时，TEX 会详尽列出其内部状态，以及已装入字体所对应的字体名，然后结束作业。

由于 IniTEX 既能装入又能转储格式，由此产生一种有趣的可能性。假设你编写了一组建立在 plain TEX 之上的宏 superplain.tex。那么你可以调用

```
% initex &plain superplain
*\dump
```

从而得到一个格式文件 superplain.fmt，它既包含全部 plain，也包含你所有的宏。

33.1.3 格式的预装

在某些系统上，可以中断一个正在运行的程序，并保存其‘核心映像’，使之能作为独立程序启动。把 TEX 程序在装入某个格式之后中断、再由其核心映像生成的可执行程序，称为预装了格式的 TEX 程序。预装格式背后的思路是：在 TEX 装入格式后中断它，并把该程序提供给用户，这样每次运行就省去了装入格式的时间。在当年计算机还相当慢的时代，这一做法是有意义的；如今看来就不那么必要了。况且，转储核心映像也并非总能做到。

33.1.4 IniTEX 的知识

如果尚未装入任何格式，IniTEX 所知极少。例如，它没有开/闭分组字符。然而它又不能完全没有任何知识，否则就无法定义任何东西。

以下是它所知的全部内容。

- \catcode`\\=0, \escapechar=`\\ （见第 24 页）。
- \catcode`\^^M=5, \endlinechar=`\^^M （见第 24 页）。
- \catcode`\ =10 （见第 24 页）。
- \catcode`\%=14 （见第 24 页）。
- \catcode`\^^?=15 （见第 24 页）。
- 对 x = `a..`z,`A..`Z，\catcode$x$=11 （见第 24 页）。
- 对所有其他字符编码，\catcode$x$=12（见第 24 页）。
- 对 x = `A..`Z，\sfcode$x$=999；对所有其他字符，\sfcode$x$=1000（见第 155 页）。
- \lccode`a..`z,`A..`Z=`a..`z, \uccode`a..`z,`A..`Z=`A..`Z；对所有其他字符，\lccode$x$=0、\uccode$x$=0 （见第 39 页）。
- \delcode`.=0；对所有其他字符，\delcode$x$=-1 （见第 159 页）。
- 对所有大小写字母,\mathcode$x$="!7100+x;对所有数字,\mathcode$x$="!7000+x；对所有其他字符，\mathcode$x$=x （见第 162 页）。
- \tolerance=10000、\mag=1000、\maxdeadcycles=25。

33.1.5 TEX 与 IniTEX 的内存大小

TEX 与 IniTEX 的主存大小由源代码中的四个常数控制：mem_bot、mem_top、mem_min 与 mem_max。对 IniTEX，mem_bot = mem_min、mem_top = mem_max；对 TEX，mem_bot 与 mem_top 记录的是用于转储该格式的 IniTEX 的主存大小。因此 TEX 与 IniTEX 的各个版本在这一方面必须相互配套。

TEX 自身的主存可以大于相应 IniTEX 的主存；一般有 mem_min ≤ mem_bot 与 mem_top ≤ mem_max。

对 IniTEX 而言，较小的主存即可满足需要，因为它通常并不真正用来排版。甚至还有可能确实需要较小的主存，因为 IniTEX 需要大量辅助存储空间用于初始化和构建断词表。

33.2 关于格式的进一步说明

33.2.1 兼容性

TEX 有一条奇特的错误信息：'Fatal format error: I'm stymied'（致命格式错误：我没辙了），当 TEX 试图装入一个由不兼容版本的 IniTEX 生成的格式时给出。关于内存大小见上文；关于散列表大小（参数 hash_size 与 hash_prime）及断词例外词典（参数 hyph_size），见第 35 章。

33.2.2 预装字体

在一次 TEX 运行期间，关于字体唯一需要的信息是 tfm 文件中的数据（见下文）。由于一次 TEX 运行（尤其是输入含数学内容时）很容易用到 30–40 个字体，读取所有 tfm 文件的磁盘开销会相当可观。因此 plain 格式与 LATEX 在 IniTEX 中就装入这些度量文件。使用这种格式的 TEX 版本便无需再装入任何 tfm 文件。

另一方面，如果某个格式可能用到一系列字体，那么在实际运行 TEX 时按需装入度量文件可能更为有利。

33.2.3 plain 格式

为 TEX 编写的第一个格式，也是后来所有格式的基础，就是 the TEX book 中描述的 plain 格式。它是以下三类内容的混合：

- 一些根本不可或缺的定义与宏，例如初始的 \catcode 赋值、所有数学定界符定义，以及 \new... 宏；
- 一些有用、但 LATEX 及其他宏包采用不同实现的结构，例如 tabbing 环境；以及
- 一些除最简单应用外并不够用的宏：\item 与 \beginsection 即属此类。

Knuth 本意是让第一类内容作为未来宏宏包的基础，使它们能和平共处（见第 31 章）。这一思路体现在：'plain' 这个名称没有大写——它就是那套基础宏集合。

33.2.4 LATEX 格式

LATEX 格式由 Digital Equipment Corporation 的 Leslie Lamport 编写，并在 [29] 中描述，约于 1985 年发布。LATEX 格式使用自己版本的 plain.tex（称为 lplain.tex），与 plain TEX 并不兼容；一些 plain 宏不可用。尽管如此，它仍包含 plain 格式的大部分内容（即使这些内容与它自身的结构有所重叠）。

LaTeX 是一个功能强大的格式，提供旁注、浮动对象、交叉引用、自动生成目录等设施。它的主要缺点是：定义实际版面的‘样式文件’相当难写（尽管 LaTeX 正在进行一次重大修订以解决这一问题；见 [34] 与 [33]）。其结果是，人们所能使用的大多是 Leslie Lamport 编写的那些样式，其版面风格相当特异。关于一次成功替换这些样式的尝试，见 [6]。

33.2.5 数学格式

有两个格式具备丰富的数学排版设施：AmsTeX [43]（源于美国数学会）与 LAMSTeX [44]。前者在数学排版方面比 plain TeX 或 LaTeX 提供更多设施，但缺少例如 LaTeX 中具有的自动编号与交叉引用等特性。LAMSTeX 则是 AmsTeX 与 LaTeX 的综合。它还包含更多数学特性，例如复杂表格与交换图。

33.2.6 其他格式

除上述之外还存在其他格式：例如 `Phyzzx` [51]、`TeXsis` [35]、Macro TeX [15]、`eplain` [4]、以及 `TeXT1` [13]。这类格式通常提供 LaTeX 的设施，但力求更便于用户自行改造。而且一般而言，它们在编写时的定位是 plain 格式的附加产品。

本书本身也是用一种‘其他格式’写成的：`lollipop` 格式。该格式不包含面向用户的宏，而是供样式设计者用以编程的工具；见 [12]。

33.3 dvi 文件

`dvi` 文件（该词代表‘device independent’，设备无关）包含一次 TeX 运行的输出：它紧凑地转储了由 `\shipout`⟨box⟩ 送到其中的盒子。送出盒子的动作通常发生在输出例程内部，但并不必然如此。

33.3.1 dvi 文件格式

`dvi` 文件是一种面向字节的文件，由一个前导区、一个尾部区以及一页页的列表组成。

后续软件访问一个已完成的 `dvi` 文件时，本质上只能严格顺序访问：各页以反向链表的方式存储。这意味着只有两种访问方式：

- 已知某页的起点后，读到该页结束码即可找到下一页；以及
- 从文件末尾出发，可较快地反向读页，因为每个页起始码都包含上一页的字节位置。

前导区与尾部区包含

- 文档的放大倍数（见下文）；
- 文档所用的计量单位；以及
- 可能的一段注释字符串。

此外，尾部区还包含文件各页上所出现字体定义的列表。

文件的前导区与尾部区都不包含各页字节位置的表。`dvi` 文件格式的完整定义见 [23]。

33.3.2 页面标识

每当发生一次 \shipout，TEX 还会把计数器 0–9 的值写入 dvi 文件和终端。通常只使用计数器 0（即页码），其余计数器为零。这些零不会输出到终端。其他计数器可用于标示文档中更细的结构。日志输出会显示非零计数器，以及它们之间的零计数器。

33.3.3 放大倍数

文档的放大倍数可由⟨integer parameter⟩（整数参数）\mag 指定，它给出放大比率的 1000 倍。

dvi 文件在其前导区与尾部区中包含文档的 \mag 值。如果不使用任何 true（真）尺寸，则 dvi 文件看起来与未放大时相同，只是前导区与尾部区中的 \mag 值不同。

每当使用一个 true 尺寸时，它会被 \mag 的值除，从而最终输出仍具有用户所规定的尺寸。在使用过某个 true 尺寸之后，或在第一页已经送出到 dvi 文件之后，\mag 参数不能再更改。

plain TEX 提供\magnification 宏，用于在不改变页面物理尺寸的情况下对文档整体缩放：

```
\def\magnification{\afterassignment\m@g\count@}
\def\m@g{\mag\count@
  \hsize6.5truein\vsize8.9truein\dimen\footins8truein}
```

解释如下：命令 \m@g 通过 \afterassignment 被保存，放大值（即实际放大因子的 1000 倍）被赋给 \count@。在该赋值之后，宏 \m@g 把放大值赋给 \mag，并把水平尺寸与竖直尺寸重置回原来的值 6.5truein 与 8.9truein。\footins 也一并重置。

33.4 特殊命令

TEX 在很大程度上是与机器无关的，但它仍需要一个用于机器相关扩展的钩子。这就是\special 命令，它向 dvi 文件写入一段⟨balanced text⟩。TEX 并不解释这个记号列表：它假定打印驱动知道该如何处理。\special 命令不应改变页面上的 x、y 位置，从而使 TEX 的实现独立于实际处理该 \special 的设备驱动。。

特殊命令最常见的应用大概是嵌入图形材料，这些材料以某种页面描述语言书写，例如 PostScript。图形的尺寸通常可从包含它的文件中确定（对封装式 PostScript 则通过其 ‘边界框’ 数据），因此 TEX 可以为这类材料预留空间。

33.5 时间

TEX 有四个参数：\year、\month、\day 与\time，它们告知当前作业开始的时刻。在作业运行期间，这些参数不会被更新；用户可随时修改它们。

这四个参数都是整数；\time 给出当前作业开始时刻距午夜的分钟数。

33.6 字体

在 TEX 系统中，字体信息被分为两部分：度量信息（一个字符的高、宽、深），以及某字体中各字符的实际描述。排版程序 TEX 只需要度量信息；打印驱动与屏幕预览程序则需要字符描述。采用这种做法，例如可以让 TEX 相对方便地使用打印机的驻留字体。

33.6.1 字体度量

TEX 字体的度量信息存放在 `tfm` 文件中，该词代表 'TEX font metric'（TEX 字体度量）文件。度量文件包含以下信息（完整定义见 [23]）：

- 字体的设计尺寸；
- `\fontdimen` 参数的值（见第 4 章）；
- 各个字符的高、深、宽以及斜体修正量；
- 字距表；
- 连字表；
- 数学字符的后继字符与延伸字符信息（见第 21 章）。

度量文件采用紧凑格式，但可由辅助程序 `tftopl` 与 `pltotf` 在紧凑格式与可读格式之间相互转换（见 [26]）。其中 `pl` 代表 'property list'（属性表），该术语源自 Lisp 编程语言。`pl` 格式的文件就是纯文本，因此易于编辑；转换之后又可重新作为 `tfm` 文件使用。

33.6.2 虚拟字体

借助 '虚拟字体'（见 [24]），可以实现这样的情形：在 TEX 看来是一个字体，实际上却分布在多个物理字体文件中。此外，虚拟字体还可用来在效果上改变字体文件的内部组织。

对 TEX 本身而言，虚拟字体的存在没有任何区别：一切仍以包含度量信息的 `tfm` 文件为基础。然而，把生成的 `dvi` 文件显示在屏幕上或打印机上的屏幕/打印驱动会查找扩展名为 `.vf` 的文件，以确定各字符应如何解释。例如，`vf` 文件可以指示驱动把某个字符解释为某字体文件中的某个位置，把一个字符解释为多个位置（这是一种构造带重音字符的方式），或嵌入 `\special` 信息（例如设置灰度等级）。

`vf` 文件的可读变体扩展名为 `vpl`，类似于 `tfm` 文件对应的 `pl` 文件（见上文）。`vf` 与 `vpl` 文件之间的转换可由程序 `vftovp` 与 `vptovf` 完成。

然而，由于虚拟字体是设备驱动层面的事情，本书不再给出更多细节。

33.6.3 字体文件

字符描述存放在三类文件中。

gf Generic Font（通用字体）文件。这是 Metafont 程序生成的文件类型。直接使用这种文件的预览程序或打印驱动并不多。

pxl Pixel（像素）文件。`pxl` 格式是一种纯位图格式。因此很容易从例如扫描图像生成 `pxl` 文件。
该格式应被 `pk` 格式取代。像素文件可能相当大，因为其大小随字符尺寸呈平方增长。

pk Packed（压缩）文件。像素文件可通过一种行程编码压缩：不再存储完整位图，而只存储黑白像素 '行程' 的起始位置与长度。这使得 `pk` 文件的大小大致随字符尺寸线性增长。不过，使用压缩字体文件的预览程序或打印驱动必须先把它解压缩才能使用。

存在以下转换程序：`gftopxl`、`gftopk`、`pktopxl`、`pxltopk`。

33.6.4 Computer Modern

标准发行版中随 TEX 附带的唯一字体族是 'Computer Modern' 族。这是 Donald Knuth 依据 [42] 中的术语，对 Monotype Modern 8A 字体的改造，该字体曾用于他的 *Art of Computer Programming* 系列第一卷。这些 '现代风格字体' 都源于 1780 至 1800 年间由法国的 Firmin Didot、意大利的 Giambattista Bodoni 以及德国的 Justus Erich Walbaum 所刻制的字体。前两者所刻的这些字体也被称为 'Didone' 字体。这个名称源自 Vox 字体分类法 [50]。Didone 字体的最终灵感来源是 'Romain du Roi'——约 1692 年由 Nicolas Jaugeon 为法国皇家印刷厂设计的字体。

Didone 字体的特征是强烈的竖向取向与纤细的细线。曲线内部被磨平，进一步强化了竖向感。只要印刷精细、用纸精良，其结果就是一页清晰而亮丽的版面。然而它们相当脆弱；[48] 把它们比作同时代那些名贵却易碎的家具，说人往往不敢使用它们，'因为两者似乎都有碎成一片的危险'。在如今低分辨率（约 300 dpi）打印机大量普及的情况下，Computer Modern 多少是个不幸的选择。

最近，Donald Knuth 通过改变 Computer Modern 族中的参数，开发出一种新字体（更确切地说是一个字体子族）。其结果是一种所谓 'Egyptian' 字体：Computer Concrete [22]。这个名称源于它主要是为 *Concrete Mathematics* 一书设计的。Egyptian（埃及体）在 Vox 分类法中属于 'Mécanes' 类，意为构造出来的、并非由手写字母演化而来；它们笔画非常均匀，衬线呈方形。它们与埃及并无任何关系；只是在 19 世纪上半叶埃及学兴起并流行时，这类字体恰好也流行起来。

33.7 TEX 与 web

TEX 程序是用 WEB 编写的，WEB 是一种编程语言可视为 Pascal 的一个子集，外加一个预处理器。

TEX 没有使用 Pascal 的某些特性，以便于移植到它最初设计所针对的 Pascal 系统之外的其他系统，甚至便于自动翻译为 C 等其他编程语言。例如，它没有使用 Pascal 的 `With` 结构。此外，过程没有输出参数；除写入全局变量外，返回值的唯一途径是通过 `Function` 值。

实际上，WEB 不仅仅是 Pascal 某个子集的超集（更确切地说，它也可与其他编程语言配合使用）；它是一套 '结构化文档系统'。这意味着 WEB 程序员在一个文件中交替编写程序代码及其文档。这种 '文学化编程' 的思想是在 [19] 中提出的；更多信息见 [41]。

然后可用两个辅助程序 Tangle 与 Weave：前者剥离文档并把 WEB 转换为普通 Pascal，后者则把 WEB 文件转换为一个 TEX 文件，用以排版该程序及其文档。

WEB 程序的可移植性由 'change file'（变更文件）机制实现。变更文件是一份对 WEB 文件所做更改的清单，有点像流编辑器脚本。这些更改既可以包括让 WEB 文件适应所用 Pascal 编译器的调整，也可以包括对 TEX 的缺陷修复。因此 `TeX.web` 文件本身从不需要编辑。

33.8 TEX 用户组织

在过去十年中，TEX 用户组成了若干用户组织。存在许多国家级或语言级的用户组织，其中不少还出版通讯。历史最悠久的 TEX 用户组织干脆就叫这个名字：TEX 用户组织（TEX Users Group），简称 TUG，其刊物名为 *TUGboat*。你可以写信至

TEX Users Group
P.O. Box 2311
Portland, OR 97208-2311, USA

或通过 Internet 发送电子邮件至 `office@tug.org`。

第 34 章

追踪与统计

TEX 的实际工作过程往往与程序员所预期的大相径庭，因此需要有办法查明 TEX 是如何得到最终结果的。\tracing... 命令会把某一类的全部信息写入日志文件（若 \tracingonline 为正值，则同时写入终端），而若干 \show... 命令可用于查询 TEX 各项内容的当前状态或值。

在下面的列表中，默认只有 \show 与 \showthe 把输出显示在终端上；其他 \show... 与 \tracing... 命令写入日志文件。若 \tracingonline 为正值，它们还会同时写到终端。

\meaning 以字符串形式给出一个控制序列的含义。
\show 显示一个控制序列的含义。
\showthe 显示把 \the 加在某个记号之前所得的结果。
\showbox 显示一个盒子的内容。
\showlists 显示当前在各种模式中正在构建的部分列表的内容。这在第 63 页讨论。
\tracingcommands 若为 1，TEX 显示所执行的原语命令；若为 2 或更大，还记录条件语句的结果。
\tracingmacros 若为 1，TEX 显示所执行的宏展开以及参数的实际值；若为 2 或更大，还追踪 \output、\everypar 等⟨token parameter⟩（记号参数）。
\tracingoutput 若为正值，日志文件显示送出到 dvi 文件的盒子的转储。
\showboxdepth 显示盒子时所展示的盒子嵌套层数。
\showboxbreadth 显示盒子时，每一层所展示的连续元素个数。
\tracingonline 若此参数为正值，TEX 除日志文件外还把追踪信息写到终端。
\tracingparagraphs 若此参数为正值，TEX 生成断行算法的追踪。
\tracingpages 若此参数为正值，TEX 生成分页算法的追踪。
\tracinglostchars 若此参数为正值，每当访问某个字体中不存在的字符时，TEX 给出诊断信息。plain 默认值： 1。
\tracingrestores 若此参数为正值，TEX 报告分组结束时所恢复的全部值。
\tracingstats 若此参数为 1，TEX 在作业结束时报告各内部数组的使用情况；若为 2，则每送出一页都给出内存需求。

34.1 含义与内容：\show、\showthe、\meaning

控制序列的含义，以及那些代表内部量的控制序列的内容，可由原语命令 \show、\showthe 与 \meaning 取得。

控制序列 \show 与 \meaning 相似：前者将输出到日志文件和终端，而后者产生同样的记号，但这些记号被放入 TEX 的输入流中。

TEX 原语命令的含义就是该命令本身：

```
\show\baselineskip
```

给出

```
\baselineskip=\baselineskip
```

已定义量的含义是其定义：

```
\show\pageno
```

给出

```
\pageno=\count0
```

宏的含义是其参数文本与替换文本：

```
\def\foo#1?#2\par{\set{#1!}\set{#2?}}
\show\foo
```

给出

```
\foo=macro:
#1?#2\par ->\set {#1!}\set {#2?}
```

对没有参数的宏，箭头之前的部分（参数文本）为空。

\showthe 命令会在日志文件和终端上显示 \the 所产生的记号。在执行 \show、\showthe、\showbox 与 \showlists 时，TEX 会请求用户输入；可以通过指定 \scrollmode 来避免。由 \meaning 与 \the 生成的字符类别码为 12，空格除外（见第 28 页）；表示控制序列时使用 \escapechar 的值。

34.2 显示盒子：\showbox、\tracingoutput

若 \tracingoutput 为正值，日志文件会收到所有由 \shipout 写入 dvi 文件的盒子的转储表示。命令 \showbox⟨8-bit number⟩ 使用同样的表示。

在第一种情形下，TEX 会报告 'Completed box being shipped out'（正在送出完成的盒子）；在第二种情形下，它会进入 \errorstopmode，并告诉用户 'OK. (see the transcript file)'。若 \tracingonline 为正值，盒子也会显示在终端上；若已指定 \scrollmode，TEX 不会停下来等待输入。

所转储的嵌套盒子层数的上限是 \showboxdepth;每访问一层,最多显示 \showboxbreadth 个项目，列表的其余部分用 etc. 概括。对每个盒子，依次给出其高、深、宽；对字符则说明取自哪个字体。

例如

```
\font\tenroman=cmr10 \tenroman
\setbox0=\hbox{g}
\showbox0
```

日志文件将显示

```
\hbox(4.30554+1.94444)x5.00002
.\tenroman g
```

表示该盒子高 4.30554pt、深 1.94444pt、宽 5.00002pt，其中包含来自字体 \tenroman 的字符 'g'。注意：所有尺寸的第五位小数可能被四舍五入，因为 TEX 以 2^{-16}pt 的倍数为单位工作。

下一个例子含嵌套盒子：

```
\vbox{\hbox{g}\hbox{o}}
```

它在两个盒子之间包含 \baselineskip 胶。执行 \showbox 命令时，日志文件输出为：

```
\vbox(16.30554+0.0)x5.00002
.\hbox(4.30554+1.94444)x5.00002
..\tenroman g
.\glue(\baselineskip) 5.75002
.\hbox(4.30554+0.0)x5.00002
..\tenroman o
```

每进入一个新层级，行首就多一个点。注意 TEX 明确指出该胶是 \baselineskip 胶；对所有自动插入的胶，它都会像这样标注名称。此处 baselineskip 胶的值使得各盒子的基线相距 12 磅。

现在来看显式的（用户给出的）胶。TEX 会指出它被拉伸或收缩的比率。

例如

```
\hbox to 20pt {\kern10pt \hskip0pt plus 5pt}
```

给出（表示可用拉伸量被乘以 2.0）：

```
\hbox(0.0+0.0)x20.0, glue set 2.0
.\kern 10.0
.\glue 0.0 plus 5.0
```

而

```
\hbox to 0pt {\kern10pt \hskip0pt minus 20pt}
```

给出（收缩量被乘以 0.5）

```
\hbox(0.0+0.0)x0.0, glue set - 0.5
.\kern 10.0
.\glue 0.0 minus 20.0
```

。

这是一个含可无限拉伸或收缩的胶的例子：

```
\hbox(4.00000+0.14000)x15.0, glue set 9.00000fil
```

这表示该水平盒子中含有 fil 胶，其排版结果宽度为 9pt。

不足盒子与其他盒子一样被转储，但会给出通常的 'Underfull hbox detected at line...' 信息。溢出水平盒子中含有一条宽为 \overfullrule 的竖直引线：

```
\hbox to 5pt {\kern10pt}
```

给出

```
\hbox(0.0+0.0)x5.0
.\kern 10.0
.\rule(*+*)x5.0
```

盒子引线不会被完整转储：

```
.\leaders 40.0
..\hbox(4.77313+0.14581)x15.0, glue set 9.76852fil
...\tenrm a
...\glue 0.0 plus 1.0fil
```

是下面这一句的转储

```
\leaders\hbox to 15pt{\tenrm a\hfil}\hskip 40pt
```

引线盒子前后的胶也不标注。

34.3 总体统计

参数\tracingstats 可用于让 TEX 在作业结束时报告资源的总体使用情况。有些生产版本的 TEX 可能没有此项。

作为例子，下面是本书的统计数据：

```
Here is how much of TeX's memory you used:
```

字符串内存（上限为 'pool size'）：

```
 877 strings out of 4649
 9928 string characters out of 61781
```

主存、控制序列、字体内存：

```
 53071 words of memory out of 262141
 2528 multiletter control sequences out of 9500
 20137 words of font info for 70 fonts,
             out of 72000 for 255
```

断词：

```
 14 hyphenation exceptions out of 607
```

栈：依次为输入栈、嵌套栈、参数栈、缓冲栈与保存栈

```
 17i,6n,19p,245b,422s stack positions out of
 300i,40n,60p,3000b,4000s
```

第 35 章

错误、灾难与求助

TEX 运行时可能出现各种错误。本章讨论输入中的错误如何显示，以及 TEX 内部数据结构可能发生何种溢出。

\errorcontextlines（仅 TEX3）错误信息中额外显示的上下文行数。
\errmessage 报告一个错误，把本命令的参数作为消息给出。
\errhelp 当用户在 \errmessage 之后请求进一步帮助时所显示的记号。

35.1 错误信息

当 TEX 运行在 \errorstopmode 下时（通常都是如此；其他运行模式见第 32 章），发生的错误会报告到用户终端，且 TEX 会请求用户给出进一步指示。错误的发生，要么是由于 TEX 的某种内部状况，要么是由于某个宏发出了\errmessage 命令。

发生错误时，TEX 会显示出错所在的那一行输入；如果出错命令不在该行，而是例如在某个可能从这一行间接调用的宏中，那么该命令所在的行也会显示出来。如果出错命令是被间接调用的，则还会显示前若干次宏调用，行数由\errorcontextlines 指定。

\errorcontextlines = 0 时，只打印... 作为存在上下文的唯一标示；负值则连这一点也抑制。

对于导致出错命令的调用序列中的每个宏，TEX 会尝试显示其前后的一些记号。先显示一行，以（间接）出错命令结尾；再在下一行给出其后的一些记号。

```
This paragraph ends \vship1cm with a skip.
```

给出

```
! Undefined control sequence.
l.1 This paragraph ends \vship
                                    1cm with a skip.
```

如果 TEX 不是运行在某种非停止模式下，用户便有机会修补错误或请求进一步信息。一般可使用以下选项：

⟨**return**⟩ TEX 将继续处理。如果错误无关紧要，TEX 要么能忽略、要么能自行修补，这是最省事的出路。

h 给出关于该错误的进一步细节。如果错误是由 \errmessage 命令引起的，则\errhelp 记号会在此显示。

i 插入（Insert）。用户可插入一些内容。例如，如果某个控制序列拼写错误，有时可以直接插入正确命令，如上面这个例子中的

```
i\vskip
```

此外，这也是插入 `\show` 命令以检查 TEX 内部状态的机会。然而，如果 TEX 正处在扫描某个复杂结构的过程中，这类命令不会被执行，甚至会添乱。

`s`（`\scrollmode`）滚动跳过后续错误，但仍显示消息。TEX 会自行修补之后的所有错误。这是个方便的选项，例如当错误发生在对齐中时，因为随后出现的错误往往相当多。

`r`（`\nonstopmode`）不停地运行。TEX 不会再为用户交互而停止。

`q`（`\batchmode`）安静运行。TEX 不会再为用户交互而停止，也不再给出任何终端输出。

`x` 退出（Exit）。中止本次 TEX 运行。

`e` 编辑（Edit）。并非所有 TEX 系统都提供此选项。若提供，则中止本次 TEX 运行并启动一个编辑器，打开输入文件，甚至可能直接定位到出错行。

35.2 溢出错误

严酷的现实对 TEX 工作过程的复杂程度施加了一些限制。有些限制由编译时常量决定，因此相当宽松；另一些则在很大程度上依赖于具体的计算机实现。

下面列出所有会促使 TEX 报告 'Capacity exceeded'（容量超限）的溢出错类别。所涉及的大多数上限是由编译时常量决定的；此处括号中给出的值，是 TEX 源码列表 [25] 中所用的值。实际值可能不同，而且很可能就是如此。请记住：TEX 诞生于当年，那时即便大型计算机也相当小。

35.2.1 缓冲区大小 (500)

所有已打开文件的当前行都保存在 TEX 的输入缓冲区中,用`\csname...\endcsname` 正在构建的控制序列名也同样如此。

35.2.2 例外词典 (307)

由 `\hyphenation` 指定的断词例外数目上限必须是素数。系统会分配两个以此数量为长度的半字数组。

改变此数值会使格式互不兼容；也就是说，TEX 只能使用由具有相同此常量值的 IniTEX 所生成的格式。

35.2.3 字体内存 (20 000)

关于字体的信息存放在一个内存字数组中。在 IniTEX 中预装过多字体很容易使其溢出。

35.2.4 分组层级

已打开分组的数目应能记录在一个四分之一字中。没有与之对应的编译时常量。

35.2.5 散列表大小 (2100)

控制序列数目的上限。建议此数不超过主存大小的 10%。TEX 与 IniTEX 中的该值应一致；`hash_prime` 值也应一致。

此值相当小；对于比 plain TEX 更复杂的宏宏包，取约 3000 更为现实。

35.2.6 字符串数目 (3000)

字符串数目的最大值应能记录在一个半字中。

35.2.7 输入栈大小 (200)

对每个输入源，都会在输入栈上分配一项。典型的输入源是输入文件（但同时打开的文件数目更有限，见下文），以及记号列表，例如记号变量、宏替换文本和对齐模板。具有 '失控递归' 的宏（例如`\def\mac{\mac}` ）会使此栈溢出。

TEX 在此做了一些优化：在某个记号列表的最后一次调用之前，会清理所有以该调用结尾的记号列表。这一过程类似于 '消解尾递归'（见第 11 章）。

35.2.8 主存大小 (30 000)

TEX 几乎所有 '动态' 对象，例如宏定义文本和当前页上的全部材料，都存放在主存数组中。格式仅用于宏定义就可能占用 20 000 字主存，而包含例如 LATEX picture 环境的复杂页面很容易使此数组溢出。

TEX 的主存以字为单位划分，一个半字应能寻址整个内存。因此在当前的 32 位计算机上，最常见的选择是让主存大小不超过 64K 字节。这样半字地址便可存于 16 位，即半个机器字。

然而存在所谓 'Big TEX' 实现，其主存大于 64K 字。此时大多数编译器会分配 32 位字来寻址该内存，即使（比如说）18 位就已足够。因此 Big TEX 一旦越过 64K 门槛就会立刻大出许多。所以它们通常不出现在微型计算机上，尽管为这类机器实现虚拟内存方案是可能的；例如见 [45]。

TEX 可以有比 IniTEX 更大的主存；详见第 33 章。

35.2.9 参数栈大小 (60)

宏参数中可能包含带有更多参数的宏调用。可嵌套出现的参数数目受参数栈大小限制。

35.2.10 模式内存 (8000)

断词模式存放在一个 trie（检索树）数组中。对英语或意大利语，默认的 8000 个断词模式似乎足够；但对荷兰语或德语则不够。

35.2.11 每种语言的模式内存 op 数

断词 op 的数目（见关于断词的文献：[30] 及 [25] 附录 H）应能记录在一个四分之一字中。没有与之对应的编译时常量。TEX 第 2 版具有相同的上限，但溢出时不给出错误信息。同样，对荷兰语、德语等语言，此上限过低。有些 TEX 版本在此处有更高的上限。

35.2.12 字符串池大小 (32 000)

字符串即错误信息与控制序列名。它们按每字符一个字节存放。TEX 初始有约 23 000 个字符的字符串。

如果用户在一个相当庞大的宏宏包之上又定义大量控制序列，字符串池就会溢出。然而，即使用户不定义任何新命令，也可能发生溢出：交叉引用方案同样是通过定义控制序列来工作的。对大型文档，40 000 或 60 000 的池大小大概足够。

35.2.13 保存栈大小 (600)

在分组内被赋值的量，必须在该分组结束后恢复。保存栈用于存放待恢复的值；保存栈的大小限制了可恢复值的数目。

对某个量交替进行全局赋值与局部赋值会导致 '保存栈堆积'：每次在一次全局赋值之后的局部赋值，都会保存该变量的先前值。因此这种赋值的交替会使保存栈上的项无谓地膨胀。

35.2.14 语义嵌套栈大小 (40)

每当 TEX 切换到嵌套在另一模式内部的模式时（例如在`\vbox` 中处理`\hbox` 时），当前状态就被压入语义嵌套栈。语义嵌套栈大小是可压入层级的最大数目。

35.2.15 文本输入层级 (6)

`\input` 文件的嵌套层数必须非常有限，因为所有当前行都要保存在输入缓冲区中。

第 36 章

TEX 的语法

本书许多章节都包含定义 TEX 形式语法的文法片段。本章解释该文法重写规则的结构，并介绍一些关键概念。

在 the TEX book 中，文法出现在第 24–27 章。一份更为严谨的 TEX 文法可见于 [1]。本书给出的文法与 the TEX book 中的几乎相同。

36.1 记号

文法的基础是以下要素

文法符号 这些符号用尖括号括起：

⟨term⟩

控制序列 这些符号以等宽字体给出，并用反斜杠表示转义字符：

`\command`

最后还有

关键字 同样以等宽字体给出：

`keyword`

这是一批有限的词，在特定上下文中对 TEX 具有特殊含义；见下文。

文法的这三种要素用于语法规则中：

⟨snark⟩ ⟶ `boojum` | ⟨empty⟩

这条规则表示：文法实体⟨snark⟩ 要么是关键字 `boojum`，要么是文法实体⟨empty⟩。

还有另外两条记号约定。第一，双引号用于表示十六进制（以 16 为底）记法。例如`"ab56` 代表 $10 \times 16^3 + 11 \times 16^2 + 5 \times 16^1 + 6 \times 16^0$。第二，用下标表示类别码。因此 $\mathtt{a}_{12}$ 表示一个类别码为 12 的 ‘a’。

36.2 关键字

关键字是一个字符（或字符记号）序列，其类别码可以是除 13（活动字符）外的任何值。与控制序列不同，TEX 在关键字中不区分大小写。关键字中的大写字符通过加 32 转换为小写；此处不使用 `\lccode` 与 `\uccode`。此外，任何关键字之前都可以有若干可选空格。

因此 `true cm` 与 `truecm` 都是合法的。然而迄今最奇特的例子来自文法规则

⟨fil unit⟩ ⟶ `fil` | ⟨fil unit⟩`l`

它意味着`fil L l`也是一个合法的⟨fil dimen⟩。由此可能产生奇怪的错误；例子见第 107 页。

以下是全部关键字的完整列表：`at`、`bp`、`by`、`cc`、`cm`、`dd`、`depth`、`em`、`ex`、`fil`、`height`、`in`、`l`、`minus`、`mm`、`mu`、`pc`、`plus`、`pt`、`scaled`、`sp`、`spread`、`to`、`true`、`width`。

36.3 特定文法符号

有些文法符号出现在大量规则中。其中之一是⟨optional spaces⟩（可选空格）。其含义大概 显而易见，但以下是其形式定义：

⟨optional spaces⟩ ⟶ ⟨empty⟩ | ⟨space token⟩⟨optional spaces⟩

也就是说，⟨optional spaces⟩ 就是零个或多个空格记号。

其他一些符号则不那么一目了然。下面介绍其中几个。

36.3.1 ⟨equals⟩

在赋值中等号是可选的；因此 TEX 的文法中有这样一个符号：

⟨equals⟩ ⟶ ⟨optional spaces⟩ | ⟨optional spaces⟩$=_{12}$

36.3.2 ⟨filler⟩、⟨general text⟩

比⟨optional spaces⟩ 更隐晦的，是在某些位置允许出现的空格与 `\relax` 记号的组合，例如

```
\setbox0= \relax\box1
```

涉及的量是

⟨filler⟩ ⟶ ⟨optional spaces⟩ | ⟨filler⟩`\relax`⟨optional spaces⟩

⟨filler⟩ 的一个重要之处出现在

⟨general text⟩ ⟶ ⟨filler⟩`{`⟨balanced text⟩⟨right brace⟩

⟨general text⟩ 跟在 `\message`、`\uppercase`、`\mark` 等控制序列之后。⟨balanced text⟩ 两侧的花括号在下一节解释。

36.3.3 `{}` 与⟨left brace⟩⟨right brace⟩

TEX 文法对花括号采用了一个或许有些不妥的约定。首先，

`{` 与`}`

代表的花括号既可以是显式的开/闭分组字符，也可以是由 `\let` 定义的控制序列，例如

```
\let\bgroup={ \let\egroup=}
```

而文法符号

⟨left brace⟩ 与⟨right brace⟩

代表显式的开/闭分组字符，即类别码分别为 1 和 2 的字符。

这两种花括号有各种组合。盒子两侧的花括号可以是隐式的：

`\hbox`⟨box specification⟩`{`⟨horizontal mode material⟩`}`

宏定义两侧则必须是显式花括号：

⟨definition text⟩ ⟶ ⟨parameter text⟩⟨left brace⟩⟨balanced text⟩⟨right brace⟩

最后，上面提到的⟨general text⟩ 必须显式闭合，但可以隐式开启：

⟨general text⟩ ⟶ ⟨filler⟩{⟨balanced text⟩⟨right brace⟩

⟨general text⟩ 的闭花括号必须是显式的，因为 general text 是一个记号列表，其中可能含有 `\egroup` 记号。TEX 会通过展开来寻找⟨general text⟩ 的开花括号。

36.3.4 ⟨math field⟩

在数学模式中，上下标、加 `\underline` 等各种运算所取的参数是一个⟨math field⟩：要么是单个符号，要么是一个分组。确切定义如下。

⟨math field⟩ ⟶ ⟨math symbol⟩ | ⟨filler⟩{⟨math mode material⟩}
⟨math symbol⟩ ⟶ ⟨character⟩ | ⟨math character⟩

⟨character⟩ 见第 38 页，⟨math character⟩ 见第 158 页。

36.4 TEX 第 2 版与第 3 版的差异

1989 年，Knuth 发布了 TEX 第 3.0 版，这是自 1986 年发布的第 2.0 版以来 TEX 的首次真正变化（TEX 第 0 版于 1982 年发布；关于 TEX 历史的更多信息见 [18]）。其间的各中间版本都只是缺陷修复。

第 2 版与第 3 版的主要区别在于：8 位输入成为可能。与之相关，原先为 127 或 128 的若干量分别被提高到 255 或 256。以下是简要列表，完整描述见 [20]。

- 所有‘码’（`\catcode`、`\sfcode` 等；见第 40 页）现在作用于 256 个字符编码，而不再是 128 个。
- 一个编码为 `\endlinechar` 的字符会被附加到行尾，除非该参数为负或大于 255（原先为 127）（见第 23 页）。
- 如果 `\escapechar` 为负或大于 255（原先为 127），`\write` 及其他命令不输出转义字符（见第 28 页）。
- `^^` 替换机制已被扩展（见第 26 页）。
- 新增参数 `\language`、`\inputlineno`、`\errorcontextlines`、`\lefthyphenmin`、`\righthyphenmin`、`\badness`、`\holdinginserts`、`\emergencystretch`，以及命令 `\noboundary`、`\setlanguage`。
- 当分页不是发生在罚分项上时，`\outputpenalty` 的值不再为零；相反它为 10 000（见第 187 页）。

plain 格式也做了更新，主要是为 `\lefthyphenmin` 等参数设置默认值，但也新增了若干宏。

第 37 章

TEX 原语术语表

本章给出 TEX 所有原语的列表。每个控制序列之后给出该命令或参数的语法类别，并附简短描述。部分命令还给出其使用语法。

对于参数，给出其所属的类别。在 the TEX book 中没有语法类别的命令，在本列表中标记为 ‘⟨expandable command⟩’ 或 ‘⟨primitive command⟩’。

语法术语如⟨equals⟩ 和⟨optional space⟩ 在第 36 章中解释。

`\-` ⟨horizontal command⟩ 自定连字符；等价于 `\discretionary""{-}{}{}`。可用于指示单词中可断字的位置。

`\char32` ⟨horizontal command⟩ 控制空格。插入与空格记号在 `\spacefactor` = 1000 时相同的间距。

`\char47` ⟨primitive command⟩ 斜体修正：插入由前一个字符指定的紧排。每个字符在 `tfm` 文件中都规定了一个斜体修正量（可能为零）。对于倾斜字体，此修正用于补偿伸出量。

`\above`⟨`dimen`⟩ ⟨generalized fraction command⟩ 横线宽度指定的分式。

`\abovedisplayshortskip` ⟨glue parameter⟩ 当展示公式前一行较短时，展示式上方的粘连。

`\abovedisplayskip` ⟨glue parameter⟩ 展示式上方的粘连。

`\abovewithdelims`⟨$\texttt{delim}_1$⟩⟨$\texttt{delim}_2$⟩⟨`dimen`⟩ ⟨generalized fraction command⟩ 带定界符的广义分式。

`\accent`⟨`8-bit number`⟩⟨`optional assignments`⟩⟨`character`⟩ ⟨horizontal command⟩ 在字符上放置重音符号的命令。

`\adjdemerits` ⟨integer parameter⟩ 相邻行视觉上不协调时的罚值。plain TEX 中默认值为 `10000`。

`\advance`⟨`numeric variable`⟩⟨`optional by`⟩⟨`number`⟩ ⟨arithmetic assignment⟩ 用于增减一个⟨numeric variable⟩ 的算术命令，即一个⟨count variable⟩、⟨dimen variable⟩、⟨glue variable⟩，或⟨muglue variable⟩。

`\afterassignment`⟨`token`⟩ ⟨primitive command⟩ 保存下一个记号，使其在下一次赋值之后执行。用此方式只能保存一个记号。

`\aftergroup`⟨`token`⟩ ⟨primitive command⟩ 保存下一个记号，使其在当前分组结束后插入。用此方式可保存多个记号。

`\atop`⟨`dimen`⟩ ⟨generalized fraction command⟩ 将对象上下叠放。

`\atopwithdelims`⟨$\texttt{delim}_1$⟩⟨$\texttt{delim}_2$⟩ ⟨generalized fraction command⟩ 带定界符地将对象上下叠放。

`\badness` ⟨internal integer⟩ （仅 TEX3）最近构造的盒子的不良度。

\baselineskip ⟨glue parameter⟩ 竖直列表中相邻盒子之间的“理想”基线距离；plain TEX 中为 12pt。

\batchmode ⟨interaction mode assignment⟩ TEX 自行修补错误并在遇到严重错误（如缺少输入文件）时紧急停止，但不产生终端输出。

\begingroup ⟨primitive command⟩ 打开一个分组，必须用\endgroup 关闭。

\belowdisplayshortskip ⟨glue parameter⟩ 当展示公式前一行较短时，展示式下方的粘连。

\belowdisplayskip ⟨glue parameter⟩ 展示式下方的粘连。

\binoppenalty ⟨integer parameter⟩ 在未被子公式括起的二元运算符之后断行的罚值。plain TEX 默认值： 700。

\botmark ⟨expandable command⟩ 当前页上最后一个标记。

\box⟨8-bit number⟩ ⟨box⟩ 使用一个盒子寄存器，并清空它。

\boxmaxdepth ⟨dimen parameter⟩ 盒子允许的最大深度。plain TEX 中默认值为 \maxdimen。

\brokenpenalty ⟨integer parameter⟩ 在断字行之后分页的附加罚值。plain TEX 中默认值为 100。

\catcode⟨8-bit number⟩ ⟨internal integer⟩；该控制序列本身是一个 ⟨codename⟩。访问类别码。

\char⟨number⟩ ⟨character⟩ 显式指定一个待排版的字符。

\chardef⟨control sequence⟩⟨equals⟩⟨number⟩ ⟨shorthand definition⟩ 定义一个控制序列作为某个字符码的同义名。

\cleaders ⟨leaders⟩ 与\leaders 相同，但对于盒子引导符，多余空间会被均分在引导符之前和之后的粘连项中。

\closein⟨4-bit number⟩ ⟨primitive command⟩ 关闭一个输入流。

\closeout⟨4-bit number⟩ ⟨primitive command⟩ 关闭一个输出流。

\clubpenalty ⟨integer parameter⟩ 在段落第一行之后分页的附加罚值。plain TEX 中默认值为 150。

\copy⟨8-bit number⟩ ⟨box⟩ 使用一个盒子寄存器，但保留其内容。

\count⟨8-bit number⟩ ⟨internal integer⟩；该控制序列本身是一个 ⟨register prefix⟩。访问计数寄存器。

\countdef⟨control sequence⟩⟨equals⟩⟨8-bit number⟩ ⟨shorthand definition⟩；该控制序列本身是一个 ⟨registerdef⟩。定义一个控制序列作为某个 \count 寄存器的同义名。

\cr ⟨primitive command⟩ 结束一条对齐行。

\crcr ⟨primitive command⟩ 若一条对齐行尚未被 \cr 结束，则结束它。

\csname ⟨expandable command⟩ 开始构造一个控制序列的名字。必须与 \endcsname 配对使用。

\day ⟨integer parameter⟩ 当前作业的日期（日）。

\deadcycles ⟨special integer⟩ 计数器，记录输出例程被调用但未发生 \shipout 的次数。若此数达到 \maxdeadcycles，TEX 给出错误信息。plain TEX 默认值： 25。

\def ⟨def⟩ 开始一个宏定义。

\defaulthyphenchar ⟨integer parameter⟩ 装入字体时 \hyphenchar 的值。plain TEX 中的默认值为 `\-。

\defaultskewchar ⟨integer parameter⟩ 装入字体时 \skewchar 的值。plain TEX 中的默认值为 -1。

\delcode⟨8-bit number⟩ ⟨internal integer⟩；该控制序列本身是一个 ⟨codename⟩。访问用于指定某字符在 \left 或 \right 之后如何用作定界符的码。

\delimiter⟨27-bit number⟩ ⟨math character⟩ 显式指定一个定界符。

\delimiterfactor ⟨integer parameter⟩ 带定界符公式中应由定界符覆盖部分的1000 倍。plain TEX 默认值： 901。

\delimitershortfall ⟨integer parameter⟩ 带定界符公式中允许未被定界符覆盖部分的尺寸。plain TEX 默认值： 5pt。

\dimen⟨8-bit number⟩ ⟨internal dimen⟩；该控制序列本身是一个 ⟨register prefix⟩。访问尺寸寄存器。

\dimendef⟨control sequence⟩⟨equals⟩⟨8-bit number⟩ ⟨shorthand definition⟩；该控制序列本身是一个 ⟨registerdef⟩。定义一个控制序列作为某个 \dimen 寄存器的同义名。

\discretionary{*pre-break*}{*post-break*}{*no-break*} ⟨horizontal command⟩ 指定字符序列在断行处如何拆分。

\displayindent ⟨dimen parameter⟩ 展示式居中盒子因悬挂缩进而缩进的距离。此值对每个展示式自动设置。

\displaylimits ⟨primitive command⟩ 恢复上下限的默认位置。

\displaystyle ⟨primitive command⟩ 选择数学排版的展示样式。

\displaywidowpenalty ⟨integer parameter⟩ 在展示公式上方最后一行之前分页的附加罚值。plain TEX 中默认值为 50。

\displaywidth ⟨dimen parameter⟩ 展示式居中盒子的宽度。此值对每个展示式自动设置。

\divide⟨numeric variable⟩⟨optional by⟩⟨number⟩ ⟨arithmetic assignment⟩ 用于除以一个⟨numeric variable⟩ 的算术命令（见 \advance）。

\doublehyphendemerits ⟨integer parameter⟩ 相邻两行都以连字符结尾时的罚值。plain TEX 中默认值为 10 000。

\dp⟨8-bit number⟩ ⟨internal dimen⟩；该控制序列本身是一个 ⟨box dimension⟩。盒子寄存器中盒子的深度。

\dump ⟨vertical command⟩ 转储一个格式文件；仅在 IniTEX 中可行，不允许在分组内部使用。

\edef ⟨def⟩ 开始一个宏定义；替换文本在定义时即被展开。

\else ⟨expandable command⟩ 选择条件语句的⟨false text⟩，或 \ifcase 的默认分支。

\emergencystretch ⟨dimen parameter⟩ （仅 TEX3）断行算法第三趟处理时，段落各行所假定的额外伸长量。

\end ⟨vertical command⟩ 结束本次运行。

\endcsname ⟨expandable command⟩ 结束由 \csname 开始的控制序列名。

\endgroup ⟨primitive command⟩ 结束由\begingroup 打开的分组。

\endinput ⟨expandable command⟩ 在当前行之后结束当前文件的输入。

\endlinechar ⟨integer parameter⟩ 附加到输入行末的行尾字符的字符码。IniTEX 默认值： 13。

\eqno⟨math mode material⟩$$ ⟨eqno⟩ 在展示公式右侧放置公式编号。

\errhelp ⟨token parameter⟩ 当用户在 \errmessage 之后请求帮助时将显示的记号。

\errmessage⟨general text⟩ ⟨primitive command⟩ 报告错误并给予用户采取行动的机会。

\errorcontextlines ⟨integer parameter⟩ （仅 TEX3）错误信息中额外显示的上下文行数。

\errorstopmode ⟨interaction mode assignment⟩ 遇到错误时请求用户输入。

\escapechar ⟨integer parameter⟩ 控制序列转换为字符记号时所用转义字符的编号。IniTEX 默认值： 92。

\everycr ⟨token parameter⟩ 在每个 \cr 或非多余的 \crcr 之后插入的记号列表。

\everydisplay ⟨token parameter⟩ 在展示式开头插入的记号列表。
\everyhbox ⟨token parameter⟩ 在水平盒子开头插入的记号列表。
\everyjob ⟨token parameter⟩ 在每次作业开头插入的记号列表。
\everymath ⟨token parameter⟩ 在非展示式数学开头插入的记号列表。
\everypar ⟨token parameter⟩ 在段落文本之前插入的记号列表。
\everyvbox ⟨token parameter⟩ 在竖直盒子开头插入的记号列表。
\exhyphenpenalty ⟨integer parameter⟩ 当断前文本为空时，在自定断行处断行的罚值。plain TEX 中默认值为 50。
\expandafter ⟨expandable command⟩ 取下两个记号，将第二个记号的展开结果置于第一个记号之后。
\fam ⟨integer parameter⟩ 当前字体族的编号。
\fi ⟨expandable command⟩ 所有条件语句的结束定界符。
\finalhyphendemerits ⟨integer parameter⟩ 当段落倒数第二行以连字符结尾时附加的罚值。plain TEX 默认值 5000。
\firstmark ⟨expandable command⟩ 当前页上第一个标记。
\floatingpenalty ⟨integer parameter⟩ 当插入物被拆分时，加到 \insertpenalties 上的罚值。
\font⟨control sequence⟩⟨equals⟩⟨file name⟩⟨at clause⟩ ⟨simple assignment⟩ 将一个控制序列与一个 tfm 文件关联。单独使用时，该控制序列是一个⟨font⟩，表示当前字体。
\fontdimen⟨number⟩⟨font⟩ ⟨internal dimen⟩ 访问字体的各种参数。
\fontname⟨font⟩ ⟨primitive command⟩ 字体的外部名称。
\futurelet⟨control sequence⟩⟨token$_1$⟩⟨token$_2$⟩ ⟨let assignment⟩ 将⟨token$_2$⟩ 的含义赋予⟨control sequence⟩。
\gdef ⟨def⟩ \global\def 的同义命令。
\global ⟨prefix⟩ 使下一个定义、算术语句或赋值成为全局的。
\globaldefs ⟨integer parameter⟩ 覆盖 \global 设定：此参数为正值时使所有赋值成为全局的，为负值时使所有赋值成为局部的。
\halign⟨box specification⟩{⟨alignment material⟩} ⟨vertical command⟩ 水平对齐。展示式对齐：

```
$$\halign⟨box specification⟩{...}⟨optional assignments⟩$$
```

\hangafter ⟨integer parameter⟩ 若为正，表示缩进开始之前的行数；若为负，其绝对值表示从段落首行开始的缩进行数。每段之后恢复默认值 1。
\hangindent ⟨dimen parameter⟩ 若为正，表示从左页边缩进的距离；若为负，表示从右页边缩进距离的负值。每段之后恢复默认值 0pt。
\hbadness ⟨integer parameter⟩ TEX 不报告水平盒子过松或过满的阈值。plain TEX 默认值： 1000。
\hbox⟨box specification⟩{⟨horizontal material⟩} ⟨box⟩ 构造一个水平盒子。
\hfil ⟨horizontal command⟩ 等价于\hskip 0cm plus 1fil 的水平跳跃。
\hfill ⟨horizontal command⟩ 等价于\hskip 0cm plus 1fill 的水平跳跃。
\hfilneg ⟨horizontal command⟩ 等价于\hskip 0cm minus 1fil 的水平跳跃。
\hfuzz ⟨dimen parameter⟩ TEX 判定水平盒子过满之前所容忍的超出量。plain TEX 默认值： 0.1pt。
\hoffset ⟨dimen parameter⟩ 页面相对于参考点向右偏移的距离，参考点位于距左页边一英寸处。
\holdinginserts ⟨integer parameter⟩ （仅 TEX3）若此值为正，则在插入 \output 记号时，插入物不放入其盒子中。
\hrule ⟨vertical command⟩ 沿水平方向伸展的线。
\hsize ⟨dimen parameter⟩ 竖直盒子内文本排版所用的行宽。
\hskip⟨glue⟩ ⟨horizontal command⟩ 在水平模式下插入一个粘连项。

\hss ⟨horizontal command⟩ 等价于\hskip 0cm plus 1fil minus 1fil 的水平跳跃。

\ht⟨8-bit number⟩ ⟨internal dimen⟩；该控制序列本身是一个 ⟨box dimension⟩。盒子寄存器中盒子的高度。

\hyphenation⟨general text⟩ ⟨hyphenation assignment⟩ 为当前 \language 值定义断字例外词。

\hyphenchar⟨font⟩ ⟨internal integer⟩ 在其后插入\discretionary{}{}{} 的字符编号。

\hyphenpenalty ⟨integer parameter⟩ 一般情况下在自定断行处断行的罚值。plain TEX 中默认值为 50。

\if⟨token$_1$⟩⟨token$_2$⟩ ⟨expandable command⟩ 测试字符码是否相等。

\ifcase⟨number⟩⟨case$_0$⟩\or...\or⟨case$_n$⟩\else⟨other cases⟩\fi ⟨expandable command⟩ 枚举式分支语句。

\ifcat⟨token$_1$⟩⟨token$_2$⟩ ⟨expandable command⟩ 测试两个字符是否具有相同的类别码。

\ifdim⟨dimen$_1$⟩⟨relation⟩⟨dimen$_2$⟩ ⟨expandable command⟩ 比较两个尺寸。

\ifeof⟨4-bit number⟩ ⟨expandable command⟩ 测试一个文件是否已读完，或不存在。

\iffalse ⟨expandable command⟩ 此测试恒为假。

\ifhbox⟨8-bit number⟩ ⟨expandable command⟩ 测试盒子寄存器中是否含有水平盒子。

\ifhmode ⟨expandable command⟩ 测试当前模式是否为（可能是受限的）水平模式。

\ifinner ⟨expandable command⟩ 测试当前模式是否为内部模式。

\ifmmode ⟨expandable command⟩ 测试当前模式是否为（可能是展示式）数学模式。

\ifnum⟨number$_1$⟩⟨relation⟩⟨number$_2$⟩ ⟨expandable command⟩ 测试数值之间的关系。

\ifodd⟨number⟩ ⟨expandable command⟩ 测试一个数是否为奇数。

\iftrue ⟨expandable command⟩ 此测试恒为真。

\ifvbox⟨8-bit number⟩ ⟨expandable command⟩ 测试盒子寄存器中是否含有竖直盒子。

\ifvmode ⟨expandable command⟩ 测试当前模式是否为（可能是内部的）竖直模式。

\ifvoid⟨8-bit number⟩ ⟨expandable command⟩ 测试盒子寄存器是否为空。

\ifx⟨token$_1$⟩⟨token$_2$⟩ ⟨expandable command⟩ 测试宏展开是否相等，或字符码与类别码是否相等。

\ignorespaces ⟨primitive command⟩ 展开其后的记号，直到遇到非 ⟨space token⟩ 的内容为止。

\immediate ⟨primitive command⟩ 前缀，用于使输出操作立即执行。

\indent ⟨primitive command⟩ 切换到水平模式并插入宽度为 \parindent 的盒子。在竖直模式下，此命令会在⟨horizontal command⟩ 之前自动插入。

\input⟨file name⟩ ⟨expandable command⟩ 读取指定文件作为 TEX 输入。

\inputlineno ⟨internal integer⟩ （仅 TEX3）当前输入行的行号。

\insert⟨8-bit number⟩{⟨vertical mode material⟩} ⟨primitive command⟩ 开始一个插入项。

\insertpenalties ⟨special integer⟩ 拆分插入物的罚值总和。在输出例程内部，表示暂留插入物的数量。

\interlinepenalty ⟨integer parameter⟩ 在段落各行之间分页的罚值。plain TEX 中默认值为 0。

\jobname ⟨expandable command⟩ 正在处理的主 TEX 文件的名称。
\kern⟨dimen⟩ ⟨kern⟩ 向列表中添加一个尺寸为指定⟨dimen⟩ 的紧排项；在水平模式和竖直模式下均可使用。
\language ⟨integer parameter⟩ （仅 TEX3）选择一组断字模式和例外词。
\lastbox ⟨box⟩ 若当前列表中最后添加的元素是盒子，则该寄存器保存它。
\lastkern ⟨internal dimen⟩ 若列表最后一项是紧排项，则为其尺寸。
\lastpenalty ⟨internal integer⟩ 若列表最后一项是罚值，则为其值。
\lastskip ⟨internal glue⟩ 或⟨internal muglue⟩。若列表最后一项是跳跃项，则为其尺寸。
\lccode⟨8-bit number⟩ ⟨internal integer⟩；该控制序列本身是一个 ⟨codename⟩。访问某给定字符码对应的小写字符码。
\leaders⟨box or rule⟩⟨vertical/horizontal/mathematical skip⟩ ⟨leaders⟩ 用一条线或盒子的副本填满指定空间。
\left ⟨primitive command⟩ 将其后的字符用作开始定界符。
\lefthyphenmin ⟨integer parameter⟩ （仅 TEX3）断字之前最少保留的字符数。
\leftskip ⟨glue parameter⟩ 放置在所有行左侧的粘连。
\leqno⟨math mode material⟩$$ ⟨eqno⟩ 在展示公式左侧放置公式编号。
\let⟨control sequence⟩⟨equals⟩⟨token⟩ ⟨let assignment⟩ 将一个控制序列定义为某个记号；若该记号是命令或宏，则赋予其含义。
\limits ⟨primitive command⟩ 将上下限置于大运算符的上方和下方。这是展示样式下的默认位置。
\linepenalty ⟨integer parameter⟩ 与每次断行关联的罚值。plain TEX 中默认值为 10。
\lineskip ⟨glue parameter⟩ 当相邻盒子底部与顶部之间的距离小于 \lineskiplimit 时添加的粘连。plain TEX 中默认值为 1pt。
\lineskiplimit ⟨dimen parameter⟩ 竖直列表中相邻盒子底部与顶部之间应保持的距离。plain TEX 中默认值为 0pt。
\long ⟨prefix⟩ 指示待定义宏的参数中允许包含 \par 记号。
\looseness ⟨integer parameter⟩ 本段应比理想长度多出（若为负则少）的行数。
\lower⟨dimen⟩⟨box⟩ ⟨primitive command⟩ 在水平模式下调整盒子的竖直位置。
\lowercase⟨general text⟩ ⟨primitive command⟩ 将参数转换为小写形式。
\mag ⟨integer parameter⟩ 文档放大倍数的 1000 倍。IniTEX 中默认值为 1000。
\mark⟨general text⟩ ⟨primitive command⟩ 指定一个标记文本。
\mathaccent⟨15-bit number⟩⟨math field⟩ ⟨primitive command⟩ 在数学模式下放置重音符号。
\mathbin⟨math field⟩ ⟨math atom⟩ 使其后的⟨math field⟩ 作为二元运算起作用。
\mathchar⟨15-bit number⟩ ⟨primitive command⟩ 显式指定一个数学字符。
\mathchardef⟨control sequence⟩⟨equals⟩⟨15-bit number⟩ ⟨shorthand definition⟩ 定义一个控制序列作为某个数学字符码的同义名。
\mathchoice{*D*}{*T*}{*S*}{*SS*} ⟨primitive command⟩ 为四种数学排版样式给出公式的四个变体。
\mathclose⟨math field⟩ ⟨math atom⟩ 使其后的⟨math field⟩ 作为闭符号起作用。
\mathcode⟨8-bit number⟩ ⟨internal integer⟩；该控制序列本身是一个 ⟨codename⟩。决定某字符在数学模式下处理方式的字符码。
\mathinner⟨math field⟩ ⟨math atom⟩ 使其后的⟨math field⟩ 作为内部公式起作用。
\mathop⟨math field⟩ ⟨math atom⟩ 使其后的⟨math field⟩ 作为大运算符起作用。
\mathopen⟨math field⟩ ⟨math atom⟩ 使其后的⟨math field⟩ 作为开符号起作用。
\mathord⟨math field⟩ ⟨math atom⟩ 使其后的⟨math field⟩ 作为普通对象起作用。

\mathpunct⟨math field⟩ ⟨math atom⟩ 使其后的⟨math field⟩ 作为标点符号起作用。
\mathrel⟨math field⟩ ⟨math atom⟩ 使其后的⟨math field⟩ 作为关系符起作用。
\mathsurround ⟨dimen parameter⟩ 行间公式前后放置的紧排量。
\maxdeadcycles ⟨integer parameter⟩ 允许输出例程在未发生 \shipout 的情况下被调用的最多次数。IniTEX 默认值： 25。
\maxdepth ⟨dimen parameter⟩ 页面盒子的最大深度。plain TEX 中默认值为 4pt。
\meaning ⟨expandable command⟩ 以字符串形式给出一个控制序列的含义。
\medmuskip ⟨muglue parameter⟩ 中等数量的 mu 粘连。plain TEX 中的默认值：4mu plus 2mu minus 4mu
\message⟨general text⟩ ⟨primitive command⟩ 向终端写入一条信息。
\mkern ⟨primitive command⟩ 插入以 mu 单位计量的紧排。
\month ⟨integer parameter⟩ 当前作业的月份。
\moveleft⟨dimen⟩⟨box⟩ ⟨primitive command⟩ 在竖直模式下调整盒子的水平位置。
\moveright⟨dimen⟩⟨box⟩ ⟨primitive command⟩ 在竖直模式下调整盒子的水平位置。
\mskip ⟨mathematical skip⟩ 插入以 mu 单位计量的粘连。
\multiply⟨numeric variable⟩⟨optional by⟩⟨number⟩ ⟨arithmetic assignment⟩ 用于乘以一个⟨numeric variable⟩ 的算术命令（见 \advance）。
\muskip⟨8-bit number⟩ ⟨internal muglue⟩；该控制序列本身是一个 ⟨register prefix⟩。访问以 mu 单位计量的跳跃寄存器。
\muskipdef⟨control sequence⟩⟨equals⟩⟨8-bit number⟩ ⟨shorthand definition⟩；该控制序列本身是一个 ⟨registerdef⟩。定义一个控制序列作为某个 \muskip 寄存器的同义名。
\newlinechar ⟨integer parameter⟩ 在 \write 语句中触发换行的字符编号。plain TEX 默认值 −1。
\noalign⟨filler⟩{⟨vertical (horizontal) mode material⟩} ⟨primitive command⟩ 指定竖直（水平）材料，置于 \halign（\valign）的行（列）之间。
\noboundary ⟨horizontal command⟩ （仅 TEX3）省略隐式边界字符。
\noexpand⟨token⟩ ⟨expandable command⟩ 不展开下一个记号。
\noindent ⟨primitive command⟩ 以空水平列表切换到水平模式。
\nolimits ⟨primitive command⟩ 将大运算符的上下限作为下标和上标表达式放置。这是文本样式下的默认位置。
\nonscript ⟨primitive command⟩ 若下一个粘连项出现在下标样式或二级下标样式中，则取消它。
\nonstopmode ⟨interaction mode assignment⟩ TEX 尽其所能力修复错误，在需要用户干预时紧急停止。
\nulldelimiterspace ⟨dimen parameter⟩ 空定界符所占的宽度。plain TEX 中默认值为 1.2pt。
\nullfont ⟨fontdef token⟩ TEX 在紧急情况下使用的空字体的名称。
\number⟨number⟩ ⟨expandable command⟩ 将一个⟨number⟩ 转换为十进制表示。
\omit ⟨primitive command⟩ 省略一个对齐条目的模板。
\openin⟨4-bit number⟩⟨equals⟩⟨filename⟩ ⟨primitive command⟩ 打开一个输入流。
\openout⟨4-bit number⟩⟨equals⟩⟨filename⟩ ⟨primitive command⟩ 打开一个输出流。
\or ⟨primitive command⟩ \ifcase 各分支之间的分隔符。
\outer ⟨prefix⟩ 指示待定义的宏只能出现在外层。
\output ⟨token parameter⟩ 包含输出页面指令的记号列表。

\outputpenalty ⟨integer parameter⟩ 当前分页处的罚值；若分页处不在罚值处，则为 10 000。

\over ⟨generalized fraction command⟩ 分式。

\overfullrule ⟨dimen parameter⟩ 用于标示过满水平盒子所打印竖线的宽度。plain TEX 默认值： 5pt。

\overline⟨math field⟩ ⟨math atom⟩ 在其后的⟨math field⟩ 上方加上横线。

\overwithdelims⟨delim$_1$⟩⟨delim$_2$⟩ ⟨generalized fraction command⟩ 带定界符的分式。

\pagedepth ⟨special dimen⟩ 当前页的深度。

\pagefilllstretch ⟨special dimen⟩ 当前页累积的三阶伸长量。

\pagefillstretch ⟨special dimen⟩ 当前页累积的二阶伸长量。

\pagefilstretch ⟨special dimen⟩ 当前页累积的一阶伸长量。

\pagegoal ⟨special dimen⟩ 页面盒子的目标高度。初始为 \vsize，并因插入项的高度而减小。

\pageshrink ⟨special dimen⟩ 当前页累积的收缩量。

\pagestretch ⟨special dimen⟩ 当前页累积的零阶伸长量。

\pagetotal ⟨special dimen⟩ 当前页累积的自然高度。

\par ⟨primitive command⟩ 结束一个段落并进入竖直模式。

\parfillskip ⟨glue parameter⟩ 放置在段落最后一个元素与行末之间的粘连。plain TEX 默认值： 0pt plus 1fil。

\parindent ⟨dimen parameter⟩ 段落前方所加缩进盒子的尺寸。

\parshape ⟨internal integer⟩ 用于一般段落形状的命令：

\parshape⟨equals⟩$n\ i_1\ \ell_1\ \ldots\ i_n\ \ell_n$

指定行数 n，以及 n 对缩进量与行长度。

\parskip ⟨glue parameter⟩ 段落开始时添加到竖直列表中的粘连量；plain TEX 中的默认值为 0pt plus 1pt。

\patterns⟨general text⟩ ⟨hyphenation assignment⟩ 为当前 \language 值定义一组断字模式；仅在 IniTEX 中允许。

\pausing ⟨integer parameter⟩ 指定 TEX 在从文件读入每一行之后暂停。

\penalty ⟨primitive command⟩ 指定不在此处断行的期望程度。

\postdisplaypenalty ⟨integer parameter⟩ 放置在展示式下方竖直列表中的罚值。

\predisplaypenalty ⟨integer parameter⟩ 放置在展示式上方竖直列表中的罚值。plain TEX 默认值： 10 000。

\predisplaysize ⟨dimen parameter⟩ 展示式前一行的有效宽度。

\pretolerance ⟨integer parameter⟩ 不使用断字时段落的容差值。plain TEX 中默认值为 100。

\prevdepth ⟨special dimen⟩ TEX 所感知的、添加到竖直列表中最后一个盒子的深度。

\prevgraf ⟨special integer⟩ 最近添加到竖直列表中的段落的行数。

\radical⟨24-bit number⟩ ⟨primitive command⟩ 用于排版根号等内容的命令。

\raise⟨dimen⟩⟨box⟩ ⟨primitive command⟩ 在水平模式下调整盒子的竖直位置。

\read⟨number⟩to⟨control sequence⟩ ⟨simple assignment⟩ 从一个流中读取一行到一个控制序列。

\relax ⟨primitive command⟩ 什么也不做。

\relpenalty ⟨integer parameter⟩ 在未被子公式括起的二元关系符之后断行的罚值。plain TEX 默认值： 500。

\right ⟨primitive command⟩ 将其后的字符用作闭定界符。

\righthyphenmin ⟨integer parameter⟩ （仅 TEX3）断字之后最少保留的字符数。

\rightskip ⟨glue parameter⟩ 放置在所有行右侧的粘连。

\romannumeral⟨number⟩ ⟨expandable command⟩ 将一个正的⟨number⟩ 转换为小写罗马数字表示。

\scriptfont⟨4-bit number⟩ ⟨family member⟩；该控制序列本身是一个 ⟨font range⟩。访问字体族的下标字体。

\scriptscriptfont⟨4-bit number⟩ ⟨family member⟩；该控制序列本身是一个 ⟨font range⟩。访问字体族的二级下标字体。

\scriptscriptstyle ⟨primitive command⟩ 选择数学排版的二级下标样式。

\scriptspace ⟨dimen parameter⟩ 下标和上标之后的额外间距。plain TEX 中默认值为 .5pt。

\scriptstyle ⟨primitive command⟩ 选择数学排版的下标样式。

\scrollmode ⟨interaction mode assignment⟩ TEX 自行修补错误，但在缺少文件时会询问用户。

\setbox⟨8-bit number⟩⟨equals⟩⟨box⟩ ⟨simple assignment⟩ 将一个盒子赋给一个盒子寄存器。

\setlanguage⟨number⟩ ⟨primitive command⟩ （仅 TEX3）插入一个 whatsit，将当前语言重置为指定的⟨number⟩。

\sfcode⟨8-bit number⟩ ⟨internal integer⟩；该控制序列本身是一个 ⟨codename⟩。访问与某字符关联的 \spacefactor 值。

\shipout⟨box⟩ ⟨primitive command⟩ 将一个盒子输出到 dvi 文件。

\show⟨token⟩ ⟨primitive command⟩ 在屏幕上显示一个记号的含义。

\showbox⟨8-bit number⟩ ⟨primitive command⟩ 将一个盒子的内容写入日志文件。

\showboxbreadth ⟨integer parameter⟩ 当\tracingoutput 为正时，每访问一层所显示的连续元素数量。plain TEX 默认值： 5。

\showboxdepth ⟨integer parameter⟩ 当\tracingoutput 为正时所显示的层数。plain TEX 默认值： 3。

\showlists ⟨primitive command⟩ 将所有模式下当前构建中的部分列表内容写入日志文件。

\showthe⟨internal quantity⟩ ⟨primitive command⟩ 在终端上显示将一个记号前加 \the 的结果。

\skewchar⟨font⟩ ⟨internal integer⟩ 后置重音符号的字体位置。

\skip⟨8-bit number⟩ ⟨internal glue⟩；该控制序列本身是一个 ⟨register prefix⟩。访问跳跃寄存器。

\skipdef⟨control sequence⟩⟨equals⟩⟨8-bit number⟩ ⟨shorthand definition⟩；该控制序列本身是一个 ⟨registerdef⟩。定义一个控制序列作为某个 \skip 寄存器的同义名。

\spacefactor ⟨special integer⟩ 词间粘连的伸长分量应乘以的比率的 1000 倍。

\spaceskip ⟨glue parameter⟩ 若非零，则为词间粘连。

\span ⟨primitive command⟩ 连接两个相邻的对齐条目，或（在前导码中）展开下一个记号。

\special⟨general text⟩ ⟨primitive command⟩ 将一个记号列表写入 dvi 文件。

\splitbotmark ⟨expandable command⟩ 被拆分出页面上的最后一个标记。

\splitfirstmark ⟨expandable command⟩ 被拆分出页面上的第一个标记。

\splitmaxdepth ⟨dimen parameter⟩ 经 \vsplit 操作拆分出盒子的最大深度。plain TEX 中默认值为 4pt。

\splittopskip ⟨glue parameter⟩ 经 \vsplit 操作后剩余部分顶部与该盒子中第一个元素之间的最小距离。plain TEX 中默认值为 10pt。

\string⟨token⟩ ⟨expandable command⟩ 将一个记号转换为一个或多个字符组成的字符串。

\tabskip ⟨glue parameter⟩ \halign（\valign）各列（行）之间的粘连量。

\textfont⟨4-bit number⟩ ⟨family member⟩；该控制序列本身是一个 ⟨font range⟩。访问字体族的文本字体。

\textstyle ⟨primitive command⟩ 选择数学排版的文本样式。

\the⟨internal quantity⟩ ⟨primitive command⟩ 将 TEX 中各种量的值展开为（字符）记号串。

\thickmuskip ⟨muglue parameter⟩ 大量的 mu 粘连。plain TEX 中的默认值：5mu plus 5mu。

\thinmuskip ⟨muglue parameter⟩ 少量的 mu 粘连。plain TEX 中的默认值：3mu。

\time ⟨integer parameter⟩ 当前作业自午夜起经过的分钟数。

\toks⟨8-bit number⟩ ⟨register prefix⟩ 访问一个记号列表寄存器。

\toksdef⟨control sequence⟩⟨equals⟩⟨8-bit number⟩ ⟨shorthand definition⟩；该控制序列本身是一个 ⟨registerdef⟩。将一个控制序列指派给某个 \toks 寄存器。

\tolerance ⟨integer parameter⟩ 使用断字时段落各行的容差值。plain TEX 中默认值为 200，IniTEX 中为 10 000。

\topmark ⟨expandable command⟩ 上一页的最后一个标记。

\topskip ⟨glue parameter⟩ 页面盒子顶部与页面上第一个盒子基线之间的最小距离。plain TEX 中默认值为 10pt。

\tracingcommands ⟨integer parameter⟩ 当此值为 1 时，TEX 显示已执行的原语命令；当此值为 2 或更大时，还会记录条件语句的结果。

\tracinglostchars ⟨integer parameter⟩ 若此参数为正，则每当访问某字体中不存在的字符时，TEX 都会给出诊断信息。plain TEX 默认值： 1。

\tracingmacros ⟨integer parameter⟩ 当此值为 1 时，日志文件显示所执行宏的展开以及参数的实际值；当此值为 2 或更大时，\output 和 \everypar 等⟨token parameter⟩ 也会被跟踪。

\tracingonline ⟨integer parameter⟩ 若此参数为正，TEX 还会将跟踪信息写入终端。

\tracingoutput ⟨integer parameter⟩ 若此参数为正，日志文件显示输出到 dvi 文件的盒子转储。

\tracingpages ⟨integer parameter⟩ 若此参数为正，TEX 生成分页算法的跟踪信息。

\tracingparagraphs ⟨integer parameter⟩ 若此参数为正，TEX 生成断行算法的跟踪信息。

\tracingrestores ⟨integer parameter⟩ 若此参数为正，TEX 报告在某一层分组结束时恢复的所有值。

\tracingstats ⟨integer parameter⟩ 若此参数为正，TEX 在作业结束时报告各种内部数组的使用情况。

\uccode⟨8-bit number⟩ ⟨internal integer⟩；该控制序列本身是一个 ⟨codename⟩。访问某给定字符码对应的大写字符码。

\uchyph ⟨integer parameter⟩ 若允许对以大写字母开头的单词断字，则此值为正。plain TEX 默认值 1。

\underline⟨math field⟩ ⟨math atom⟩ 在其后的⟨math field⟩ 下方加下划线。

\unhbox⟨8-bit number⟩ ⟨horizontal command⟩ 拆开一个含有水平盒子的盒子寄存器，将其内容追加到列表中，并清空该寄存器。

\unhcopy⟨8-bit number⟩ ⟨horizontal command⟩ 与 \unhbox 相同，但不清空该寄存器。

\unkern ⟨primitive command⟩ 若列表最后一项是紧排项，则移除它。

\unpenalty ⟨primitive command⟩ 若列表最后一项是罚值，则移除它。

\unskip ⟨primitive command⟩ 若列表最后一项是跳跃项，则移除它。

\unvbox⟨8-bit number⟩ ⟨vertical command⟩ 拆开一个含有竖直盒子的盒子寄存器，将其内容追加到列表中，并清空该寄存器。

\unvcopy⟨8-bit number⟩ ⟨vertical command⟩ 与 \unvbox 相同，但不清空该寄存器。

\uppercase⟨general text⟩ ⟨primitive command⟩ 将参数转换为大写形式。

\vadjust⟨filler⟩{⟨vertical mode material⟩} ⟨primitive command⟩ 在水平模式下为外层竖直列表指定材料。

\valign⟨box specification⟩{⟨alignment material⟩} ⟨horizontal command⟩ 竖直对齐。

\vbadness ⟨integer parameter⟩ 不报告过满或过松竖直盒子的阈值。plain TEX 默认值： 1000。

\vbox⟨box specification⟩{⟨vertical material⟩} ⟨primitive command⟩ 构造一个参考点位于最后一项的竖直盒子。

\vcenter⟨box specification⟩{⟨vertical material⟩} ⟨primitive command⟩ 构造一个以数学轴为竖直中心的竖直盒子。

\vfil ⟨vertical command⟩ 等价于\vskip 0cm plus 1fil 的竖直跳跃。

\vfill ⟨vertical command⟩ 等价于\vskip 0cm plus 1fill 的竖直跳跃。

\vfilneg ⟨vertical command⟩ 等价于\vskip 0cm minus 1fil 的竖直跳跃。

\vfuzz ⟨dimen parameter⟩ TEX 判定竖直盒子过满之前所容忍的超出量。plain TEX 默认值： 0.1pt。

\voffset ⟨dimen parameter⟩ 页面相对于参考点向下偏移的距离，参考点位于距页面顶部一英寸处。

\vrule ⟨horizontal command⟩ 沿竖直方向伸展的线。

\vsize ⟨dimen parameter⟩ 页面盒子的高度。

\vskip⟨glue⟩ ⟨vertical command⟩ 在竖直模式下插入一个粘连项。

\vsplit⟨8-bit number⟩to⟨dimen⟩ ⟨primitive command⟩ 从一个竖直盒子上拆分出顶部。

\vss ⟨vertical command⟩ 等价于\vskip 0cm plus 1fil minus 1fil 的竖直跳跃。

\vtop⟨box specification⟩{⟨vertical material⟩} ⟨primitive command⟩ 构造一个参考点位于第一项的竖直盒子。

\wd⟨8-bit number⟩ ⟨internal dimen⟩；该控制序列本身是一个 ⟨box dimension⟩。盒子寄存器中盒子的宽度。

\widowpenalty ⟨integer parameter⟩ 在段落最后一行之前分页的附加罚值。plain TEX 中默认值为 150。

\write⟨number⟩⟨general text⟩ ⟨primitive command⟩ 生成一个 whatsit 项，其中包含要写入终端或文件的记号列表。

\xdef ⟨def⟩ \global\edef 的同义命令。

\xleaders ⟨leaders⟩ 与 \leaders 相同，但对于盒子引导符，多余空间会均匀分布在各盒子之间。

\xspaceskip ⟨glue parameter⟩ 若非零且 \spacefactor $\geq$ 2000，则为词间粘连。

\year ⟨integer parameter⟩ 当前作业的年份。

第 38 章

表格

38.1 字符表

ASCII 控制码

dec
字符
hexoct

b7 b6 b5 位 b4 b3 b2 b1	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
	控制符		符号 数字		大写字母		小写字母	
0 0 0 0	0 0 NUL 0	16 10 DLE 20	32 20 SP 40	48 30 0 60	64 40 @ 100	80 50 P 120	96 60 ‘ 140	112 70 p 160
0 0 0 1	1 1 SOH 1	17 11 DC1 21	33 21 ! 41	49 31 1 61	65 41 A 101	81 51 Q 121	97 61 a 141	113 71 q 161
0 0 1 0	2 2 STX 2	18 12 DC2 22	34 22 ” 42	50 32 2 62	66 42 B 102	82 52 R 122	98 62 b 142	114 72 r 162
0 0 1 1	3 3 ETX 3	19 13 DC3 23	35 23 # 43	51 33 3 63	67 43 C 103	83 53 S 123	99 63 c 143	115 73 s 163
0 1 0 0	4 4 EOT 4	20 14 DC4 24	36 24 $ 44	52 34 4 64	68 44 D 104	84 54 T 124	100 64 d 144	116 74 t 164
0 1 0 1	5 5 ENQ 5	21 15 NAK 25	37 25 % 45	53 35 5 65	69 45 E 105	85 55 U 125	101 65 e 145	117 75 u 165
0 1 1 0	6 6 ACK 6	22 16 SYN 26	38 26 & 46	54 36 6 66	70 46 F 106	86 56 V 126	102 66 f 146	118 76 v 166
0 1 1 1	7 7 BEL 7	23 17 ETB 27	39 27 ’ 47	55 37 7 67	71 47 G 107	87 57 W 127	103 67 g 147	119 77 w 167
1 0 0 0	8 8 BS 10	24 18 CAN 30	40 28 (50	56 38 8 70	72 48 H 110	88 58 X 130	104 68 h 150	120 78 x 170
1 0 0 1	9 9 HT 11	25 19 EM 31	41 29) 51	57 39 9 71	73 49 I 111	89 59 Y 131	105 69 i 151	121 79 y 171
1 0 1 0	10 A LF 12	26 1A SUB 32	42 2A * 52	58 3A : 72	74 4A J 112	90 5A Z 132	106 6A j 152	122 7A z 172
1 0 1 1	11 B VT 13	27 1B ESC 33	43 2B + 53	59 3B ; 73	75 4B K 113	91 5B [133	107 6B k 153	123 7B { 173
1 1 0 0	12 C FF 14	28 1C FS 34	44 2C , 54	60 3C < 74	76 4C L 114	92 5C \ 134	108 6C l 154	124 7C | 174
1 1 0 1	13 D CR 15	29 1D GS 35	45 2D − 55	61 3D = 75	77 4D M 115	93 5D] 135	109 6D m 155	125 7D } 175
1 1 1 0	14 E SO 16	30 1E RS 36	46 2E . 56	62 3E > 76	78 4E N 116	94 5E ^ 136	110 6E n 156	126 7E ~ 176
1 1 1 1	15 F SI 17	31 1F US 37	47 2F / 57	63 3F ? 77	79 4F O 117	95 5F ← 137	111 6F o 157	127 7F DEL 177

TEX 字符代码

dec
字符
hexoct

b7 b6 b5 位 b4 b3 b2 b1	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
	控制符		符号 数字		大写字母		小写字母	
0 0 0 0	0 0 ^@ 0	16 10 ^P 20	32 20 SP 40	48 30 0 60	64 40 @ 100	80 50 P 120	96 60 ` 140	112 70 p 160
0 0 0 1	1 1 ^A 1	17 11 ^Q 21	33 21 ! 41	49 31 1 61	65 41 A 101	81 51 Q 121	97 61 a 141	113 71 q 161
0 0 1 0	2 2 ^B 2	18 12 ^R 22	34 22 " 42	50 32 2 62	66 42 B 102	82 52 R 122	98 62 b 142	114 72 r 162
0 0 1 1	3 3 ^C 3	19 13 ^S 23	35 23 # 43	51 33 3 63	67 43 C 103	83 53 S 123	99 63 c 143	115 73 s 163
0 1 0 0	4 4 ^D 4	20 14 ^T 24	36 24 $ 44	52 34 4 64	68 44 D 104	84 54 T 124	100 64 d 144	116 74 t 164
0 1 0 1	5 5 ^E 5	21 15 ^U 25	37 25 % 45	53 35 5 65	69 45 E 105	85 55 U 125	101 65 e 145	117 75 u 165
0 1 1 0	6 6 ^F 6	22 16 ^V 26	38 26 & 46	54 36 6 66	70 46 F 106	86 56 V 126	102 66 f 146	118 76 v 166
0 1 1 1	7 7 ^G 7	23 17 ^W 27	39 27 ' 47	55 37 7 67	71 47 G 107	87 57 W 127	103 67 g 147	119 77 w 167
1 0 0 0	8 8 ^H 10	24 18 ^X 30	40 28 (50	56 38 8 70	72 48 H 110	88 58 X 130	104 68 h 150	120 78 x 170
1 0 0 1	9 9 ^I 11	25 19 ^Y 31	41 29) 51	57 39 9 71	73 49 I 111	89 59 Y 131	105 69 i 151	121 79 y 171
1 0 1 0	10 A ^J 12	26 1A ^Z 32	42 2A * 52	58 3A : 72	74 4A J 112	90 5A Z 132	106 6A j 152	122 7A z 172
1 0 1 1	11 B ^K 13	27 1B ^[33	43 2B + 53	59 3B ; 73	75 4B K 113	91 5B [133	107 6B k 153	123 7B { 173
1 1 0 0	12 C ^L 14	28 1C ^\ 34	44 2C , 54	60 3C < 74	76 4C L 114	92 5C \ 134	108 6C l 154	124 7C | 174
1 1 0 1	13 D ^M 15	29 1D ^] 35	45 2D - 55	61 3D = 75	77 4D M 115	93 5D] 135	109 6D m 155	125 7D } 175
1 1 1 0	14 E ^N 16	30 1E ^^ 36	46 2E . 56	62 3E > 76	78 4E N 116	94 5E ^ 136	110 6E n 156	126 7E ~ 176
1 1 1 1	15 F ^O 17	31 1F ^_ 37	47 2F / 57	63 3F ? 77	79 4F O 117	95 5F _ 137	111 6F o 157	127 7F ^? 177

38.2 计算机现代字体

COMPUTER MODERN ROMAN 字体布局

0 0 Γ 0	16 10 ı 20	32 20 ˊ 40	48 30 0 60	64 40 @ 100	80 50 P 120	96 60 ‘ 140	112 70 p 160
1 1 Δ 1	17 11 ȷ 21	33 21 ! 41	49 31 1 61	65 41 A 101	81 51 Q 121	97 61 a 141	113 71 q 161
2 2 Θ 2	18 12 ˋ 22	34 22 ” 42	50 32 2 62	66 42 B 102	82 52 R 122	98 62 b 142	114 72 r 162
3 3 Λ 3	19 13 ˊ 23	35 23 # 43	51 33 3 63	67 43 C 103	83 53 S 123	99 63 c 143	115 73 s 163
4 4 Ξ 4	20 14 ˇ 24	36 24 $ 44	52 34 4 64	68 44 D 104	84 54 T 124	100 64 d 144	116 74 t 164
5 5 Π 5	21 15 ˘ 25	37 25 % 45	53 35 5 65	69 45 E 105	85 55 U 125	101 65 e 145	117 75 u 165
6 6 Σ 6	22 16 ¯ 26	38 26 & 46	54 36 6 66	70 46 F 106	86 56 V 126	102 66 f 146	118 76 v 166
7 7 Υ 7	23 17 ˚ 27	39 27 ’ 47	55 37 7 67	71 47 G 107	87 57 W 127	103 67 g 147	119 77 w 167
8 8 Φ 10	24 18 ¸ 30	40 28 (50	56 38 8 70	72 48 H 110	88 58 X 130	104 68 h 150	120 78 x 170
9 9 Ψ 11	25 19 ß 31	41 29) 51	57 39 9 71	73 49 I 111	89 59 Y 131	105 69 i 151	121 79 y 171
10 A Ω 12	26 1A æ 32	42 2A * 52	58 3A : 72	74 4A J 112	90 5A Z 132	106 6A j 152	122 7A z 172
11 B ff 13	27 1B œ 33	43 2B + 53	59 3B ; 73	75 4B K 113	91 5B [133	107 6B k 153	123 7B ˝ 173
12 C fi 14	28 1C ø 34	44 2C ’ 54	60 3C ¡ 74	76 4C L 114	92 5C “ 134	108 6C l 154	124 7C ¯ 174
13 D fl 15	29 1D Æ 35	45 2D - 55	61 3D = 75	77 4D M 115	93 5D] 135	109 6D m 155	125 „ 7D 175
14 E ffi 16	30 1E Œ 36	46 2E · 56	62 3E ¿ 76	78 4E N 116	94 5E ˆ 136	110 6E n 156	126 ˜ 7E 176
15 F ffl 17	31 1F Ø 37	47 2F / 57	63 3F ? 77	79 4F O 117	95 5F ˙ 137	111 6F o 157	127 ¨ 7F 177

COMPUTER MODERN TYPEWRITER 字体布局

0	16	32	48	64	80	96	112
0 0	10 20	20 40	30 0 60	40 @ 100	50 P 120	60 ` 140	70 p 160
1	17	33	49	65	81	97	113
1 1	11 21	21 ! 41	31 1 61	41 A 101	51 Q 121	61 a 141	71 q 161
2	18	34	50	66	82	98	114
2 2	12 22	22 " 42	32 2 62	42 B 102	52 R 122	62 b 142	72 r 162
3	19	35	51	67	83	99	115
3 3	13 23	23 # 43	33 3 63	43 C 103	53 S 123	63 c 143	73 s 163
4	20	36	52	68	84	100	116
4 4	14 24	24 $ 44	34 4 64	44 D 104	54 T 124	64 d 144	74 t 164
5	21	37	53	69	85	101	117
5 5	15 25	25 % 45	35 5 65	45 E 105	55 U 125	65 e 145	75 u 165
6	22	38	54	70	86	102	118
6 6	16 26	26 & 46	36 6 66	46 F 106	56 V 126	66 f 146	76 v 166
7	23	39	55	71	87	103	119
7 7	17 27	27 ' 47	37 7 67	47 G 107	57 W 127	67 g 147	77 w 167
8	24	40	56	72	88	104	120
8 10	18 30	28 (50	38 8 70	48 H 110	58 X 130	68 h 150	78 x 170
9	25	41	57	73	89	105	121
9 11	19 31	29) 51	39 9 71	49 I 111	59 Y 131	69 i 151	79 y 171
10	26	42	58	74	90	106	122
A 12	1A 32	2A * 52	3A : 72	4A J 112	5A Z 132	6A j 152	7A z 172
11	27	43	59	75	91	107	123
B 13	1B 33	2B + 53	3B ; 73	4B K 113	5B [133	6B k 153	7B { 173
12	28	44	60	76	92	108	124
C 14	1C 34	2C , 54	3C < 74	4C L 114	5C \ 134	6C l 154	7C | 174
13	29	45	61	77	93	109	125
D 15	1D 35	2D - 55	3D = 75	4D M 115	5D] 135	6D m 155	7D } 175
14	30	46	62	78	94	110	126
E 16	1E 36	2E . 56	3E > 76	4E N 116	5E ^ 136	6E n 156	7E ~ 176
15	31	47	63	79	95	111	127
F 17	1F 37	2F / 57	3F ? 77	4F O 117	5F – 137	6F o 157	7F 177

COMPUTER MODERN ITALIC 字体布局

0 0 Γ 0	16 10 ı 20	32 20 ˊ 40	48 30 0 60	64 40 @ 100	80 50 P 120	96 60 ‘ 140	112 70 p 160
1 1 Δ 1	17 11 ȷ 21	33 21 ! 41	49 31 1 61	65 41 A 101	81 51 Q 121	97 61 a 141	113 71 q 161
2 2 Θ 2	18 12 ˋ 22	34 22 ” 42	50 32 2 62	66 42 B 102	82 52 R 122	98 62 b 142	114 72 r 162
3 3 Λ 3	19 13 ˊ 23	35 23 # 43	51 33 3 63	67 43 C 103	83 53 S 123	99 63 c 143	115 73 s 163
4 4 Ξ 4	20 14 ˇ 24	36 24 £ 44	52 34 4 64	68 44 D 104	84 54 T 124	100 64 d 144	116 74 t 164
5 5 Π 5	21 15 ˘ 25	37 25 % 45	53 35 5 65	69 45 E 105	85 55 U 125	101 65 e 145	117 75 u 165
6 6 Σ 6	22 16 ¯ 26	38 26 & 46	54 36 6 66	70 46 F 106	86 56 V 126	102 66 f 146	118 76 v 166
7 7 Υ 7	23 17 ˚ 27	39 27 ’ 47	55 37 7 67	71 47 G 107	87 57 W 127	103 67 g 147	119 77 w 167
8 8 Φ 10	24 18 ¸ 30	40 28 (50	56 38 8 70	72 48 H 110	88 58 X 130	104 68 h 150	120 78 x 170
9 9 Ψ 11	25 19 ß 31	41 29) 51	57 39 9 71	73 49 I 111	89 59 Y 131	105 69 i 151	121 79 y 171
10 A Ω 12	26 1A æ 32	42 2A * 52	58 3A : 72	74 4A J 112	90 5A Z 132	106 6A j 152	122 7A z 172
11 B ff 13	27 1B œ 33	43 2B + 53	59 3B ; 73	75 4B K 113	91 5B ⌈ 133	107 6B k 153	123 7B ¯ 173
12 C fi 14	28 1C ø 34	44 2C , 54	60 3C < 74	76 4C L 114	92 5C “ 134	108 6C l 154	124 7C ¯ 174
13 D fl 15	29 1D Æ 35	45 2D - 55	61 3D = 75	77 4D M 115	93 5D ⌋ 135	109 6D m 155	125 7D ˝ 175
14 E ffi 16	30 1E Œ 36	46 2E . 56	62 3E ¿ 76	78 4E N 116	94 5E ˆ 136	110 6E n 156	126 7E ˜ 176
15 F ffl 17	31 1F Ø 37	47 2F / 57	63 3F ? 77	79 4F O 117	95 5F ˙ 137	111 6F o 157	127 7F ¨ 177

COMPUTER MODERN SYMBOL 字体

0 0 − 0	16 10 ≍ 20	32 20 ← 40	48 30 ′ 60	64 40 ℵ 100	80 50 $\mathcal{P}$ 120	96 60 ⊢ 140	112 70 √ 160
1 1 · 1	17 11 ≡ 21	33 21 → 41	49 31 ∞ 61	65 41 $\mathcal{A}$ 101	81 51 $\mathcal{Q}$ 121	97 61 ⊣ 141	113 71 ⊔ 161
2 2 × 2	18 12 ⊆ 22	34 22 ↑ 42	50 32 ∈ 62	66 42 $\mathcal{B}$ 102	82 52 $\mathcal{R}$ 122	98 62 ⌊ 142	114 72 ∀ 162
3 3 ∗ 3	19 13 ⊇ 23	35 23 ↓ 43	51 33 ∋ 63	67 43 $\mathcal{C}$ 103	83 53 $\mathcal{S}$ 123	99 63 ⌋ 143	115 73 ∫ 163
4 4 ÷ 4	20 14 ≤ 24	36 24 ↔ 44	52 34 △ 64	68 44 $\mathcal{D}$ 104	84 54 $\mathcal{T}$ 124	100 64 ⌈ 144	116 74 ⊔ 164
5 5 ⋄ 5	21 15 ≥ 25	37 25 ↗ 45	53 35 ▽ 65	69 45 $\mathcal{E}$ 105	85 55 $\mathcal{U}$ 125	101 65 ⌉ 145	117 75 ⊓ 165
6 6 ± 6	22 16 ≺ 26	38 26 ↘ 46	54 36 / 66	70 46 $\mathcal{F}$ 106	86 56 $\mathcal{V}$ 126	102 66 { 146	118 76 ⊑ 166
7 7 ∓ 7	23 17 ≻ 27	39 27 ≃ 47	55 37 ′ 67	71 47 $\mathcal{G}$ 107	87 57 $\mathcal{W}$ 127	103 67 } 147	119 77 ⊒ 167
8 8 ⊕ 10	24 18 ∼ 30	40 28 ⇐ 50	56 38 ∀ 70	72 48 $\mathcal{H}$ 110	88 58 $\mathcal{X}$ 130	104 68 ⟨ 150	120 78 § 170
9 9 ⊖ 11	25 19 ≈ 31	41 29 ⇒ 51	57 39 ∃ 71	73 49 $\mathcal{I}$ 111	89 59 $\mathcal{Y}$ 131	105 69 ⟩ 151	121 79 † 171
10 A ⊗ 12	26 1A ⊂ 32	42 2A ⇑ 52	58 3A ¬ 72	74 4A $\mathcal{J}$ 112	90 5A $\mathcal{Z}$ 132	106 6A ∣ 152	122 7A ‡ 172
11 B ⊘ 13	27 1B ⊃ 33	43 2B ⇓ 53	59 3B ∅ 73	75 4B $\mathcal{K}$ 113	91 5B ∪ 133	107 6B ∥ 153	123 7B ¶ 173
12 C ⊙ 14	28 1C ≪ 34	44 2C ⇔ 54	60 3C ℜ 74	76 4C $\mathcal{L}$ 114	92 5C ∩ 134	108 6C ↕ 154	124 7C ♣ 174
13 D ◯ 15	29 1D ≫ 35	45 2D ↖ 55	61 3D ℑ 75	77 4D $\mathcal{M}$ 115	93 5D ⊎ 135	109 6D ⇕ 155	125 7D ♢ 175
14 E ∘ 16	30 1E ≺ 36	46 2E ↙ 56	62 3E ⊤ 76	78 4E $\mathcal{N}$ 116	94 5E ∧ 136	110 6E ∖ 156	126 7E ♡ 176
15 F • 17	31 1F ≻ 37	47 2F ∝ 57	63 3F ⊥ 77	79 4F $\mathcal{O}$ 117	95 5F ∨ 137	111 6F ≀ 157	127 7F ♠ 177

COMPUTER MODERN 数学扩展字体

0	16	32	48	64	80	96	112
0 (0	10 (20	20 (40	30 (60	40 \ 100	50 $\sum$ 120	60 $\bigcup$ 140	70 $\sqrt{}$ 160
1	17	33	49	65	81	97	113
1) 1	11) 21	21) 41	31) 61	41 / 101	51 $\prod$ 121	61 $\bigcup$ 141	71 $\sqrt{}$ 161
2	18	34	50	66	82	98	114
2 [2	12 (22	22 [42	32 [62	42 ' 102	52 $\int$ 122	62 $\widehat{}$ 142	72 $\sqrt{}$ 162
3	19	35	51	67	83	99	115
3] 3	13) 23	23] 43	33] 63	43 ' 103	53 $\bigcup$ 123	63 $\widehat{}$ 143	73 $\sqrt{}$ 163
4	20	36	52	68	84	100	116
4 $\lfloor$ 4	14 [24	24 $\vert$ 44	34 $\lfloor$ 64	44 $\langle$ 104	54 $\bigcap$ 124	64 $\widehat{}$ 144	74 \ 164
5	21	37	53	69	85	101	117
5 ⌋ 5	15] 25	25 $\vert$ 45	35 ⌋ 65	45 $\rangle$ 105	55 $\biguplus$ 125	65 $\sim$ 145	75 ' 165
6	22	38	54	70	86	102	118
6 ⌈ 6	16 $\lfloor$ 26	26 [46	36 ' 66	46 $\bigcup$ 106	56 $\bigwedge$ 126	66 $\sim$ 146	76 ⌈ 166
7	23	39	55	71	87	103	119
7 ⌉ 7	17] 27	27 $\vert$ 47	37 ' 67	47 $\bigsqcup$ 107	57 $\bigvee$ 127	67 $\sim$ 147	77 $\Vert$ 167
8	24	40	56	72	88	104	120
8 { 10	18 ⌈ 30	28 { 50	38 ` 70	48 $\oint$ 110	58 $\sum$ 130	68 ⌈ 150	78 $\uparrow$ 170
9	25	41	57	73	89	105	121
9 } 11	19 $\vert$ 31	29 } 51	39 ' 71	49 $\oint$ 111	59 $\prod$ 131	69 ⌉ 151	79 $\downarrow$ 171
10	26	42	58	74	90	106	122
A $\langle$ 12	1A { 32	2A $\langle$ 52	3A ' 72	4A $\odot$ 112	5A $\int$ 132	6A $\lfloor$ 152	7A ´ 172
11	27	43	59	75	91	107	123
B $\rangle$ 13	1B } 33	2B $\rangle$ 53	3B ' 73	4B $\odot$ 113	5B $\bigcup$ 133	6B $\lfloor$ 153	7B ` 173
12	28	44	60	76	92	108	124
C ' 14	1C $\langle$ 34	2C / 54	3C { 74	4C $\oplus$ 114	5C $\bigcap$ 134	6C ⌈ 154	7C ` 174
13	29	45	61	77	93	109	125
D $\Vert$ 15	1D $\rangle$ 35	2D \ 55	3D } 75	4D $\bigoplus$ 115	5D $\biguplus$ 135	6D ⌉ 155	7D ´ 175
14	30	46	62	78	94	110	126
E / 16	1E / 36	2E / 56	3E ' 76	4E $\otimes$ 116	5E $\bigwedge$ 136	6E { 156	7E $\Uparrow$ 176
15	31	47	63	79	95	111	127
F \ 17	1F \ 37	2F \ 57	3F ' 77	4F $\bigotimes$ 117	5F $\bigvee$ 137	6F } 157	7F $\Downarrow$ 177

38.3 Plain TEX 数学符号

38.3.1 数学字符代码

下列字符已通过

\mathcode⟨8-bit number⟩⟨equals⟩⟨15-bit number⟩

赋值定义。

字符	\mathcode	类别	族	十六进制位置
.	"013A	普通	1	3A
/	"013D			3D
\	"026E		2	6E
|	"026A			6A
+	"202B	二元运算	0	2B
-	"2200		2	00
*	"2203			03
:	"303A	关系符	0	3A
=	"303D			3D
<	"313C		1	3C
>	"313E			3E
(	"4028	开符号	0	28
[	"405B			5B
{	"4266		2	66
!	"5021	闭符号	0	21
)	"5029			29
?	"503F			3F
]	"505D			5D
}	"5267		2	67
;	"603B	标点	0	3B
,	"613B		1	3B
	"8000			
'	"8000			
_	"8000			

38.3.2 定界符代码

下列字符已通过

\delcode⟨8-bit number⟩⟨equals⟩⟨24-bit number⟩

赋值定义。它们可与 \left 和 \right 配合使用。

字符	\delcode	小型变体 族	小型变体 十六进制位置	大型变体 族	大型变体 十六进制位置
(	"028300	0	28	3	00
)	"029301	0	29	3	01
[	"05B302	0	5B	3	02
]	"05D303	0	5D	3	03
<	"26830A	2	68	3	0A
>	"26930B	2	69	3	0B
/	"02F30E	0	2F	3	0E
|	"26A30C	2	6A	3	0C
\	"26E30F	2	6E	3	0F

38.3.3 ⟨mathchardef⟩ 记号：普通符号

下列字符已通过

\mathchardef⟨control sequence⟩⟨equals⟩⟨15-bit number⟩

赋值定义。

符号	控 制 序 列	\mathcode	族	十六进制位置
∂	\partial	"0140	1	40
$\flat$	\flat	"015B		5B
$\natural$	\natural	"015C		5C
$\sharp$	\sharp	"015D		5D
ℓ	\ell	"0160		60
$\imath$	\imath	"017B		7B
$\jmath$	\jmath	"017C		7C
$\wp$	\wp	"017D		7D
$\prime$	\prime	"0230	2	30
∞	\infty	"0231		31
$\triangle$	\triangle	"0234		34
$\forall$	\forall	"0238		38
$\exists$	\exists	"0239		39
$\neg$	\neg	"023A		3A
$\emptyset$	\emptyset	"023B		3B
$\Re$	\Re	"023C		3C
$\Im$	\Im	"023D		3D
$\top$	\top	"023E		3E
$\bot$	\bot	"023F		3F
$\aleph$	\aleph	"0240		40
∇	\nabla	"0272		72
$\clubsuit$	\clubsuit	"027C		7C
$\diamondsuit$	\diamondsuit	"027D		7D
$\heartsuit$	\heartsuit	"027E		7E
$\spadesuit$	\spadesuit	"027F		7F

38.3.4 ⟨mathchardef⟩ 记号：大型算符

下列字符已通过

\mathchardef⟨control sequence⟩⟨equals⟩⟨15-bit number⟩

赋值定义。

符 号	控制序列	\mathcode	族	十六进制位置
$\int\!\!\int$	\smallint	"1273	2	73
$\bigsqcup$	\bigsqcup	"1346	3	46
$\oint$	\ointop	"1348		48
$\bigodot$	\bigodot	"134A		4A
$\bigoplus$	\bigoplus	"134C		4C
$\bigotimes$	\bigotimes	"134E		4E
$\sum$	\sum	"1350		50
$\prod$	\prod	"1351		51
$\int$	\intop	"1352		52
$\bigcup$	\bigcup	"1353		53
$\bigcap$	\bigcap	"1354		54
$\biguplus$	\biguplus	"1355		55
$\bigwedge$	\bigwedge	"1356		56
$\bigvee$	\bigvee	"1357		57
$\coprod$	\coprod	"1360		60

38.3.5 ⟨mathchardef⟩ 记号：二元运算

下列字符已通过

\mathchardef⟨control sequence⟩⟨equals⟩⟨15-bit number⟩

赋值定义。

符号	控制序列	\mathcode	族	十六进制位置
▷	\triangleright	"212E	1	2E
◁	\triangleleft	"212F		2F
⋆	\star	"213F		3F
⋅	\cdot	"2201	2	01
×	\times	"2202		02
∗	\ast	"2203		03
÷	\div	"2204		04
◇	\diamond	"2205		05
±	\pm	"2206		06
∓	\mp	"2207		07
⊕	\oplus	"2208		08
⊖	\ominus	"2209		09
⊗	\otimes	"220A		0A
⊘	\oslash	"220B		0B
⊙	\odot	"220C		0C
◯	\bigcirc	"220D		0D
∘	\circ	"220E		0E
•	\bullet	"220F		0F
△	\bigtriangleup	"2234		34
▽	\bigtriangledown	"2235		35
∪	\cup	"225B		5B
∩	\cap	"225C		5C
⊎	\uplus	"225D		5D
∧	\wedge	"225E		5E
∨	\vee	"225F		5F
\	\setminus	"226E		6E
≀	\wr	"226F		6F
∐	\amalg	"2271		71
⊔	\sqcup	"2274		74
⊓	\sqcap	"2275		75
†	\dagger	"2279		79
‡	\ddagger	"227A		7A

38.3.6 ⟨mathchardef⟩ 记号：关系符

下列字符已通过

\mathchardef⟨control sequence⟩⟨equals⟩⟨15-bit number⟩

赋值定义。

符号	控制序列	\mathcode	族	十六进制位置
↼	\leftharpoonup	"3128	1	28
↽	\leftharpoondown	"3129		29
⇀	\rightharpoonup	"312A		2A
⇁	\rightharpoondown	"312B		2B
⌣	\smile	"315E		5E
⌢	\frown	"315F		5F
≍	\asymp	"3210	2	10
≡	\equiv	"3211		11
⊆	\subseteq	"3212		12
⊇	\supseteq	"3213		13
≤	\leq	"3214		14
≥	\geq	"3215		15
⪯	\preceq	"3216		16
⪰	\succeq	"3217		17
∼	\sim	"3218		18
≈	\approx	"3219		19
⊂	\subset	"321A		1A
⊃	\supset	"321B		1B
≪	\ll	"321C		1C
≫	\gg	"321D		1D
≺	\prec	"321E		1E
≻	\succ	"321F		1F
←	\leftarrow	"3220		20
→	\rightarrow	"3221		21
↔	\leftrightarrow	"3224		24
↗	\nearrow	"3225		25
↘	\searrow	"3226		26
≃	\simeq	"3227		27
⇐	\Leftarrow	"3228		28
⇒	\Rightarrow	"3229		29
⇔	\Leftrightarrow	"322C		2C
↖	\nwarrow	"322D		2D
↙	\swarrow	"322E		2E
∝	\propto	"322F		2F
∈	\in	"3232		32
∋	\ni	"3233		33
̸	\not	"3236		36
↦	\mapstochar	"3237		37
⊥	\perp	"323F		3F
⊢	\vdash	"3260		60
⊣	\dashv	"3261		61

∣	\mid	"326A	6A
∥	\parallel	"326B	6B
⊑	\sqsubseteq	"3276	76
⊒	\sqsupseteq	"3277	77

38.3.7 \delimiter 宏

下列字符已通过

\def⟨control sequence⟩{\delimiter⟨27-bit number⟩}

赋值定义。

定界符号	控制序列	十六进制代码	功能
⎧	\lmoustache	"4000340	开符号
⎫	\rmoustache	"5000341	闭符号
⎛	\lgroup	"400033A	开符号
⎞	\rgroup	"500033B	闭符号
∣	\arrowvert	"33C	普通
∥	\Arrowvert	"33D	普通
⎪	\bracevert	"33E	普通
∥	\Vert	"26B30D	普通
∣	\vert	"26A30C	普通
↑	\uparrow	"3222378	关系符
↓	\downarrow	"3223379	关系符
↕	\updownarrow	"326C33F	关系符
⇑	\Uparrow	"322A37E	关系符
⇓	\Downarrow	"322B37F	关系符
⇕	\Updownarrow	"326D377	关系符
\	\backslash	"26E30F	普通
⟩	\rangle	"526930B	闭符号
⟨	\langle	"426830A	开符号
}	\rbrace	"5267309	闭符号
{	\lbrace	"4266308	开符号
⌉	\rceil	"5265307	闭符号
⌈	\lceil	"4264306	开符号
⌋	\rfloor	"5263305	闭符号
⌊	\lfloor	"4262304	开符号

索引

\-, 151
\/, 39
LaTeX, 222
TeX, 219
TeX version 3, 48, 153
TeX 第 3 版, 207
TeX version 2, 237
\{, 87
\}, 87
\ , 18, 24, 157
\^, 171
^^ replacement, 18
_, 171
WEB, 226
dvi file, 222
tfm files, 38
~, 157

\above, 175
\abovedisplayshortskip, 179
\abovedisplayskip, 179
\abovewithdelims, 175
\accent, 31
accents in math mode, 163
\active, 17
\adjdemerits, 149
\advance, 65, 71
\afterassignment, 49, 103
\aftergroup, 104
\allowbreak, 176
arithmetic, 65
    fixed-point, 66
    floating-point, 65
    on glue, 71
\atop, 175
\atopwithdelims, 175

badness
    and line breaking, 148
    calculation, 75
\badness, 48
baseline
    distance, 129
\baselineskip, 129
\batchmode, 218
\begingroup, 85
\belowdisplayshortskip, 179
\belowdisplayskip, 179
\bgroup, 85
\big etc., 163
big TeX, 233
\binoppenaly, 176
\botmark, 202
box, 41
    dimensions, 44
    registers, 42
    unboxing, 50
\box, 43
boxes
    text in, 51
\boxmaxdepth, 45
\brokenpenalty, 195
\bye, 218

\c, 31
\cases, 189
\catcode, 17
category codes, 16
\centering, 189
\char, 29
character
    escape, 19, 21
    hyphen, 151
    parameter, 20, 92
    space, 20
\chardef, 29
\cleaders, 81
\closein, 210
\closeout, 210
\clubpenalty, 195
Computer Modern typefaces, 225
conditional, 114

conditionals
evaluation of, 119
control
sequence, 18
space, 18, 157
symbol, 18
\copy, 43
\count, 64
\countdef, 65
\cr, 189
\crcr, 189
\csname, 93

date, 224
\day, 224
\deadcycles, 201
\def, 90
\defaulthyphenchar, 151
\defaultskewchar, 164
\delcode, 162
\delimiter, 162
delimiter codes, 162
delimiter sizes, 162
\delimiterfactor, 162
delimiters, 161
\delimitershortfall, 162
demerits, 149
device drivers, 224, 225
\dimen, 71
\dimendef, 71
discardable items, 56
\discretionary, 151
discretionary hyphen, 151
discretionary item, 151
\displayindent, 179
\displaylimits, 173
\displaystyle, 171
\displaywidowpenalty, 195
\displaywidth, 179
\divide, 65, 72
\doublehyphendemerits, 149
\dp, 47
\dump, 220

\edef, 90, 103
\egroup, 85
\else, 114
\emergencystretch, 150
\end, 218
\endcsname, 93
\endgroup, 85
\endinput, 209
\endinsert, 208
\endlinechar, 16
\eqalignno, 189
\eqno, 180
\errhelp, 232
\errmessage, 231
error patching, 231
\errorcontextlines, 231
\errorstopmode, 218
\escapechar, 34
evaluation
conditionals, 119
\everycr, 189
\everydisplay, 179
\everyhbox, 49
\everyjob, 217
\everymath, 170
\everypar, 134
\everyvbox, 49
\exhyphenpenalty, 151
\expandafter, 102
extension fonts, 162

\fam, 166
\fi, 114
\finalhyphendemerits, 149
\firstmark, 202
\floatingpenalty, 207
font
dimensions, 38
\font, 37
font files, 225
font metrics, 224
\fontdimen, 38
\fontname, 38
fonts, 36
format files, 219
frenchspacing, 158
\frenchspacing, 158
\futurelet, 95

\gdef, 90
\global, 85, 89
\globaldefs, 85
glue
setting, 74
arithmetic on, 71
shrink component of, 73

stretch component of, 73
glue
interline, 129

\halign, 183
\hangafter, 142
\hangindent, 142
hanging
indentation, 142
\hbadness, 49
\hbox, 44
\hfil, 68
\hfill, 68
\hfilneg, 68
\hglue, 75
\hidewidth, 188
\hoffset, 191
\holdinginserts, 207
horizontal commands, 56
\hrule, 78
\hsize, 142
\hskip, 73
\hss, 68
\ht, 47
hyphenation, 152
\hyphenation, 153
\hyphenchar, 151
\hyphenpenalty, 151

I/O, 209
\if, 115
\ifcase, 117
\ifcat, 115
\ifdim, 117
\ifeof, 117
\iffalse, 118
\ifhbox, 117
\ifhmode, 116
\ifinner, 116
\ifmmode, 116
\ifnum, 116
\ifodd, 117
\iftrue, 118
\ifvbox, 117
\ifvmode, 116
\ifvoid, 117
\ifx, 116
\ignorespaces, 23
\immediate, 211
\indent, 134
IniTEX, 219
\input, 209
\inputlineno, 211
\insert, 205
\insertpenalties, 207
integers, 61
\interlinepenalty, 195
italic correction, 39

job, 217
\jobname, 217

\kern, 75
kerning, 39
keywords, 235

language, 153
\language, 153, 154
language
current, 154
languages, 151
\lastbox, 44
\lastkern, 76
\lastpenalty, 195
\lastskip, 76
\lccode, 32
\leaders, 80
\leavevmode, 50
\left, 161
\lefthyphenmin, 153
\leftskip, 142
\leqno, 180
\let, 30, 94
ligatures, 39
\limits, 172
line
end, 15, 24
input, 15
width, 142
line
empty, 21
line breaking
badness, 148
\linepenalty, 149
\lineskip, 130
\lineskiplimit, 130
list
horizontal, 55
token, 125
vertical, 56

lists
horizontal
breakpoints in, 148
penalties in , 148
\llap, 48
Lollipop, 222
\long, 89
\loop, 96
\looseness, 149
\lower, 47
\lowercase, 33

machine independence, 15
\mag, 223
magnification, 223
\magnification, 223
\mark, 201
math characters, 161
\mathaccent, 163
\mathbin, 172
\mathchar, 161
\mathchardef, 161
\mathchoice, 171
\mathclose, 172
\mathcode, 161
\mathinner, 172
\mathop, 172
\mathopen, 172
\mathord, 172
\mathpunct, 172
\mathrel, 172
\mathsurround, 175
\matrix, 189
\maxdeadcycles, 201
\maxdepth, 192
\meaning, 228
\medmuskip, 174
\message, 212
\midinsert, 208
migrating material, 58
\mkern, 174
mode, 55
mode
horizontal, 55
internal, 57
restricted, 57
vertical, 56
\month, 224
\moveleft, 45, 47
\moveright, 45, 47
\mskip, 174
mu 黏合, 173
\multiply, 65, 72
\multispan, 188
\muskip, 174
\muskipdef, 174

\newbox, 42
\newcount, 65
\newdimen, 71
\newfam, 166
\newif, 118
\newinsert, 206
\newlanguage, 214
\newlinechar, 212
\newmuskip, 174
\newread, 210
\newskip, 71
\newtoks, 126
\newwrite, 210
\noalign, 184
\noboundary, 40
\noexpand, 105
\noindent, 134
\nointerlineskip, 131
\nolimits, 172
\nonfrenchspacing, 158
\nonscript, 175
\nonstopmode, 218
\nulldelimiterspace, 162
\nullfont, 38
number
conversion, 64
roman numerals, 64
\number, 64
numbers, 61

\obeylines, 25
\offinterlineskip, 131
\omit, 187
\openin, 210
\openout, 210
\openup, 132
\or, 117
\outer, 89
\output, 200
\outputpenalty, 194
\over, 175
overflow errors, 232
\overfullrule, 49

\overline, 175
\overwithdelims, 175

\pagedepth, 194
\pagefilllstretch, 194
\pagefillstretch, 194
\pagefilstretch, 194
\pagegoal, 206
\pageinsert, 208
\pageshrink, 194
\pagestretch, 194
\pagetotal, 194
\par, 21
paragraph
  breaking into lines, 150
  end, 138
  shape, 142
  start, 134
\parfillskip, 139
\parindent, 134
\parshape, 143
\parskip, 134
Pascal, 226
\patterns, 153
\pausing, 211
\penalty, 148, 195
\postdisplaypenalty, 179
PostScript, 224
\predisplaypenalty, 179
\predisplaysize, 179
\pretolerance, 150
\prevdepth, 130, 131
\prevgraf, 149
\protect, 110

\radical, 163
radicals, 163
\raggedbottom, 192
\raggedright, 142
\raise, 47
\read, 210
registers, allocation of, 214
\relax, 106
\relpenalty, 176
\removelastskip, 76
\right, 161
\righthyphenmin, 153
\rightskip, 142
\rlap, 48
\romannumeral, 64
run modes, 218

\scriptfont, 166
\scriptscriptfont, 166
\scriptscriptstyle, 171
\scriptspace, 171
\scriptstyle, 171
\scrollmode, 218
\setbox, 43, 49
\sfcode, 156, 157
\shipout, 201
\show, 228
\showbox, 228
\showboxbreadth, 228
\showboxdepth, 228
\showlists, 59
\showthe, 228
shrink, 73
\skew, 163
\skewchar, 164
\skip, 71
\skipdef, 71
\smash, 47
space
  factor, 156
  optional , 22
  token, 24
space
  control –, 157
  funny, 24
space, optional, 236
\spacefactor, 156
spacefactor code, 157
\spaceskip, 156
spacing, 155
\span, 186, 187
\special, 223
specials, 223
\splitbotmark, 202
\splitfirstmark, 202
\splitmaxdepth, 196
\splittopskip, 197
state
  internal, 18
statistics, 227
stretch, 73
\string, 34, 105
\strut, 188
successors, 162

\tabskip, 186
\textfont, 166
\textstyle, 171
\the, 107
\thickmuskip, 174
\thinmuskip, 174
tie, 157
time, 224
\time, 224
token
    lists, 125
token
    space, 22
\toks, 126
\toksdef, 126
\tolerance, 150
\topmark, 202
\topskip, 191
tracing, 227
\tracingcommands, 227
\tracinglostchars, 227
\tracingmacros, 227
\tracingonline, 227
\tracingoutput, 228
\tracingpages, 227
\tracingparagraphs, 150
\tracingrestores, 227
\tracingstats, 230
TUG, 226
TUGboat, 226

\uccode, 32
\uchyph, 152
\underline, 175
\unhbox, 50
\unhcopy, 50
units of measurement, 72
\unkern, 76
\unpenalty, 195
\unskip, 76
\unvbox, 50
\unvcopy, 50
\uppercase, 33

\vadjust, 59
\valign, 183
\vbadness, 49
\vbox, 45
\vcenter, 173
verbatim 模式, 100
vertical
    commands, 56
\vfil, 68
\vfill, 68
\vfilneg, 68
\vglue, 75
VirTEX, 219
virtual fonts, 224
\voffset, 191
\vrule, 78
\vsize, 192
\vskip, 73
\vsplit, 196
\vss, 68
\vtop, 45

\wd, 47
whatsit, 211
\widehat, 164
\widetilde, 164
\widowpenalty, 195
\write, 109, 211

\xdef, 90
\xleaders, 81
\xspaceskip, 156

\year, 224

上标, 171
下标, 171
代码名, 34
保存栈, 84

全局语句, 85
公式编号, 180
分组, 84
分页, 195
前缀！宏, 89
原语命令, 95
参数, 90
参数！定界, 91
参数！未定界, 90
命令！原语, 95
大写, 32
字体族, 165
字体表, 254
字符
    代码, 28
    隐式, 30
字符！活动, 95

字符！活动，与 \noexpand, 105
宏, 88
宏！外层, 89
定义！宏, 88
定界符！分组, 85
对齐, 183
对齐中的线, 188
对齐定位符, 185
小写, 32
局部语句, 85
展开, 101
展开！可展开控制序列, 101
展示公式, 178
展示公式，非居中, 181
展示对齐, 184
广义分式, 175
引线, 79
引线！线段, 80
当前页面, 193

插入, 205
数学公式的居中, 173
数学公式的轴, 173
数学列表中的断点, 176
数学单位, 173
数学样式, 170
数学模式, 170
数学模式中的惩罚, 176
数学移位字符, 170
数学符号，列表, 259
数学类别, 172
数学间距, 173
断点的计算, 195
机器依赖性, 180
标记, 201
水平对齐, 184
活动字符, 95
流, 210

竖直列表中的断点, 195
竖直对齐, 184
竖直模式中的惩罚, 195
符号字体, 176
紧缩样式, 171
线段, 78

胶, 68
自变量, 90
花括号, 86
表格, 183

表，ASCII, 252
表，字体, 254
表，字符代码, 253
语句！全局, 85
语句！局部, 85
输入文件, 209
输入！栈, 96
输出例程, 200
近期贡献, 193

递归, 96
重音, 31
页码, 202
页面定位, 191
页面构造器, 193
页面深度, 192
页面长度, 194

参考文献

[1] W. Appelt. *TEX für Fortgeschrittene*. Addison-Wesley Verlag, 1988. 227

[2] B. Beeton. Controlling `<ctrl-M>`; ruling the depths. *TUGboat*, 9:182–183, 1988. 32

[3] B. Beeton. Additional font and glyph attributes for processing of mathematics, 1991. document N1174 Rev.,of ISO/IEC JTC1/SC18/WG8. 171

[4] K. Berry. Eplain. *TUGboat*, 11:571–572, 1990. 214

[5] J. Braams. Babel, a language option for LATEX. *TUGboat*, 12:291–301, 1991. 149

[6] J. Braams, V. Eijkhout, and N.A.F.M. Poppelier. The development of national LATEX styles. *TUGboat*, 10:401–406, 1989. 214

[7] M.J. Downes. Line breaking in `\unhbox`ed text. *TUGboat*, 11:605–612. 55

[8] V. Eijkhout. An indentation scheme. *TUGboat*, 11:613–616. 135

[9] V. Eijkhout. A paragraph skip scheme. *TUGboat*, 11:616–619. 136

[10] V. Eijkhout. Unusual paragraph shapes. *TUGboat*, 11:51–53. 58, 144

[11] V. Eijkhout. Oral TEX. *TUGboat*, 12:272–276, 1991. 114, 123

[12] V. Eijkhout and A. Lenstra. The document style designer as a separate entity. *TUGboat*, 12:31–34, 1991. 214

[13] D. Guenther. TEX T1 goes public domain. *TUGboat*, 11:54–55, 1990. 214

[14] *Hart's Rules for Compositors and Readers at the Oxford University Press*. Oxford University Press, 1983. 39th edition. 156

[15] A. Hendrikson. *MacroTEX, A TEX Macro Toolkit*. TEX__nology Inc, 1991. 214

[16] A. Jeffrey. Lists in TEX's mouth. *TUGboat*, 11:237–245, 1990. 114

[17] D.E. Knuth. *Computer Modern Typefaces*. Addison-Wesley. 45

[18] D.E. Knuth. The errors of TEX. *Software Practice and Experience*, 19:607–681. 229

[19] D.E. Knuth. Literate programming. *Computer J.*, 27:97–111. 217

[20] D.E. Knuth. The new versions of TEX and Metafont. *TUGboat*, 10:325–327. 45, 229

[21] D.E. Knuth. A torture test for TEX. Technical report, Stanford Computer Science Report 1027, Stanford, California. 69

[22] D.E. Knuth. Typesetting concrete mathematics. *TUGboat*, 10:31–36. 217

[23] D.E. Knuth. *TEX: the Program*. Addison-Wesley, 1986. 36, 44, 211, 214, 216

[24] D.E. Knuth. Virtual fonts: more fun for grand wizards. *TUGboat*, 11:13–23, 1990. 42, 216

[25] D.E. Knuth. *The TEX book*. Addison-Wesley, reprinted with corrections 1989. 31, 40, 224, 225

[26] D.E. Knuth and D.R. Fuchs. TEX ware. Technical report, 1986. Stanford Computer Science report 86–1097. 216

[27] D.E. Knuth and M.F. Plass. Breaking paragraphs into lines. *Software practice and experience*, 11:1119–1184, 1981. 146, 147
[28] G. Kuiken. Additional hyphenation patterns. *TUGboat*, 11:24–25, 1990. 150
[29] L. Lamport. *LaTeX, a Document Preparation System*. Addison-Wesley, 1986. 213
[30] F.M. Liang. *Word hy-phen-a-tion by com-pu-ter*. PhD thesis, 1983. 150, 225
[31] S. Maus. Looking ahead for a ⟨box⟩. *TUGboat*, 11:612–613, 1990. 106
[32] S. Maus. An expansion power lemma. *TUGboat*, 12:277, 1991. 114
[33] F. Mittelbach and R. Schöpf. LaTeX3. *TUGboat*, 12. 214
[34] F. Mittelbach and R. Schöpf. With LaTeX into the nineties. *TUGboat*, 10:681–690, 1989. 214
[35] E. Myers and F.E. Paige. TeX sis – TeX macros for physicists. Macros and manual available by anonymous ftp from lifshitz.ph.utexas.edu (128.83.131.57). 214
[36] H. Partl. German TeX. *TUGboat*, 9:70–72, 1988. 149
[37] Z. Rubinstein. Printing annotated chess literature in natural notation. *TUGboat*, 10:387–390, 1989. 98
[38] D. Salomon. Output routines: Examples and techniques. part i: Introduction and examples. *TUGboat*, 11:69–85, 1990. 197
[39] D. Salomon. Output routines: Examples and techniques. part ii: OTR techniques. *TUGboat*, 11:212–236, 1990. 197
[40] D. Salomon. Output routines: Examples and techniques. part iii: Insertions. *TUGboat*, 11:588–605, 1990. 201
[41] W. Sewell. *Weaving a Program: Literate Programming in* `WEB`. Van Nostrand Reinhold, 1989. 217
[42] R. Southall. Designing a new typeface with metafont. In *TeX for scientific documentation, Lecture Notes in Computer Science 236*. Springer Verlag, 1984. 42, 217
[43] M. Spivak. *The Joy of TeX*. American Mathematical Society, 1986. 214
[44] M. Spivak. *LAMSTeX, the Synthesis*. The TeX plorators Corporation, 1989. 214
[45] K. Thull. The virtual memory management of public TeX. *TUGboat*, 10:15–22, 1989. 225
[46] J. Tschichold. *Ausgewählte Aufsätze über Fragen der Gestalt des Buches und der Typographie*. Birkhäuser Verlag, 1975. 139
[47] P. Tutelaers. A font and a style for typesetting chess using LaTeX or plain TeX. *TUGboat*, 13, 1991. 98
[48] D.B. Updike. *Printing Types*. Harvard University Press, 1937. (reprinted 1980, New York NY: Dover Publications). 217
[49] S. von Bechtolsheim. A tutorial on `\futurelet`. *TUGboat*, 9:276–278, 1988. 100
[50] M. Vox. *Caractère*, 1955. 217
[51] M. Weinstein. Everything you wanted to know about phyzzx but didn't know to ask. Technical report, 1984. Stanford Linear Accelerator Publication, SLAC-TN-84-7. 214
[52] J.V. White. *Graphic Design for the Electronic Age*. Watson-Guptill, 1988. 131

变更日志

版本 1.1

新增了一条关于宏定义之后 `\afterassignment` 的小注。

修正了索引宏的问题，但愿如此。

分出 letter 与 a4 两种纸张版本。

改进了关于间距的第 20 章的引言。